0{"sid":"HSJR3YuCOE9jPqHEAAAA","upgrades":
["websocket"],"pingInterval":25000,"pingTimeout":20000,"maxPayload":1000000}

The Socket.IO Book

From "what is real-time?" to running Socket.IO at scale: every concept explained simply and then in depth, with production practices, a troubleshooting manual, labs, and an interview question bank for entry-level to senior roles.

24 chapters Socket.IO **4.8.4** Engine.IO protocol **v4** · Socket.IO protocol **v5**

Examples run, not just written

Contents

PART I · THE CORE

01 Foundations: What "Real-Time" Means and Why Socket.IO Exists

02 Getting Started: Your First Server and Client

03 Events, Emitting, Listening and Acknowledgements

04 Rooms, Namespaces and Broadcasting

05 Connection Lifecycle: Connect, Disconnect, Reconnect, Heartbeats, Recovery and Delivery Guarantees

06 Middleware, Authentication and Security

07 Under the Hood: Engine.IO, Transports, the Protocol, Parsers and Adapters

08 Scaling: Multiple Servers, Sticky Sessions, Adapters, Load Balancers and Kubernetes

09 TypeScript, Frontend Frameworks, Mobile and Other Languages

10 Testing, Debugging and Monitoring

11 Performance Tuning and Running in Production

12 Troubleshooting: Common Bugs, Symptoms, Causes and Fixes

13 Architecture, Patterns, Organisational Best Practices and Full Projects

14 Interview Question Bank (Entry Level → Senior/Architect)

15 Cheat Sheet, API Reference and Glossary

PART II · BEYOND THE CORE

16 Native Mobile Clients: Android (Java/Kotlin), iOS (Swift) and Flutter (Dart)

17 Socket.IO Servers and Clients in Other Languages (Python, Go, Rust, Java)

18 WebTransport in Practice (HTTP/3)

19 Writing a Custom Adapter and a Custom Parser

20 Other Runtimes: Electron, Web/Shared/Service Workers, Edge, Bun and Deno

21 Offline-First Clients and End-to-End Encryption

22 Complete Options Reference (Server and Client)

23 Hands-On Labs and Exercises (with Solutions)

24 Visual Guide (Diagrams)

How this book was verified

- Every protocol example (handshake packets, wire format) was captured from a real running Socket.IO 4.8.4 server, and every default value was checked against the library's source.
- Code was **run**, not just written: integration tests, labs and exercise solutions; servers in **Python** (python-socketio, Flask-SocketIO), **Go**, **Rust** (socketioxide) and **Java** (netty-socketio) tested against the official JS client; the **Java client**; **WebTransport** over HTTP/3; a **custom adapter** across two servers via Redis; a **custom parser**; **Bun** and **Deno**; the **offline outbox** and **E2EE** examples.
- The **nginx and HAProxy** configurations were run in front of three nodes with the Redis adapter (Chapter 8, Lab B in Chapter 23).
- Version history was checked against the published npm packages, and the security table (Chapter 6.10) against the npm/GitHub advisory database.
- Not executable in the build environment, and checked against source code/docs instead: the Swift and Dart clients, Electron, and the `deno.land/x` Deno port. These spots are marked in their chapters.

How to read it

If you are…	Read
A complete beginner	Chapters 1 → 5 in order, then Chapter 13 (projects)
A developer preparing for interview	Chapters 1–8 skim, then Chapter 14 (questions) fully
Working on a big production system	Chapters 5, 6, 8, 10, 11, 12, 13
Debugging something right now	Chapter 12 (Troubleshooting) first
Want a quick reference	Chapter 15 (Cheat sheet) and Chapter 22 (all options)
Building mobile apps	Chapter 16
Not using Node.js on the server	Chapter 17
Learning by doing	Chapter 23 (labs & exercises), Chapter 24 (diagrams)

If you remember one paragraph

Socket.IO is a library that gives you **a two-way, event-based, real-time channel between a client (usually a browser or mobile app) and a server.** Under the hood it uses a lower layer called **Engine.IO**, which opens a connection using **HTTP long-polling** or **WebSocket** (or **WebTransport**), keeps it alive with **heartbeats**, and **upgrades** to the best transport available. On top of that, Socket.IO adds **named events**, **acknowledgements** (request/response), **automatic reconnection**, **rooms** and

namespaces (grouping), **broadcasting**, **middleware**, **binary support** and **multi-server scaling via adapters** (e.g. Redis). It is **not** a plain WebSocket library: a plain WebSocket client cannot talk to a Socket.IO server and vice-versa.

CHAPTER 01

Foundations: What "Real-Time" Means and Why Socket.IO Exists

> **Goal of this chapter:** understand the problem Socket.IO solves *before* touching any code. Most interview failures on Socket.IO come from not understanding this chapter, not from forgetting an API name.

1.1 The "explain like I'm 5" version

Imagine you are waiting for a pizza.

- **Way 1 — Keep asking (Polling):** every 10 seconds you phone the pizza shop and ask, "Is my pizza ready?" Most of the time they say "No." You waste a lot of phone calls, and if the pizza becomes ready 1 second after you hung up, you only find out 9 seconds later.
- **Way 2 — Ask and wait on the line (Long-polling):** you call and say, "Don't answer me until my pizza is ready." The shop keeps you on hold. As soon as the pizza is ready they tell you, you hang up, and you immediately call again to wait for the *next* news.
- **Way 3 — An open walkie-talkie (WebSocket):** you and the shop each hold a walkie-talkie. Either of you can talk at any time, instantly, without dialling again.

Socket.IO is like a very smart helper who:

1. Tries to give you the walkie-talkie (WebSocket). If walkie-talkies don't work in your building (old proxy, strict firewall), it falls back to "ask and wait on the line" (long-polling) so you **still** get your news.
2. Keeps checking "are you still there?" (heartbeats) so it knows when the line is dead.
3. If the line breaks, it **automatically calls back** (reconnection).
4. Lets you give each message a **name** ("pizza-ready", "driver-left") — those are **events**.
5. Lets you ask a question and get a reply ("Did you get my order?" → "Yes!") — those are **acknowledgements**.
6. Lets the shop shout to **groups** of customers at once ("Everyone on Main Street, your driver is near!") — those are **rooms** and **broadcasts**.

Keep this picture in mind. Every chapter just zooms into one of these points.

1.2 How the normal web works: request/response (HTTP)

The web was built on **HTTP**, a *request/response* protocol:

```
Browser  ── "GET /messages" ──►  Server
Browser  ◄── "here are your messages" ── Server
(connection is idle or closed; the server cannot speak first)
```

Key property: **the client always speaks first.** The server can only *answer*. A server cannot say "Hey, a new message just arrived!" on its own.

This is perfect for loading pages, but terrible for:

- chat apps (new messages from other people),
- notifications ("Someone liked your post"),
- live dashboards (stock prices, sports scores, server metrics),
- multiplayer games,
- collaborative editing (Google-Docs-style),
- delivery / ride tracking (driver's position updates),
- live auctions / bidding,
- IoT device telemetry and commands.

All of these need the **server to push data to the client the moment something happens.** That is what "real-time" means on the web: *low-latency, server-initiated (push) communication*, usually *bi-directional*.

1.3 The techniques for real-time on the web (history and comparison)

You must know all of these for interviews. Socket.IO uses two of them internally.

1.3.1 Short polling

The client asks every N seconds with a normal HTTP request.

```
setInterval(async () ⇒ {
  const res = await fetch("/api/messages?since=" + lastId);
  const messages = await res.json();
  render(messages);
}, 5000);
```

Pros	Cons
Dead simple, works everywhere	Wasteful: most requests return nothing
Stateless, easy to scale	Latency up to N seconds
	Every request carries full HTTP headers (cookies, etc.)

1.3.2 Long polling (a.k.a. "Comet")

The client sends a request; the server **holds it open** until it has data (or a timeout happens). When the response arrives, the client immediately sends a new request.

```
Client: GET /poll ──────────────(server waits… 12 s…)──▶ response: [msg]
Client: GET /poll (immediately again) ──(waits)──────▶ …
```

Pros	Cons
Near-instant delivery	One HTTP request per message batch (header overhead)
Works through almost every proxy/firewall	Client→server messages need separate requests
Uses plain HTTP	Server must hold many open requests

Socket.IO's HTTP long-polling transport is exactly this. It uses `GET` requests to *receive* data (held open by the server) and `POST` requests to *send* data.

1.3.3 Server-Sent Events (SSE / `EventSource`)

A standard browser API where the server keeps an HTTP response open and streams text events down it.

```
const es = new EventSource("/stream");
es.onmessage = (e) ⇒ console.log(e.data);
```

Pros	Cons
Simple, standard, auto-reconnects with `Last-Event-ID`	**One-way only** (server → client)
Plain HTTP, works with HTTP/2	Text only (no binary)
	Under HTTP/1.1, browsers limit ~6 connections per domain

Socket.IO does **not** use SSE.

1.3.4 WebSocket

A standard protocol (RFC 6455) that starts as an HTTP request with an `Upgrade: websocket` header, then switches the TCP connection into a **persistent, full-duplex** (both sides talk at the same time) message channel.

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After `101 Switching Protocols`, both sides exchange small **frames** (2–14 bytes of overhead instead of hundreds of bytes of HTTP headers).

```
const ws = new WebSocket("wss://example.com/chat");
ws.onopen = () => ws.send("hello");
ws.onmessage = (e) => console.log(e.data);
```

Pros	Cons
True bi-directional, very low overhead and latency	**No** built-in reconnection
Text and binary	**No** built-in events/rooms/acks — just raw messages
Supported in all modern browsers	Some corporate proxies/firewalls/antivirus break it
	No built-in heartbeat at the application level (browsers can't send ping frames)
	You must build scaling/broadcast across servers yourself

1.3.5 WebTransport

A newer API built on **HTTP/3 (QUIC)**. Supports multiple streams and unreliable datagrams. Socket.IO supports it since **v4.7.0** (opt-in; it requires HTTP/3 and TLS and a separate HTTP/3 server such as `@fails-components/webtransport`). Browser support is still not universal (notably Safari support has lagged), so it is an *option*, not the default.

1.3.6 Comparison table (memorise this)

Feature	Short polling	Long polling	SSE	WebSocket	Socket.IO
Direction	C → S (pull)	S → C push (emulated)	S → C only	Both	Both
Latency	High	Low	Low	Lowest	Lowest (on WS)
Overhead per message	High	High	Low	Very low	Low (small protocol header)
Binary	Yes (HTTP)	Yes	No	Yes	Yes
Auto-reconnect	N/A	Manual	Yes	**No**	**Yes**
Fallback when WS blocked	—	—	—	**No**	**Yes** (polling)
Named events	No	No	Yes (event field)	No	**Yes**
Request/response (acks)	Yes (HTTP)	No	No	No	**Yes**
Rooms / broadcast	No	No	No	No	**Yes**
Multi-server broadcasting	—	—	—	Build it yourself	**Adapters** (Redis etc.)

1.4 So what exactly is Socket.IO?

Official definition: *Socket.IO is a library that enables low-latency, bidirectional and event-based communication between a client and a server.*

It has two parts:

1. **Server library** — `socket.io` (Node.js). There are also community/official implementations in other languages (Python `python-socketio`, Java `netty-socketio`, Go, Rust `socketioxide`, Deno, etc.).
2. **Client library** — `socket.io-client` (browser & Node.js). There are also official clients for Java/Android, Swift/iOS, C++, Dart, Python (`python-socketio` also includes a client), .NET (community), Rust, etc.

And it is built in **two layers**:

```
┌──────────────────────────────────────────────────────────┐
| Your application code: socket.emit("chat", msg)          |
├──────────────────────────────────────────────────────────┤
| Socket.IO  (events, acks, rooms, namespaces, broadcasting,  |
|             reconnection, middleware, binary, adapters)    |
├──────────────────────────────────────────────────────────┤
| Engine.IO  (the low-level connection: handshake, transports |
|             polling / websocket / webtransport, upgrade,    |
|             heartbeats, packet encoding)                    |
├──────────────────────────────────────────────────────────┤
| HTTP long-polling   |   WebSocket   |   WebTransport       |
├──────────────────────────────────────────────────────────┤
| TCP (or QUIC for WebTransport)                           |
└──────────────────────────────────────────────────────────┘
```

ELI5: Engine.IO is the *phone line* (it makes sure there is a working line and checks it's still alive). Socket.IO is the *language you speak on the phone* (with names for messages, ways to reply, groups of listeners, etc.).

1.4.1 What Socket.IO is NOT (very common interview trap)

1. **It is not a WebSocket implementation.** It *uses* WebSocket as one transport, but it adds its own protocol on top. Every message has extra metadata (packet type, namespace, ack id). So: - A plain `new WebSocket("ws://server")` client **cannot** connect to a Socket.IO server (well — it can open the TCP/WebSocket connection to `/socket.io/?EIO=4&transport=websocket`, but it won't speak the protocol unless you implement it by hand). - A Socket.IO client **cannot** connect to a plain WebSocket server (like one written with `ws`).
2. **It is not a message queue / broker** (like Kafka or RabbitMQ). By default there is **no persistence** and **at-most-once delivery**: if the client is disconnected when you emit, the message is gone (unless you use connection state recovery or build your own storage — see Chapter 5).
3. **It is not meant for server-to-server communication at scale** (use gRPC, HTTP, a message broker). It *can* be used from Node to Node with `socket.io-client`, but that is not its primary purpose.
4. **It is not HTTP/REST replacement for everything.** Loading data, uploads of large files, cacheable resources → use HTTP. Use Socket.IO for *push* and *live interaction*.

1.5 Why use Socket.IO instead of raw WebSocket? (the "features" list)

Each of these is explained in depth in later chapters, but you should be able to recite this list in an interview:

1. **HTTP long-polling fallback** — if a WebSocket connection cannot be established (proxies, firewalls, antivirus, old networks), the connection still works.
2. **Automatic reconnection** — with exponential back-off and jitter.
3. **Heartbeat / disconnection detection** — Engine.IO pings regularly so both sides know if the other is gone, even when TCP doesn't notice (e.g. a laptop lid closed).
4. **Packet buffering** — events emitted while disconnected are buffered on the client and sent upon reconnection (by default).
5. **Acknowledgements** — callback-based or Promise-based request/response over the socket.
6. **Named events** — `emit("order:created", data)` instead of parsing raw strings.
7. **Automatic serialization** — send JS objects, arrays, numbers, `Buffer` / `ArrayBuffer` / `Blob`; no manual `JSON.stringify`.
8. **Binary support** mixed with JSON.
9. **Broadcasting** — to all clients, to all except one, to rooms.
10. **Rooms** — server-side groups of sockets (e.g. a chat channel, a user's devices).
11. **Namespaces** — split one connection into separate logical channels (multiplexing), each with its own handlers and middleware.
12. **Middleware** — authentication and validation at connection time and per event.
13. **Horizontal scaling** — adapters (Redis, Redis Streams, MongoDB, Postgres, cluster, cloud pub/sub) to broadcast across many server processes.
14. **Connection state recovery** (v4.6+) — after a short disconnection, restore the socket's id, rooms, data, and missed packets.
15. **Admin UI**, TypeScript types, many client implementations.

When raw WebSocket might be better:

- You need minimal overhead and control (e.g. high-frequency trading feeds, games with custom binary protocols) and you are happy to implement reconnection/heartbeats.
- You need to interoperate with non-Socket.IO clients (third-party devices, standard protocols like GraphQL subscriptions over `graphql-ws`, MQTT over WebSocket).
- You have extreme connection counts and want to use something like `uWebSockets.js` directly (although Socket.IO can also run on uWebSockets.js — see Chapter 11).

Alternatives you should be able to name: raw `ws` library, `uWebSockets.js`, SockJS, SignalR (.NET), Phoenix Channels (Elixir), Action Cable (Rails), Centrifugo, Pusher/Ably (hosted), Firebase Realtime Database / Firestore listeners, Supabase Realtime, AWS API Gateway WebSocket APIs, AWS AppSync, Azure Web PubSub (which can even host Socket.IO), GraphQL subscriptions, MQTT (IoT), gRPC streaming, SSE.

1.6 Key vocabulary (you will see these words everywhere)

Term	ELI5 meaning	Technical meaning
Server (`io`)	The pizza shop	The `Server` instance from `socket.io`, attached to an HTTP server
Client	You	The `socket.io-client` running in a browser/app
Socket	Your personal phone line with the shop	One connection between one client and one namespace. On the server, one `Socket` object per client per namespace
Event	A message with a label	A named message: `socket.emit("name", ...args)`
emit	Say something	Send an event
on	Listen for something	Register a handler for an event
Acknowledgement (ack)	"Got it!" reply	A callback the receiver calls to respond to a specific emit
Broadcast	Shout to many	Send an event to many sockets at once
Room	A group chat	A server-side named group of sockets you can broadcast to
Namespace	A separate department in the shop	A communication channel (`/` , `/admin` , `/chat`) sharing the same underlying connection
Transport	The type of phone line	`polling` , `websocket` , `webtransport`
Upgrade	Switching from hold-music line to walkie-talkie	Moving from polling to WebSocket after connecting
Heartbeat (ping/pong)	"Still there?" "Yes!"	Periodic packets to detect dead connections
Handshake	Saying hello and exchanging IDs	The first request; server returns session id (`sid`) and settings
Adapter	The shop's internal messenger between branches	The component that stores rooms and forwards broadcasts (in-memory, or Redis etc. across servers)
Middleware	The bouncer at the door	Functions run before a connection is accepted or before an event is handled
Sticky session	Always talking to the same shop branch	Load-balancer setting that routes all requests of a client to the same server

1.7 The life of a Socket.IO connection in 10 steps (big picture)

You'll see each of these in detail later. Here is the whole story once:

1. The client calls `io("https://example.com")`.
2. The client sends an **HTTP GET** to `/socket.io/?EIO=4&transport=polling` (the Engine.IO **handshake**). (*If you configured `transports: ["websocket"]`, it opens a WebSocket directly instead.*)
3. The server replies with an Engine.IO **OPEN** packet containing a session id and timing settings. A real response captured from Socket.IO 4.8.4:

```
0{"sid":"HSJR3YuCOE9jPqHEAAAA","upgrades":
["websocket"],"pingInterval":25000,"pingTimeout":20000,"maxPayload":1000000}
```

4. The client sends a Socket.IO **CONNECT** packet for the namespace (`40` for the main namespace `/`, optionally with an `auth` payload).
5. The server runs **middleware** (e.g. auth). If accepted, it replies `40{"sid":"..."}` (a CONNECT with the **Socket.IO** socket id — which is different from the Engine.IO sid) and fires `io.on("connection")`. If rejected, it replies with a **CONNECT_ERROR** (`44`) and the client receives `connect_error`.
6. Meanwhile the client tries to **upgrade** to WebSocket in the background. If it works, all further traffic moves to the WebSocket; polling is closed.
7. Both sides exchange **events** (`42["chat","hi"]`) and **acks** (`43...`).
8. Every `pingInterval` (25 s) the **server** sends a **PING** (`2`); the client must reply **PONG** (`3`) within `pingTimeout` (20 s), or the server considers it disconnected. The client also expects pings: if none arrives in `pingInterval + pingTimeout`, it considers the connection dead.
9. If the connection dies, the server fires `disconnect` with a reason, and the client starts **reconnecting** with exponential back-off.
10. On reconnection, the client gets a **new socket id** (unless connection state recovery succeeded), so the server must rejoin rooms etc. (often in the `connection` handler, e.g. based on the authenticated user id).

1.8 Versions and compatibility (frequently asked)

Socket.IO server	Engine.IO server	Socket.IO protocol	Engine.IO protocol	Notes
1.x	1.x	3	3	Ancient

Socket.IO server	Engine.IO server	Socket.IO protocol	Engine.IO protocol	Notes
2.x	3.x	4	3	Client sends pings; reconnect events on socket; auto-CORS for all origins
3.x	4.x	5	4	Big breaking release (Nov 2020)
4.x	5.x / 6.x	5	4	Current; v4 is backward-compatible with v3 clients

Important compatibility facts:

- A **v3/v4 client** works with a **v3/v4 server** (same protocols).
- A **v2 client** can connect to a **v3/v4 server** only if the server enables `allowEIO3: true`.
- A **v3/v4 client cannot** connect to a **v2 server**.
- **Client and server versions are independent packages** but the *protocol* must match.

Big changes in v3 (vs v2) — classic interview question:

1. **CORS is no longer enabled by default.** You must configure the `cors` option.
2. **Heartbeat direction reversed** — the *server* sends PING, the client answers PONG (in v2 the client pinged). This works better with browsers that throttle timers in background tabs.
3. **Reconnection events moved** from the `Socket` to the `Manager` (`socket.io.on("reconnect", ...)` instead of `socket.on("reconnect", ...)`).
4. `socket.rooms` **is a** `Set` (was an object).
5. `io.set()` **removed**; configuration is via constructor options.
6. **No more implicit connection to the default namespace** when connecting to another namespace.
7. `socket.binary()` **removed**, binary detection is automatic.
8. **The** `auth` **option**: credentials sent in the CONNECT packet, read via `socket.handshake.auth`.
9. **Payload encoding** changed (in Engine.IO v4: record separator `\x1e` instead of length prefixes, binary as base64 with `b` prefix in polling).
10. **Default** `maxHttpBufferSize` **reduced from 100 MB to 1 MB.**
11. **The** `cookie` **(io cookie) is disabled by default.**
12. `perMessageDeflate` **disabled by default** (WebSocket compression).
13. **Middleware errors**: `next(err)` sends a CONNECT_ERROR; the client receives `connect_error` with `err.message` and `err.data`.
14. **The** `Socket#use()` **per-socket middleware** signature changed and was later re-added in 3.0.5.
15. Written in **TypeScript**.

Additions during v4 (approximate release markers — know the features, versions are a bonus):

Version	Notable additions
4.0 (Mar 2021)	Typed events (TypeScript generics), `io.except()`, utility methods `io.socketsJoin()` / `socketsLeave()` / `disconnectSockets()` / `fetchSockets()`
4.1	`io.serverSideEmit()` (events between Socket.IO servers in a cluster)
4.4	`io.attachApp()` — run on **uWebSockets.js**; `socket.timeout(ms)` for acks on the client
4.5	Acks with timeout on the server (`socket.timeout(ms).emit(...)`), broadcast with acks (`io.timeout(ms).emit(...)`), `onAnyOutgoing()` / `prependAnyOutgoing()`, `serverSideEmit()` with ack
4.6 (Feb 2023)	**Connection state recovery**, `emitWithAck()` (Promise API), client `retries` + `ackTimeout` (at-least-once from client), `io.engine.use()` for Express middleware, `cleanupEmptyChildNamespaces`
4.7 (Jun 2023)	**WebTransport** support
4.8 (Sep 2024)	Client: fetch-based polling transport (useful in service workers / runtimes without `XMLHttpRequest`), custom transport implementations, `tryAllTransports` option

> Interview tip: you will not be failed for not knowing the exact minor version. You *will* be asked "what changed from v2 to v3/v4?" and "what is connection state recovery?".

1.9 When should you (and shouldn't you) choose Socket.IO? — decision guide

Choose Socket.IO when:

- You build a web/mobile product needing bi-directional real-time features (chat, notifications, presence, live updates, collaboration, light games).
- Your users may be behind restrictive networks (enterprises, schools, hotels) — fallback matters.
- You want rooms, broadcasting and multi-server fan-out without building them.
- Your team uses Node.js (or a language with a mature Socket.IO server).

Consider alternatives when:

- One-way server→client updates only and HTTP/2 → **SSE** might be simpler.
- You must interoperate with standard WebSocket clients → raw **WebSocket** / `ws`.
- You need durable messaging with replay and exactly-once processing → a **broker** (Kafka, RabbitMQ, NATS JetStream) *behind* your Socket.IO servers.
- You don't want to operate infrastructure → hosted services (Ably, Pusher, Azure Web PubSub, AWS AppSync/API Gateway WebSockets, Firebase).

- Extremely latency-sensitive games → custom UDP / WebRTC data channels / WebTransport datagrams.

1.10 Chapter summary

- HTTP is request/response: the server can't speak first. Real-time apps need push.
- Techniques: short polling, long polling, SSE, WebSocket, WebTransport.
- Socket.IO = Engine.IO (connection, transports, heartbeats, upgrade) + Socket.IO layer (events, acks, rooms, namespaces, broadcast, reconnection, middleware, adapters).
- Socket.IO is **not** WebSocket; it's a protocol on top of WebSocket/polling.
- Default delivery guarantee: **at-most-once**, **ordered**.
- v3 was the big breaking release (CORS, heartbeat reversal, reconnect events on Manager, rooms as Set, auth option, 1 MB buffer default). v4 is compatible with v3 clients.

Next: Chapter 2 — Getting started

CHAPTER 02

Getting Started: Your First Server and Client

> **Goal:** build a working real-time app from scratch, understand every line, and learn the different ways to set up the server (standalone, with `http`, with Express, with HTTPS, with other frameworks) and the client (CDN, bundler, Node).

2.1 Install

```
# server
npm install socket.io

# client (when using a bundler such as Vite/Webpack, or in Node.js)
npm install socket.io-client
```

- `socket.io` (server) **also serves the client bundle** by default at `/socket.io/socket.io.js` (option `serveClient: true`). Handy for demos; in real projects you normally install `socket.io-client` via npm and bundle it.
- Requirements: a supported Node.js version (Node 10+ is technically supported by socket.io 4; use a current LTS in practice).

Optional performance add-ons for the `ws` WebSocket engine (native modules):

```
npm install --save-optional bufferutil utf-8-validate
```

(See Chapter 11. On recent Node versions `ws` can use Node's built-in `buffer.isUtf8()`, so `utf-8-validate` matters less; `bufferutil` still speeds up frame masking/unmasking.)

2.2 The smallest possible server

```
// server.js  (CommonJS)
const { Server } = require("socket.io");

const io = new Server(3000); // creates an HTTP server listening on port 3000
```

```
io.on("connection", (socket) => {
  console.log("a client connected:", socket.id);

  socket.emit("welcome", "Hello from the server!"); // send to THIS client

  socket.on("ping-me", (msg) => {                    // listen to THIS client
    console.log("client says:", msg);
  });

  socket.on("disconnect", (reason) => {
    console.log("client left:", socket.id, reason);
  });
});
```

ESM version:

```
// server.mjs
import { Server } from "socket.io";
const io = new Server(3000);
```

Line-by-line (ELI5):

- `new Server(3000)` — "Open a pizza shop at door number 3000."
- `io.on("connection", (socket) => {...})` — "Every time a new customer walks in, give me their personal phone line (`socket`)."
- `socket.id` — a random unique id for this connection (e.g. `"KykvYCRwmIsIifIyAAAA"`). **It changes on every reconnection** — never use it as a user id.
- `socket.emit("welcome", ...)` — "Say 'welcome' to *this* customer only."
- `socket.on("ping-me", ...)` — "When *this* customer says 'ping-me', do this."
- `socket.on("disconnect", ...)` — "When this customer leaves (or the line dies)."

2.3 The smallest possible client (Node.js)

```
// client.js
const { io } = require("socket.io-client");

const socket = io("http://localhost:3000");

socket.on("connect", () => {
  console.log("connected with id", socket.id);
  socket.emit("ping-me", "hi server!");
});

socket.on("welcome", (msg) => console.log(msg));
```

```
socket.on("disconnect", (reason) => console.log("disconnected:", reason));
```

Run `node server.js` then `node client.js`. You'll see both sides talk.

2.4 The browser client — three ways

(a) Served by the server itself

```
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io(); // no URL → connects to the host that served the page
</script>
```

(b) From a CDN

```
<script src="https://cdn.socket.io/4.8.1/socket.io.min.js"
        integrity="(use the SRI hash from the official docs)"
        crossorigin="anonymous"></script>
<script>
  const socket = io("https://api.example.com");
</script>
```

(Also available from jsDelivr/cdnjs/unpkg. Pin the exact version.)

(c) With a bundler (the normal way in React/Vue/Angular apps)

```
import { io } from "socket.io-client";
const socket = io("https://api.example.com");
```

Bundle size: the 4.8.4 `socket.io.min.js` is ~47 KB minified, ~14.8 KB gzipped. There is also a `socket.io.msgpack.min.js` build using the msgpack parser, and an "es" build for tree-shaking.

URL forms of `io()`

```
io();                                // same origin, main namespace "/"
io("/admin");                        // same origin, namespace "/admin"
io("https://api.example.com");       // other origin, namespace "/"
io("https://api.example.com/admin"); // other origin, namespace "/admin"
io("https://api.example.com", { path: "/my-custom-path/" }); // custom HTTP path
```

> **Very common confusion:** the part after the host in the URL is the **namespace**, not the HTTP path! `io("https://example.com/chat")` connects to namespace `/chat` using the HTTP path `/socket.io/`. To change the HTTP path you use the `path` option (and the same `path` on the server).

2.5 Attaching to an existing HTTP server (the usual way)

Your app usually already has an HTTP server (for REST endpoints, static files...). Attach Socket.IO to it so both share the **same port**.

With Node's `http`

```
const { createServer } = require("node:http");
const { Server } = require("socket.io");

const httpServer = createServer((req, res) => {
  res.end("regular HTTP response");
});

const io = new Server(httpServer, { /* options */ });

io.on("connection", (socket) => { /* ... */ });

httpServer.listen(3000);
```

Socket.IO intercepts only requests whose path starts with `/socket.io/` (configurable); all other requests go to your handler.

With Express (most common in interviews and in companies)

```
const express = require("express");
const { createServer } = require("node:http");
const { Server } = require("socket.io");

const app = express();
const httpServer = createServer(app);   // ⟵ wrap the express app in an http server
const io = new Server(httpServer, {
  cors: { origin: "http://localhost:5173" }, // the frontend dev server origin
});
```

```
app.get("/", (req, res) => res.send("API is up"));

io.on("connection", (socket) => {
  console.log("connected", socket.id);
});

httpServer.listen(3000, () => console.log("listening on :3000"));
```

> **Bug #1 of all time:** calling `app.listen(3000)` instead of `httpServer.listen(3000)`. `app.listen()` creates a **new, different** HTTP server that Socket.IO is not attached to, so the client gets `404` on `/socket.io/...`. Always listen on the server you passed to `new Server()`.
>
> (Alternatively: `const server = app.listen(3000); const io = new Server(server);` — this works because `app.listen` *returns* the http server it created.)

With HTTPS

```
const { readFileSync } = require("node:fs");
const { createServer } = require("node:https");
const { Server } = require("socket.io");

const httpsServer = createServer({
  key: readFileSync("/path/to/key.pem"),
  cert: readFileSync("/path/to/cert.pem"),
});
const io = new Server(httpsServer);
httpsServer.listen(443);
```

In production TLS is usually terminated by a load balancer / reverse proxy (nginx, ALB, Cloudflare), and Node listens on plain HTTP behind it. The client still uses `https://` / `wss://`.

With HTTP/2

```
const { readFileSync } = require("node:fs");
const { createSecureServer } = require("node:http2");
const { Server } = require("socket.io");

const httpServer = createSecureServer({
  allowHTTP1: true, // REQUIRED: WebSocket upgrade still uses HTTP/1.1
  key: readFileSync("key.pem"),
  cert: readFileSync("cert.pem"),
});
const io = new Server(httpServer);
httpServer.listen(3000);
```

With other frameworks

- **Fastify:** `fastify-socket.io` plugin, or `new Server(fastify.server)` after `fastify.ready()`.
- **Koa:** `const server = http.createServer(app.callback()); new Server(server)`.
- **NestJS:** `@nestjs/websockets` + `@nestjs/platform-socket.io`, `@WebSocketGateway()` (Chapter 9).
- **Next.js:** a custom server, or a separate Socket.IO service (Chapter 9). Serverless functions (Vercel functions, AWS Lambda) are **not** suitable for hosting long-lived Socket.IO servers.
- **Deno / Bun:** there is an official Deno port of the server; Bun runs the Node package through its Node compatibility layer. Test your exact runtime version before relying on it in production.
- **uWebSockets.js:** `io.attachApp(uwsApp)` (Chapter 11).

Standalone (Socket.IO creates the HTTP server)

```
const io = new Server(3000, { /* options */ });
// or
const io = new Server({ /* options */ });
io.listen(3000);
```

2.6 CORS — you will hit this on day one

If the page is served from `http://localhost:5173` and the Socket.IO server runs at `http://localhost:3000`, the browser treats that as **cross-origin**. Since v3 Socket.IO does **not** allow cross-origin requests by default. Symptom in the browser console:

```
Access to XMLHttpRequest at 'http://localhost:3000/socket.io/?EIO=4&transport=polling&t=...'
from origin 'http://localhost:5173' has been blocked by CORS policy
```

Fix on the server:

```
const io = new Server(httpServer, {
  cors: {
    origin: ["http://localhost:5173", "https://app.example.com"],
    methods: ["GET", "POST"],      // optional; defaults of the `cors` package are fine
    credentials: true,             // only if you need cookies (see below)
  },
});
```

The `cors` option is passed to the popular `cors` npm package, so it accepts the same options (`origin` can be a string, array, RegExp, or a function `(origin, callback) ⇒ callback(null, true|false)`).

If you need cookies (e.g. session-cookie authentication), the **client** must also set:

```
const socket = io("https://api.example.com", { withCredentials: true });
```

and the server must set `credentials: true` **and** a specific origin (not `"*"` — browsers reject `Access-Control-Allow-Origin: *` with credentials).

> **Important nuance (interview favourite):** CORS applies to the **HTTP long-polling** requests (they are XHR/fetch). The **WebSocket** handshake is **not** subject to CORS in browsers — any website can open a WebSocket to your server. So to protect WebSocket connections from malicious sites you must check the `Origin` header yourself (with `allowRequest`) and/or use proper authentication (tokens). See Chapter 6 (Cross-Site WebSocket Hijacking).

2.7 A complete first project: a mini chat (with explanations)

Server — `index.js`

```
const express = require("express");
const { createServer } = require("node:http");
const { join } = require("node:path");
const { Server } = require("socket.io");

const app = express();
const server = createServer(app);
const io = new Server(server);

app.get("/", (req, res) => {
  res.sendFile(join(__dirname, "index.html"));
});

io.on("connection", (socket) => {
  // 1. Tell everybody else someone joined
  socket.broadcast.emit("system", "A user joined");

  // 2. When this user sends a chat message, send it to EVERYONE (including the sender)
  socket.on("chat message", (msg, ack) => {
    if (typeof msg !== "string" || msg.length === 0 || msg.length > 500) {
      return typeof ack === "function" && ack({ ok: false, error: "invalid message" });
    }
    io.emit("chat message", { text: msg, from: socket.id, at: Date.now() });
    if (typeof ack === "function") ack({ ok: true });
```

```
  });

  // 3. Typing indicator — to everyone except the sender, and it's OK to lose it
  socket.on("typing", () => socket.broadcast.volatile.emit("typing", socket.id));

  socket.on("disconnect", () => io.emit("system", "A user left"));
});

server.listen(3000, () => console.log("http://localhost:3000"));
```

Client — `index.html`

```
<!doctype html>
<html>
  <body>
    <ul id="messages"></ul>
    <p id="typing"></p>
    <form id="form"><input id="input" autocomplete="off" /><button>Send</button></form>

    <script src="/socket.io/socket.io.js"></script>
    <script>
      const socket = io();
      const form = document.getElementById("form");
      const input = document.getElementById("input");
      const messages = document.getElementById("messages");

      function addLine(text) {
        const li = document.createElement("li");
        li.textContent = text;           // textContent, NOT innerHTML → prevents XSS
        messages.appendChild(li);
      }

      form.addEventListener("submit", (e) => {
        e.preventDefault();
        if (!input.value) return;
        socket.emit("chat message", input.value, (res) => {
          if (!res.ok) alert(res.error);   // server acknowledged
        });
        input.value = "";
      });

      input.addEventListener("input", () => socket.emit("typing"));

      socket.on("chat message", (m) => addLine(m.from.slice(0, 5) + ": " + m.text));
      socket.on("system", (t) => addLine("[system] " + t));
      socket.on("typing", (id) => {
        document.getElementById("typing").textContent = id.slice(0, 5) + " is typing…";
        clearTimeout(window.__t);
        window.__t = setTimeout(() => (document.getElementById("typing").textContent = ""),
1000);
      });
    </script>
```

```
  </body>
</html>
```

What you just used (each is a later chapter):

Code	Meaning
`socket.emit`	send to one client
`io.emit`	send to all connected clients (of the main namespace)
`socket.broadcast.emit`	send to all **except** this socket
`volatile`	OK to drop if the client isn't ready to receive
`(msg, ack) ⇒ ack(...)`	acknowledgement (request/response)
`textContent`	basic XSS prevention for user-generated content

2.8 The "emit cheat-sheet" (memorise — asked in nearly every interview)

```
io.on("connection", (socket) ⇒ {
  // to the sender only
  socket.emit("hello", "you");

  // to all clients in the current namespace, including the sender
  io.emit("hello", "everyone");

  // to all clients except the sender
  socket.broadcast.emit("hello", "everyone but you");

  // to all clients in room "game", except the sender
  socket.to("game").emit("hello", "players but you");

  // to all clients in room "game1" and/or "game2", except the sender
  socket.to("game1").to("game2").emit("hello");

  // to all clients in room "game", including the sender
  io.in("game").emit("hello");          // .in() is an alias of .to()

  // to all clients in namespace "/admin"
  io.of("/admin").emit("hello");

  // to room "notifications" in namespace "/admin"
  io.of("/admin").to("notifications").emit("hello");
```

```
  // to an individual socket by id (private message) — each socket auto-joins a room named by
its id
  io.to(otherSocketId).emit("hello");

  // to everyone except those in room "banned"
  io.except("banned").emit("hello");

  // with acknowledgement
  socket.emit("question", "do you like pizza?", (answer) => {});

  // with acknowledgement + timeout
  socket.timeout(5000).emit("question", (err, answer) => {});

  // promise style
  const answer = await socket.timeout(5000).emitWithAck("question");

  // without compression
  socket.compress(false).emit("uncompressed", "that's rough");

  // may be dropped if the client is not ready to receive
  socket.volatile.emit("maybe", "do you really need it?");

  // to all clients on THIS node only (when using multiple nodes + adapter)
  io.local.emit("hi", "my lovely babies");

  // to all connected clients
  io.emit("an event sent to all connected clients");
});
```

Client side:

```
socket.emit("hello", "server");                      // basic
socket.emit("question", (answer) => {});             // with ack
socket.timeout(5000).emit("question", (err, answer) => {}); // ack + timeout
const answer = await socket.emitWithAck("question");  // promise (no timeout unless
ackTimeout/timeout set)
socket.volatile.emit("mouse-move", { x, y });        // droppable
socket.compress(false).emit("x");
```

> A **client cannot broadcast.** `socket.broadcast` exists **only on the server.** If a client wants to send to others, it sends to the server and the server relays.

2.9 Server options at a glance (details in later chapters)

```
const io = new Server(httpServer, {
  // ---- Socket.IO-level options ----
  path: "/socket.io/",                // HTTP path handled
  serveClient: true,                  // serve /socket.io/socket.io.js
  adapter: undefined,                 // e.g. createAdapter(pub, sub) for Redis
  parser: undefined,                  // custom parser e.g. require("socket.io-msgpack-parser")
  connectTimeout: 45000,              // ms a client has to join a namespace after the
Engine.IO handshake
  connectionStateRecovery: undefined, // { maxDisconnectionDuration: 120000, skipMiddlewares:
true }
  cleanupEmptyChildNamespaces: false, // for dynamic namespaces

  // ---- Engine.IO-level options ----
  pingInterval: 25000,                // ms between server pings
  pingTimeout: 20000,                 // ms to wait for a pong before closing
  upgradeTimeout: 10000,              // ms to complete a transport upgrade
  maxHttpBufferSize: 1e6,             // max message size in bytes (1 MB) before closing the
socket
  allowRequest: undefined,            // (req, callback) => callback(err, allowed)
  transports: ["polling", "websocket"], // (+ "webtransport" if enabled)
  allowUpgrades: true,
  perMessageDeflate: false,           // WebSocket compression (off by default)
  httpCompression: true,              // gzip/deflate for polling (threshold 1024 bytes)
  wsEngine: require("ws").Server,     // WebSocket server implementation
  cors: undefined,                    // options passed to the `cors` package
  cookie: false,                      // the "io" cookie (sticky session helper)
  allowEIO3: false,                   // accept Socket.IO v2 clients
  initialPacket: undefined,
});
```

All defaults above were verified against `socket.io@4.8.4` / `engine.io@6.6.11` source.

2.10 Client options at a glance

```
const socket = io("https://api.example.com", {
  // ---- Manager (connection/reconnection) options ----
  reconnection: true,
  reconnectionAttempts: Infinity,
  reconnectionDelay: 1000,     // initial delay
  reconnectionDelayMax: 5000,  // max delay
  randomizationFactor: 0.5,    // jitter: delay ± 50%
  timeout: 20000,              // connection timeout (before connect_error)
  autoConnect: true,           // connect immediately on io()
  parser: undefined,
```

```
  // ---- Socket options ----
  auth: { token: "abc" },       // or (cb) => cb({ token: getToken() })
  ackTimeout: undefined,        // default timeout for acks (used with retries)
  retries: undefined,           // number of retries for emits (at-least-once)

  // ---- Engine.IO (low-level) options ----
  path: "/socket.io/",
  transports: ["polling", "websocket", "webtransport"], // order to try (client default)
  upgrade: true,                // try to upgrade polling → websocket
  rememberUpgrade: false,       // skip polling next time if WS worked before
  tryAllTransports: false,      // (4.8+) try next transport if the first fails
  query: { roomHint: "x" },     // extra query params on the handshake
  extraHeaders: { "x-foo": "bar" }, // NOT applied to browser WebSocket (browser API can't set
headers)
  withCredentials: false,       // send cookies cross-origin (polling)
  forceNew: false,              // create a new Manager (separate connection)
  multiplex: true,              // share connection across namespaces
  closeOnBeforeunload: false,   // close silently on page unload
  timestampRequests: true,      // cache-buster `t=` on polling requests
});
```

> On the client the default `transports` includes `"webtransport"`, but it is only tried if the server enables it — with the default server setup you get polling → websocket.

2.11 Project structure for anything bigger than a demo

Don't put everything inside `io.on("connection")`. A maintainable layout:

```
src/
  server.ts              # creates http server + io, wires everything
  socket/
    index.ts             # createSocketServer(httpServer) → io
    middlewares/
      auth.ts            # io.use(authMiddleware)
      rateLimit.ts
    handlers/
      chat.handler.ts    # registerChatHandlers(io, socket)
      presence.handler.ts
      notification.handler.ts
    schemas/             # zod/joi payload schemas
    events.ts            # TypeScript event maps (shared with client!)
  services/              # business logic (NOT socket-aware)
  repositories/          # DB access
```

```
// socket/handlers/chat.handler.js
module.exports = (io, socket) => {
  const sendMessage = async (payload, ack) => { /* validate → service → emit */ };
  const deleteMessage = async (payload, ack) => { /* ... */ };

  socket.on("chat:send", sendMessage);
  socket.on("chat:delete", deleteMessage);
};

// socket/index.js
const registerChatHandlers = require("./handlers/chat.handler");
const registerPresenceHandlers = require("./handlers/presence.handler");

io.on("connection", (socket) => {
  registerChatHandlers(io, socket);
  registerPresenceHandlers(io, socket);
});
```

This is the structure recommended by the official docs ("application structure") and used by most companies. Chapter 13 expands it into a full organisational standard.

2.12 Chapter summary

- `new Server(httpServer, options)` attaches Socket.IO to an existing server; **listen on that same server.**
- The client connects with `io(url, options)`; the URL path is the **namespace**; the HTTP path is the `path` option.
- CORS must be configured since v3; WebSocket handshakes are not protected by CORS.
- Emit cheat sheet: `socket.emit`, `io.emit`, `socket.broadcast.emit`, `socket.to(room)`, `io.to(room)`, `io.of(nsp)`, `io.except()`, acks, `timeout`, `emitWithAck`, `volatile`, `compress`, `local`.
- Organise handlers by feature, keep business logic out of socket handlers.

Next: Chapter 3 — Events and acknowledgements

CHAPTER 03

Events, Emitting, Listening and Acknowledgements

> **Goal:** master the core of Socket.IO — sending and receiving events, what data you can send, request/response with acknowledgements, timeouts, promises, catch-all listeners, volatile and compressed events, reserved event names, and ordering.

3.1 ELI5: events are labelled envelopes

Every message in Socket.IO is like an envelope with:

- a **label** written on the front (the event name, e.g. `"chat:message"`),
- **things inside** (zero or more arguments: strings, numbers, objects, files...),
- optionally a **reply slip** ("please sign and send back") — the acknowledgement.

The receiver has a box per label (`socket.on("chat:message", handler)`); when an envelope with that label arrives, the handler opens it.

3.2 `emit` and `on` — the basics

```
// sender
socket.emit("hello", "world");

// receiver
socket.on("hello", (arg) ⇒ {
  console.log(arg); // "world"
});
```

Works in **both directions** with the same API:

Direction	Sender	Receiver
client → server	`socket.emit()` in browser	`socket.on()` inside `io.on("connection")`
server → client	`socket.emit()` / `io.emit()` on server	`socket.on()` in browser

Multiple arguments

You can send **any number of arguments**:

```
socket.emit("move", 10, 20, { piece: "knight" }, Uint8Array.from([1, 2, 3]));

socket.on("move", (x, y, meta, bytes) => { /* ... */ });
```

> **Best practice:** prefer **one object argument** for anything non-trivial (`emit("move", { x, y, piece })`). It's self-documenting, easier to validate, extend and version, and avoids argument-order bugs.

What data can you send? (serialization rules)

Socket.IO's default parser serializes arguments with `JSON.stringify` semantics, **plus special handling for binary**.

Type	Supported?	What arrives
`string` , `number` , `boolean` , `null`	✅	same
plain objects, arrays	✅	deep copies
`Buffer` (Node), `ArrayBuffer` , typed arrays (`Uint8Array` ...), `Blob` / `File` (browser)	✅ binary	In Node receiver: `Buffer` ; in browser receiver: `ArrayBuffer` (default `binaryType`)
`Date`	⚠️	becomes an **ISO string** (`toJSON`) — convert back yourself
`Map` , `Set`	❌	become `{}` — convert to arrays/objects first (`[...map]` , `Object.fromEntries(map)`)
`undefined`	⚠️	object properties with `undefined` are dropped; `undefined` in arrays becomes `null`
`NaN` , `Infinity`	⚠️	become `null`
`BigInt`	❌	`JSON.stringify` throws `TypeError` — convert to string
functions	❌ (except as the **last** argument, which becomes an **ack callback**)	—

Type	Supported?	What arrives
class instances	⚠️	only own enumerable properties; prototype/methods lost; `toJSON()` is honoured
circular references	❌	throws

```
// ❌ Common mistake
socket.emit("stats", new Map([["a", 1]])); // receiver gets {}

// ✅ Fix
socket.emit("stats", Object.fromEntries(map));
```

> With the **msgpack parser** (`socket.io-msgpack-parser`) the rules differ (e.g. Dates and binary are encoded natively by msgpack). Both sides must use the **same parser**.

Binary data example

```
// server: send a file
const fs = require("node:fs");
socket.emit("image", fs.readFileSync("cat.png")); // Buffer

// browser
socket.on("image", (buf) => {              // ArrayBuffer
  const blob = new Blob([buf], { type: "image/png" });
  img.src = URL.createObjectURL(blob);
});

// browser: upload a file selected in <input type="file">
input.addEventListener("change", () => {
  socket.emit("upload", input.files[0], (status) => console.log(status));
});

// server
socket.on("upload", (file, ack) => {         // Buffer
  fs.writeFile("/tmp/upload", file, (err) => ack({ ok: !err }));
});
```

Binary is sent as separate WebSocket binary frames (a `BINARY_EVENT` packet with placeholders + attachments — see Chapter 7). Over polling it is base64-encoded.

> **Limits:** a single message larger than `maxHttpBufferSize` (default **1 MB**) causes the server to **close the connection** (the client sees a disconnect, typically `"transport close"` / `"transport error"`). For big files use **HTTP uploads** (multipart, pre-signed S3 URLs) or **chunk** the file into pieces with acks for flow control (Chapter 13 shows a chunked upload pattern).

3.3 Listener management

Server-side and client-side `Socket` listener methods

Both server and client sockets behave like an `EventEmitter`:

```
socket.on("event", listener);        // add listener
socket.once("event", listener);      // add one-time listener
socket.off("event", listener);       // remove a specific listener (alias: removeListener)
socket.off("event");                 // remove all listeners for this event (client)
socket.removeAllListeners("event");  // remove all listeners for this event
socket.removeAllListeners();         // remove ALL listeners (careful: also reserved ones like
disconnect)
socket.listeners("event");           // array of listeners
```

On the **client**, `socket.off()` with no arguments removes all listeners.

Catch-all listeners (`onAny`) — great for logging/debugging

```
// incoming
socket.onAny((eventName, ...args) => {
  console.log("IN ", eventName, args);
});

// outgoing (v4.5+)
socket.onAnyOutgoing((eventName, ...args) => {
  console.log("OUT", eventName, args);
});

socket.prependAny(listener);         // run before other catch-all listeners
socket.offAny(listener);             // remove one
socket.offAny();                     // remove all
socket.listenersAny();               // list
socket.prependAnyOutgoing(listener);
socket.offAnyOutgoing(listener);
socket.listenersAnyOutgoing();
```

Notes: - `onAny` does **not** receive acknowledgement packets, and the ack callback (if any) is the last element of `args`. - Reserved events (`connect`, `disconnect`, ...) are **not** passed to `onAny`.

Server-side per-event middleware: `socket.use()`

Runs for **every incoming event** on that socket before the handler (Chapter 6 covers it fully):

```
socket.use(([event, ...args], next) => {
  if (event === "admin:action" && !socket.data.user?.isAdmin) {
    return next(new Error("unauthorized"));
  }
  next();
});

// errors from socket.use() are emitted as an "error" event on the server socket
socket.on("error", (err) => {
  if (err && err.message === "unauthorized") socket.emit("app:error", "unauthorized");
});
```

3.4 Acknowledgements (request/response)

ELI5

Normally, `emit` is "fire and forget" — like throwing a paper plane over a wall. You don't know if anyone caught it. An **acknowledgement** is like attaching a self-addressed envelope: the receiver puts their answer inside and sends it back to *you specifically*.

Callback style (works in both directions)

```
// client
socket.emit("order:create", { item: "pizza" }, (response) => {
  console.log(response); // { ok: true, orderId: 42 }
});

// server
socket.on("order:create", async (payload, callback) => {
  const order = await orders.create(payload);
  callback({ ok: true, orderId: order.id });
});
```

Server → client works the same:

```
// server
socket.emit("are-you-there", (answer) => console.log(answer));

// client
socket.on("are-you-there", (callback) => callback("yes"));
```

Rules (all asked in interviews):

1. The ack function **must be the last argument** of `emit`.
2. The receiver gets it as the **last argument** of the handler.
3. Call it **at most once.** Calling twice: the second call is ignored (the ack id is consumed after the first reply).
4. You can pass **multiple arguments** to the callback: `callback(err, data)`.
5. Arguments follow the same serialization rules (JSON + binary).
6. **Not calling the callback** means the sender waits forever (memory held for the pending ack) unless a timeout is used. Always reply — even on errors.
7. The handler cannot know in advance whether the sender passed an ack, so validate: `if (typeof callback !== "function") return;` (malicious or buggy clients might not pass one, and calling `undefined()` throws inside your handler).

The error-first convention

Socket.IO itself doesn't impose a format. Pick one convention for the whole organisation. Two popular choices:

```
// (a) Node-style error-first
callback(null, { orderId: 42 });           // success
callback({ code: "OUT_OF_STOCK" });         // error

// (b) Envelope style (recommended — easy to type & evolve)
callback({ ok: true, data: { orderId: 42 } });
callback({ ok: false, error: { code: "OUT_OF_STOCK", message: "Pizza unavailable" } });
```

Acks with timeout

What if the other side never answers (network loss, buggy handler)? Use `timeout()`:

```
// client or server (server-side since v4.5)
socket.timeout(5000).emit("order:create", payload, (err, response) => {
  if (err) {
    // the other side did not acknowledge the event in the given delay
    return showError("Server did not respond");
  }
```

```
  console.log(response);
});
```

When `timeout()` is used, the callback gets an **extra first argument** `err` (an `Error` with message `"operation has timed out"`) — `null` on success. This is the #1 confusion with timeouts: **with `.timeout()`, the first callback arg is the error.**

Promise style: `emitWithAck()` (v4.6+)

```
// client
try {
  const response = await socket.timeout(5000).emitWithAck("order:create", payload);
  console.log(response);
} catch (err) {
  // timed out (only possible when a timeout was set)
}

// server, to one client
const answer = await socket.timeout(3000).emitWithAck("are-you-there");
```

Without any timeout (`.timeout()` or the client's `ackTimeout` option), the promise only settles when the other side answers — it may hang forever.

Broadcast with acknowledgements (server, v4.5+)

```
io.timeout(5000).emit("health-check", (err, responses) => {
  if (err) {
    // some clients did not answer in time; `responses` has the ones that did
  }
  console.log(responses); // one entry per client that answered
});

// promise version
try {
  const responses = await io.timeout(5000).emitWithAck("health-check");
} catch (e) {
  // some clients did not acknowledge in time
}

// to a room
const answers = await io.to("room-42").timeout(5000).emitWithAck("vote");
```

- **Promise vs callback when some clients don't answer (verified):** `emitWithAck` *rejects* on timeout, so you lose the answers that did arrive. The callback form gives you both: `io.timeout(1000).emit("poll", q, (err, answers) => …)` — `err` is set, and `answers` still contains every reply received in time. Use the callback form for polls, health checks and anything where partial results are useful.

- **Always use `.timeout()` with broadcast acks.** (In the 4.8 source, the broadcast timer is `setTimeout(..., this.flags.timeout)`; without a timeout it fires almost immediately with an error.)
- Works across a cluster with adapters that support it (Redis adapter etc.).

Acks don't work with `socket.broadcast` the same way?

`socket.broadcast.timeout(…).emit(…, cb)` and `socket.to(room).timeout(…).emit(…, cb)` are also `BroadcastOperator`s, so they behave like `io.timeout()`: one callback with an array of responses.

Client-side automatic retries — at-least-once from client (v4.6+)

```
const socket = io({
  ackTimeout: 10000, // wait up to 10 s for the server to ack each emit
  retries: 3,        // then retry up to 3 times
});

// this event will be retried until acknowledged (or 3 retries exhausted)
socket.emit("payment:submit", payload, (err, res) => {
  if (err) {/* all retries failed */}
});
```

How it works:

- Packets are queued and sent **one at a time** (in order); the next packet waits for the previous one's ack.
- If no ack arrives within `ackTimeout`, the packet is re-sent (up to `retries` times).
- **The server may receive the same event more than once** → your server handler must be **idempotent** (e.g. dedupe by a client-generated message id). That's the definition of *at-least-once*.

```
// client: include a unique id
socket.emit("message:send", { id: crypto.randomUUID(), text });

// server: dedupe
socket.on("message:send", async (msg, ack) => {
  const inserted = await db.messages.insertIfNotExists(msg.id, msg); // unique index on id
  ack({ ok: true, duplicate: !inserted });
});
```

You can also apply retries to a single emit with a timeout: `socket.timeout(5000).emit(...)` (no retries) vs. the `retries` option (global).

Ack patterns in production

1. **"Command" events** (create order, send message, join room) → **always ack**, with a timeout, with an envelope `{ ok, data | error }`.
2. **"Fire-and-forget" signals** (typing, cursor move, mouse position) → no ack, often `volatile`.
3. **Server-originated notifications** → generally no ack (client may be offline). If you need confirmation of delivery, store in DB and let the client "mark as read" via an event or HTTP call.

3.5 Volatile events

```
socket.volatile.emit("cursor", { x, y });
io.volatile.emit("price", tick);
```

ELI5: "If you can't hear me right now, forget it; I'll say a newer thing soon anyway."

Behaviour:

- **Server side:** the packet is **discarded** if the underlying transport is not currently writable (e.g. a polling request isn't open, or the socket is buffering) — instead of being buffered.
- **Client side:** the packet is discarded if the low-level transport is not currently writable — which includes the time when the socket is disconnected — instead of being added to `sendBuffer` like a normal emit.

Use for **high-frequency, superseded data**: positions, typing indicators, live prices, game state ticks, sensor readings. Never for things that matter individually (payments, chat messages).

3.6 Compression

```
socket.compress(false).emit("blob", bigBuffer); // disable compression for this packet
```

- Applies to **WebSocket per-message deflate** (if enabled with `perMessageDeflate` — **off by default** on the server since v3) and to **HTTP compression** of polling responses (`httpCompression`, on by default, threshold 1 KB).
- Default `compress` flag is `true` (i.e. "compress if the transport supports it and it is enabled").
- Disable for already-compressed data (images, video, gzip) → saves CPU.

Why is `perMessageDeflate` off by default? Each WebSocket with compression keeps a zlib context (~hundreds of KB of memory per connection with default window bits), and compression costs CPU — at 10k+ connections it can be a major memory/CPU cost (and historically caused memory fragmentation issues in Node). Enable it only when messages are big and compressible, and tune `threshold`, `zlibDeflateOptions`, `serverNoContextTakeover`.

3.7 Reserved event names (cannot emit these)

Calling `socket.emit("connect")` throws: `"connect" is a reserved event name`.

Side	Reserved names
Server `Socket`	`connect`, `connect_error`, `disconnect`, `disconnecting`, `newListener`, `removeListener`
Client `Socket`	`connect`, `connect_error`, `disconnect`, `disconnecting`, `newListener`, `removeListener`

Also avoid naming your own events like Manager events (`reconnect`, `reconnect_attempt`, `error`, `ping`) — they're on a different object (`socket.io`), so it technically works, but it confuses readers. The built-in `"message"` event is used by `socket.send()`:

```
socket.send("hi");        // same as socket.emit("message", "hi")
socket.on("message", ...);
```

3.8 Event naming conventions (organisational best practice)

Pick a convention and enforce it (lint rule / TypeScript event maps):

- `domain:action` — `chat:message`, `chat:typing`, `order:created`, `room:join`. (Recommended in the official docs' application-structure guide.)
- Use **past tense for facts the server announces** (`order:created`, `user:joined`) and **imperative for commands the client requests** (`order:create`, `room:join`).
- Lowercase, no spaces, no dynamic parts in names (put ids in the payload, not in the event name — `room:message` + `{ roomId }`, not `room-123-message`). Dynamic event names break typing, documentation and metrics, and can cause listener leaks.
- Version breaking payload changes: `order:created:v2` or a `version` field.

- Keep a **single shared file** describing events (TypeScript interfaces shared by client and server, or an AsyncAPI document).

3.9 Ordering guarantees

- Socket.IO guarantees **message ordering** for a given socket (packets are sent and received in emit order), **in both directions**, regardless of the transport and even during the polling → WebSocket upgrade (Engine.IO pauses polling and flushes before switching).
- Ordering is **not** guaranteed **across different sockets** (e.g. events from user A and user B), and **not across server processes** in a cluster for broadcasts originating on different nodes.
- With **async handlers**, *processing* may finish out of order even if *arrival* was in order:

```
// ⚠️ arrival order: A then B — completion order can be B then A
socket.on("doc:edit", async (op) ⇒ {
  await db.apply(op);          // A may take longer than B
});
```

If processing order matters, serialize per socket (or per document) with a queue:

```
const queue = [];
let busy = false;
socket.on("doc:edit", (op, ack) ⇒ { queue.push([op, ack]); drain(); });

async function drain() {
  if (busy) return;
  busy = true;
  while (queue.length) {
    const [op, ack] = queue.shift();
    try { await db.apply(op); ack?.({ ok: true }); }
    catch (e) { ack?.({ ok: false, error: e.message }); }
  }
  busy = false;
}
```

(Libraries like `p-queue` with `concurrency: 1` do this neatly. For collaborative editing you'd also use OT/CRDT — Chapter 13.)

3.10 Async handlers and errors — a production must-know

Socket.IO **does not catch errors thrown inside your event handlers** for you, and it does not catch rejected promises from `async` handlers:

```
// ❌ An exception here goes up to the EventEmitter → can crash the process
socket.on("x", (data) => {
  JSON.parse(data); // throws on bad input
});

// ❌ An unhandled rejection: in Node 15+ unhandled rejections crash the process by default
socket.on("y", async () => {
  await somethingThatRejects();
});
```

Fix: wrap every handler. A small helper:

```
const safe = (handler) => async (...args) => {
  const ack = typeof args[args.length - 1] === "function" ? args.pop() : null;
  try {
    const result = await handler(...args);
    ack?.({ ok: true, data: result });
  } catch (err) {
    logger.error({ err }, "socket handler failed");
    ack?.({ ok: false, error: { code: err.code ?? "INTERNAL", message: "Something went wrong" }
});
  }
};

socket.on("order:create", safe(async (payload) => {
  const valid = OrderSchema.parse(payload);   // zod validation throws on bad input
  return orderService.create(socket.data.user.id, valid);
}));
```

This gives you: validation, consistent ack envelopes, logging, no crashes. Combine with `process.on("unhandledRejection")` logging as a last resort (not as error handling).

3.11 `send()` and `message`

```
socket.send("hello");              // emits a "message" event
socket.on("message", (data) => {}); // receives it
io.send("to everyone");
```

Rarely used in modern code; named events are clearer.

3.12 Emitting from outside a connection handler

You often need to emit from a REST controller, a cron job, or a queue consumer.

```
// keep a reference to io (module export, DI container, app.set("io", io) in Express…)
app.post("/api/orders", async (req, res) => {
  const order = await createOrder(req.body);
  req.app.get("io").to(`user:${order.userId}`).emit("order:created", order);
  res.status(201).json(order);
});
```

From a **different process** (worker, microservice) that is *not* a Socket.IO server, use an **emitter** (`@socket.io/redis-emitter`, or the Postgres/MongoDB emitters) — see Chapter 8.

Emitting to a **user** rather than a socket: make each socket join a room named after the user id in the `connection` handler (`socket.join("user:" + userId)`), then emit to that room. This handles multiple tabs/devices and reconnections (socket ids change; user ids don't). This is *the* standard pattern.

3.13 Chapter summary

- `emit(name, ...args)` / `on(name, handler)` in both directions; prefer one object arg.
- Data: JSON-serializable + binary. Dates → strings, Map/Set → `{}`, BigInt throws.
- Acks = request/response; last argument; call once; always reply; validate it's a function.
- `timeout(ms)` adds an `err` first argument; `emitWithAck` returns a Promise.
- Broadcast acks require `.timeout()` and give an array of responses.
- Client `retries` + `ackTimeout` = at-least-once from client → make handlers idempotent.
- `volatile` = droppable; `compress(false)` for pre-compressed data.
- Reserved names: connect, connect_error, disconnect, disconnecting, newListener, removeListener.
- Order is guaranteed per socket, but async processing can reorder — queue if needed.
- Wrap handlers to catch errors (sync and async) and always ack.

Next: Chapter 4 — Rooms, namespaces and broadcasting

CHAPTER 04

Rooms, Namespaces and Broadcasting

> **Goal:** understand how to send messages to *groups* of clients. Rooms and namespaces are the most-asked Socket.IO interview topic after "what is Socket.IO".

4.1 ELI5

Imagine a big **school** (the Socket.IO server).

- The school has separate **buildings**: the main building, the sports building, the teachers' building. Those are **namespaces** (`/` , `/sports` , `/admin`). You choose which building to enter, and each building can have its own guard (middleware) and rules (handlers). Students *choose* buildings.
- Inside each building there are **classrooms**: "Math 101", "Art club". Those are **rooms**. The **principal (server)** puts students into classrooms and takes them out. Students can't walk into a room by themselves — they have to *ask* the server, which decides.
- The principal can use the **loudspeaker** to talk to: everyone in a building (`io.of("/sports").emit`), everyone in a classroom (`io.to("math-101").emit`), everyone except one kid (`socket.broadcast.emit`), and so on. That's **broadcasting**.
- Every student also has a **private room with their own name on it** (the socket's id) — so the principal can whisper to one student by sending to that room.

4.2 Rooms

What a room is

A **room** is an arbitrary **server-side** channel name that sockets can **join** and **leave**. It is used to broadcast events to a subset of clients.

Key facts:

1. Rooms are a **server-only** concept. The client doesn't know which rooms it's in (unless you tell it). There is **no** `socket.join()` on the client.
2. A room is just a **string** (any string: `"room-42"` , `"user:123"` , `"org:9:admins"`).

3. Rooms are **created automatically** when the first socket joins and **deleted automatically** when the last socket leaves.
4. Every socket **automatically joins a room named after its own id** (`socket.id`). That's how `io.to(socketId).emit()` works for private messages.
5. Sockets **automatically leave all their rooms on disconnect**. You don't need to clean up.
6. Rooms belong to a **namespace**: room `"chess"` in `/` is unrelated to room `"chess"` in `/games`.
7. A socket can be in **any number** of rooms.

Join / leave

```
io.on("connection", (socket) => {
  socket.join("some room");                 // join one room
  socket.join(["room1", "room2"]);          // join several
  socket.leave("some room");                // leave one
  console.log(socket.rooms);                // Set { <socket.id>, "room1", "room2" }
});
```

`socket.join()` returns `void` with the default in-memory adapter, but may return a **Promise** with some adapters (the type is `Promise<void> | void`). Writing `await socket.join(room)` is safe either way and future-proof.

Broadcasting to rooms

```
io.to("room1").emit("event");                    // everyone in room1 (incl. the sender, if in
it)
socket.to("room1").emit("event");                // everyone in room1 EXCEPT the sender
io.to("room1").to("room2").emit("event");        // union: in room1 OR room2 — each socket
receives it ONCE
io.to(["room1", "room2"]).emit("event");         // same, array form
io.except("room3").emit("event");                // everyone except sockets in room3
io.to("room1").except("room2").emit("event");    // in room1 but not in room2
io.to(socketId).emit("private", "hi");           // one specific socket
io.in("room1").emit("event");                    // `in` is an alias of `to`
```

> **Interview trap:** joining `to("a").to("b")` is a **union**, not an intersection, and a socket in both rooms receives the event **only once** (the adapter de-duplicates). There's no built-in intersection — implement it with a dedicated room (`"a&b"`) or `fetchSockets()` + filtering.
>
> **Interview trap 2:** `socket.to(room).emit()` excludes the *sender*. `io.to(room).emit()` includes everyone in the room (the sender too, if it's in the room).

Typical room usages (patterns)

Use case	Room naming
Chat channel / conversation	`conversation:<id>`
All devices/tabs of a user	`user:<userId>` ← **most important pattern**
Organisation/tenant broadcast	`org:<orgId>`
Role-based broadcast	`org:<orgId>:role:admin`
A document being edited	`doc:<docId>`
A game match	`match:<matchId>`
Subscribers to a resource (e.g. stock)	`ticker:AAPL`
Geographic/area	`geo:<geohash>`

The per-user room pattern (memorise)

```
io.use(authMiddleware); // sets socket.data.user

io.on("connection", (socket) => {
  const userId = socket.data.user.id;
  socket.join(`user:${userId}`);   // all tabs/devices of this user share this room
});

// anywhere in the app (REST controller, job, etc.)
io.to(`user:${userId}`).emit("notification", payload);
```

Why: socket ids change on every reconnection and a user may have several tabs/devices; the user room is stable and covers all of them. **Never** store "userId → socketId" in a plain variable map — it breaks with multiple tabs, reconnections and multiple servers.

Room lifecycle events (adapter events)

```
io.of("/").adapter.on("create-room", (room) => console.log(`room ${room} was created`));
io.of("/").adapter.on("delete-room", (room) => console.log(`room ${room} was deleted`));
io.of("/").adapter.on("join-room", (room, id) => console.log(`socket ${id} joined ${room}`));
io.of("/").adapter.on("leave-room", (room, id) => console.log(`socket ${id} left ${room}`));
```

Note: these fire for the **socket's own private room** too (`room === id`). These events are emitted **locally** on each server (with the Redis adapter each node only sees its own sockets' joins).

Inspecting rooms

```
io.of("/").adapter.rooms;   // Map<Room, Set<SocketId>>  (LOCAL node only)
io.of("/").adapter.sids;    // Map<SocketId, Set<Room>>  (LOCAL node only)

// cluster-aware (works with adapters):
const sockets = await io.in("room1").fetchSockets();
sockets.length;                                          // number of sockets in room1

const ids = await io.in("room1").allSockets();          // Set<SocketId> (older API)
```

> `adapter.rooms` contains **both** real rooms and every socket's private id-room. To list "real" rooms only, filter out keys that are also in `adapter.sids`: `js const { rooms, sids } = io.of("/").adapter; const realRooms = [...rooms.keys()].filter((r) => !sids.has(r));`

`disconnecting` vs `disconnect` — read rooms before they're gone

```
socket.on("disconnecting", (reason) => {
  // socket.rooms is still populated here
  for (const room of socket.rooms) {
    if (room !== socket.id) socket.to(room).emit("user:left", socket.id);
  }
});

socket.on("disconnect", (reason) => {
  // socket.rooms is EMPTY here
});
```

4.3 Server-side utility methods (bulk operations) — v4+

These work on the local server **and across a cluster** (when the adapter supports it):

```
// make all sockets join room1
io.socketsJoin("room1");

// make all sockets in room1 also join room2 and room3
io.in("room1").socketsJoin(["room2", "room3"]);

// make the socket with id `theSocketId` join room1 (works even if it's connected on another
node!)
io.in(theSocketId).socketsJoin("room1");
```

```
// make all sockets leave room1
io.socketsLeave("room1");
io.in("room1").socketsLeave(["room2", "room3"]);

// disconnect all sockets / those in a room
io.disconnectSockets();
io.in("room1").disconnectSockets();
io.in(theSocketId).disconnectSockets(true); // true = close the underlying connection too

// fetch sockets (RemoteSocket objects when on another node)
const sockets = await io.fetchSockets();
const roomSockets = await io.in("room1").fetchSockets();
for (const s of sockets) {
  s.id; s.handshake; s.rooms; s.data;      // readable
  s.emit("hello");                         // writable operations work remotely
  s.join("room2"); s.leave("room2");
  s.disconnect();
}
```

Practical examples

```
// a user was removed from an organisation → kick all their sockets out of the org room
io.in(`user:${userId}`).socketsLeave(`org:${orgId}`);

// a user was banned → disconnect every device
io.in(`user:${userId}`).disconnectSockets(true);

// how many people are viewing document 42 right now (cluster-wide)?
const viewers = (await io.in("doc:42").fetchSockets()).map((s) => s.data.user.name);
```

> **Performance warning:** `fetchSockets()` across a cluster asks *every* node and waits for responses (with a timeout, default 5000 ms in the Redis adapter's `requestsTimeout`). Don't call it on hot paths (every message). For presence counts at scale, maintain counters in Redis instead (Chapter 13).

`socket.data` is the place to store per-socket info (user id, name, roles). It is **serialized** and returned by `fetchSockets()` on other nodes, so keep it small and JSON-serializable.

4.4 Broadcasting — all the flavours

Broadcast = send to many sockets with one call. The server builds/encodes the packet **once** and writes it to every target socket (efficient).

```
io.emit("x");                       // all sockets in the main namespace "/"
io.of("/admin").emit("x");          // all sockets in namespace /admin
socket.broadcast.emit("x");         // all sockets in the socket's namespace except the sender
socket.to("r").emit("x");           // room r, except sender
io.to("r").emit("x");               // room r
io.except("r").emit("x");           // all except room r
io.local.emit("x");                 // only sockets connected to THIS server node (cluster)
io.volatile.emit("x");              // droppable
io.compress(false).emit("x");
io.timeout(5000).emit("x", (err, responses) => {}); // with acks
```

Modifiers chain in any order before `emit`:

```
io.of("/game").to("match:7").except("spectators").volatile.compress(false).emit("tick", state);
```

The "broadcast to everyone except the sender" is a server responsibility

The client can't broadcast. Pattern:

```
// client
socket.emit("cursor:move", { x, y });

// server: relay to others in the same doc
socket.on("cursor:move", (pos) => {
  socket.to(`doc:${socket.data.docId}`).volatile.emit("cursor:move", { id: socket.id, ...pos
});
});
```

Broadcasting from multiple servers

With several Socket.IO servers (processes/containers), `io.emit()` on server A only reaches clients connected to server A **unless you configure an adapter** (Redis etc.) that forwards the broadcast to all other servers. That's Chapter 8. `io.local` lets you deliberately skip the adapter.

4.5 Namespaces

What a namespace is

A **namespace** is a communication channel that lets you **split the logic of your application over a single shared connection** (this sharing is called **multiplexing**).

Each namespace has its own:

- event handlers (`io.of("/admin").on("connection", ...)`),
- rooms (independent per namespace),
- middleware (`io.of("/admin").use(...)`).

The default (main) namespace is `"/"` . `io.sockets` is an alias for `io.of("/")` , and `io.on("connection")` , `io.emit()` , `io.use()` , `io.to()` are shortcuts for the main namespace.

```
// server
io.of("/").on("connection", (socket) => {});        // same as io.on("connection")
io.of("/orders").on("connection", (socket) => {});
io.of("/users").on("connection", (socket) => {});

// client
const mainSocket = io("https://example.com");          // namespace "/"
const ordersSocket = io("https://example.com/orders"); // namespace "/orders"
const usersSocket = io("https://example.com/users");   // namespace "/users"
// ↑ all three share ONE underlying WebSocket connection (multiplexing)
```

When to use namespaces vs rooms (classic interview question)

	Namespace	Room
Who decides membership	**Client** chooses which namespace to connect to	**Server** joins/leaves sockets
Separate handlers/middleware	✅ Yes	❌ No (same handlers)
Scope	Separation of **concerns/features/permissions**	**Grouping of users** for broadcasting
Number	Few, mostly static (`/` , `/admin` , `/chat`) — or dynamic for multi-tenancy	Many, dynamic (one per conversation/user/doc)
Connection	Each namespace = separate `Socket` object; shared transport	Rooms live inside a namespace

	Namespace	Room
Typical use	`/admin` with admin-only auth middleware; separating product areas; multi-tenant `/tenant-<id>`	chat channels, user rooms, game matches

Rule of thumb: **use rooms by default**; use namespaces when you need different middleware/authorization or a separate set of event handlers.

Namespace middleware (per-namespace auth)

```
const admin = io.of("/admin");

admin.use(async (socket, next) => {
  const user = await verify(socket.handshake.auth.token);
  if (user?.role !== "admin") return next(new Error("forbidden"));
  socket.data.user = user;
  next();
});

admin.on("connection", (socket) => {
  socket.on("user:ban", banHandler);
});
```

Clients connecting to `/admin` without being admins get `connect_error`; their connection to `/` is unaffected.

Dynamic namespaces

Create namespaces on the fly (great for **multi-tenancy**):

```
// with a RegExp
const workspaces = io.of(/^\/workspace-\w+$/);

workspaces.use(tenantAuth);                  // middleware for every matching child namespace

workspaces.on("connection", (socket) => {
  const workspace = socket.nsp;              // the child Namespace, e.g. "/workspace-acme"
  console.log(workspace.name);
  workspace.emit("hello");                   // broadcast to everyone in THIS child namespace
});

// broadcast to ALL child namespaces matched by the parent
workspaces.emit("maintenance", "in 5 minutes");
```

```
// with a function (async check, e.g. tenant exists in DB)
io.of(async (name, auth, next) => {
```

```
  const ok = name.startsWith("/tenant-") && (await tenants.exists(name.slice(8)));
  next(null, ok);   // (error, allowed)
});
```

- Child namespaces are created on first connection. With `cleanupEmptyChildNamespaces: true` (v4.6+, server option), a child namespace with no sockets left is removed (prevents memory growth with many tenants).
- Connecting to a namespace that doesn't exist (and doesn't match any dynamic pattern) gives the client a `connect_error` with message `"Invalid namespace"`.

Multiplexing in detail (client side)

```
const a = io("https://example.com");         // Manager #1 created for this URL
const b = io("https://example.com/admin");   // reuses Manager #1 → same connection
const c = io("https://example.com/admin", { forceNew: true }); // new Manager → new connection
const d = io("https://example.com", { multiplex: false });     // new Manager too
```

- A client-side `Manager` owns the Engine.IO connection (transport, reconnection); each `Socket` belongs to a Manager and represents one namespace.
- Access the Manager via `socket.io` (yes, the property is literally named `io`).
- Two `io()` calls for the **same URL and the same namespace**: the client's lookup function notices the namespace is already used on the cached Manager and creates a **new Manager → a second, separate connection**. This is a classic source of "duplicate connections" bugs (e.g. calling `io()` inside a React component that re-renders). Create the socket **once** and share it (Chapter 9).
- Two `io()` calls for the same URL but **different namespaces** reuse the cached Manager → one shared connection (multiplexing).
- **Query parameters/ `extraHeaders` are per Manager**, shared by all namespaces on it; `auth` **is per namespace Socket**.

Namespace-level vs main-namespace properties

Property	Meaning
`socket.nsp`	the Namespace the socket belongs to
`io.of("/x").sockets`	`Map<SocketId, Socket>` of **local** sockets in this namespace
`io.of("/x").adapter`	the adapter instance for this namespace
`io._nsps`	`Map<string, Namespace>` of all namespaces (private-ish)

4.6 Socket IDs — what they are and aren't

- `socket.id` (Socket.IO) is a random 20-character id, generated per namespace connection. It is **not** the same as the Engine.IO id (`socket.conn.id`) since v3 — exposing the engine id would allow session hijacking of the underlying connection.
- On the **client**, `socket.id` is **undefined until connected** and **changes after each reconnection** (unless connection state recovery restored the session).
- Since it's random and ephemeral, **don't use it as an identity**; use your authenticated user id. Don't trust a socket id *sent by a client* as authorization ("delete messages of socket X") — always use `socket.data.user` from auth.
- You can customise the Engine.IO id with `io.engine.generateId = (req) => uuid()` . (The Socket.IO-level id is generated internally.)

4.7 Worked example: a chat with channels and private messages

```
io.use(authMiddleware); // → socket.data.user = { id, name }

io.on("connection", (socket) => {
  const { id: userId, name } = socket.data.user;
  socket.join(`user:${userId}`);

  socket.on("channel:join", async ({ channelId }, ack) => {
    if (!(await acl.canJoin(userId, channelId))) return ack?.({ ok: false, error: "FORBIDDEN"
});
    await socket.join(`channel:${channelId}`);
    socket.to(`channel:${channelId}`).emit("channel:user-joined", { channelId, userId, name });
    ack?.({ ok: true, data: { history: await messages.last(channelId, 50) } });
  });

  socket.on("channel:leave", async ({ channelId }, ack) => {
    await socket.leave(`channel:${channelId}`);
    socket.to(`channel:${channelId}`).emit("channel:user-left", { channelId, userId });
    ack?.({ ok: true });
  });

  socket.on("channel:message", async ({ channelId, text, clientMsgId }, ack) => {
    if (!socket.rooms.has(`channel:${channelId}`)) return ack?.({ ok: false, error:
"NOT_A_MEMBER" });
    const msg = await messages.save({ channelId, userId, text, clientMsgId }); // persisted
first!
    io.to(`channel:${channelId}`).emit("channel:message", msg); // includes sender's other tabs
    ack?.({ ok: true, data: { id: msg.id } });
  });
```

```
  socket.on("dm:send", async ({ toUserId, text }, ack) => {
    const msg = await dms.save({ from: userId, to: toUserId, text });
    io.to(`user:${toUserId}`).to(`user:${userId}`).emit("dm:message", msg); // both parties,
all devices
    ack?.({ ok: true });
  });
});
```

Things to notice (interviewers notice them too): - membership is **authorized server-side** before joining; - messages are **persisted before broadcasting** (so reconnecting clients can fetch them); - the sender's **other tabs** get the message (`io.to` not `socket.to`); - DMs use **user rooms**, not socket ids; - every command is **acked**.

4.8 Chapter summary

- Rooms: server-side groups; join/leave on server; auto-created/deleted; auto-left on disconnect; every socket is in its own id-room.
- `io.to()` includes sender; `socket.to()` / `socket.broadcast` excludes sender; multiple `to()` = union (no duplicates); `except()` excludes.
- Utility methods: `socketsJoin`, `socketsLeave`, `disconnectSockets`, `fetchSockets` — cluster-aware.
- Use the `user:<id>` room pattern instead of a userId → socketId map.
- Namespaces: client-chosen channels with their own handlers, middleware and rooms, multiplexed over one connection; dynamic namespaces for multi-tenancy.
- Use rooms for grouping, namespaces for separation of concerns/permissions.

Next: Chapter 5 — Connection lifecycle

CHAPTER 05

Connection Lifecycle: Connect, Disconnect, Reconnect, Heartbeats, Recovery and Delivery Guarantees

> **Goal:** know exactly what happens from the first byte to the last, every event fired on both sides, every disconnect reason, how reconnection works, how dead connections are detected, what happens to messages sent while offline, and how to get stronger delivery guarantees. This chapter is where senior interviews are won or lost.

5.1 ELI5

- **Connecting** is like dialling a friend: first you say "hello, it's me" (handshake), your friend may check who you are (middleware), then says "come in" (`connect`).
- **Heartbeat**: every 25 seconds your friend asks "still there?" (ping) and you answer "yes!" (pong). If you don't answer in 20 seconds, your friend hangs up.
- **Disconnect**: the call ends — because someone hung up on purpose, or the line broke.
- **Reconnect**: if the line broke by accident, your phone redials automatically, waiting a bit longer each time (1 s, 2 s, 4 s… up to 5 s, with a bit of randomness so a thousand phones don't all redial at the exact same moment).
- **Buffering**: things you said while the line was dead are written on a notepad and said again when the line comes back.
- **Connection state recovery**: if you were away only a short time, your friend remembers what group chats you were in and tells you what you missed.

5.2 Server-side lifecycle events

```
// Engine.IO level (low-level connection, before any namespace)
io.engine.on("initial_headers", (headers, req) ⇒ { /* modify handshake response headers */ });
io.engine.on("headers", (headers, req) ⇒ { /* modify headers of every HTTP response */ });
io.engine.on("connection", (rawSocket) ⇒ { /* Engine.IO socket established */ });
io.engine.on("connection_error", (err) ⇒ {
  console.log(err.req);      // the request object
  console.log(err.code);     // the error code, e.g. 1
```

```
  console.log(err.message);  // the error message, e.g. "Session ID unknown"
  console.log(err.context);  // extra context
});

// Socket.IO level (per namespace)
io.use((socket, next) => { /* middleware: runs before "connection" */ next(); });

io.on("connection", (socket) => {
  socket.on("disconnecting", (reason, description) => { /* still in rooms */ });
  socket.on("disconnect", (reason, description) => { /* rooms already left */ });
  socket.on("error", (err) => { /* errors from socket.use() middleware */ });
});
```

`io.on("connect")` is an alias of `io.on("connection")`.

Engine.IO `connection_error` codes (know these for debugging)

Code	Message	Typical cause
0	`Transport unknown`	Client asked for a transport not enabled on the server (e.g. server `transports: ["websocket"]`, client polling)
1	`Session ID unknown`	The `sid` isn't known by **this** server → **missing sticky sessions** in a multi-server setup, or the server restarted
2	`Bad handshake method`	Handshake not done with GET
3	`Bad request`	Malformed request (e.g. invalid `Origin` header, bad query)
4	`Forbidden`	Rejected by `allowRequest`
5	`Unsupported protocol version`	v2 client (`EIO=3`) on a v3/v4 server without `allowEIO3: true`

Server-side disconnect reasons (memorise)

Reason	Meaning	Side that initiated
`server namespace disconnect`	You called `socket.disconnect()` on the server	Server
`client namespace disconnect`	The client called `socket.disconnect()`	Client
`server shutting down`	The server is closing (`io.close()`)	Server
`ping timeout`	Client didn't answer a PING within `pingTimeout`	Network / client
`transport close`	The connection was closed (user closed tab, lost network, proxy cut it)	Network / client

Reason	Meaning	Side that initiated
`transport error`	The connection encountered an error	Network
`parse error`	Server received an invalid packet	Client bug / attack
`forced close`	The low-level Engine.IO connection was closed by the server (e.g. `socket.conn.close()`)	Server
`forced server close`	The server closed the whole underlying connection — e.g. `socket.disconnect(true)` on one namespace closes the connection, and the client's sockets in *other* namespaces get this reason	Server

The second argument `description` (when available) gives low-level details (e.g. the underlying error/close code).

> Interview note: `socket.disconnect(true)` on the server closes the **underlying connection** (all namespaces of that client); `socket.disconnect()` (or `false`) only leaves **that namespace**.

5.3 Client-side lifecycle events

```
const socket = io(URL);

// --- Socket events (per namespace) ---
socket.on("connect", () => {
  console.log(socket.id, socket.connected, socket.recovered);
});

socket.on("connect_error", (err) => {
  // fired when: low-level connection can't be established, OR middleware rejected (next(err))
  console.log(err.message);   // e.g. "xhr poll error", "websocket error", "timeout", or your middleware message
  console.log(err.data);      // extra data from server middleware (err.data = {...})
  console.log(socket.active); // true → will retry automatically; false → you must call socket.connect()
});

socket.on("disconnect", (reason, details) => {
  console.log(reason);        // see table below
  console.log(socket.active); // true if automatic reconnection will happen
});

// --- Manager events (the connection; one per multiplexed connection) ---
```

```
socket.io.on("open", () => {});                          // Engine.IO connection opened
socket.io.on("error", (err) => {});                      // connection error
socket.io.on("ping", () => {});                          // a PING was received from the server
socket.io.on("packet", (packet) => {});                  // any packet
socket.io.on("close", (reason, description) => {});      // underlying connection closed
socket.io.on("reconnect_attempt", (attempt) => {});      // before each attempt
socket.io.on("reconnect_error", (err) => {});            // an attempt failed
socket.io.on("reconnect", (attempt) => {});              // success after `attempt` tries
socket.io.on("reconnect_failed", () => {});              // gave up (reconnectionAttempts
reached)

// --- Engine.IO level (transport) ---
socket.io.engine.on("upgrade", (transport) => console.log("upgraded to", transport.name));
socket.io.engine.transport.name; // "polling" | "websocket" | "webtransport"
```

> **v2 → v3 trap:** in v2 you wrote `socket.on("reconnect", …)`. Since v3 the reconnection events are on the **Manager** (`socket.io.on("reconnect", …)`). Writing `socket.on("reconnect")` in v3/v4 silently does nothing — a very common bug when migrating.

Client-side disconnect reasons

Reason	Meaning	Auto-reconnect?
`io server disconnect`	Server called `socket.disconnect()`	**No** — call `socket.connect()` yourself if appropriate
`io client disconnect`	You called `socket.disconnect()` on the client	**No**
`ping timeout`	No PING from server within `pingInterval + pingTimeout`	Yes
`transport close`	Connection closed (user lost network, server restarted, proxy timeout)	Yes
`transport error`	Connection error (e.g. server killed during a polling cycle)	Yes
`parse error`	Client received an invalid packet	Yes

```
socket.on("disconnect", (reason) => {
  if (reason === "io server disconnect") {
    // the server kicked us (e.g. session expired, banned).
    // Decide: refresh the token and reconnect, or show a "you've been logged out" UI.
  }
  // otherwise the client will try to reconnect automatically
});
```

Client socket properties

Property	Meaning
`socket.id`	Socket id, `undefined` until connected; new value on each reconnection (unless recovered)
`socket.connected` / `socket.disconnected`	Current state
`socket.active`	`true` if the socket is connected **or will try to reconnect.** `false` after `disconnect()` or a server-side disconnect/middleware rejection
`socket.recovered`	`true` if connection state recovery succeeded on the last connect (v4.6+)
`socket.io`	The Manager
`socket.auth`	The auth payload (can be updated before reconnecting)
`socket.sendBuffer`	Packets emitted while disconnected, waiting to be sent
`socket.receiveBuffer`	Packets received before the `connect` was processed

Client methods

```
socket.connect();     // (alias: socket.open()) — connect manually (autoConnect: false, or after a server disconnect)
socket.disconnect();  // (alias: socket.close()) — disconnect; no auto-reconnection
socket.io.reconnection(false); // turn off automatic reconnection on the fly
socket.io.opts.query = { ... }; // change query for the NEXT connection attempt (engine-level)
socket.auth = { token: newToken }; // change auth for the NEXT CONNECT packet
```

5.4 The complete connection sequence (step by step with real packets)

These packets were captured from a real Socket.IO 4.8.4 server using the polling transport:

```
1) GET /socket.io/?EIO=4&transport=polling
   ← 0{"sid":"HSJR3YuCOE9jPqHEAAAA","upgrades":
["websocket"],"pingInterval":25000,"pingTimeout":20000,"maxPayload":1000000}
      ^ Engine.IO OPEN packet (type 0) with session id, possible upgrades, heartbeat settings,
max payload

2) POST /socket.io/?EIO=4&transport=polling&sid=HSJR3YuCOE9jPqHEAAAA
   body: 40
```

```
      ^ Engine.IO MESSAGE (4) containing Socket.IO CONNECT (0) to namespace "/"
   ← ok

3) GET /socket.io/?EIO=4&transport=polling&sid=HSJR3YuCOE9jPqHEAAAA
   ← 40{"sid":"KykvYCRwmIsIifIyAAAA"}
      ^ Socket.IO CONNECT reply with the Socket.IO socket id (different from the Engine.IO sid)

4) POST … body: 421["hello","world"]
      ^ MESSAGE(4) + EVENT(2) + ack id 1 + ["event", ...args]

5) GET …
   ← 431[{"ok":true}]
      ^ MESSAGE(4) + ACK(3) + ack id 1 + [response args]
```

Then, typically within milliseconds, the **upgrade** to WebSocket:

```
6) GET /socket.io/?EIO=4&transport=websocket&sid=HSJR3YuCOE9jPqHEAAAA   (Upgrade: websocket)
   → 2probe            (client sends PING with "probe" over the new WebSocket)
   ← 3probe            (server answers PONG "probe")
   (client pauses polling: waits for in-flight polling requests to finish)
   → 5                 (client sends UPGRADE packet over the WebSocket)
   (server closes the polling transport with a NOOP (6) to the pending GET if needed)
   — from now on everything flows over the WebSocket —
   ← 2                 (server PING every pingInterval)
   → 3                 (client PONG)
```

If the WebSocket fails (blocked by proxy/firewall), the client just stays on polling; the user doesn't notice except for higher latency/overhead.

Why start with polling and then upgrade? (classic question)

1. **Reliability first:** polling works nearly everywhere. Some proxies/antivirus accept the WebSocket handshake but then never deliver data — if the client went WebSocket-only, the user would wait for a timeout (could be many seconds) before falling back. Starting with polling gives an immediately working connection.
2. **Then optimise:** upgrade to WebSocket in the background for lower latency/overhead.

Trade-offs of the default:

- More HTTP requests at start; **requires sticky sessions** when you have multiple servers (the polling requests and the WebSocket upgrade must hit the same server).
- You can skip polling with `transports: ["websocket"]` on the client → no sticky sessions needed and faster connection — but **no fallback** if WebSocket is blocked. Many companies do this for internal/mobile apps where the network is known. A middle ground: `transports: ["websocket", "polling"]` (try WebSocket first) and on `connect_error` revert to `["polling", "websocket"]`:

```
const socket = io({ transports: ["websocket", "polling"] });
socket.on("connect_error", () => {
```

```
  // revert to classic upgrade
  socket.io.opts.transports = ["polling", "websocket"];
});
```

(With 4.8+, the client option `tryAllTransports: true` makes the client try the next transport in the list if the first one fails, instead of giving up.)

5.5 Heartbeat (ping/pong) mechanism in depth

Why we need heartbeats

TCP itself might not notice a dead peer for a very long time: if a phone goes into a tunnel, or a laptop sleeps, or a NAT/proxy silently drops the mapping, no FIN/RST packet is ever sent. The server would keep a "zombie" socket forever (memory leak, wrong presence status: "Alice is online" when she isn't). Proxies and load balancers also cut **idle** connections (e.g. AWS ALB default idle timeout 60 s, nginx `proxy_read_timeout` default 60 s) — regular traffic keeps them alive.

How it works in Socket.IO v3/v4 (Engine.IO v4)

```
server ──PING(2)──► client          every pingInterval (default 25 000 ms)
server ◄──PONG(3)── client          client must reply within pingTimeout (default 20 000 ms)
```

- **Server side:** after sending a PING, if no PONG arrives within `pingTimeout`, the server closes the connection with reason `ping timeout`.
- **Client side:** the client expects a PING at least every `pingInterval + pingTimeout` (45 s by default). If nothing arrives in that window, it closes with `ping timeout` and starts reconnecting.
- Since 4.8 the client also tracks the **expiration timestamp** of that timer, because timers may be throttled/frozen (laptop suspended, phone locked); when the app wakes up and tries to emit, it detects the expired heartbeat instead of writing into a dead connection (the packet is buffered and sent after reconnection).
- The values are sent to the client in the handshake (`pingInterval`, `pingTimeout`), so you configure them **on the server only**.

Worst-case detection time of a dead client on the server ≈ `pingInterval + pingTimeout` = 45 s by default.

Tuning

```
const io = new Server(httpServer, {
  pingInterval: 10000, // ping every 10 s
  pingTimeout: 5000,   // 5 s to answer
});
```

- **Lower values** → faster dead-connection detection, more traffic (tiny packets though), more false positives on slow mobile networks or with busy event loops.
- **Higher values** → fewer packets, slower detection.
- Keep `pingInterval` **below the idle timeout of every proxy/load balancer** in the path (ALB, nginx, Cloudflare (100 s), corporate proxies), otherwise idle connections get cut → `transport close` loops every N seconds. Classic bug: "clients disconnect exactly every 60 seconds" = proxy idle timeout < heartbeat interval or nginx `proxy_read_timeout`.
- **Event loop blocking:** if your Node.js process blocks the event loop for longer than `pingTimeout` (heavy sync computation, huge JSON.parse, sync crypto), it can't process PONGs/send PINGs on time → mass `ping timeout` disconnects. Monitor event loop lag.
- **Browser background tabs:** browsers throttle timers in background tabs (≥1 s, or even 1 min for intensive throttling in Chrome after 5 min hidden). Because the **server** sends pings (v3+), the client only needs to *respond*, which works fine even when throttled (receiving a message isn't throttled the same way). This was the main reason for reversing the heartbeat direction in v3/Engine.IO v4.

5.6 Automatic reconnection in depth

The algorithm (exponential back-off with jitter)

Client options (with defaults, verified in source):

```
io(URL, {
  reconnection: true,            // enable auto-reconnection
  reconnectionAttempts: Infinity,// max attempts before "reconnect_failed"
  reconnectionDelay: 1000,       // initial delay (ms)
  reconnectionDelayMax: 5000,    // max delay (ms)
  randomizationFactor: 0.5,      // jitter factor (0 ≤ x ≤ 1)
  timeout: 20000,                // each connection attempt's timeout
});
```

Delay for attempt n (0-based) is computed roughly as:

```
base   = reconnectionDelay * 2^n             // 1000, 2000, 4000, 8000, …
jitter = random * randomizationFactor * base // up to ±50%
```

```
delay  = base ± jitter, capped at reconnectionDelayMax
```

So with defaults: ~1 s (0.5–1.5 s), ~2 s (1–3 s), ~4 s (2–5 s), then 5 s max for every next attempt (the cap applies after jitter).

Why jitter? If a server restarts with 50 000 connected clients and they all reconnect after exactly 1 s, 2 s, 4 s... the server gets 50 000 simultaneous handshakes in waves — the **thundering herd**, which can crash the server again (and again). Randomization spreads reconnections over time. For very large deployments you may *increase* `reconnectionDelayMax` (e.g. 10–30 s) and keep the randomization.

What triggers / doesn't trigger reconnection

- **Triggers:** `ping timeout`, `transport close`, `transport error`, `parse error`, and failure of the initial low-level connection (server down → `connect_error` and retry).
- **Does NOT trigger:**
- client called `socket.disconnect()` → `"io client disconnect"`;
- server called `socket.disconnect()` → `"io server disconnect"`;
- **middleware rejection** (`next(new Error(...))` in `io.use`) → `connect_error` with `socket.active === false` — the client will **not** retry automatically, because retrying with the same credentials would fail again. You fix the cause (e.g. refresh token) and call `socket.connect()`.

```
socket.on("connect_error", async (err) => {
  if (socket.active) {
    // temporary failure, the socket will automatically try to reconnect
  } else {
    // the connection was denied by the server (middleware)
    if (err.message === "TOKEN_EXPIRED") {
      socket.auth.token = await refreshToken();
      socket.connect();
    } else {
      showLoginScreen();
    }
  }
});
```

State that is lost on reconnection (unless recovered)

On the **server**, a reconnection creates a **brand-new** `Socket` **object** with a new id:

- rooms are **empty** (only its own id-room),
- `socket.data` is empty (middleware runs again and must set it),
- event listeners are registered again in `io.on("connection")`.

Therefore: **derive all state from the authenticated user in the connection handler** (e.g. re-join `user:<id>` and the user's conversation rooms from the DB), and/or let the client tell the server what it wants to re-subscribe to on each `connect`:

```
// client
socket.on("connect", () => {
  // re-subscribe after every (re)connection
  socket.emit("subscribe", { rooms: [...currentSubscriptions] });
});
```

On the **client**, the same `Socket` object is reused and **your listeners stay registered** — which leads to the next famous bug.

Famous bug: registering listeners inside `connect`

```
// ❌ every reconnection adds ANOTHER "chat" listener → messages appear 2x, 3x, 4x…
socket.on("connect", () => {
  socket.on("chat", renderMessage);
});

// ✅ register once, outside
socket.on("chat", renderMessage);
socket.on("connect", () => { /* only connection-related logic */ });
```

(Same bug happens in React components that call `socket.on()` in `useEffect` without a cleanup `socket.off()` — Chapter 9.)

5.7 Client buffering (offline emits) and its dangers

By default, when the client is **disconnected**, `socket.emit()` doesn't fail — the packet is pushed into `socket.sendBuffer`, and all buffered packets are sent **when the connection is (re)established** (in order).

Good: user sends a chat message during a 2-second network blip → delivered after reconnection.

Bad: a 10-minute offline period with a "mouse position" emitted 60 times/second → 36 000 stale packets flood the server on reconnection.

Fixes:

```
// 1. Use volatile for droppable events (not buffered)
socket.volatile.emit("cursor", pos);
```

```
// 2. Check connection state
if (socket.connected) socket.emit("cursor", pos);

// 3. Clear the buffer on reconnect if stale data shouldn't be sent
socket.on("connect", () => { socket.sendBuffer = []; });  // drop what was queued while offline
```

> Server-side there is **no** per-socket offline buffer for disconnected clients: once a socket is disconnected, the server-side Socket object is gone; emits to it (or to rooms it was in) simply don't reach it. (Exception: connection state recovery, below.)

5.8 Connection State Recovery (v4.6+) — deep dive

The problem

A client loses its connection for 3 seconds (elevator, Wi-Fi handover). Without recovery:

- it reconnects with a **new id**, **no rooms**, **no** `socket.data`;
- every event broadcast during those 3 s is **lost** for that client.

The feature

```
const io = new Server(httpServer, {
  connectionStateRecovery: {
    // how long the server keeps the session (and the packets) after a disconnection
    maxDisconnectionDuration: 2 * 60 * 1000, // default: 2 minutes
    // skip the middlewares upon successful recovery
    skipMiddlewares: true,                   // default: true
  },
});

io.on("connection", (socket) => {
  if (socket.recovered) {
    // recovery was successful: socket.id, socket.rooms and socket.data were restored,
    // and missed packets were (or will be) sent to the client
  } else {
    // new or unrecoverable session → do the normal setup (join rooms, etc.)
  }
});
```

Client:

```
socket.on("connect", () => {
  if (socket.recovered) {
    // any event missed during the disconnection period has been received
  } else {
    // new or unrecoverable session → re-fetch state via HTTP/acks
  }
});
```

How it works under the hood (verified in source)

1. When CSR is enabled, the server sends the client a **private session id (`pid`)** in the CONNECT reply (alongside `sid`).
2. Every **event packet** sent through the adapter's broadcast (and, with CSR enabled, even `socket.emit()` to a single socket is routed through `adapter.broadcast` to the socket's own room) gets a unique **offset** appended as an extra last argument. The client strips it and remembers the **last offset** it processed.
3. When the socket disconnects for a **recoverable reason** (`transport error`, `transport close`, `forced close`, `ping timeout`, `server shutting down`, `forced server close`), the adapter **persists the session:** `{ sid, pid, rooms, data }`. (A deliberate `disconnect()` by client or server is **not** recoverable.)
4. On reconnection, the client sends `{ pid, offset }` in the CONNECT packet's auth payload.
5. The server looks up the session; if found, not expired, and the offset is known, it restores `id`, `rooms`, `data`, sets `socket.recovered = true`, and sends every stored packet **after** that offset that matches the socket's rooms.

Limitations (very important for real systems)

- **Packets with acknowledgements are not stored** (a callback can't be serialized), and **volatile packets are not stored**.
- The in-memory adapter keeps **all broadcast packets in memory** for `maxDisconnectionDuration` → memory cost proportional to broadcast volume. Size it.
- It's a **best-effort** optimisation, not a guarantee: it fails if the server restarted (with the in-memory adapter), the duration expired, or the offset is unknown. Your app must still handle `socket.recovered === false` (re-sync state from the DB/API).
- **Adapter support** (from each adapter's official feature table): ✅ default in-memory adapter, ✅ **Redis Streams** adapter, ✅ **MongoDB** adapter (since 0.3.0). ❌ classic **Redis pub/sub adapter** (pub/sub has no storage), ❌ **Postgres** adapter. Check your adapter's docs before relying on it.
- With `skipMiddlewares: true`, middleware doesn't re-run on recovery — fine for auth if tokens can't be revoked within 2 minutes; set it to `false` if you need re-auth.

5.9 Delivery guarantees (at-most-once vs at-least-once vs exactly-once)

Default guarantees

- **Ordering:** guaranteed (per socket connection, both directions).
- **Arrival: at-most-once**. A message may be lost if:
- the connection breaks while the message is in flight,
- the client is disconnected when the server emits (server → client),
- the server crashes after receiving but before processing,
- a volatile packet is dropped.
- **Client → server while disconnected:** buffered and sent on reconnection (but lost if the page is closed/reloaded before reconnecting, because the buffer is in memory).

Getting at-least-once: client → server

1. Use **acks**: the client knows the server got it.
2. Retry on timeout (`retries` + `ackTimeout` options in v4.6+, or manually).
3. Make server handlers **idempotent** (client-generated unique id + unique DB index).
4. Optionally persist pending messages client-side (IndexedDB/localStorage) to survive reloads ("outbox pattern").

Getting at-least-once: server → client

Option A — **connection state recovery** (best-effort, short disconnections).

Option B — **offset/sequence-based sync** (the robust, standard approach):

1. Persist every message in a database with a monotonically increasing id/timestamp per conversation (or a global sequence).
2. The client keeps track of the **last id** it processed (per conversation), stored in memory/localStorage.
3. On **every (re)connection**, the client asks "give me everything after id X" (via an ack or HTTP), then continues receiving live events.
4. De-duplicate on the client by id (an event may arrive both live and via sync).

```
// server
io.on("connection", async (socket) => {
  socket.on("sync", async ({ conversationId, afterId }, ack) => {
    if (!(await acl.canRead(socket.data.user.id, conversationId))) return ack({ ok: false });
    const missed = await messages.findAfter(conversationId, afterId, { limit: 500 });
    ack({ ok: true, data: missed });
  });
```

```
});

// client
let lastId = loadLastId();
socket.on("connect", async () => {
  const res = await socket.timeout(10000).emitWithAck("sync", { conversationId, afterId: lastId
});
  res.data.forEach(handleMessage);
});
socket.on("message", handleMessage);

const seen = new Set();
function handleMessage(m) {
  if (seen.has(m.id)) return;      // dedupe
  seen.add(m.id);
  lastId = Math.max(lastId, m.id);
  render(m);
}
```

Option C — **per-message client acks** (the server re-sends unacknowledged messages): heavier; mostly used for critical notifications.

Exactly-once?

Over a network, "exactly-once delivery" is impossible in general; what you implement is **at-least-once delivery + idempotent processing = effectively-once processing**. Say exactly this in interviews.

5.10 Timeouts summary (one table to rule them all)

Setting	Side	Default	What it controls
`pingInterval`	server	25 000 ms	Time between PINGs
`pingTimeout`	server	20 000 ms	Time to wait for PONG
`upgradeTimeout`	server	10 000 ms	Time to complete polling→WebSocket upgrade
`connectTimeout`	server	45 000 ms	Time for a client to send the CONNECT for a namespace after the Engine.IO handshake (else the raw connection is closed)
`maxDisconnectionDuration`	server (CSR)	120 000 ms	How long a session is kept for recovery

Setting	Side	Default	What it controls
`timeout`	client	20 000 ms	Connection attempt timeout → `connect_error` "timeout"
`reconnectionDelay`	client	1 000 ms	Initial reconnection delay
`reconnectionDelayMax`	client	5 000 ms	Max reconnection delay
`randomizationFactor`	client	0.5	Jitter
`ackTimeout`	client	none	Default ack timeout (used with `retries`)
`socket.timeout(ms)`	both	none	Per-emit ack timeout
`requestsTimeout`	Redis adapter	5 000 ms	Timeout for inter-server requests (fetchSockets, etc.)

5.11 Handling token expiry / re-authentication on long-lived connections

A WebSocket connection can outlive the access token used to authenticate it (tokens expire in 15 min; connections last hours). Options:

1. **Validate only at connection time** (most common): when the token expires the connection stays authenticated until it drops. Acceptable for many apps.
2. **Server-side expiry timer:** at connection, schedule a disconnect at token expiry:

```
io.use((socket, next) => { /* verify JWT → socket.data.exp = payload.exp */ next(); });
io.on("connection", (socket) => {
  const ms = socket.data.exp * 1000 - Date.now();
  const t = setTimeout(() => {
    socket.emit("auth:expired");
    socket.disconnect(true);
  }, ms);
  socket.on("disconnect", () => clearTimeout(t));
});
```

Client: on `io server disconnect` or `auth:expired`, refresh the token, set `socket.auth`, call `socket.connect()`. 3. **In-band refresh:** client sends `auth:refresh` with a new token before expiry; server verifies and updates `socket.data.exp` (and resets the timer). 4. **Revocation (logout, ban, password change):** `io.in("user:" + id).disconnectSockets(true)` — cluster-wide with an adapter. 5. **Per-event authorization checks** for sensitive operations (check permissions at the time of the action, not only at connect).

Use `auth` as a **function** so every reconnection picks up the latest token:

```
const socket = io(URL, {
  auth: (cb) => cb({ token: localStorage.getItem("accessToken") }),
});
```

5.12 Page lifecycle (browser specifics)

- **Page reload/close:** the browser closes the connection; the server sees `transport close` (it can't distinguish "closed tab" from "lost network" — use a grace period for presence, Chapter 13).
- `closeOnBeforeunload` (client option, default **false** in current versions; it was `true` in some earlier 4.x releases): when `true`, the client closes the connection silently on `beforeunload`, so every browser behaves the same (Firefox closes the connection on `beforeunload`, Chrome doesn't) and no `disconnect` event fires at the Socket.IO level while the page is unloading. Only matters if your client code reacts to `disconnect` during a reload.
- **Mobile background:** iOS/Android suspend apps; the connection dies; reconnect when foregrounded. Use **push notifications** (APNs/FCM) for delivery while the app is in the background — Socket.IO can't deliver to a suspended app.
- `navigator.onLine` / `online` **event:** you can call `socket.connect()` on the browser's `online` event to reconnect faster than waiting for the back-off.
- **Visibility:** pause high-frequency subscriptions when `document.hidden`.

5.13 Chapter summary

- Server events: `connection`, `disconnecting`, `disconnect`, `error`; Engine `connection_error` with codes 0–5 (code 1 = Session ID unknown = sticky session issue).
- Client events: `connect`, `connect_error`, `disconnect`; Manager: `reconnect*`, `error`, `ping`, `open`, `close`.
- Disconnect reasons and which ones auto-reconnect (not `io server disconnect` / `io client disconnect`, not middleware rejection).
- Heartbeat: server PINGs every 25 s, 20 s to PONG; client times out after 45 s of no PING.
- Reconnection: exponential back-off with jitter (1 s → 5 s max, ±50%).
- Client buffers emits while disconnected (use volatile or clear `sendBuffer`).
- Connection state recovery restores id, rooms, data, missed packets for short disconnections — best effort; not for packets with acks or volatile; adapter dependent.

- Default delivery: at-most-once, ordered. At-least-once = acks + retries + idempotency (client → server) and persisted messages + offset sync + dedupe (server → client).

Next: Chapter 6 — Middleware, authentication and security

CHAPTER 06

Middleware, Authentication and Security

> **Goal:** protect your real-time API like you protect your REST API — authenticate every connection, authorize every action, validate every payload, limit abuse, and know the attacks specific to WebSockets.

6.1 ELI5

Your server is a **party**.

- The **bouncer at the door** (connection middleware `io.use`) checks every guest's ticket (token/cookie) before they enter. No ticket → "Sorry, you can't come in" (`connect_error`).
- Inside, a **staff member near the VIP room** (per-event middleware `socket.use`, or checks inside handlers) checks if you're allowed to do specific things.
- Guests may try to **shout too much** (spam) → **rate limiting**.
- Guests may bring **weird packages** (bad payloads) → **validation**.
- A sneaky neighbour may try to **sneak in using your ticket** via a fake website → **Cross-Site WebSocket Hijacking** protection (origin checks).

6.2 The three middleware levels

Level	API	Runs	Typical use
HTTP / Engine.IO	`io.engine.use((req, res, next) ⇒ …)` (v4.6+), or `allowRequest` option	On every HTTP request of the Engine.IO connection (handshake, polling requests, WebSocket upgrade request)	Express middleware: `express-session`, `helmet`, `passport`, cookie parsing
Namespace (connection)	`io.use((socket, next) ⇒ …)` / `io.of("/x").use(…)`	**Once per connection** to that namespace, before `connection`	Authentication, loading the user, connection limits
Socket (per event)	`socket.use(([event, ...args], next) ⇒ …)`	Every **incoming** event for that socket	Authorization per event, rate limiting, validation, logging

6.2.1 Namespace (connection) middleware — `io.use`

```
io.use((socket, next) => {
  // runs for every incoming connection on the main namespace
  if (isValid(socket.handshake.auth.token)) {
    next();                                  // accept
  } else {
    const err = new Error("not authorized");  // reject
    err.data = { code: "AUTH_INVALID", hint: "please log in again" }; // extra details for the
client
    next(err);
  }
});
```

Rules and facts:

1. **You must call** `next()` — otherwise the client stays stuck "connecting". If it has not joined any namespace after `connectTimeout` (default 45 s), the server closes the underlying connection and the client retries — and gets stuck again. Forgetting `next()` in one code path is a classic bug ("client never connects, no error").
2. Middlewares run **in registration order**; `next(err)` stops the chain.
3. `next(err)` → the client receives `connect_error` with `err.message` and `err.data`. The client does **not** auto-reconnect after a middleware rejection (`socket.active === false`).
4. The socket is **not yet connected** in a middleware: don't emit to it or join rooms there — do that in the `connection` handler. Also, if the connection is eventually rejected, no `disconnect` event fires for it, so don't allocate resources in middleware that you only release on `disconnect`.
5. Middleware can be **async**:

```
io.use(async (socket, next) => {
  try {
    socket.data.user = await auth.verify(socket.handshake.auth.token);
    next();
  } catch (e) {
    next(new Error("unauthorized"));
  }
});
```

6. Middlewares are per namespace. `io.use` = main namespace only. To protect `/admin`, use `io.of("/admin").use(...)`. (Dynamic namespace parent: `io.of(/regex/).use`.)
7. With connection state recovery and `skipMiddlewares: true`, middleware is skipped for recovered sessions.

6.2.2 Engine-level middleware — `io.engine.use` (v4.6+)

Runs Express-compatible middleware on the raw HTTP request(s):

```
const session = require("express-session");

const sessionMiddleware = session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, secure: true, sameSite: "lax" },
});

app.use(sessionMiddleware);        // for Express routes
io.engine.use(sessionMiddleware);  // for Socket.IO requests

io.on("connection", (socket) => {
  const session = socket.request.session;   // same session as Express
  if (!session.userId) return socket.disconnect(true);
  socket.join(`user:${session.userId}`);
});
```

With **Passport** (session-based):

```
function onlyForHandshake(middleware) {
  return (req, res, next) => {
    const isHandshake = req._query.sid === undefined; // first request has no sid yet
    if (isHandshake) middleware(req, res, next);
    else next();
  };
}

io.engine.use(onlyForHandshake(sessionMiddleware));
io.engine.use(onlyForHandshake(passport.session()));
io.engine.use(onlyForHandshake((req, res, next) => {
  if (req.user) next();
  else { res.writeHead(401); res.end(); }
}));

io.on("connection", (socket) => {
  const user = socket.request.user;
});
```

`helmet` can also be applied: `io.engine.use(helmet())`.

Before 4.6, people used a wrapper: `const wrap = (mw) => (socket, next) => mw(socket.request, {}, next); io.use(wrap(sessionMiddleware));` — you'll see it in older code bases.

> Note: with the WebSocket transport, there's **no `res` object to write headers to** after the upgrade, so cookies set in middleware during a WebSocket-only connection can't be sent back. That's why session cookies should be created by your normal HTTP login route, not by the socket.

6.2.3 `allowRequest` — the earliest gate

```
const io = new Server(httpServer, {
  allowRequest: (req, callback) => {
    const origin = req.headers.origin;
    const allowed = ALLOWED_ORIGINS.includes(origin);
    callback(null, allowed);   // (err: string | null | undefined, success: boolean)
  },
});
```

- Runs for the handshake / upgrade requests, **before** any Socket.IO logic.
- A `false` result → HTTP 403 → Engine `connection_error` code 4 `Forbidden`; client `connect_error` (with `socket.active === true`, it will keep retrying — because at this level it looks like a transport failure).
- Great place for **origin checks** (against CSWSH), IP allow/deny lists, quick connection rate limiting.

6.2.4 Per-event middleware — `socket.use`

```
io.on("connection", (socket) => {
  socket.use(([event, ...args], next) => {
    logger.debug({ event, sid: socket.id }, "incoming event");
    next();
  });

  socket.use(([event], next) => {
    if (event.startsWith("admin:") && !socket.data.user.isAdmin) {
      return next(new Error("forbidden"));
    }
    next();
  });

  socket.on("error", (err) => {
    // errors passed to next(err) in socket.use() land here; they are NOT sent to the client
automatically
    socket.emit("app:error", { message: err.message });
  });
});
```

- Calling `next(err)` **prevents the event handler from running** and emits an `error` event on the server socket.
- The ack callback (if any) is the last element of `args` — you can reply directly:

```
socket.use(([event, ...args], next) => {
  const ack = typeof args.at(-1) === "function" ? args.at(-1) : null;
  if (rateLimited(socket)) return ack ? ack({ ok: false, error: "RATE_LIMITED" }) : undefined;
  next();
});
```

- Not calling `next()` silently drops the event.

6.3 What's available on `socket.handshake`

```
io.use((socket, next) => {
  socket.handshake ==== {
    headers: {/* the headers of the initial HTTP request */},
    time: "Fri Oct 16 2026 10:00:00 GMT+0000",  // creation date as string
    address: "::ffff:127.0.0.1",                // IP of the client (or of the proxy!)
    xdomain: false,                             // cross-domain?
    secure: false,                              // TLS?
    issued: 1760608800000,                      // creation date (ms timestamp)
    url: "/socket.io/?EIO=4&transport=polling&t=…", // request URL
    query: { EIO: "4", transport: "polling", t: "…" /* + your query params */ },
    auth: { token: "…" /* from the client's auth option */ },
  };
  next();
});
```

Also: `socket.request` (the first `http.IncomingMessage`), `socket.conn` (Engine.IO socket: `socket.conn.transport.name`, `socket.conn.remoteAddress`, `socket.conn.protocol`).

Getting the real client IP behind a proxy

`socket.handshake.address` is the IP of the **direct peer** — your load balancer. Read `X-Forwarded-For` **only if you trust your proxy** (otherwise anyone can spoof it):

```
function clientIp(socket) {
  const xff = socket.handshake.headers["x-forwarded-for"];
  return xff ? xff.split(",")[0].trim() : socket.handshake.address;
}
```

(With Cloudflare: `cf-connecting-ip`. With nginx, make sure it sets `X-Forwarded-For` / `X-Real-IP` and that clients cannot reach Node directly.)

6.4 Authentication strategies

6.4.1 Token in the `auth` payload (recommended for SPAs/mobile)

```
// client
const socket = io("https://api.example.com", {
  auth: { token: accessToken },
  // or dynamic (re-evaluated on every connection attempt → picks up refreshed tokens):
  // auth: (cb) => cb({ token: localStorage.getItem("accessToken") }),
});

// server
const jwt = require("jsonwebtoken");

io.use((socket, next) => {
  const token = socket.handshake.auth?.token;
  if (!token) return next(new Error("NO_TOKEN"));
  try {
    const payload = jwt.verify(token, process.env.JWT_PUBLIC_KEY, {
      algorithms: ["RS256"],            // ALWAYS pin algorithms
      audience: "realtime-api",
      issuer: "https://auth.example.com",
    });
    socket.data.user = { id: payload.sub, roles: payload.roles, exp: payload.exp };
    next();
  } catch (e) {
    const err = new Error(e.name === "TokenExpiredError" ? "TOKEN_EXPIRED" : "TOKEN_INVALID");
    next(err);
  }
});
```

Why `auth` and not the query string?

- The `auth` payload travels **inside the CONNECT packet** (the message body), not the URL.
- Query strings end up in **server/proxy/CDN access logs**, browser history and monitoring tools → token leak.
- `auth` is per **namespace** (each namespace can authenticate differently).

Why not headers? Browsers' `WebSocket` API **cannot set custom headers.** `extraHeaders` works for polling in browsers and for everything in Node.js clients, but in the browser the WebSocket handshake won't carry them. So `Authorization` headers are unreliable for browser clients → use `auth` (or cookies).

6.4.2 Cookie / session-based

- Use `io.engine.use(sessionMiddleware)` (above).

- Cross-origin: client `withCredentials: true`; server `cors: { origin: <exact>, credentials: true }`; cookie `SameSite=None; Secure` if truly cross-site.
- Same-origin deployments (frontend and API on the same domain) avoid most of this pain.
- **Cookies are sent automatically by the browser** → vulnerable to **CSWSH** (next section) unless you check `Origin`.

6.4.3 Short-lived "socket ticket" pattern (high-security)

1. Client calls `POST /api/socket-ticket` (authenticated via normal session/JWT).
2. Server returns a **single-use, 30-second** random ticket stored in Redis (`ticket → userId`).
3. Client connects with `auth: { ticket }`.
4. Middleware atomically consumes the ticket (`GETDEL`) and loads the user.

Benefits: long-lived credentials never travel over the socket; replay window tiny.

6.4.4 Authorization is not authentication

Authenticating the connection is not enough. **Every event that touches data must check permissions:**

```
socket.on("conversation:join", async ({ conversationId }, ack) => {
  // ❌ trusting the client: "I'm allowed in conversation 42"
  // ✅ checking server-side:
  const allowed = await acl.isMember(socket.data.user.id, conversationId);
  if (!allowed) return ack?.({ ok: false, error: "FORBIDDEN" });
  await socket.join(`conversation:${conversationId}`);
  ack?.({ ok: true });
});
```

Never accept identity fields from the payload (`{ userId: 5, text }`) — always use `socket.data.user.id`.

6.5 Cross-Site WebSocket Hijacking (CSWSH) — the WebSocket-specific attack

ELI5: a bad website (evil.com) tells your browser "please open a WebSocket to bank.com". The browser **automatically attaches your bank.com cookies**. If bank.com authenticates sockets only by cookie and doesn't check where the request came from, evil.com now has a live, authenticated socket as you — it can read your data and send commands.

Why it happens: **the Same-Origin Policy / CORS does not apply to WebSocket handshakes.** Browsers send the `Origin` header but don't block cross-origin WebSocket connections; the server must decide.

Note: with the default `["polling", "websocket"]` transports, the initial polling request **is** subject to CORS, which helps — but an attacker can force `transports: ["websocket"]` to skip polling entirely, so you must protect the WebSocket path too.

Proof (tested on socket.io 4.8.4): with `cors: { origin: "http://localhost:5173" }`, a raw WebSocket handshake to `/socket.io/?EIO=4&transport=websocket` carrying `Origin: https://evil.example` was **accepted** — the server answered with the Engine.IO open packet `0{"sid":…}`. The `cors` option only adds CORS headers to HTTP polling responses; it does not reject WebSocket upgrades. (python-socketio, by contrast, rejected the same request with HTTP 403 — see Chapter 17.)

Defences (use several):

1. **Check** `Origin` on the handshake and upgrade (`allowRequest`) against an allow-list.
2. **Token-based auth** in `auth` (not automatically sent by the browser) instead of, or in addition to, cookies.
3. `SameSite=Lax` / `Strict` cookies (cross-site WebSocket requests won't carry `Strict` / `Lax` cookies in modern browsers — but don't rely on this alone).
4. Short-lived tickets.

```
const ALLOWED = new Set(["https://app.example.com", "https://admin.example.com"]);

const io = new Server(httpServer, {
  cors: { origin: [...ALLOWED], credentials: true },
  allowRequest: (req, cb) => cb(null, ALLOWED.has(req.headers.origin)),
});
```

Verified result of this exact configuration (4.8.4): allowed origin → accepted; forged `Origin: https://evil.example` → polling request answered **HTTP 403** `{"code":4,"message":"Forbidden"}`, WebSocket upgrade refused (**HTTP 400**), and `io.engine.on("connection_error")` logged code `4 Forbidden`.

(Non-browser clients — mobile apps, Node scripts — may not send `Origin`. Decide explicitly: allow missing origin only if authentication is token-based.)

6.6 Input validation (never trust the client)

Anything the client sends can be malicious: wrong types, huge strings, prototype pollution keys (`__proto__`), negative numbers, deeply nested objects.

```
const { z } = require("zod");

const SendMessage = z.object({
  conversationId: z.string().uuid(),
  text: z.string().trim().min(1).max(2000),
  clientMsgId: z.string().uuid(),
});

socket.on("message:send", async (raw, ack) => {
  if (typeof ack !== "function") return;          // require ack for commands
  const parsed = SendMessage.safeParse(raw);
  if (!parsed.success) return ack({ ok: false, error: "VALIDATION", details:
parsed.error.flatten() });
  // ... use parsed.data only
});
```

Checklist:

- Validate **types, lengths, ranges, formats**, and **number of arguments**.
- Reject unknown keys (`.strict()`), especially before spreading into DB queries (NoSQL injection: `{ "$gt": "" }`).
- Sanitize/escape user content **at render time** on clients (`textContent` , React's escaping) to prevent **XSS** — broadcasting HTML received from one user to everyone is a fast way to an XSS worm.
- Use parameterized queries (SQL injection still exists over sockets!).
- Limit the size of **arrays**/nesting to avoid CPU exhaustion.
- Validate **binary** payloads (type, size, magic bytes) before storing.

6.7 Rate limiting and abuse prevention

Per-socket event rate limiting

```
const { RateLimiterMemory } = require("rate-limiter-flexible"); // or RateLimiterRedis for
clusters

const limiter = new RateLimiterMemory({ points: 20, duration: 1 }); // 20 events / second

io.on("connection", (socket) => {
  socket.use(async ([event, ...args], next) => {
    try {
      await limiter.consume(socket.data.user?.id ?? socket.id); // key per user (or per IP)
      next();
    } catch {
      const ack = typeof args.at(-1) === "function" ? args.at(-1) : null;
      ack?.({ ok: false, error: "RATE_LIMITED" });
```

```
      // optionally disconnect repeat offenders
    }
  });
});
```

Use **per-user** keys (users can open many sockets) and **Redis-backed** limiters in a cluster (`RateLimiterRedis`) so limits are global, not per process.

Connection limits

```
// limit concurrent connections per user (register this as the LAST middleware)
io.use(async (socket, next) => {
  const key = `conn:${socket.data.user.id}`;
  const count = await redis.incr(key);
  await redis.expire(key, 3600);
  if (count > 10) { await redis.decr(key); return next(new Error("TOO_MANY_CONNECTIONS")); }
  next();
});

io.on("connection", (socket) => {
  socket.on("disconnect", () => redis.decr(`conn:${socket.data.user.id}`));
});
```

(Careful: process crashes leave counters inflated → in real systems prefer a Redis sorted set of `socketId → lastSeen` cleaned by expiry, so stale entries disappear by themselves.)

Connection-rate limits per IP at the **edge** (nginx `limit_conn` / `limit_req` , WAF, Cloudflare) are cheaper than in Node.

Payload size

- `maxHttpBufferSize` (default **1 MB**) — max size of a single message. Larger → the server **closes the connection**. Lower it (e.g. `1e5` = 100 KB) if you only send small JSON, to limit memory abuse.
- On polling, this also limits the request body size.

Other DoS considerations

- **Slow consumers**: a client that doesn't read causes server-side write buffers to grow. Monitor `socket.conn.transport.writable` /buffer sizes; disconnect clients whose buffers exceed limits (e.g. check `socket.conn.writeBuffer.length`).
- **Too many rooms / namespaces** created by client input (dynamic namespace names or room names from user input) → memory growth. Validate names, cap counts, use `cleanupEmptyChildNamespaces` .
- **Expensive handlers**: an event that triggers a heavy DB query can be spammed → rate limit and cache.

- **Pending acks**: server-side `socket.emit(..., ack)` without timeout to a client that never answers keeps callbacks in memory → always use `timeout()`.

6.8 Transport security

- **Always use TLS** (`https://` → `wss://`) in production. Without it, tokens and data are readable by anyone on the network, and many proxies mangle unencrypted WebSocket traffic (another reason WebSocket "doesn't work" on some networks — TLS hides it from them).
- Terminate TLS at the load balancer or in Node (`https.createServer`).
- HSTS on your domains.
- Don't disable certificate validation in Node clients (`rejectUnauthorized: false`) except in local development.

6.9 Information leakage

- Don't send **internal error messages/stack traces** to clients (`err.message` from DB drivers can leak schema/SQL). Map to error codes.
- Don't broadcast **full user objects** (emails, phone numbers) to rooms — send minimal public projections.
- `socket.data` is sent between servers with `fetchSockets()` — fine, but don't put secrets there unnecessarily.
- Remove `serveClient` (set `false`) if you bundle the client yourself — less attack surface, no version disclosure.
- The **Admin UI** must be protected with authentication (`auth: { type: "basic", … }`) and preferably not exposed publicly (Chapter 10).

6.10 Keep dependencies updated — real vulnerabilities have happened

Real advisories have been published for every layer of the stack. This table was pulled from the **npm / GitHub advisory database** (October 2026); each row links to the advisory by its GHSA id (`https://github.com/advisories/<id>`):

Package	Affected versions	Severity	Issue	GHSA id
`socket.io`	< 2.4.0	moderate	CORS misconfiguration (all origins allowed by default)	GHSA-fxwf-4rqh-v8g3
`socket.io`	< 2.5.1, and ≥ 3.0.0 < 4.6.2	moderate	Unhandled `error` event (can crash the process)	GHSA-25hc-qcg6-38wj
`engine.io`	≥ 4.0.0 < 6.2.1	moderate	Uncaught exception from a crafted request	GHSA-r7qp-cfhv-p84w
`engine.io`	≥ 5.1.0 < 6.4.2	moderate	Uncaught exception vulnerability	GHSA-q9mw-68c2-j6m5
`engine.io`	≥ 4.1.0 < 6.6.7	**high**	Polling transport connection exhaustion	GHSA-r635-g3xr-vw7x
`socket.io-parser`	≥ 4.0.0 < 4.0.5	**critical**	Insufficient validation when decoding a packet	GHSA-qm95-pgcg-qqfq
`socket.io-parser`	≥ 4.0.4 < 4.2.3	moderate	Insufficient validation when decoding a packet	GHSA-cqmj-92xf-r6r9
`socket.io-parser`	≥ 4.0.0 < 4.2.6	**high**	Unbounded number of binary attachments	GHSA-677m-j7p3-52f9
`socket.io-parser`	≥ 4.0.0 < 4.2.7	**high**	Zero-attachment memory exhaustion	GHSA-2m8v-j782-fhvr
`ws`	≥ 8.0.0 < 8.17.1	**high**	DoS with a request containing many HTTP headers	GHSA-3h5v-q93c-6h6q
`ws`	≥ 8.0.0 < 8.20.1	moderate	Uninitialized memory disclosure	GHSA-58qx-3vcg-4xpx
`ws`	≥ 8.0.0 < 8.21.0	**high**	Memory exhaustion DoS from tiny fragments	GHSA-96hv-2xvq-fx4p

(Older major lines — e.g. `ws` 5–7, `socket.io-parser` 3.x — had their own advisories and backported fixes; check the advisory pages for your exact version.)

A fresh `npm install socket.io@4.8.4` in October 2026 resolved to `engine.io@6.6.11`, `socket.io-parser@4.2.7` and `ws@8.21.3` — all past the fixes above. **But transitive versions are pinned by your lockfile**: an old `package-lock.json` can keep a vulnerable `ws` or `socket.io-parser` even after you bump `socket.io`. Run `npm ls ws engine.io socket.io-parser` to see what you actually ship.

Notice the pattern: almost every issue is "a malicious client sends a malformed or oversized packet and crashes or exhausts the server". That's why `maxHttpBufferSize`, rate limits and patching matter even for internal apps.

Lessons: run `npm audit` / Dependabot / Snyk in CI, keep `socket.io`, `engine.io`, `socket.io-parser` and `ws` patched, and add a `process.on("uncaughtException")` logger + process manager restarts as defence in depth.

6.11 Security checklist (copy into your team's PR template)

- [ ] TLS everywhere (`wss://`).
- [ ] `cors.origin` is an explicit allow-list (never `"*"` with credentials).
- [ ] `allowRequest` checks `Origin` (CSWSH protection), especially with cookie auth.
- [ ] Connection authenticated in `io.use` (token in `auth`, not query string).
- [ ] Token expiry handled (disconnect/refresh), revocation via `disconnectSockets`.
- [ ] Every event handler authorizes the action and validates input (schema).
- [ ] No identity taken from payloads; use `socket.data.user`.
- [ ] Room joins authorized server-side.
- [ ] Rate limits per user/IP (Redis-backed in clusters); connection caps.
- [ ] `maxHttpBufferSize` sized for your messages.
- [ ] Errors mapped to codes, no stack traces to clients.
- [ ] User-generated content escaped on render (XSS).
- [ ] Admin UI protected or disabled in production.
- [ ] `serveClient: false` if not needed.
- [ ] Dependencies scanned and up to date.
- [ ] Handlers wrapped (no crash on throw/reject).

6.12 Chapter summary

- Three middleware levels: `io.engine.use` (HTTP), `io.use` (connection), `socket.use` (per event), plus the `allowRequest` gate.
- `next(err)` in connection middleware → client `connect_error` with `message` / `data`, no auto-reconnect.
- Authenticate with `auth` payload (or cookies + origin checks); never trust identity from payloads; authorize every action.
- CSWSH: WebSocket isn't covered by CORS → check `Origin`.
- Validate inputs, rate-limit, limit sizes, escape output, keep deps patched.

Next: Chapter 7 — Under the hood

CHAPTER 07

Under the Hood: Engine.IO, Transports, the Protocol, Parsers and Adapters

> **Goal:** understand exactly what travels on the wire and which internal component does what. This is what lets you debug anything with the browser's Network tab, and what senior interviews probe ("explain the upgrade mechanism", "what's an adapter?", "why sticky sessions?").
>
> All packet captures below come from a real Socket.IO **4.8.4** server.

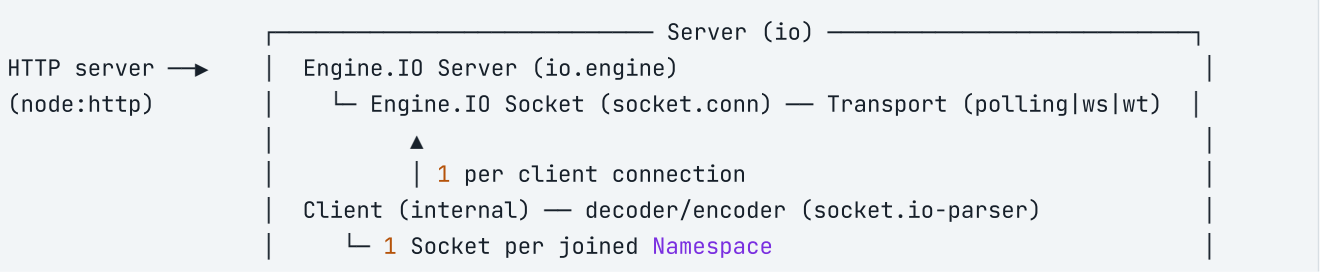
7.1 ELI5

Sending a Socket.IO message is like sending a parcel:

1. You write a **letter** ("chat", "hi") — your event.
2. The **Socket.IO parser** puts it in an **envelope** and writes on it: "this is an EVENT, for department `/admin`, reply slip #7" → `42/admin,7["chat","hi"]`.
3. **Engine.IO** puts the envelope in a **box** marked "MESSAGE" → adds a `4` in front.
4. The **transport** is the **delivery truck**: either a long-polling truck (HTTP requests) or a WebSocket truck (an always-open road). If the road is blocked, the old truck still delivers.
5. On arrival, everything is unpacked in reverse order.

The **adapter** is the **post office sorting room**: it knows which people are in which groups (rooms) and, with several post offices (servers), it forwards parcels between them.

7.2 The architecture (server side)



```
|  Namespace "/" ─── Adapter (rooms: Map<Room, Set<SocketId>>)      |
|  Namespace "/admin" ─── Adapter                                   |
└──────────────────────────────────────────────────────────────────┘
```

Component	Package	Responsibility
`Server`	`socket.io`	Entry point, options, namespaces, attaches to HTTP server
`Namespace`	`socket.io`	Group of sockets with own handlers/middleware/adapter
`Socket`	`socket.io`	One client in one namespace; emit/on/join/leave
`Client`	`socket.io` (internal)	One per Engine.IO connection; demultiplexes packets to namespace Sockets
`Adapter`	`socket.io-adapter` (+ Redis etc.)	Stores room membership, performs broadcasts, inter-server communication
Parser	`socket.io-parser`	Encodes/decodes Socket.IO packets (JSON + binary attachments)
Engine.IO `Server` / `Socket`	`engine.io`	Handshake, transports, upgrade, heartbeat, low-level packets
Engine.IO parser	`engine.io-parser`	Encodes Engine.IO packets and payloads
`ws`	`ws`	Default WebSocket server implementation (`wsEngine`)

Client side mirrors it: `Socket` (per namespace) → `Manager` (connection, reconnection) → `engine.io-client` `Socket` → transports.

7.3 Engine.IO protocol (v4)

Engine.IO packet types

Type	Name	Direction	Purpose
`0`	**open**	S → C	Handshake response (sid, upgrades, pingInterval, pingTimeout, maxPayload)
`1`	**close**	both	Request to close the transport
`2`	**ping**	S → C (also `2probe` C → S during upgrade)	Heartbeat
`3`	**pong**	C → S (also `3probe` S → C during upgrade)	Heartbeat reply

Type	Name	Direction	Purpose
`4`	**message**	both	Carries an application (Socket.IO) packet
`5`	**upgrade**	C → S	Completes the transport upgrade
`6`	**noop**	S → C	Used to close a pending poll during upgrade

The handshake (open packet) — real capture

```
GET /socket.io/?EIO=4&transport=polling&t=<cache-buster>
← 0{"sid":"HSJR3YuCOE9jPqHEAAAA","upgrades":
["websocket"],"pingInterval":25000,"pingTimeout":20000,"maxPayload":1000000}
```

Query parameters:

Param	Meaning
`EIO=4`	Engine.IO protocol version (v3/v4 clients send 4; v2 clients send 3)
`transport`	`polling` / `websocket` / `webtransport`
`sid`	Session id (on every request after the handshake)
`t`	Cache-busting timestamp (polling)
+ your `query` option params	

Note `"upgrades":["websocket"]` — what this transport may be upgraded to. When the handshake itself happens over WebSocket the list is empty (nothing to upgrade to):

```
(WebSocket-only connection) ← 0{"sid":"u1NewvylZUVY8NNBAAAA","upgrades":
[],"pingInterval":2000,"pingTimeout":1000,"maxPayload":1000000}
```

`maxPayload` tells the client the max bytes per polling payload (= server's `maxHttpBufferSize`), so the client can split its writes across several requests.

HTTP long-polling transport

- **Receiving (server → client):** the client keeps **one `GET` pending.** The server answers it when it has packets (or when it sends a ping). Then the client sends a new `GET` immediately.
- **Sending (client → server):** the client sends `POST` requests with packets in the body. Only one POST at a time; packets emitted meanwhile are batched in the next POST.
- **Payload format (EIO v4):** multiple packets in one HTTP body are separated by the **record separator character** `\x1e` (ASCII 30).

- **Binary over polling:** encoded as **base64 with a `b` prefix.**

Real capture: one polling GET returning **three** packets (CONNECT reply, a binary event header, and its binary attachment), separated by `\x1e`:

```
40/admin,{"sid":"l_dmkQgfXzZ8Ld6XAAAB"}␞451-/admin,["bin",
{"_placeholder":true,"num":0},"txt"]␞bAQID
```

`bAQID` = `b` + base64(`01 02 03`) — the bytes `[1,2,3]`.

Why polling needs **sticky sessions**: the session (`sid`) lives in the memory of **one** server. Each GET/POST is a separate HTTP request; if the load balancer sends one to another server, that server answers **400** `{"code":1,"message":"Session ID unknown"}`.

WebSocket transport

- One WebSocket per connection; each Engine.IO packet = **one WebSocket frame.**
- Text packets are sent as text frames (`4…` for messages, `2` / `3` for ping/pong).
- **Binary** packets are sent as **binary frames** with the raw bytes (no prefix, no base64).

Real capture over WebSocket (connecting to namespace `/admin`, receiving a binary event, sending an event with an ack, heartbeat):

```
→ 40/admin,
← 0{"sid":"u1NewvylZUVY8NNBAAAA","upgrades":
[],"pingInterval":2000,"pingTimeout":1000,"maxPayload":1000000}
← 40/admin,{"sid":"O8qU3PmyyZeZYZZHAAAA"}
← 451-/admin,["bin",{"_placeholder":true,"num":0},"txt"]
← (binary frame) 01 02 03
→ 42/admin,7["q"]
← 43/admin,7["answer"]
← 2
→ 3
```

The upgrade mechanism (polling → WebSocket)

```
Client                                          Server
  |--- GET  ?transport=polling  (handshake) ----->|
  |<-- 0{"sid":…,"upgrades":["websocket"]…} ------|
  |--- POST 40 (CONNECT) ------------------------>|
  |<-- 40{"sid":…} (via pending GET) -------------|
  |                                               |
  |=== WS  ?transport=websocket&sid=… (upgrade) ==>|   (101 Switching Protocols)
  |=== 2probe ===================================>|   test the new road
  |<== 3probe ====================================|   it works!
  |  (client pauses polling: waits for the pending  |
```

```
|   GET/POST to complete; server ends the pending |
|   GET with a 6 "noop")                          |
|═══ 5 (upgrade) ================================>|   switch!
|<═══ buffered packets now flow over WebSocket ═══|
```

- The `upgradeTimeout` (default 10 s) limits how long the server waits for the upgrade to complete; on timeout the WebSocket is closed and the client stays on polling.
- Because polling is paused and flushed before switching, **no packet is lost or reordered** during the upgrade.
- `rememberUpgrade: true` (client) → if a previous connection successfully used WebSocket, the next one starts directly with WebSocket (stored in memory per page — uses a static flag).
- `upgrade: false` (client) → stay on the first transport.

WebTransport (v4.7+)

- HTTP/3 (QUIC) based transport; requires HTTPS and an HTTP/3 server (e.g. `@fails-components/webtransport`), plus enabling `"webtransport"` in the server's `transports`.
- The client lists `"webtransport"` in its default transports; it's used if the server advertises it in `upgrades`.
- Benefits: no head-of-line blocking across streams, faster connection setup (0/1-RTT). Use for advanced cases; WebSocket is the practical default.

7.4 Socket.IO protocol (v5)

Socket.IO packet types

Type	Name	Example	Meaning
`0`	**CONNECT**	`0` / `0/admin,{"token":"…"}` (C→S); `0{"sid":"…"}` (S→C)	Connect to a namespace (with auth payload); server reply carries the socket id (and `pid` with CSR)
`1`	**DISCONNECT**	`1/admin,`	Leave a namespace
`2`	**EVENT**	`2["hello","world"]` / `2/admin,13["q"]`	An event, optional ack id
`3`	**ACK**	`313["answer"]`	Acknowledgement for ack id 13
`4`	**CONNECT_ERROR**	`4{"message":"not authorized","data":{…}}`	Namespace connection refused (middleware error / invalid namespace)

Type	Name	Example	Meaning
`5`	**BINARY_EVENT**	`51-["bin",{"_placeholder":true,"num":0}]`	Event with N binary attachments
`6`	**BINARY_ACK**	`61-15[{"_placeholder":true,"num":0}]`	Ack with binary attachments

On the wire each Socket.IO packet travels inside an Engine.IO **message** (`4`), which is why you see `40` , `42` , `43` , `44` , `45` … in the Network tab.

Packet format

```
<packet type>[<# of binary attachments>-][<namespace>,][<ack id>][JSON-stringified payload]
```

- Namespace is omitted when it's the main namespace `/` .
- Ack id is present only when the sender expects an acknowledgement.

Decoding real examples:

Raw (as seen in DevTools)	Engine.IO	Socket.IO	Meaning
`40`	message	CONNECT `/`	client asks to join `/`
`40{"sid":"KykvYCRwmIsIifIyAAAA"}`	message	CONNECT reply	joined `/` , socket id
`421["hello","world"]`	message	EVENT, ack id 1	`emit("hello","world",cb)`
`431[{"ok":true}]`	message	ACK 1	`cb({ok:true})`
`42/admin,7["q"]`	message	EVENT in `/admin` , ack 7	
`451-/admin,["bin",{"_placeholder":true,"num":0},"txt"]`	message	BINARY_EVENT, 1 attachment, `/admin`	followed by 1 binary frame
`44{"message":"not authorized"}`	message	CONNECT_ERROR	middleware rejected
`41`	message	DISCONNECT `/`	
`2` / `3`	ping / pong	—	heartbeat

Binary handling

1. The parser walks the arguments, finds `Buffer` / `ArrayBuffer` / `Blob` etc.
2. Replaces each with a placeholder `{"_placeholder":true,"num":<index>}`.
3. Sends the JSON header packet (`BINARY_EVENT` with the count) followed by each binary **attachment** as a separate Engine.IO message (binary frame / `b`-base64 in polling).
4. The receiver buffers the header until all attachments arrive, then reconstructs the arguments.

Implications: - One logical binary event = several frames. - A message with **many small buffers** is less efficient than one combined buffer. - Security: attackers can send malformed attachment counts — keep `socket.io-parser` patched (historic CVEs) and keep `maxHttpBufferSize` sane.

Why can't a plain WebSocket client talk to Socket.IO?

Because it would need to: open the right URL (`/socket.io/?EIO=4&transport=websocket`), parse the `0{...}` open packet, send `40` to join a namespace, answer every `2` with `3` (or be disconnected after `pingTimeout`), and encode/decode `42[...]` /binary attachments. That is the Socket.IO protocol — which is exactly what client libraries implement.

7.5 Parsers

The **parser** turns packets into wire format and back. Both sides **must use the same parser**.

Parser	Package	Notes
Default	`socket.io-parser`	JSON + separate binary attachments; human-readable in DevTools
MessagePack	`socket.io-msgpack-parser`	Binary encoding of everything; smaller payloads for some data, handles binary inline (no attachments); not human-readable; numbers/strings encoding differs slightly
Custom	implement `Encoder` / `Decoder`	e.g. protobuf-based, compression, encryption

```
// server
const customParser = require("socket.io-msgpack-parser");
const io = new Server(httpServer, { parser: customParser });

// client
import customParser from "socket.io-msgpack-parser";
const socket = io(URL, { parser: customParser });
```

When does msgpack help? Payloads with lots of numbers/binary; when it doesn't: mostly textual payloads (similar size, more CPU in JS). **Measure** before switching. Also note the Redis adapter has its own `parser` option for messages between servers (msgpack by default via `notepack.io`).

7.6 Adapters (internals)

What an adapter does

Every namespace has an **adapter** instance (`io.of("/").adapter`). Its job:

1. Store **room membership**: `rooms: Map<Room, Set<SocketId>>` and `sids: Map<SocketId, Set<Room>>`.
2. **Broadcast**: given `{ rooms, except, flags }`, compute the target sockets and write the (once-encoded) packet to each.
3. In a cluster: **forward** broadcasts and requests (fetchSockets, socketsJoin, serverSideEmit, broadcast acks...) to other servers and handle theirs.
4. For connection state recovery: persist sessions and packets (`SessionAwareAdapter` in the in-memory adapter, or Redis Streams / MongoDB adapters).

The **default in-memory adapter** does (1) and (2) only for the local process — which is why a multi-server deployment needs another adapter.

Broadcast algorithm (simplified, in-memory adapter)

```
broadcast(packet, { rooms, except, flags }):
  encoded = encoder.encode(packet)        # encode ONCE
  if rooms is empty → targets = all sockets in namespace
  else targets = union of sockets of each room (each socket once)
  remove sockets in any "except" room
  for each target socket → socket.client.writeToEngine(encoded, flags)
```

Encoding once and reusing the buffer for every recipient is a key performance property (`io.emit` to 10 000 sockets doesn't serialize 10 000 times).

Redis adapter internals (pub/sub)

```
Server A: io.to("room1").emit("x")
   ├─ broadcast locally (A's sockets in room1)
   └─ PUBLISH socket.io#/#room1# <msgpack([serverUid, packet, {rooms, except, flags}])>
      (to a single room → channel "socket.io#/#room1#"; otherwise "socket.io#/#")
```

```
Server B and C: PSUBSCRIBE "socket.io#/#*" → receive → ignore if it's their own uid
                → broadcast locally to THEIR sockets in room1
```

- The classic adapter uses a **pattern subscription** (`PSUBSCRIBE socket.io#/#*`), so **every server receives every broadcast**, even for rooms where it has no sockets. At very large scale this fan-out becomes the bottleneck.
- Requests like `fetchSockets()` / `socketsJoin()` / `serverSideEmit()` use dedicated channels: `socket.io-request#/#` and `socket.io-response#/#` , with `requestsTimeout` (default 5 s).
- The **sharded** adapter (`createShardedAdapter` , Redis 7+ sharded pub/sub) has a `subscriptionMode` option: `"static"` (fixed channels per namespace), `"dynamic"` (default — one channel per *public* room, a server subscribes only to rooms it has sockets in), `"dynamic-private"` (also creates channels for private socket-id rooms).
- Redis **pub/sub is fire-and-forget**: if a server is disconnected from Redis momentarily, it misses broadcasts during that time. The Redis **Streams** adapter fixes this (it can resume reading the stream from the last offset) and supports connection state recovery.

More in Chapter 8.

Writing a custom adapter (rare, but shows mastery)

Adapters extend `Adapter` from `socket.io-adapter` and override methods like `broadcast` , `addAll` , `del` , `delAll` , `fetchSockets` , `addSockets` , `delSockets` , `disconnectSockets` , `serverSideEmit` , `broadcastWithAck` , `persistSession` , `restoreSession` . The `ClusterAdapter` / `ClusterAdapterWithHeartbeat` base classes in `socket.io-adapter` let you implement only the transport (how messages travel between nodes) — that's how the Redis Streams, Postgres, MongoDB, cluster and cloud adapters are built.

7.7 What happens during `socket.emit("chat", "hi")` (server → client), step by step

1. `Socket#emit` checks the name isn't reserved, builds `{ type: EVENT, data: ["chat","hi"], nsp: "/" }` , and registers an ack callback if the last arg is a function (assigns an id).
2. (With connection state recovery enabled, it goes through `adapter.broadcast` to the socket's own room so the packet gets an offset and is stored.)
3. The parser **encodes**: `2["chat","hi"]` (plus attachments if binary).
4. `Client#writeToEngine` → if `volatile` and transport not writable → dropped.
5. Engine.IO wraps it as a message: `42["chat","hi"]` , pushes it to the `writeBuffer` , and **flushes** when the transport is writable (immediately for WebSocket; on the next pending GET for polling — several packets may be batched with `\x1e`).
6. `ws` writes a WebSocket frame (compressed if `perMessageDeflate` enabled and above threshold).

7. Client side: engine.io-client receives the frame, decodes the Engine.IO packet type `4`, passes `2["chat","hi"]` to the Manager's decoder.
8. The decoder yields the packet; the Manager routes it to the Socket for namespace `/`.
9. If the socket isn't connected yet, the packet is stored in `receiveBuffer`; otherwise `onAny` listeners run, then `chat` listeners with `"hi"`.

7.8 Debugging the wire yourself

- **Browser DevTools → Network → filter "socket.io"**:
- polling requests appear as XHR/fetch with the payloads in Response/Payload tabs;
- the WebSocket appears as `?EIO=4&transport=websocket` with a **Messages** tab showing every frame (`2`, `3`, `42[...]`). Use this to verify what's actually sent.
- **Node:** `DEBUG=socket.io*,engine*` (server) or `DEBUG=socket.io-client:*,engine.io-client:*` (Node client); in the browser `localStorage.debug = "socket.io-client:*"` then reload. (Chapter 10.)
- `curl` **handshake test** (is the server reachable and speaking Socket.IO?):

```
curl "https://api.example.com/socket.io/?EIO=4&transport=polling"
# expect: 0{"sid":"…","upgrades":["websocket"],"pingInterval":25000,…}
```

If you get HTML/404 → wrong path or proxy routing. `{"code":0,"message":"Transport unknown"}` → transport disabled. `{"code":5,…}` → protocol version mismatch.

7.9 Chapter summary

- Two protocols: **Engine.IO v4** (open/close/ping/pong/message/upgrade/noop = 0–6) and **Socket.IO v5** (CONNECT/DISCONNECT/EVENT/ACK/CONNECT_ERROR/BINARY_EVENT/BINARY_ACK = 0–6).
- `42["event",…]` = Engine.IO message (4) + Socket.IO event (2).
- Polling: GET to receive (held open), POST to send, `\x1e` separator, binary as `b`+base64.
- WebSocket: one frame per packet, binary frames raw.
- Upgrade: `2probe` / `3probe`, pause polling, `5`, no loss/reordering.
- Parsers: default JSON+attachments, msgpack optional; both sides must match.
- Adapters: room storage + broadcast + inter-server communication; in-memory by default.

Next: Chapter 8 — Scaling

CHAPTER 08

Scaling: Multiple Servers, Sticky Sessions, Adapters, Load Balancers and Kubernetes

> **Goal:** run Socket.IO on many processes and machines correctly. The two words you must say in any scaling interview are "**sticky sessions**" and "**adapter**" — and you must be able to explain *why* each is needed.

8.1 ELI5

One pizza shop (server) can only answer so many phones. So you open **more branches**. Two new problems appear:

1. "**Which branch am I talking to?**" If you use the "ask and wait on the line" method (long-polling), every call must go to the **same branch**, because only that branch remembers your order. The receptionist who routes calls must always send you to the same branch → **sticky sessions**.
2. "**How do I tell everyone in every branch?**" If branch A wants to announce "free pizza for room 'Main Street'", customers of Main Street might be on branch B and C. Branches need an **intercom** connecting them → the **adapter** (e.g. Redis pub/sub).

8.2 How far can one server go?

A single Node.js process can typically hold **tens of thousands** of mostly-idle WebSocket connections (10k–100k+ depending on memory, message rate, payload size, compression, and your handler logic). The bottlenecks are usually:

- **CPU** (single-threaded event loop): JSON encoding/decoding, broadcast fan-out, your handler logic, TLS (if terminated in Node).
- **Memory**: per-connection buffers (more with `perMessageDeflate`), `socket.data`, rooms, pending acks, connection-state-recovery packet storage.
- **File descriptors**: each connection = 1 fd → raise `ulimit -n`.
- **Fan-out**: broadcasting 1 message to 10 000 sockets = 10 000 writes. At 100 msg/s → 1 M writes/s. *Messages × recipients* is the metric that matters.

Load test your real workload (Chapter 10) — don't trust generic numbers.

Then you scale **horizontally**: several Node processes (on one machine via cluster/PM2) and/or several machines/containers behind a load balancer.

8.3 Problem 1 — Sticky sessions (session affinity)

Why they are required

With the **HTTP long-polling** transport, one Socket.IO connection = **many HTTP requests** (handshake GET, data GETs, POSTs, then the WebSocket upgrade request). The Engine.IO session (`sid`) lives in the **memory of the server that did the handshake.**

```
Client ── GET handshake ──▶ LB ──▶ Server A   (creates sid=abc)
Client ── POST sid=abc ──▶ LB ──▶ Server B   ✗ 400 {"code":1,"message":"Session ID unknown"}
```

So the load balancer must route **all requests of a session to the same server**.

Even when polling upgrades to WebSocket quickly, the first few requests are polling, and the WebSocket upgrade request itself must reach the server that holds the `sid` .

When you DON'T need sticky sessions

If clients use **WebSocket only** (`transports: ["websocket"]`), one connection = one TCP connection that stays on one server → no sticky sessions needed.

```
// client
const socket = io(URL, { transports: ["websocket"] });
// (optionally also restrict the server: new Server({ transports: ["websocket"] }))
```

Trade-off: no polling fallback (users behind WebSocket-hostile proxies fail to connect). Many companies accept this for mobile apps and internal tools.

Ways to get sticky sessions

Method	How	Notes
IP hash	nginx `ip_hash` / `hash $remote_addr consistent` ; HAProxy `balance source`	Simple; uneven distribution behind NAT (whole office on one server); breaks if a client's IP changes (mobile)

Method	How	Notes
Cookie-based	LB sets a cookie identifying the server (AWS ALB stickiness, HAProxy `cookie`, nginx Plus `sticky`, Traefik sticky cookies, ingress-nginx affinity)	Best distribution. **Cross-origin:** client needs `withCredentials: true`, server CORS `credentials: true` so the browser sends the LB cookie on polling requests
Socket.IO `cookie` option	`new Server({ cookie: { name: "io", httpOnly: true, sameSite: "strict", maxAge: 86400 } })` → Engine.IO sets an `io=<sid>` cookie that the LB can hash on	Useful with LBs that can route on an arbitrary cookie value
Node cluster	`@socket.io/sticky` (master routes by sid) + `@socket.io/cluster-adapter`	Single machine, multiple cores
PM2	`@socket.io/pm2` (a PM2 fork with sticky sessions) + cluster adapter	Single machine
Query/URL-based routing	e.g. route by `?shard=` or by subdomain	Custom architectures

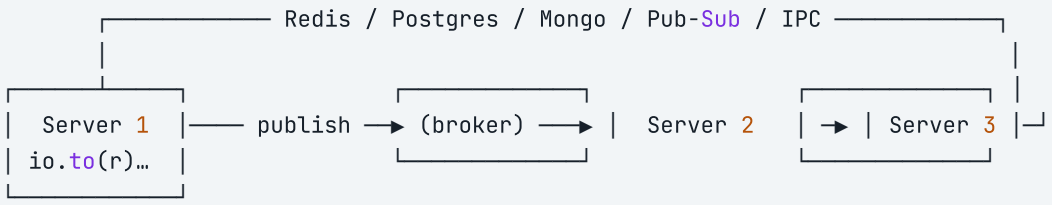
8.4 Problem 2 — Broadcasting across servers: adapters

Why it's required

```
User A connected to Server 1, in room "game:7"
User B connected to Server 2, in room "game:7"

Server 1: io.to("game:7").emit("move")   → reaches only User A ✗
```

The default **in-memory adapter** only knows local sockets. A distributed **adapter** forwards the broadcast to all other servers, which deliver it to their local sockets.



Official / well-known adapters

Adapter	Package	Backend	Highlights
Redis	`@socket.io/redis-adapter`	Redis Pub/Sub	Most popular; also a **sharded** variant for Redis 7 sharded pub/sub; ❌ no connection state recovery
Redis Streams	`@socket.io/redis-streams-adapter`	Redis Streams	Survives temporary Redis disconnections without losing packets; ✅ connection state recovery
MongoDB	`@socket.io/mongo-adapter`	MongoDB change streams (replica set required)	✅ CSR (since 0.3.0)
Postgres	`@socket.io/postgres-adapter`	`LISTEN/NOTIFY` (+ attachment table for payloads > 8 KB)	❌ CSR
Cluster	`@socket.io/cluster-adapter`	Node.js `cluster` IPC	Multiple cores on one machine
GCP Pub/Sub	`@socket.io/gcp-pubsub-adapter`	Google Cloud Pub/Sub	
AWS SQS	`@socket.io/aws-sqs-adapter`	AWS SNS + SQS	
Azure Service Bus	`@socket.io/azure-service-bus-adapter`	Azure Service Bus	
Azure Web PubSub	`@azure/web-pubsub-socket.io`	Managed service	Azure hosts the connections for you
Community	e.g. AMQP (RabbitMQ), NATS adapters		Check maintenance status

All official ones support "socket management" (`fetchSockets` , `socketsJoin` , ...), inter-server communication (`serverSideEmit`) and broadcast with acks.

Redis adapter — setup (`redis` package)

```
const { createServer } = require("node:http");
const { Server } = require("socket.io");
const { createClient } = require("redis");
const { createAdapter } = require("@socket.io/redis-adapter");

async function main() {
  const pubClient = createClient({ url: process.env.REDIS_URL });
  const subClient = pubClient.duplicate();   // a subscriber connection can't publish → 2
clients
```

```
  pubClient.on("error", (err) => logger.error({ err }, "redis pub error"));
  subClient.on("error", (err) => logger.error({ err }, "redis sub error"));

  await Promise.all([pubClient.connect(), subClient.connect()]);

  const httpServer = createServer();
  const io = new Server(httpServer, {
    adapter: createAdapter(pubClient, subClient, {
      key: "socket.io",          // channel prefix (namespace your apps if they share Redis!)
      requestsTimeout: 5000,     // for fetchSockets etc.
    }),
  });

  httpServer.listen(3000);
}
main();
```

With **ioredis**:

```
const { Redis } = require("ioredis");
const pubClient = new Redis(process.env.REDIS_URL);
const subClient = pubClient.duplicate();
const io = new Server({ adapter: createAdapter(pubClient, subClient) });
```

With a **Redis Cluster** (ioredis):

```
const { Cluster } = require("ioredis");
const pubClient = new Cluster([{ host: "redis-1", port: 6379 }, { host: "redis-2", port: 6379
}]);
const subClient = pubClient.duplicate();
```

Sharded pub/sub (Redis ≥ 7, with the `redis` package):

```
const { createShardedAdapter } = require("@socket.io/redis-adapter");
const io = new Server({ adapter: createShardedAdapter(pubClient, subClient /*, {
subscriptionMode: "dynamic" } */) });
```

> Gotchas: - **Always attach `error` listeners** to Redis clients; an unhandled `error` event crashes Node. - Two different apps sharing one Redis must use different `key` prefixes, or they'll receive each other's broadcasts. - The **subscriber** connection can't run normal commands; use the pub client (or another client) for app data. - If Redis goes down: local broadcasts still work; cross-server broadcasts are lost while it's down (pub/sub has no buffering). - With the classic adapter every node receives every broadcast (pattern subscription) — the sharded adapter in `dynamic` mode reduces this.

Redis Streams adapter

```
const { createClient } = require("redis");
const { createAdapter } = require("@socket.io/redis-streams-adapter");

const redisClient = createClient({ url: process.env.REDIS_URL });
await redisClient.connect();

const io = new Server({
  adapter: createAdapter(redisClient, {
    streamName: "socket.io",    // default
    maxLen: 10_000,             // approximate stream trimming
    readCount: 100,             // XREAD batch size
  }),
  connectionStateRecovery: { maxDisconnectionDuration: 2 * 60 * 1000 },
});
```

- Uses **one** Redis connection (it reads with `XREAD BLOCK`).
- A node that loses Redis briefly **resumes from its last stream offset** → no lost broadcasts (as long as `maxLen` wasn't exceeded).
- Sessions for connection state recovery are stored as Redis keys (`sio:session:` prefix).

Postgres adapter (if you already run Postgres and don't want Redis)

```
const { Pool } = require("pg");
const { createAdapter } = require("@socket.io/postgres-adapter");

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
await pool.query(`
  CREATE TABLE IF NOT EXISTS socket_io_attachments (
    id bigserial UNIQUE,
    created_at timestamptz DEFAULT NOW(),
    payload bytea
  );
`);
const io = new Server({ adapter: createAdapter(pool) });
```

NOTIFY payloads are limited to 8000 bytes; bigger packets go through the attachments table. Fine for moderate traffic; Redis is usually faster for high volume.

Node.js cluster (multi-core on one machine)

```
const cluster = require("node:cluster");
const http = require("node:http");
const { availableParallelism } = require("node:os");
const { Server } = require("socket.io");
const { setupMaster, setupWorker } = require("@socket.io/sticky");
```

```
const { createAdapter, setupPrimary } = require("@socket.io/cluster-adapter");

if (cluster.isPrimary) {
  const httpServer = http.createServer();
  setupMaster(httpServer, { loadBalancingMethod: "least-connection" }); // sticky sessions
  setupPrimary();                                                       // adapter IPC hub
  httpServer.listen(3000);
  for (let i = 0; i < availableParallelism(); i++) cluster.fork();
  cluster.on("exit", (worker) => { console.log(`worker ${worker.process.pid} died`);
cluster.fork(); });
} else {
  const httpServer = http.createServer();
  const io = new Server(httpServer);
  io.adapter(createAdapter());
  setupWorker(io);              // receives connections handed over by the primary
  io.on("connection", (socket) => { /* ... */ });
}
```

> Why can't plain `cluster` work? Node's cluster module distributes **TCP connections** round-robin; HTTP polling requests of one session use *different* TCP connections and would land on different workers. `@socket.io/sticky` makes the primary inspect the `sid` and route to the right worker.

8.5 Emitting from outside Socket.IO servers: emitters

A background worker (e.g. processing payments) wants to notify users but is **not** a Socket.IO server (no HTTP server, no client connections). Use an **emitter**: it publishes into the adapter's channel; Socket.IO servers deliver.

```
const { Emitter } = require("@socket.io/redis-emitter");
const { createClient } = require("redis");

const redisClient = createClient({ url: process.env.REDIS_URL });
await redisClient.connect();
const emitter = new Emitter(redisClient /*, { key: "socket.io" } must match the adapter */);

emitter.to(`user:${userId}`).emit("payment:succeeded", { orderId });
emitter.of("/admin").emit("stats", stats);
emitter.except("banned").emit("announcement", text);
emitter.in("room1").socketsJoin("room2");
emitter.in(`user:${userId}`).disconnectSockets();
emitter.serverSideEmit("cache:invalidate", key);
```

Limitations: emitters **can't receive acks** and can't fetch sockets (no response channel). Equivalent packages exist for Postgres (`@socket.io/postgres-emitter`) and MongoDB (`@socket.io/mongo-emitter`). The emitter must match the adapter: the Redis emitter speaks the classic Redis **pub/sub** adapter's format, so check your adapter's docs for a compatible emitter before mixing them (e.g. with the Redis *Streams* adapter).

Architecture tip: many companies use a **message broker** (Kafka/RabbitMQ/SQS) for domain events, and a thin "realtime gateway" service (Socket.IO servers) that consumes those events and emits to rooms. The gateway owns connections; other services never talk to sockets directly.

8.6 Inter-server communication: `serverSideEmit`

```
// on any server
io.serverSideEmit("config:reload", { version: 42 });

// every OTHER server receives it (not the sender)
io.on("config:reload", ({ version }) => reloadConfig(version));

// with acknowledgement (v4.5+)
io.timeout(5000).serverSideEmit("ping", (err, responses) => {
  // responses: one per other server
});
const responses = await io.timeout(5000).serverSideEmitWithAck("health");
```

Use for cluster-wide cache invalidation, config reloads, coordination. Reserved names apply (`connection` , `connect` , `new_namespace`).

8.7 Load balancer and reverse proxy configuration

Lab-verified (October 2026): the nginx and HAProxy configurations in this section were run (nginx 1.24, HAProxy 2.8) in front of three Socket.IO 4.8.4 nodes sharing the Redis adapter. Results:

Setup	Result
nginx config below (hash on client IP, Upgrade headers)	connected, **upgraded to WebSocket**
HAProxy round-robin + `cookie SERVERID insert`, polling-only client with `withCredentials: true`	connected every time, connections spread over nodes 5001/5002/5003
Same HAProxy, client **without** `withCredentials`	**failed** every time (`xhr poll error` / `xhr post error`); nodes logged `engine error code=1 Session ID unknown` — the LB cookie wasn't sent back, so requests landed on random nodes
HAProxy round-robin **without** stickiness, polling-only	**failed:** `Session ID unknown` (code 1)
HAProxy round-robin without stickiness, **WebSocket-only** client	connected — no stickiness needed
Client on node 5001 broadcasts, client on node 5002 listening	received via the Redis adapter

The full lab (configs, node script, test script) is in Chapter 23 so you can reproduce it.

nginx

```
http {
  map $http_upgrade $connection_upgrade {   # recommended: only set "upgrade" when needed
    default upgrade;
    ''      close;
  }

  upstream socket_nodes {
    # sticky sessions (pick one)
    hash $remote_addr consistent;   # whole client IP
    # ip_hash;                      # first 3 octets of IPv4 (coarser)
    # sticky cookie srv_id expires=1h; # nginx Plus only

    server app01:3000;
    server app02:3000;
    server app03:3000;
  }
```

```
  server {
    listen 443 ssl;
    server_name api.example.com;
    # ssl_certificate ... ssl_certificate_key ...

    location /socket.io/ {
      proxy_pass http://socket_nodes;

      proxy_http_version 1.1;                          # REQUIRED for WebSocket
      proxy_set_header Upgrade $http_upgrade;          # REQUIRED
      proxy_set_header Connection $connection_upgrade; # REQUIRED

      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;

      proxy_read_timeout 120s;   # must exceed pingInterval (25 s) comfortably; default is 60 s
      proxy_send_timeout 120s;
      proxy_buffering off;       # don't buffer long-polling responses
    }
  }
}
```

Missing `proxy_http_version 1.1` or the `Upgrade` / `Connection` headers is the **#1 reason** "WebSocket doesn't work behind nginx" (clients silently stay on polling, or get `websocket error`). In DevTools you'll see polling requests every ~25 s and no `transport=websocket` 101 response.

If you serve Socket.IO under a sub-path (`/api/socket.io/`), set `path` on both client and server, or rewrite carefully.

HAProxy

```
frontend http_front
  bind *:443 ssl crt /etc/ssl/site.pem
  default_backend socket_nodes
  timeout client 120s

backend socket_nodes
  balance roundrobin
  cookie SERVERID insert indirect nocache   # cookie-based stickiness
  timeout server 120s
  timeout tunnel 1h                          # for long-lived WebSocket connections
  server app01 app01:3000 check cookie app01
  server app02 app02:3000 check cookie app02
```

(Or `balance source` for IP-based stickiness.)

Apache httpd

Needs `mod_proxy`, `mod_proxy_http`, `mod_proxy_wstunnel` (or `ProxyPass … upgrade=websocket` in 2.4.47+), and `mod_proxy_balancer` with `stickysession` for multiple nodes.

AWS

- **ALB** (Application Load Balancer): supports WebSocket natively. Enable **target group stickiness** (load-balancer-generated cookie `AWSALB`, or application cookie). For cross-origin, set client `withCredentials: true` and server CORS `credentials: true` so the ALB cookie is sent on polling requests. **Idle timeout default 60 s** → keep `pingInterval` (25 s) below it (it is, by default) or raise it.
- **NLB** (Network Load Balancer, layer 4): passes TCP through; stickiness by source IP; idle timeout for TCP is 350 s.
- **API Gateway WebSocket APIs** are **not** Socket.IO servers (custom WS protocol with routes) — you can't point a Socket.IO client at them.
- **Elastic Beanstalk**: enable WebSocket in its nginx proxy config.
- **ECS/EKS**: same as below for Kubernetes.

Cloudflare / CDNs

- WebSockets supported (proxy enabled). Cloudflare closes idle connections after ~100 s of inactivity → heartbeats (25 s) keep them alive.
- Cloudflare **load balancing** has session affinity (cookie) options.
- Long-polling requests through CDNs: ensure they're not cached (`t=` param and headers handle this).

Heroku / Render / Railway / Fly.io etc.

Most support WebSockets. Heroku's router supports **session affinity** as a feature flag (`heroku features:enable http-session-affinity`); its 55-second idle timeout is reset by heartbeat traffic.

8.8 Kubernetes

Typical setup: Deployment with N replicas → Service → Ingress (ingress-nginx) → clients.

Ingress-nginx sticky sessions + timeouts:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```yaml
  name: realtime
  annotations:
    nginx.ingress.kubernetes.io/affinity: "cookie"
    nginx.ingress.kubernetes.io/session-cookie-name: "route"
    nginx.ingress.kubernetes.io/session-cookie-max-age: "172800"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
spec:
  ingressClassName: nginx
  rules:
    - host: rt.example.com
      http:
        paths:
          - path: /socket.io/
            pathType: Prefix
            backend:
              service: { name: realtime, port: { number: 3000 } }
```

(ingress-nginx handles the `Upgrade` headers for WebSocket automatically.)

Other notes:

- `Service.spec.sessionAffinity: ClientIP` gives IP stickiness at the Service level, but behind an Ingress all traffic comes from the ingress pods — use Ingress-level affinity.
- **Graceful shutdown on rolling deploys:** when a pod is terminated, every client on it disconnects and reconnects elsewhere. Spread this out:
- `terminationGracePeriodSeconds` long enough,
- a `preStop` hook (e.g. `sleep 10`) so the endpoint is removed from the LB first,
- on `SIGTERM`: stop accepting new connections, optionally notify clients, then `io.close()` (Chapter 11),
- roll pods slowly (`maxSurge`, `maxUnavailable`) to avoid a reconnection storm on the remaining pods.
- **Autoscaling:** CPU-based HPA is often misleading for socket servers (idle connections use memory, not CPU). Scale on **connection count** / event-loop lag via custom metrics (Prometheus adapter). Note that scale-out doesn't move existing connections — new pods only get *new* connections. Scale-in disconnects clients.
- **Readiness probes:** mark not-ready during shutdown so no new connections arrive.
- **Liveness probes:** don't make them depend on Redis; a Redis outage shouldn't restart all pods.

8.9 Capacity planning & OS tuning checklist

- `ulimit -n` (open files) ≥ expected connections × ~1.2 (e.g. 100 000+); systemd `LimitNOFILE`.

- Kernel: `net.core.somaxconn`, `net.ipv4.tcp_max_syn_backlog` for connection bursts; `net.ipv4.ip_local_port_range` on **proxies** (each upstream connection uses an ephemeral port — a single nginx → single Node upstream is limited to ~28k–64k connections per destination IP:port; add more upstream IPs/ports or use more Node processes).
- Memory: measure per-connection memory in a load test (heap snapshot); disable `perMessageDeflate` unless needed; keep `socket.data` small.
- Node: `--max-old-space-size` appropriate to container memory.
- Redis: monitor pub/sub throughput, `client-output-buffer-limit pubsub` (slow subscribers get disconnected when their output buffer exceeds limits → that node silently stops receiving broadcasts until reconnect!).

8.10 Scaling patterns and anti-patterns

Patterns

- **Stateless socket servers**: keep durable state in DB/Redis; any node can serve any user after reconnection.
- **User rooms** (`user:<id>`) instead of id maps → works across nodes.
- **Presence in Redis** (sets/sorted sets with TTL heartbeats) rather than `fetchSockets()` on every request.
- **Partition by tenant/region** for very large systems (separate clusters per region; route via DNS/geo).
- **Gateway pattern**: Socket.IO servers only handle connections; business logic in services; communication via broker/emitter.
- **Backpressure**: throttle/coalesce high-frequency updates (send state every 100 ms, not every change).

Anti-patterns

- Storing `userId → socket` objects in a local `Map` and trying to emit from another node.
- Calling `io.fetchSockets()` on hot paths.
- Broadcasting huge payloads to huge rooms (send a "changed" notification + let clients fetch via HTTP/CDN instead).
- One giant room everybody is in, with chatty events.
- Forgetting sticky sessions → random `400 Session ID unknown` and reconnect loops.
- No `error` handler on Redis clients.

8.11 Chapter summary

- Two problems at scale: **sticky sessions** (for polling) and **cross-node broadcast** (adapter).
- Sticky sessions via IP hash, LB cookies, `@socket.io/sticky`; or skip them with WebSocket-only transports.
- Adapters: Redis (classic, sharded), Redis Streams (resilient, CSR), Mongo, Postgres, cluster, cloud. Emitters let non-socket processes broadcast.
- `serverSideEmit` for server-to-server events.
- Proxy config: HTTP/1.1 + `Upgrade` / `Connection` headers + timeouts > ping interval.
- Kubernetes: ingress cookie affinity, long timeouts, graceful shutdown, scale on connections.

Next: Chapter 9 — TypeScript, frameworks and clients

CHAPTER 09

TypeScript, Frontend Frameworks, Mobile and Other Languages

> **Goal:** use Socket.IO correctly in real stacks: fully typed events with TypeScript, React (hooks, context, StrictMode), Vue, Angular, Next.js, NestJS, React Native, native mobile, and servers/clients in other languages.

9.1 TypeScript — fully typed events

Since v4, both `Server` and client `Socket` accept **generic event maps**. Define them **once** in a shared package and import from both sides — this is the single most valuable practice for large teams (typos and payload mismatches become compile errors).

Shared event definitions

```
// packages/realtime-contracts/src/events.ts
export interface Message {
  id: string;
  conversationId: string;
  authorId: string;
  text: string;
  createdAt: string; // ISO (Dates become strings on the wire!)
}

export type Ack<T> = (res: { ok: true; data: T } | { ok: false; error: { code: string;
message?: string } }) => void;

// events the SERVER emits to clients (client listens)
export interface ServerToClientEvents {
  "message:new": (msg: Message) => void;
  "user:typing": (p: { conversationId: string; userId: string }) => void;
  "presence:update": (p: { userId: string; online: boolean }) => void;
  "noArg": () => void;
  "withAck": (question: string, callback: (answer: number) => void) => void;
}

// events the CLIENT emits to the server (server listens)
export interface ClientToServerEvents {
  "message:send": (p: { conversationId: string; text: string; clientMsgId: string }, ack: Ack<{
id: string }>) => void;
  "conversation:join": (p: { conversationId: string }, ack: Ack<{ history: Message[] }>) =>
```

```
void;
  "typing": (p: { conversationId: string }) => void;
}

// events between Socket.IO servers (serverSideEmit)
export interface InterServerEvents {
  "cache:invalidate": (key: string) => void;
}

// type of socket.data
export interface SocketData {
  user: { id: string; name: string; roles: string[] };
}
```

Typed server

```
import { Server, Socket } from "socket.io";
import type { ClientToServerEvents, ServerToClientEvents, InterServerEvents, SocketData } from
"@acme/realtime-contracts";

const io = new Server<ClientToServerEvents, ServerToClientEvents, InterServerEvents,
SocketData>(httpServer, {
  cors: { origin: process.env.WEB_ORIGIN },
});

type AppSocket = Socket<ClientToServerEvents, ServerToClientEvents, InterServerEvents,
SocketData>;

io.on("connection", (socket: AppSocket) => {
  socket.data.user.name;                                    // typed

  socket.on("message:send", async (payload, ack) => {       // payload & ack typed
    const msg = await messages.create(socket.data.user.id, payload);
    io.to(`conversation:${payload.conversationId}`).emit("message:new", msg); // typed
    ack({ ok: true, data: { id: msg.id } });
  });

  // @ts-expect-error — unknown event
  socket.emit("mesage:new", {});
});

io.serverSideEmit("cache:invalidate", "users:1");           // typed
io.on("cache:invalidate", (key) => cache.delete(key));      // typed
```

Typed client

```
import { io, Socket } from "socket.io-client";
import type { ClientToServerEvents, ServerToClientEvents } from "@acme/realtime-contracts";
```

```
// NOTE: generic order is REVERSED on the client: <ListenEvents, EmitEvents>
const socket: Socket<ServerToClientEvents, ClientToServerEvents> = io(URL);

socket.on("message:new", (msg) => render(msg));          // msg: Message
socket.emit("typing", { conversationId: "c1" });

const res = await socket.timeout(5000).emitWithAck("message:send", { conversationId: "c1",
text: "hi", clientMsgId: crypto.randomUUID() });
```

> **Interview / bug note:** on the server the order is `Server<ListenEvents (client→server), EmitEvents (server→client), …>`; on the client it's `Socket<ListenEvents (server→client), EmitEvents (client→server)>`. Both are "what I listen to, what I emit" from that side's perspective. Mixing them up gives confusing type errors.

Typing acks with timeout

When you use `socket.timeout(ms).emit(..., cb)`, the type system automatically adds the `err` first parameter to the callback (`DecorateAcknowledgements`). With broadcast, `io.timeout(ms).emit(...)` types the callback as `(err, responses[])`.

Runtime validation is still needed

Types vanish at runtime. A malicious client can send anything. Pair the TS interfaces with runtime schemas (zod/valibot/typebox) — or **derive** TS types from schemas:

```
const SendSchema = z.object({ conversationId: z.string().uuid(), text: z.string().max(2000),
clientMsgId: z.string().uuid() });
type SendPayload = z.infer<typeof SendSchema>;
```

9.2 React

The rules (each prevents a real, common bug)

1. **Create the socket once** — at module level or in a context provider — **not** inside a component body (that creates a new connection per render).
2. **Register listeners in `useEffect` and remove them in the cleanup** (`socket.off(event, handler)` with the **same function reference**) — otherwise listeners accumulate on every re-mount → duplicated messages and memory leaks.

3. **React 18+ StrictMode** mounts/unmounts/re-mounts effects in development → effects run twice. Correct cleanup makes this harmless. Don't "fix" it by removing StrictMode.
4. Avoid **stale closures:** handlers registered once capture old state — use functional `setState(prev => …)` or refs.
5. Decide **where** connect/disconnect happens (e.g. connect after login, disconnect on logout) — `autoConnect: false` helps.

socket module

```
// src/socket.ts
import { io, Socket } from "socket.io-client";
import type { ServerToClientEvents, ClientToServerEvents } from "@acme/realtime-contracts";

export const socket: Socket<ServerToClientEvents, ClientToServerEvents> =
io(import.meta.env.VITE_WS_URL, {
  autoConnect: false,                                  // connect explicitly after login
  auth: (cb) => cb({ token: localStorage.getItem("token") }), // fresh token on each
(re)connect
  transports: ["websocket", "polling"],
});
```

Connection state hook

```
import { useEffect, useState } from "react";
import { socket } from "./socket";

export function useSocketStatus() {
  const [connected, setConnected] = useState(socket.connected);

  useEffect(() => {
    const onConnect = () => setConnected(true);
    const onDisconnect = () => setConnected(false);
    socket.on("connect", onConnect);
    socket.on("disconnect", onDisconnect);
    return () => {
      socket.off("connect", onConnect);       // same references → removes exactly these
      socket.off("disconnect", onDisconnect);
    };
  }, []);

  return connected;
}
```

Generic event hook

```
import { useEffect, useRef } from "react";
import { socket } from "./socket";

export function useSocketEvent<E extends keyof ServerToClientEvents>(event: E, handler:
ServerToClientEvents[E]) {
  const handlerRef = useRef(handler);
  handlerRef.current = handler;                // always call the latest handler (no stale
closure)

  useEffect(() => {
    const listener = ((...args: any[]) => (handlerRef.current as any)(...args)) as any;
    socket.on(event, listener);
    return () => { socket.off(event, listener); };
  }, [event]);
}

// usage
function Chat({ conversationId }: { conversationId: string }) {
  const [messages, setMessages] = useState<Message[]>([]);

  useSocketEvent("message:new", (msg) => {
    if (msg.conversationId === conversationId) setMessages((prev) => [...prev, msg]); //
functional update
  });

  useEffect(() => {
    let cancelled = false;
    socket.timeout(5000).emitWithAck("conversation:join", { conversationId })
      .then((res) => { if (!cancelled && res.ok) setMessages(res.data.history); })
      .catch(() => {/* timeout */});
    return () => { cancelled = true; /* optionally emit "conversation:leave" */ };
  }, [conversationId]);

  return <MessageList messages={messages} />;
}
```

Connect on login, disconnect on logout

```
useEffect(() => {
  if (user) socket.connect();
  return () => { socket.disconnect(); };
}, [user]);
```

Where to put socket data in state management

- Small apps: component state + context.

- Redux Toolkit: a **middleware** that owns the socket and dispatches actions on events (don't put the socket object in the store — it's not serializable).
- React Query/TanStack Query: on events, `queryClient.setQueryData` or `invalidateQueries` → server state stays the source of truth.
- Zustand: a store action subscribes once and updates state.

Next.js

- **Client components only** (`"use client"`) for `socket.io-client` — sockets don't exist during server rendering. Guard against SSR (`useEffect` runs only in the browser).
- Hosting the **server**:
- Use a **custom server** (`server.js` that creates the Next handler and a Socket.IO `Server` on the same `http` server), or
- better for production: run Socket.IO as a **separate service** (its own deployment) and point the client at it (CORS).
- **Serverless/edge functions (Vercel functions, Lambda) can't host Socket.IO** — they are short-lived and don't keep connections. Use a long-running Node host, or a managed service.

```
// server.js (custom Next.js server)
const { createServer } = require("node:http");
const next = require("next");
const { Server } = require("socket.io");

const app = next({ dev: process.env.NODE_ENV !== "production" });
const handler = app.getRequestHandler();

app.prepare().then(() => {
  const httpServer = createServer(handler);
  const io = new Server(httpServer);
  io.on("connection", (socket) => { /* ... */ });
  httpServer.listen(3000);
});
```

9.3 Vue 3

```
// socket.js
import { reactive } from "vue";
import { io } from "socket.io-client";

export const state = reactive({ connected: false, messages: [] });

export const socket = io(import.meta.env.VITE_WS_URL, { autoConnect: false });
```

```
socket.on("connect", () => { state.connected = true; });
socket.on("disconnect", () => { state.connected = false; });
socket.on("message:new", (m) => { state.messages.push(m); });   // registered ONCE at module
level
```

In components using component-scoped listeners, register in `onMounted` and remove in `onBeforeUnmount` / `onUnmounted` with `socket.off(event, handler)`. With **Pinia**, subscribe once in a store action (`bindEvents()`), guarded against double binding.

9.4 Angular

Wrap the socket in an injectable **service**, expose events as **RxJS Observables**, and let subscriptions handle cleanup:

```
@Injectable({ providedIn: "root" })
export class RealtimeService implements OnDestroy {
  private socket = io(environment.wsUrl, { autoConnect: false, auth: (cb) => cb({ token:
this.auth.token }) });

  constructor(private auth: AuthService, private zone: NgZone) {}

  connect() { this.socket.connect(); }

  on<T>(event: string): Observable<T> {
    return new Observable<T>((subscriber) => {
      const handler = (data: T) => this.zone.run(() => subscriber.next(data)); // run inside
Angular zone → change detection
      this.socket.on(event, handler);
      return () => this.socket.off(event, handler);                          // teardown on
unsubscribe
    });
  }

  emitWithAck<T>(event: string, payload: unknown): Promise<T> {
    return this.socket.timeout(5000).emitWithAck(event, payload);
  }

  ngOnDestroy() { this.socket.disconnect(); }
}
```

(`ngx-socket-io` is a popular wrapper library.) With zoneless Angular / signals, update signals in the handler instead of `zone.run` .

9.5 NestJS (server)

```
// chat.gateway.ts
import {
  WebSocketGateway, WebSocketServer, SubscribeMessage, MessageBody, ConnectedSocket,
  OnGatewayInit, OnGatewayConnection, OnGatewayDisconnect,
} from "@nestjs/websockets";
import { Server, Socket } from "socket.io";

@WebSocketGateway({ namespace: "/chat", cors: { origin: process.env.WEB_ORIGIN } })
export class ChatGateway implements OnGatewayInit, OnGatewayConnection, OnGatewayDisconnect {
  @WebSocketServer() server: Server;  // the Namespace ("/chat") instance when a namespace is
set

  afterInit(server: Server) { /* e.g. server.use(authMiddleware) */ }

  handleConnection(client: Socket) { /* ... */ }
  handleDisconnect(client: Socket) { /* ... */ }

  @SubscribeMessage("message:send")
  async onMessage(@MessageBody() dto: SendMessageDto, @ConnectedSocket() client: Socket) {
    const msg = await this.messages.create(client.data.user.id, dto);
    this.server.to(`conversation:${dto.conversationId}`).emit("message:new", msg);
    return { ok: true, data: { id: msg.id } };   // returned value becomes the ACK response
  }
}
```

NestJS specifics to know:

- Packages: `@nestjs/websockets` + `@nestjs/platform-socket.io`.
- **Return value of a `@SubscribeMessage` handler = acknowledgement** (if the client passed a callback). Returning `{ event, data }` (a `WsResponse`) emits an event back instead.
- Guards (`@UseGuards(WsAuthGuard)`), Pipes (`ValidationPipe` with class-validator DTOs), Interceptors and **exception filters** (`WsException`, `BaseWsExceptionFilter`) work on gateways. Note: guards run per message, **not** on connection — do connection auth in `handleConnection` (disconnect if invalid) or in a `server.use()` middleware in `afterInit`/a custom adapter.
- **Redis adapter** in Nest: extend `IoAdapter`, override `createIOServer` to call `server.adapter(createAdapter(pub, sub))`, then `app.useWebSocketAdapter(redisIoAdapter)`.

```
export class RedisIoAdapter extends IoAdapter {
  private adapterConstructor: ReturnType<typeof createAdapter>;
  async connectToRedis() {
    const pubClient = createClient({ url: process.env.REDIS_URL });
    const subClient = pubClient.duplicate();
    await Promise.all([pubClient.connect(), subClient.connect()]);
    this.adapterConstructor = createAdapter(pubClient, subClient);
  }
  createIOServer(port: number, options?: ServerOptions) {
    const server = super.createIOServer(port, options);
```

```
    server.adapter(this.adapterConstructor);
    return server;
  }
}
// main.ts
const redisIoAdapter = new RedisIoAdapter(app);
await redisIoAdapter.connectToRedis();
app.useWebSocketAdapter(redisIoAdapter);
```

9.6 React Native / Expo

```
import { io } from "socket.io-client";
const socket = io("https://api.example.com", { transports: ["websocket"] });
```

- `transports: ["websocket"]` is commonly recommended in React Native: the polling transport relies on XHR implementations that sometimes behave differently, and WebSocket is reliable on mobile networks.
- Use the device's real address (not `localhost`) when testing on a device/emulator (Android emulator → `10.0.2.2`, or your LAN IP). Android blocks cleartext `http://` by default (use HTTPS or allow cleartext for dev).
- Handle app state: disconnect/pause when the app goes to background (`AppState`), and reconnect when it returns; use push notifications (FCM/APNs) for background delivery.

9.7 Native mobile clients

Platform	Official client	Notes
iOS / macOS	**Socket.IO-Client-Swift**	`SocketManager(socketURL:config:)`, `manager.defaultSocket`; config options like `.log(true)`, `.compress`, `.forceWebsockets(true)`; check version compatibility (v16+ for Socket.IO v3/v4 servers)
Android / Java	**socket.io-client-java**	`IO.socket(uri, options)`; v2.x of the Java client for Socket.IO v3/v4 servers; callbacks run on a background thread → post UI updates to the main thread
C++	socket.io-client-cpp	v3.x for Socket.IO v3/v4
Dart / Flutter	`socket_io_client` (community)	Match its major version to your server version per its compatibility table
Python	`python-socketio`	Both client and server

Platform	Official client	Notes
.NET	`SocketIOClient` (community)	
Rust	`rust_socketio` (client), `socketioxide` (server)	
Go	e.g. `googollee/go-socket.io` (server; historically v2-only for a long time — check protocol support), others	Verify protocol v5 support before choosing

Always check each client's compatibility table with your server version — mismatched protocol versions are a top cause of "it connects in the browser but not on Android/iOS".

9.8 Server implementations in other languages

Language	Library	Notes
Python	`python-socketio` (+ ASGI/WSGI integrations: aiohttp, FastAPI via ASGI app, Flask-SocketIO)	Mature, supports Redis/Kafka/RabbitMQ managers for scaling
Java	`netty-socketio` (historically v2 protocol; check forks for v4), Spring integrations	Check protocol version
Go	various (`googollee/go-socket.io` , `zishang520/socket.io`)	Check protocol support
Rust	`socketioxide`	Tower/axum/hyper integration
Deno	official port (`deno.land/x/socket_io`)	
PHP	community (`workerman/phpsocket.io`)	
Elixir/.NET	prefer native solutions (Phoenix Channels, SignalR)	

Interop works as long as both sides implement the same **protocol versions** (Engine.IO v4 / Socket.IO v5 for modern clients).

9.9 Node.js-to-Node.js (server-side client)

Use `socket.io-client` in Node to connect services or to write integration tests/load tests:

```
const { io } = require("socket.io-client");
const socket = io("https://rt.example.com", {
  transports: ["websocket"],
```

```
  auth: { token: SERVICE_TOKEN },
  extraHeaders: { "x-service": "billing" }, // works in Node (not in browsers' WebSocket)
});
```

Note: a Node client keeps the process alive while connected; call `socket.disconnect()` to let it exit.

9.10 Chapter summary

- Share typed event maps between client and server; remember the reversed generic order on the client; still validate at runtime.
- React: one socket instance, listeners in `useEffect` with `off` cleanup, functional updates/refs to avoid stale closures, StrictMode-safe.
- Vue/Angular: register once in a module/service; Angular → Observables + teardown.
- Next.js: client components only; host Socket.IO on a long-running server, not serverless.
- NestJS: gateways, return value = ack, guards per message, Redis via custom `IoAdapter`.
- Mobile: prefer WebSocket transport; check client/server compatibility tables.

Next: Chapter 10 — Testing, debugging and monitoring

CHAPTER 10

Testing, Debugging and Monitoring

> **Goal:** be able to prove your real-time code works (unit, integration, end-to-end and load tests), find out why it doesn't (debug logs, DevTools, Admin UI), and know what's happening in production (metrics, logs, traces, alerts).

10.1 ELI5

- **Testing** = practising the pizza phone call with a pretend customer before opening the shop.
- **Debugging** = recording the phone calls to find out where the order got mixed up.
- **Monitoring** = a big screen in the shop showing how many phones are ringing, how many calls dropped, and how long customers wait.

10.2 Unit testing handlers (no network)

Keep business logic in plain functions/services, and make handlers thin. Then unit-test handlers with fake sockets:

```
// handlers/chat.js
exports.makeSendMessage = ({ messages, io }) => async (socket, payload, ack) => {
  if (!payload?.text) return ack({ ok: false, error: { code: "VALIDATION" } });
  const msg = await messages.create(socket.data.user.id, payload);
  io.to(`conversation:${payload.conversationId}`).emit("message:new", msg);
  ack({ ok: true, data: { id: msg.id } });
};

// handlers/chat.test.js (jest/vitest style)
test("sends and broadcasts", async () => {
  const emit = vi.fn();
  const io = { to: vi.fn(() => ({ emit })) };
  const messages = { create: vi.fn(async () => ({ id: "m1" })) };
  const ack = vi.fn();
  const socket = { data: { user: { id: "u1" } } };

  await makeSendMessage({ messages, io })(socket, { conversationId: "c1", text: "hi" }, ack);

  expect(io.to).toHaveBeenCalledWith("conversation:c1");
  expect(emit).toHaveBeenCalledWith("message:new", { id: "m1" });
```

```
  expect(ack).toHaveBeenCalledWith({ ok: true, data: { id: "m1" } });
});
```

There are also community mocking libraries (e.g. `socket.io-mock`), but fake objects as above are usually enough.

10.3 Integration testing with a real server and real clients

Spin up a real server on a **random port** (`listen(0)`), connect real clients, assert on events. The test below was **run and passes** on socket.io 4.8.4 with Node's built-in test runner (`node --test`); the same structure works in Jest, Vitest, Mocha.

```
const { test, before, after } = require("node:test");
const assert = require("node:assert/strict");
const { createServer } = require("node:http");
const { Server } = require("socket.io");
const { io: ioc } = require("socket.io-client");

function waitFor(socket, event) {
  return new Promise((resolve) => socket.once(event, resolve));
}

let io, serverSocket, clientSocket, port;

before(async () => {
  const httpServer = createServer();
  io = new Server(httpServer);
  io.use((socket, next) => {
    if (socket.handshake.auth.token === "good") { socket.data.user = { id: "u1" }; return
next(); }
    const err = new Error("unauthorized"); err.data = { code: "AUTH" }; next(err);
  });
  await new Promise((resolve) => httpServer.listen(0, resolve));   // random free port
  port = httpServer.address().port;
  io.on("connection", (socket) => {
    serverSocket = socket;
    socket.on("sum", (a, b, ack) => ack(a + b));
    socket.on("join", (room, ack) => { socket.join(room); ack(); });
  });
  clientSocket = ioc(`http://localhost:${port}`, { auth: { token: "good" }, transports:
["websocket"] });
  await waitFor(clientSocket, "connect");
});

after(() => { io.close(); clientSocket.disconnect(); });   // ALWAYS clean up, or the runner
hangs
```

```
test("server → client event", async () => {
  const p = waitFor(clientSocket, "hello");   // start listening BEFORE triggering
  serverSocket.emit("hello", "world");
  assert.equal(await p, "world");
});

test("client → server with ack", async () => {
  const result = await clientSocket.timeout(1000).emitWithAck("sum", 2, 3);
  assert.equal(result, 5);
});

test("server → client with ack", async () => {
  clientSocket.once("question", (cb) => cb(42));
  const answer = await serverSocket.timeout(1000).emitWithAck("question");
  assert.equal(answer, 42);
});

test("rejects bad token", async () => {
  const bad = ioc(`http://localhost:${port}`, { auth: { token: "bad" }, transports:
["websocket"] });
  const err = await waitFor(bad, "connect_error");
  assert.equal(err.message, "unauthorized");
  assert.deepEqual(err.data, { code: "AUTH" });
  assert.equal(bad.active, false);           // middleware rejection → no auto-reconnect
  bad.disconnect();
});

test("room broadcast excludes sender", async () => {
  const other = ioc(`http://localhost:${port}`, { auth: { token: "good" }, transports:
["websocket"] });
  await waitFor(other, "connect");
  await clientSocket.emitWithAck("join", "r1");
  await other.emitWithAck("join", "r1");
  let selfGot = false;
  clientSocket.once("news", () => (selfGot = true));
  const p = waitFor(other, "news");
  io.of("/").sockets.get(clientSocket.id).to("r1").emit("news", "x"); // the SENDER's server-
side socket
  assert.equal(await p, "x");
  await new Promise((r) => setTimeout(r, 50));
  assert.equal(selfGot, false);
  other.disconnect();
});
```

> **A real lesson from writing this test:** the first version used the global `serverSocket` for the room test — but `serverSocket` had been **overwritten** when the second client connected, so it pointed to the *other* client's socket, and `socket.to("r1")` excluded the very client that was supposed to receive the event. The test hung forever. Moral: in tests (and in production code!), never keep "the current socket" in a shared variable; look sockets up by id (`io.of("/").sockets.get(id)`) or keep per-connection state inside the `connection` closure.

Integration testing tips

- `transports: ["websocket"]` in tests makes them faster and deterministic (no upgrade). Also test polling at least once if your users rely on it.
- Register the listener **before** triggering the action (race conditions otherwise).
- Use **timeouts** on acks in tests, so failures fail fast instead of hanging.
- Always `disconnect()` clients and `io.close()` the server in `after` / `afterAll`; otherwise Jest says "did not exit one second after the test run" / runners hang.
- Use `forceNew: true` when a test needs several independent clients to the same URL and namespace — or simply call `ioc()` again (a second `io()` to the same namespace creates a new Manager anyway — see Chapter 4).
- Testing **multi-node behaviour**: start 2 servers in the test with the Redis adapter (Redis via Docker/Testcontainers), connect clients to different ports, assert that a broadcast on server A reaches a client on server B.
- Testing **reconnection**: `io.close()` or `serverSocket.disconnect(true)` (note: a server-side `disconnect()` gives `io server disconnect` → no auto-reconnect; to simulate network loss use `serverSocket.conn.close()` → client sees `transport close` and reconnects).
- Testing **timeouts**: use fake timers carefully — Socket.IO uses real timers internally for heartbeats; prefer small real timeouts in tests.

10.4 End-to-end tests (browser)

Use Playwright/Cypress against the real app:

- Open **two browser contexts** (two users) and assert that a message sent in one appears in the other.
- Use `page.waitForEvent("websocket")` (Playwright) to inspect frames, or assert on UI.
- Simulate offline: `context.setOffline(true)` → assert "reconnecting..." UI → back online → assert recovery/sync.

```
// Playwright sketch
const alice = await browser.newContext(); const bob = await browser.newContext();
```

```
const pa = await alice.newPage(); const pb = await bob.newPage();
await Promise.all([pa.goto(url + "/chat/c1"), pb.goto(url + "/chat/c1")]);
await pa.fill("#input", "hello bob"); await pa.press("#input", "Enter");
await expect(pb.locator(".message").last()).toHaveText(/hello bob/);
```

10.5 Load testing

What to measure: **max concurrent connections**, **connection rate** (handshakes/s), **message throughput** (in and out, especially broadcast fan-out), **latency percentiles** (p50/p95/p99 of ack round-trip or emit→receive), **memory per connection**, **CPU**, **event loop lag**, **Redis throughput**, **reconnection storm behaviour** (kill a node and watch).

Tools:

- **Artillery** — has a built-in Socket.IO engine (`engine: socketio`), scenario files with `emit`, acks, think times. Very common in interviews and companies.

```
config:
  target: "https://rt.staging.example.com"
  phases: [{ duration: 300, arrivalRate: 50 }]   # 50 new virtual users per second for 5 min
  engines: { socketio: {} }
  socketio: { transports: ["websocket"] }
scenarios:
  - engine: socketio
    flow:
      - emit: { channel: "conversation:join", data: { conversationId: "load-1" } }
      - loop:
          - emit: { channel: "message:send", data: { conversationId: "load-1", text: "hi" } }
          - think: 5
        count: 20
```

- **Custom Node scripts** with `socket.io-client` (spawn N clients across several machines; a single Node process can simulate several thousand clients).
- **k6** speaks raw WebSocket; it doesn't implement the Socket.IO protocol natively (you'd have to hand-encode `40`, `42[...]`, answer pings with `3`) — prefer Artillery or custom scripts.

Load-testing pitfalls: the **load generator** becomes the bottleneck (CPU, ephemeral ports, file descriptors), so distribute it; test through the **real load balancer**; include realistic **idle** connections (most real connections are idle); test the **polling** path too if you allow it (much more expensive server-side).

10.6 Debugging

Debug logs (the `debug` module)

Socket.IO, Engine.IO and the client use the `debug` package — logs are off by default.

Server (Node):

```
DEBUG=socket.io* node server.js                # Socket.IO layer (server, namespace, socket,
parser, adapter)
DEBUG=engine,socket.io* node server.js         # + Engine.IO server
DEBUG=* node server.js                         # everything (very noisy)
```

Node client:

```
DEBUG=socket.io-client:*,engine.io-client:* node client.js
```

Browser client:

```
localStorage.debug = "socket.io-client:*";  // then reload the page
localStorage.debug = "*";                   // everything
```

Typical log lines: `socket.io:socket emitting packet with ack id 1`, `engine:socket transport is now writable`, `engine upgrade completed`.

> Note: the minified browser bundle (`socket.io.min.js`) strips debug logs; use the non-minified build or an npm-bundled dev build to see them.

Browser DevTools

1. **Network tab**, filter `socket.io`: - Handshake: `GET …?EIO=4&transport=polling` → response `0{"sid":…}`. If it's HTML/404/CORS error → path/URL/proxy/CORS problem. - WebSocket: `… transport=websocket` → status **101**. Click it → **Messages** tab shows every frame (`2`, `3`, `40`, `42[…]`). - Repeated polling requests and no 101 → WebSocket upgrade failing (proxy headers). - `400` with `{"code":1,"message":"Session ID unknown"}` → sticky sessions missing.
2. **Console**: CORS errors, `WebSocket connection to … failed`, mixed content (`ws://` from an `https://` page is blocked — use `wss://`).

Logging every event (temporary, dev/staging)

```
io.on("connection", (socket) => {
  socket.onAny((event, ...args) => logger.debug({ sid: socket.id, event, args }, "in"));
  socket.onAnyOutgoing((event, ...args) => logger.debug({ sid: socket.id, event }, "out"));
});

// client
socket.onAny((event, ...args) => console.debug("⬇", event, args));
socket.onAnyOutgoing((event, ...args) => console.debug("⬆", event, args));
```

Don't log full payloads in production (PII, volume).

Engine-level errors on the server

```
io.engine.on("connection_error", (err) => {
  logger.warn({ code: err.code, message: err.message, context: err.context, url: err.req?.url
}, "engine connection error");
});
```

The error code tells you the cause (0 Transport unknown, 1 Session ID unknown → sticky sessions, 3 Bad request, 4 Forbidden → `allowRequest`, 5 protocol mismatch).

A systematic debugging procedure (use in incidents)

1. **Can the client reach the server?** `curl https://host/socket.io/?EIO=4&transport=polling` → expect `0{…}`.
2. **Is it CORS?** browser console; check `cors.origin` exactly matches (scheme + host + port, no trailing slash).
3. **Is the path right?** client `path` == server `path`; URL path is the namespace.
4. **Versions compatible?** v2 client vs v4 server → `allowEIO3` or upgrade.
5. **Does middleware reject?** `connect_error` message/data; server logs.
6. **Does WebSocket upgrade?** DevTools 101; proxy `Upgrade` headers.
7. **Multi-node?** `Session ID unknown` → sticky sessions; events not reaching other node's clients → adapter misconfigured / different Redis `key`.
8. **Disconnect reason?** log `disconnect` reasons on both sides (`ping timeout` → event-loop blocking or network; `transport close` every N seconds → proxy idle timeout).
9. **Turn on DEBUG** on both sides and reproduce.

10.7 Admin UI (`@socket.io/admin-ui`)

A dashboard showing servers, connected sockets (with handshake details, rooms, data), rooms, events, and lets you join/leave/disconnect sockets.

```
const { instrument } = require("@socket.io/admin-ui");

const io = new Server(httpServer, {
  cors: { origin: ["https://admin.socket.io"], credentials: true },
});

instrument(io, {
  auth: {
    type: "basic",
    username: "admin",
    password: process.env.ADMIN_UI_BCRYPT_HASH,   // bcrypt hash ($2b$…; $2y$ not supported by bcrypt pkg)
  },
  mode: process.env.NODE_ENV === "production" ? "production" : "development", // production = less detail, less memory
  readonly: true,                                 // forbid join/leave/disconnect from the UI
  namespaceName: "/admin",                        // default; creates a namespace for the UI
  // serverId: `${os.hostname()}#${process.pid}`,     // distinct ids if several servers per machine
  // store: new RedisStore(redisClient),              // for basic auth across multiple servers
});
```

Then open https://admin.socket.io (a static site; it connects from your browser to your server) or self-host the UI files from the package's `ui/dist` folder. `auth` is **mandatory** (`auth: false` explicitly disables authentication — only in local dev). Works with v4 servers and in clusters (with an adapter). Security: it creates an `/admin` namespace on your server — protect it, consider disabling it in production, or expose it only on an internal network.

10.8 Monitoring in production

Key metrics (put these on a dashboard)

Metric	How to get it	Why
Connected clients (Engine.IO)	`io.engine.clientsCount`	Capacity; sudden drops = incident

Metric	How to get it	Why
Sockets per namespace	`io.of("/x").sockets.size` (local)	Feature usage
Connections/disconnections per second	counters in `connection` / `disconnect`	Reconnection storms
Disconnect reasons	counter labelled by `reason`	`ping timeout` spikes → event loop/network; `transport close` periodic → proxy timeouts
`connect_error` rate & messages	client telemetry; server middleware rejections	Auth problems, outages
Transport distribution	`socket.conn.transport.name`, `socket.conn.on("upgrade")`	% stuck on polling (proxy issues, cost)
Events in/out per second (by name)	`socket.onAny` / `onAnyOutgoing` counters	Traffic shape, abuse
Ack latency (p50/p95/p99)	measure time from receive to ack, and client-side emit→ack	User-perceived latency
Event loop lag	`perf_hooks.monitorEventLoopDelay()`	Root cause of ping timeouts and latency
Heap/RSS memory	`process.memoryUsage()`	Leaks (listeners, rooms, pending acks)
Redis adapter health	Redis latency, pub/sub channels, client output buffers	Cross-node delivery
Handler errors	counter in your `safe()` wrapper	Bugs

Prometheus example (`prom-client`)

```
const client = require("prom-client");

const connected = new client.Gauge({ name: "socketio_connected_clients", help: "Connected
Engine.IO clients" });
const disconnects = new client.Counter({ name: "socketio_disconnects_total", help:
"Disconnects", labelNames: ["reason"] });
const eventsIn = new client.Counter({ name: "socketio_events_in_total", help: "Incoming
events", labelNames: ["event"] });
const handlerDuration = new client.Histogram({
  name: "socketio_handler_duration_seconds", help: "Handler duration", labelNames: ["event"],
  buckets: [0.005, 0.01, 0.05, 0.1, 0.25, 0.5, 1, 2.5],
});

setInterval(() => connected.set(io.engine.clientsCount), 5000).unref();

io.on("connection", (socket) => {
  socket.on("disconnect", (reason) => disconnects.inc({ reason }));
  socket.onAny((event) => eventsIn.inc({ event }));   // event names must be a FIXED set (label
```

```
cardinality!)
});

app.get("/metrics", async (_req, res) => {
  res.set("Content-Type", client.register.contentType);
  res.end(await client.register.metrics());
});
```

> Label **cardinality**: never use socket ids, user ids or dynamic event names as metric labels — it explodes your time-series database. This is another reason event names must be static.

Tracing

OpenTelemetry has a Socket.IO instrumentation (`@opentelemetry/instrumentation-socket.io`) that creates spans for emits and received events. Propagate a **correlation id** in payloads (or in the ack envelope) to connect HTTP requests, queue messages and socket events in logs.

Structured logging

Log with context on every line: `socketId`, `userId`, `namespace`, `event`, `transport`, `ip` (careful with PII), `nodeId`/hostname. Log connection lifecycle at info level, events at debug level.

Client-side telemetry

Send to your analytics/RUM: time to connect, `connect_error` messages, reconnect counts, time spent disconnected, transport used, ack latency. Many real outages (corporate proxy blocking WebSocket, a CDN misconfiguration) are only visible from the client side.

Alerts (examples)

- Connected clients drop > 30% in 5 min.
- `ping timeout` disconnect rate > baseline × 3.
- Event loop lag p99 > 200 ms for 5 min.
- Handler error rate > 1%.
- Redis adapter errors / reconnects.
- `connect_error` rate from clients > X%.

10.9 Chapter summary

- Unit-test thin handlers with fakes; integration-test with a real server on port 0 and real clients; always clean up; register listeners before triggering.
- E2E with two browser contexts; load test with Artillery/custom clients through the real LB.
- Debug with `DEBUG=…`, `localStorage.debug`, DevTools Network/Messages, `onAny`, engine `connection_error` codes, and a systematic checklist.
- Admin UI for live inspection (protect it).
- Monitor connections, disconnect reasons, transports, event rates, ack latency, event loop lag, memory, Redis; keep metric labels low-cardinality.

Next: Chapter 11 — Performance and production

CHAPTER 11

Performance Tuning and Running in Production

> **Goal:** make Socket.IO fast and cheap, avoid memory leaks, deploy and shut down safely, evolve your event API without breaking old clients, and operate it like a senior engineer.

11.1 ELI5

A fast pizza shop: phones with good batteries (efficient WebSocket engine), short sentences (small payloads), one loudspeaker announcement instead of calling each customer (broadcast to rooms, encode once), not answering every tiny question immediately but grouping them (batching/throttling), no forgotten orders piling up in the corner (memory leaks), and a calm closing routine at night that tells customers "we're closing, call the other branch" (graceful shutdown).

11.2 Server-level performance tuning

1. Native add-ons for `ws`

```
npm install --save-optional bufferutil utf-8-validate
```

- `bufferutil` speeds up masking/unmasking of WebSocket frames.
- `utf-8-validate` speeds up UTF-8 validation of text frames (less important on recent Node versions that provide a fast built-in validator).
- They're optional: `ws` falls back to JS implementations if they're missing.

2. Use uWebSockets.js (v4.4+)

`uWebSockets.js` is a C++ HTTP/WebSocket server with much lower memory per connection and higher throughput than Node's `http` + `ws`.

```
const { App } = require("uWebSockets.js");   // installed from GitHub, not npm registry
const { Server } = require("socket.io");
```

```
const app = App();
const io = new Server();

io.attachApp(app);

io.on("connection", (socket) => { /* ... */ });

app.listen(3000, (token) => {
  if (!token) console.warn("port already in use");
});
```

Caveats: different HTTP API (Express middleware doesn't plug in directly), native dependency to build/ship, fewer people know it. Use when connection density/cost matters.

3. Alternative `wsEngine`

The `wsEngine` option lets you replace `ws` with a compatible implementation (e.g. a native one). Verify maintenance status before adopting any alternative engine.

4. Discard the initial HTTP request object

Each Engine.IO connection keeps a reference to its first HTTP request (`socket.request`), which holds headers, the parsed URL, etc. — memory per connection. If you don't need it after the handshake:

```
io.engine.on("connection", (rawSocket) => {
  rawSocket.request = null;
});
```

(Verified on 4.8.4: `socket.handshake.headers` is still available in your `connection` handler; only `socket.request` becomes `null` . Don't do this if you use `socket.request.session` / `socket.request.user` from `io.engine.use()` middleware.)

5. Compression trade-offs

- `perMessageDeflate` (WebSocket) is **off** by default. Turning it on reduces bandwidth for large compressible text but costs CPU and **significant memory per connection**. If you enable it, tune:

```
new Server(httpServer, {
  perMessageDeflate: {
    threshold: 2048,                // only compress messages larger than 2 KB
    zlibDeflateOptions: { level: 3 }, // cheaper compression
    serverNoContextTakeover: true,   // don't keep compression context between messages → less
memory
    clientNoContextTakeover: true,
```

```
  },
});
```

- `httpCompression` (polling) is on by default with a 1 KB threshold — fine.
- Use `socket.compress(false)` for already-compressed binary.

6. Parser choice

The default JSON parser is fast in V8. `socket.io-msgpack-parser` can reduce size for numeric/binary-heavy payloads. Measure with your real messages; switch only with evidence (and switch both client and server).

7. Heartbeat settings

Shorter `pingInterval` = more packets (tiny) and faster detection; longer = fewer wakeups. Defaults are reasonable; with 1M connections, 25 s interval = 40k pings/s per cluster — still fine. Don't go below a few seconds.

8. Node process settings

- Run **one process per CPU core** (cluster/PM2/containers) with sticky sessions + adapter.
- Set container memory limits and `--max-old-space-size` consistently.
- Keep the **event loop free**: CPU-heavy work (image processing, big JSON, crypto, PDF) → worker threads or other services. A blocked loop delays **every** socket and causes `ping timeout` storms.

11.3 Application-level performance

Encode once: broadcast instead of looping

```
// ❌ serializes N times, N writes through the full emit path
for (const s of await io.in("room").fetchSockets()) s.emit("update", bigPayload);

// ✅ encodes once, adapter fans out (and works across nodes)
io.to("room").emit("update", bigPayload);
```

Send less data

- Send **diffs/patches**, not entire objects (`{ id, field: newValue }`).

- Short property names only if traffic is huge (readability matters more usually).
- Don't broadcast large payloads to large rooms: broadcast a **small notification** ("doc 42 changed, version 17") and let clients fetch details via HTTP (cacheable, CDN).
- Strip server-only fields; use DTO projections.

Throttle, debounce, batch, coalesce

```
// client: throttle cursor updates to 20/s and use volatile
let last = 0;
document.addEventListener("mousemove", (e) => {
  const now = performance.now();
  if (now - last < 50) return;
  last = now;
  socket.volatile.emit("cursor", { x: e.clientX, y: e.clientY });
});
```

```
// server: coalesce many updates into one tick per room every 100 ms
const pending = new Map(); // room → latest state
function queueUpdate(room, state) { pending.set(room, state); }
setInterval(() => {
  for (const [room, state] of pending) io.to(room).volatile.emit("state", state);
  pending.clear();
}, 100);
```

Rooms over filtering

Let the adapter select recipients (rooms) instead of iterating sockets and checking conditions. Precompute rooms for common audiences (`org:1:admins`).

Avoid expensive per-event work

- Cache permission checks (e.g. "is user in conversation" cached in `socket.data` or Redis with TTL; invalidate on membership change).
- Avoid DB queries on very high-frequency events (typing, cursor).
- Batch DB writes for high-volume streams.

Backpressure / slow clients

A client on a bad network can't keep up; the server buffers outgoing packets for it (`socket.conn.writeBuffer`). For high-throughput streams:

- prefer `volatile` (drop when not writable),
- send snapshots instead of every delta,

- disconnect consumers whose buffers grow beyond a limit:

```
setInterval(() => {
  for (const [, s] of io.of("/").sockets) {
    if (s.conn.writeBuffer.length > 1000) s.disconnect(true);
  }
}, 10_000).unref();
```

11.4 Memory leaks — the usual suspects

Leak	Example	Fix
Listeners on **global** emitters per connection	`bus.on("price", (p) => socket.emit("price", p))` in `connection` handler, never removed	Remove on `disconnect`, or better: use rooms (`io.to("prices").emit`)
Timers per socket	`setInterval(() => socket.emit(...), 1000)`	`clearInterval` on `disconnect`
Maps of sockets/users never cleaned	`users.set(socket.id, …)` without delete	Delete on `disconnect` (and prefer rooms/Redis)
Pending acks without timeout	`socket.emit("x", cb)` to unresponsive clients	`socket.timeout(ms).emit(…)`
Client-side listener accumulation	`socket.on` inside React effect without `off` / inside `connect` handler	Cleanup / register once
Huge `socket.data`	storing user profile + permissions blob	Keep ids only
Connection state recovery buffer	high broadcast volume × 2-minute window	Lower `maxDisconnectionDuration`, use Streams adapter with `maxLen`
Dynamic namespaces never removed	tenants created on the fly	`cleanupEmptyChildNamespaces: true`
`perMessageDeflate`	zlib contexts per connection	Disable or tune
Retained `socket.request`	per-connection request objects	`rawSocket.request = null` (if unused)

```
// ✅ correct per-socket resource management
io.on("connection", (socket) => {
  const onPrice = (p) => socket.volatile.emit("price", p);
  priceBus.on("price", onPrice);
  const timer = setInterval(() => socket.emit("tick"), 5000);

  socket.on("disconnect", () => {
```

```
    priceBus.off("price", onPrice);
    clearInterval(timer);
  });
});
```

Diagnose with heap snapshots (Chrome DevTools via `node --inspect`, or `heapdump`), comparing two snapshots after many connect/disconnect cycles; look for retained `Socket` objects. Node's `MaxListenersExceededWarning` on a global emitter is a strong leak hint.

11.5 Graceful shutdown and deployments

When a server restarts, all its clients disconnect and reconnect (to other nodes, with back-off). Make it smooth:

```
let shuttingDown = false;

app.get("/health/ready", (_req, res) => res.status(shuttingDown ? 503 : 200).end());

async function shutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;
  logger.info({ signal }, "shutting down");

  // 1) give the load balancer time to stop sending NEW connections (readiness probe fails)
  await new Promise((r) => setTimeout(r, 10_000));

  // 2) optionally tell clients so they can show a notice / reconnect elsewhere with jitter
  io.emit("server:restarting");

  // 3) close Socket.IO: disconnects all sockets ("server shutting down") and closes the HTTP
server
  await io.close();

  // 4) close other resources
  await Promise.allSettled([pubClient.quit(), subClient.quit(), db.end()]);
  process.exit(0);
}

process.on("SIGTERM", () => shutdown("SIGTERM"));
process.on("SIGINT", () => shutdown("SIGINT"));
```

Notes:

- `io.close()` closes all client connections **and the underlying HTTP server** (returns a Promise in v4).

- Clients see `transport close` (or the reason relayed) and reconnect automatically with jitter. With **connection state recovery** + a shared store (Redis Streams adapter), `server shutting down` is a recoverable reason, so sessions can resume on another node.
- **Rolling deploys:** replace pods/instances gradually; each step moves ~1/N of the connections; watch connection counts before continuing.
- **Blue/green with long-lived connections:** old (blue) servers keep their connections until you drain them — plan a deliberate drain step.
- **Spread reconnection storms:** if you disconnect many clients at once, consider disconnecting in batches over a few seconds.

11.6 Evolving your event API (versioning & backward compatibility)

Mobile apps and long-open browser tabs run **old client versions** for weeks. Rules:

1. **Additive changes only** (new optional fields, new events) are safe.
2. **Never rename/remove** an event or required field without a deprecation period.
3. Introduce breaking changes as **new events** (`order:updated:v2`) or with a `v` field; emit both old and new during migration.
4. Send the **client version** at connection (`auth: { token, clientVersion }`); reject unsupported versions with a clear error:

```
io.use((socket, next) => {
  if (semver.lt(socket.handshake.auth.clientVersion ?? "0.0.0", MIN_CLIENT)) {
    const err = new Error("UPGRADE_REQUIRED");
    err.data = { minVersion: MIN_CLIENT };
    return next(err);
  }
  next();
});
```

5. Keep the event contracts in a **shared, versioned package** (TS types + runtime schemas) and/or an **AsyncAPI** specification (the OpenAPI equivalent for event-driven APIs) — generate docs from it.
6. Upgrading Socket.IO itself: v3 → v4 is non-breaking for clients; v2 → v4 requires `allowEIO3: true` during the migration window, then migrating clients.
7. When you deploy a new **web** client, old tabs keep the old JS: either keep backward compatibility, or emit `app:new-version` to prompt a reload.

11.7 Configuration per environment

```
const io = new Server(httpServer, {
  path: process.env.SIO_PATH ?? "/socket.io/",
  cors: { origin: process.env.WEB_ORIGINS.split(","), credentials: true },
  serveClient: false,
  pingInterval: Number(process.env.SIO_PING_INTERVAL ?? 25000),
  pingTimeout: Number(process.env.SIO_PING_TIMEOUT ?? 20000),
  maxHttpBufferSize: Number(process.env.SIO_MAX_BUFFER ?? 1e6),
  transports: (process.env.SIO_TRANSPORTS ?? "polling,websocket").split(","),
  connectionStateRecovery: process.env.SIO_CSR === "1" ? { maxDisconnectionDuration: 60_000 } :
undefined,
  adapter: await buildAdapter(),  // in-memory locally, Redis in staging/prod
});
```

Keep dev/staging/prod as similar as possible (same adapter type, same proxy) — many production-only bugs come from "it works on localhost" without a proxy or a second node.

11.8 Cost optimisation

- Idle connections cost memory: scale servers on **memory/connection count**.
- Prefer WebSocket-only for clients that can (polling costs many HTTP requests).
- Reduce message volume (throttle, coalesce, volatile, diffs).
- Consider uWebSockets.js for high connection density.
- Disconnect idle users after long inactivity (e.g. background tabs for > 30 min) and reconnect on visibility change — can cut concurrent connections dramatically.
- Managed alternatives (Azure Web PubSub for Socket.IO, Ably, Pusher) trade money for ops effort — evaluate total cost.

11.9 Production readiness checklist

Correctness - [ ] Handlers wrapped (sync + async errors), always ack commands. - [ ] Inputs validated, actions authorized, idempotency for retried commands. - [ ] State derived from DB/user on (re)connection; client re-syncs on `connect`. - [ ] Client listeners registered once, cleaned up in UI frameworks.

Scalability - [ ] Adapter configured (Redis/Streams/...) with error handlers and unique `key`. - [ ] Sticky sessions or WebSocket-only transport. - [ ] Proxy: HTTP/1.1 + Upgrade headers + timeouts > ping interval. - [ ] No local userId → socket maps; user rooms instead. - [ ] Load tested at 2–3× expected peak,

including reconnection storms.

Reliability - [ ] Graceful shutdown; readiness probe; rolling deploy strategy. - [ ] Reconnection back-off sane; jitter on. - [ ] Missed-message strategy (CSR and/or offset sync).

Security — see Chapter 6 checklist.

Observability - [ ] Metrics: connections, disconnect reasons, transports, event rates, ack latency, event loop lag, memory, errors. - [ ] Structured logs with socket/user ids; DEBUG toggle available. - [ ] Alerts and runbooks (Chapter 12).

11.10 Chapter summary

- Speed: `bufferutil`, uWebSockets.js via `attachApp`, discard `request`, careful compression, measured parser choice, keep event loop free.
- App-level: broadcast via rooms (encode once), smaller payloads, throttle/coalesce, volatile, backpressure handling.
- Leaks: global listeners, timers, maps, pending acks, client listener accumulation.
- Graceful shutdown with readiness, `io.close()`, staged drains; rolling deploys.
- Version your event API; clients lag behind.

Next: Chapter 12 — Troubleshooting

CHAPTER 12

Troubleshooting: Common Bugs, Symptoms, Causes and Fixes

> **Goal:** a field manual. Each entry: **Symptom → Likely causes → How to confirm → Fix.** Read it once fully; come back to it during incidents. Entries are grouped by area.

12.0 First-aid kit (do these first, every time)

1. **Log the reasons.** Server: `disconnect` reason + `io.engine.on("connection_error")` (code/message). Client: `connect_error` (`err.message` , `err.data` , `socket.active`) and `disconnect` reason.
2. **Open DevTools → Network → filter `socket.io`.** Look at the handshake response, the WebSocket (101?) and its Messages tab.
3. **`curl` the handshake:** `curl -i "https://host/socket.io/?EIO=4&transport=polling"` → must return `0{"sid":…}`.
4. **Enable debug logs:** `DEBUG=engine,socket.io*` (server), `localStorage.debug = "socket.io-client:*"` (browser).
5. **Check versions** of client and server (`npm ls socket.io socket.io-client`).
6. **Reproduce without the proxy** (directly against Node) to split app bugs from infrastructure bugs.

A. Connection problems

A1. Client can't connect at all — `xhr poll error` / `websocket error` in `connect_error`

- **Causes:** wrong URL/port; server not running; server attached to a different HTTP server (`app.listen` bug); firewall; HTTPS page connecting to `http://` / `ws://` (mixed content blocked); certificate errors.
- **Confirm:** curl the handshake; check the browser console for mixed-content/TLS errors.
- **Fix:** `httpServer.listen(...)` (the one passed to `new Server`); use `https://` ; correct URL; valid certificate.

A2. 404 on `/socket.io/?EIO=4…`

- **Causes:** `app.listen()` instead of `httpServer.listen()`; client `path` ≠ server `path`; proxy not forwarding `/socket.io/`; serving behind a sub-path (`/api`) without adjusting `path`; Next.js/Vite dev server receiving the request instead of the Socket.IO server.
- **Fix:** listen on the correct server; align `path` on both sides (e.g. `path: "/api/socket.io/"`); proxy rule for the path; in Vite dev, configure `server.proxy` with `ws: true` for `/socket.io`.

A3. CORS error in the browser

```
Access to XMLHttpRequest at '…/socket.io/?EIO=4&transport=polling…' from origin '…' has been
blocked by CORS policy
```

- **Causes:** no `cors` option (v3+ default is no CORS); origin mismatch (`http` vs `https`, port, trailing slash `https://app.com/` ≠ `https://app.com`); `credentials: true` with `origin: "*"`; client `withCredentials: true` without server `credentials: true`.
- **Fix:** `cors: { origin: ["https://app.example.com"], credentials: true }` exactly matching; or serve frontend and API on the same origin. Note: a CORS error can also be a **masked 4xx/5xx** from a proxy that doesn't add CORS headers on errors — check the actual status.

A4. `connect_error: "Invalid namespace"`

- **Cause:** the client connects to a namespace the server doesn't define — often by accident: `io("https://api.example.com/api")` thinking `/api` is a path.
- **Fix:** define `io.of("/api")`, or use `io("https://api.example.com", { path: "/api/socket.io/" })`.

A5. `connect_error` with your middleware's message (e.g. `"unauthorized"`)

- **Cause:** `io.use` called `next(err)`.
- **Behaviour:** `socket.active` `===` `false` → **no automatic retry**.
- **Fix:** refresh credentials then `socket.connect()`; show login. Check token presence/format (`auth` vs `query`), clock skew for JWT `exp` / `nbf`, algorithm/secret mismatch between services.

A6. Client stays "connecting" forever, no error, then reconnects every ~45 s

- **Cause:** a connection middleware never calls `next()` in some code path (e.g. an `await` that never resolves, a missing `return next()` after a condition, an exception in async middleware that's swallowed).

- **Confirm:** server debug logs show the handshake but no `connection`; the raw connection is closed after `connectTimeout` (45 s).
- **Fix:** ensure every path calls `next()` / `next(err)`; wrap async middleware in try/catch.

A7. `connect_error: "timeout"`

- **Cause:** the connection attempt exceeded the client `timeout` (20 s): server overloaded (event loop blocked), network very slow, proxy buffering.
- **Fix:** investigate server load/event loop lag; proxy buffering off; increase `timeout` only if the network is genuinely slow.

A8. `{"code":5,"message":"Unsupported protocol version"}` / client says "v2.x server with a v3.x client"

- **Cause:** version mismatch. v2 client → v3/v4 server needs `allowEIO3: true`. v3/v4 client → v2 server is impossible (upgrade the server).
- **Fix:** align versions; use `allowEIO3` temporarily during migrations. Third-party clients (Swift/Java/Flutter/Python/Go): check their compatibility table.

A9. `{"code":0,"message":"Transport unknown"}`

- **Cause:** client requests a transport disabled on the server (e.g. server `transports: ["websocket"]` but client starts with polling).
- **Fix:** align `transports` on both sides.

A10. Works locally, fails in production

- **Usual suspects:** proxy WebSocket headers; sticky sessions; CORS with the real domain; TLS; `path` behind a prefix; CDN; corporate proxy at the client; different env vars.
- **Fix:** go through the first-aid kit against production; compare proxy configs.

A11. Works in the browser, fails from a mobile app

- **Causes:** client library version incompatible with server protocol; cleartext HTTP blocked on Android/iOS (ATS); `localhost` used on a device; self-signed certs; polling issues on React Native.
- **Fix:** compatible client version; HTTPS; real host/IP; `transports: ["websocket"]` on RN.

B. Transport / upgrade / proxy problems

B1. Never upgrades to WebSocket (stays on polling)

- **Symptoms:** DevTools shows a polling request every ~25 s and no `transport=websocket` 101; higher latency; `socket.io.engine.transport.name === "polling"`.
- **Causes:** proxy missing `proxy_http_version 1.1` + `Upgrade` / `Connection` headers; load balancer without WebSocket support; corporate proxy/antivirus; `upgrade: false` or `transports: ["polling"]` somewhere; server `allowUpgrades: false`.
- **Fix:** proxy configuration (Chapter 8); verify with `wscat` / `websocat` directly.

B2. Random `400 Bad Request` with `{"code":1,"message":"Session ID unknown"}`

- **Cause: missing sticky sessions** with multiple nodes (polling requests reaching a node that didn't create the session), or the node restarted.
- **Fix:** enable sticky sessions (cookie/IP hash, `@socket.io/sticky` for cluster/PM2), or `transports: ["websocket"]`. With ALB cookies cross-origin → `withCredentials: true`.

B3. Clients disconnect every 60 s (or another fixed interval) with `transport close`

- **Cause:** a proxy/LB **idle timeout** or `proxy_read_timeout` lower than the traffic interval, or a misconfigured heartbeat (`pingInterval` raised above the LB timeout).
- **Fix:** proxy timeouts > `pingInterval` (e.g. 120 s+); keep defaults for heartbeats.

B4. WebSocket works but some users can't connect at all

- **Cause:** you forced `transports: ["websocket"]` and those users are behind WebSocket-blocking networks.
- **Fix:** allow polling fallback (`["websocket", "polling"]` + fallback on `connect_error`, or `tryAllTransports: true` in 4.8+), with sticky sessions.

B5. Mixed content: `ws://` blocked on `https://` pages

- **Fix:** use `https://` URL in `io()` (the client derives `wss://`), terminate TLS at the proxy.

B6. `WebSocket is closed before the connection is established`

- **Causes:** the socket disconnected during connection (component unmounted and `socket.disconnect()` called — e.g. React StrictMode double effect), proxy rejected the upgrade, server crashed. Mostly harmless in dev StrictMode.
- **Fix:** don't create/destroy the socket in effects that StrictMode double-invokes, or accept the dev-only warning; check proxy upgrade config.

C. Events not received / received wrongly

C1. Events are received 2×, 3×, N× (duplicates growing over time)

- **Causes (by frequency):** 1. Listener registered **inside** `socket.on("connect")` → one extra listener per reconnection. 2. React/Vue component registering `socket.on` without `socket.off` cleanup (or with a *different* function reference) → one extra per mount/re-render. 3. Multiple `io()` calls (each creates a new connection) → each connection gets the event. 4. Server registers handlers in a loop or per-request (e.g. `io.on("connection")` inside an Express route). 5. Client-side retries (`retries` option) without server idempotency.
- **Confirm:** `socket.listeners("event").length`; count connections in DevTools.
- **Fix:** register once / cleanup with the same reference; single socket instance; dedupe by message id.

C2. Client never receives an event the server "sent"

- Event name **typo / case mismatch** (`"chatMessage"` vs `"chat:message"`). → shared typed event map.
- Emitting to the **wrong namespace** (`io.emit` = main namespace; the client is on `/chat`).
- Emitting to a **room the socket isn't in** (join failed/authorization; or socket reconnected → new socket not re-joined).
- Using `socket.to(room)` when the **sender itself** should receive it (`socket.to` excludes the sender).
- Emitting **before the client registered** its listener (e.g. emitted in `connection` handler, but the client attaches `socket.on("welcome")` asynchronously later) → register listeners before connecting (`autoConnect: false` then `connect()`), or have the client request the data (`emitWithAck("init")`).
- **Multi-node without adapter**: the target client is on another server.
- Different Redis `key` prefix between nodes / emitter.
- The client was **disconnected** at that moment (at-most-once) → sync strategy.
- `volatile` packet dropped.
- Payload > `maxHttpBufferSize` → the connection was closed instead.

C3. Server never receives a client event

- **Handlers registered after an `await` in an `async` connection handler.** The client emits right after `connect`; the packet arrives while the server is still awaiting (DB call), finds no listener and is **silently dropped** (verified on 4.8.4):

```
// ❌ events sent during the await are lost
io.on("connection", async (socket) => {
  const rooms = await db.loadRooms(socket.data.user.id);
  socket.on("hello", handler);
});
// ✅ register synchronously first, then do async work (or do it in io.use middleware)
io.on("connection", (socket) => {
  socket.on("hello", handler);
  db.loadRooms(socket.data.user.id).then((rooms) => socket.join(rooms));
});
```

- Handler registered on the wrong namespace's connection handler.
- Client emitting before connection *and* `volatile` (dropped) — or emits buffered but page reloaded before reconnecting.
- A `socket.use()` middleware not calling `next()`, or calling `next(err)` (check the server socket's `error` event).
- Rate limiter dropping silently.
- Event name is reserved → `emit` threw on the client (check console).

C4. The ack callback never fires

- Receiver handler doesn't call the callback in some path (error path!), or the handler threw before calling it.
- Receiver expected the callback at a different argument position (e.g. client sent `emit("x", cb)` but server handler is `(data, cb)` → `data` is the callback, `cb` undefined → `cb is not a function`).
- Middleware dropped the event.
- Connection lost in between.
- **Fix:** always use `timeout()` / `ackTimeout` on the sender; validate `typeof ack === "function"` on the receiver; wrap handlers so errors still ack.

C5. Ack callback's first argument is an Error/null unexpectedly

- **Cause:** you used `.timeout()` — callback signature becomes `(err, ...responses)`.
- **Fix:** `(err, response) => {}` with timeout; `(response) => {}` without.

C6. Broadcast ack returns immediately with "operation has timed out"

- **Cause:** `io.emit("x", cb)` / `io.to(room).emit("x", cb)` **without** `.timeout()`.
- **Fix:** `io.timeout(5000).emit("x", (err, responses) ⇒ …)`.

C7. Received data looks wrong (Date is a string, Map is `{}`, BigInt error)

- **Cause:** JSON serialization rules (Chapter 3).
- **Fix:** convert explicitly; send ISO strings/epoch numbers; arrays for Map/Set; strings for BigInt.

C8. Events arrive in the "wrong order" in processing

- **Cause:** async handlers completing out of order; or events from different sockets/ nodes (no global ordering).
- **Fix:** per-key queue; sequence numbers; server-assigned versions.

C9. "Cannot emit reserved event" error

- **Cause:** `socket.emit("connect" | "disconnect" | "connect_error" | "disconnecting" | "newListener" | "removeListener")`.
- **Fix:** rename your event.

D. Disconnections and reconnection

D1. Mass `ping timeout` disconnections

- **Causes: event loop blocked** on the server (sync CPU work, huge JSON, GC pauses from memory pressure); overloaded Node; network congestion; client devices sleeping (normal in small amounts).
- **Confirm:** event loop lag metric spikes at the same time; CPU 100%.
- **Fix:** move CPU work off the loop; scale out; reduce message volume; fix memory pressure.

D2. Client doesn't reconnect after disconnect

- **Cause:** reason `io server disconnect` (server called `disconnect()`), `io client disconnect`, or middleware rejection — auto-reconnect is intentionally disabled.

- **Fix:** handle those cases explicitly (`socket.connect()` after fixing auth).

D3. `socket.on("reconnect")` never fires

- **Cause:** v3+ moved it to the Manager.
- **Fix:** `socket.io.on("reconnect", …)` , or simply use `socket.on("connect")` (fires on every successful connection, including reconnections).

D4. After reconnection, the user no longer receives room messages

- **Cause:** a reconnection creates a **new server-side socket** with no rooms; you joined rooms only on explicit client actions.
- **Fix:** re-join rooms in `connection` based on the user (DB), or have the client re-subscribe on every `connect` ; or enable connection state recovery (and still handle `recovered === false`).

D5. Reconnection storm after a deploy/outage (servers overloaded, crash loop)

- **Cause:** all clients reconnect simultaneously; expensive handshake (auth DB queries, loading rooms).
- **Fix:** keep jitter; increase `reconnectionDelayMax` for large fleets; make connection setup cheap (cache, JWT instead of DB lookups); rate-limit handshakes at the edge; scale ahead of deploys; roll deploys slowly; CSR with `skipMiddlewares` .

D6. `socket.id` changed / server lost my identity

- **Cause:** ids change on reconnection (expected).
- **Fix:** use authenticated user ids and user rooms.

D7. A flood of old events hits the server when a client comes back online

- **Cause:** client `sendBuffer` accumulated emits while offline.
- **Fix:** `volatile` for droppable events; `if (socket.connected)` guards; clear `socket.sendBuffer` on reconnect when stale.

D8. Presence shows users "online" long after they left (or flickers)

- **Causes:** dead connections only detected after `pingInterval + pingTimeout` ; presence tracked per socket instead of per user (multiple tabs); node crash left stale Redis entries; immediate "offline" on brief reconnections → flicker.

- **Fix:** count sockets per user (online if ≥ 1); TTL-based heartbeats in Redis; grace period (e.g. 5–10 s) before broadcasting "offline" (Chapter 13).

E. Scaling / multi-node problems

E1. Broadcasts reach only some users

- **Cause:** no adapter (in-memory per node), adapter not applied to the namespace in use, Redis connection broken, different `key` prefixes, emitter/adapter mismatch.
- **Confirm:** connect two clients to two different nodes and test; check Redis `PUBSUB CHANNELS socket.io*`.
- **Fix:** configure the adapter on every node; same `key`; Redis error handlers.

E2. `fetchSockets()` is slow or times out

- **Cause:** one node slow/dead → waits `requestsTimeout` (5 s); huge numbers of sockets.
- **Fix:** don't use on hot paths; maintain presence/counters in Redis; tune timeout.

E3. Redis adapter suddenly stops delivering cross-node messages

- **Causes:** subscriber disconnected (and not resubscribed), Redis `client-output-buffer-limit pubsub` exceeded → Redis kills slow subscribers; network partition.
- **Fix:** monitor/alert on Redis client reconnects; raise limits carefully; consider the Redis Streams adapter (resumes from offset).

E4. Process crashes with an unhandled `'error'` event from Redis

- **Fix:** `pubClient.on("error", …)` and `subClient.on("error", …)`.

E5. Uneven load across nodes

- **Cause:** IP-hash stickiness behind NAT (corporate networks); long-lived connections stay where they started after scale-out.
- **Fix:** cookie-based stickiness; least-connection balancing for new connections; periodic gentle rebalancing (disconnect a fraction of clients on overloaded nodes).

F. Performance and memory

F1. Memory grows steadily until OOM

- **Causes:** see Chapter 11 leak table (global listeners, timers, maps, pending acks, perMessageDeflate, CSR buffers, dynamic namespaces).
- **Confirm:** heap snapshots; `MaxListenersExceededWarning`; memory vs connection count correlation (if memory grows while connections are flat → leak).
- **Fix:** clean up on `disconnect`; timeouts on acks; disable/tune compression.

F2. High CPU with few messages

- **Causes:** heavy compression, huge payloads JSON-encoded often, many polling clients (HTTP overhead), TLS in Node, logging every event at info level, `fetchSockets` loops.
- **Fix:** profile (`node --cpu-prof`, clinic.js flame graphs) and fix hot spots.

F3. Latency spikes

- **Causes:** event loop blocking, GC, Redis latency, big broadcasts, Nagle/proxy buffering.
- **Fix:** measure event loop lag; split work; `proxy_buffering off`; smaller messages.

F4. Server closes the connection when sending a file/large message

- **Cause:** payload > `maxHttpBufferSize` (1 MB default).
- **Fix:** increase deliberately, chunk with acks, or use HTTP uploads.

G. Framework-specific

G1. React: multiple connections in DevTools

- **Cause:** `io()` inside a component body or in an effect without cleanup; StrictMode double-invoking effects; hot module reload creating new sockets.
- **Fix:** module-level singleton or context; cleanup with `disconnect()` if you create in effects.

G2. React: handler sees stale state

- **Cause:** closure captured old state when the listener was registered.

- **Fix:** functional `setState(prev ⇒ …)`, refs, or re-register on deps change with proper cleanup.

G3. Angular: UI not updating on socket events

- **Cause:** handler running outside Angular zone.
- **Fix:** `NgZone.run`, signals, or `ChangeDetectorRef.markForCheck()`.

G4. Next.js: "window is not defined" / socket created on the server

- **Fix:** client components only, create sockets in effects or lazily; don't import the client socket in server components.

G5. Jest: "did not exit one second after the test run has completed"

- **Fix:** disconnect clients and `io.close()` in `afterAll`; `forceExit` is a last resort.

G6. NestJS: guard doesn't run on connection

- **Cause:** guards run per message, not on the connection handshake.
- **Fix:** authenticate in `handleConnection` or with `server.use()` middleware in `afterInit`.

H. Security incidents

H1. Someone receives other users' messages

- **Causes:** room join not authorized (client asked to join any room); identity taken from payload; `io.emit` instead of `io.to(room)`; room name collision (`"1"` used for both user 1 and conversation 1 → prefix rooms by type: `user:1`, `conv:1`).
- **Fix:** authorize joins; prefixes; audit emits.

H2. Abuse / spam floods

- **Fix:** per-user Redis rate limits; connection caps per user/IP; edge rate limiting; disconnect offenders; reduce `maxHttpBufferSize`.

H3. Server crashes on a malicious packet

- **Fix:** update `socket.io`, `engine.io`, `socket.io-parser`, `ws` (historic CVEs); handlers wrapped; process manager auto-restart; alerting.

12.9 Incident runbook template (copy to your wiki)

```
Title: Realtime connections dropping / not connecting
1. Impact: % of clients affected? (connected clients graph, connect_error rate)
2. Recent changes? (deploys of app/proxy/LB/Redis, certificate renewals, config)
3. Check disconnect reasons distribution (ping timeout / transport close / server disconnect)
4. Check event loop lag, CPU, memory per node
5. Check Redis health (latency, connected clients, pubsub buffers)
6. Check LB/proxy error logs (502/400 Session ID unknown/timeouts)
7. curl the handshake from outside; test WebSocket with wscat/websocat
8. Mitigate: roll back / scale out / raise timeouts / disable heavy feature flag
9. Post-incident: add the missing metric/alert, update this runbook
```

12.10 Chapter summary

Most real-world Socket.IO problems come from a short list: **wrong server listened on, CORS, path vs namespace confusion, proxy WebSocket headers/timeouts, missing sticky sessions, missing adapter, listeners registered repeatedly, rooms lost on reconnection, acks without timeouts, blocked event loop, and version mismatches.** Know these by heart and you can fix 95% of issues in minutes.

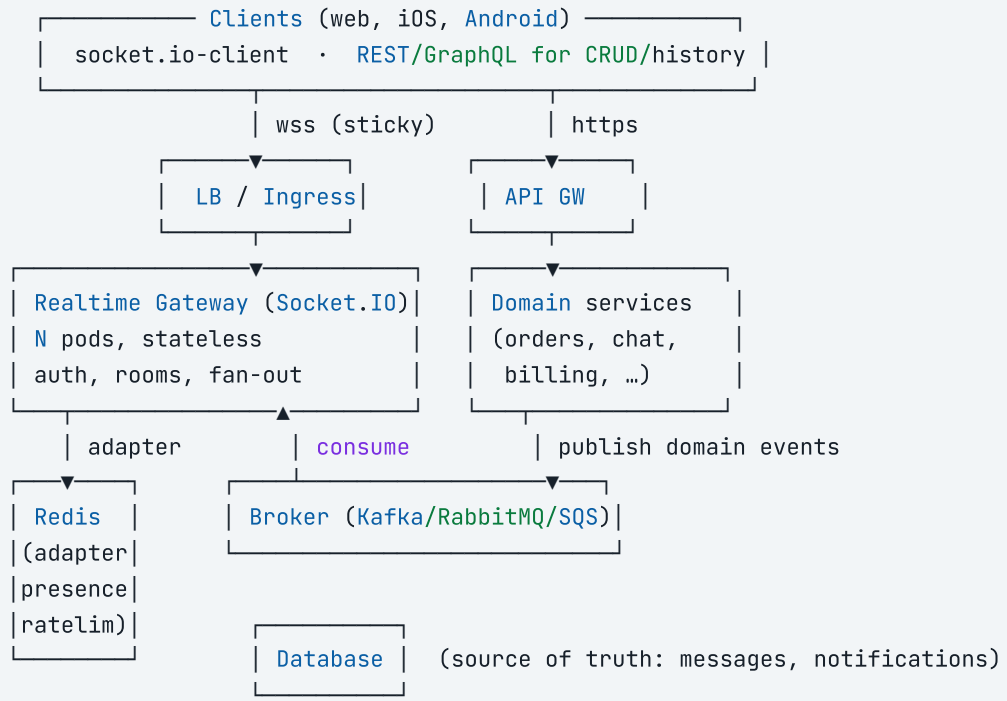
Next: Chapter 13 — Architecture, patterns and projects

CHAPTER 13

Architecture, Patterns, Organisational Best Practices and Full Projects

> **Goal:** design real-time features the way experienced teams do: proven patterns (presence, typing, read receipts, notifications, live data, collaboration, games, tracking, uploads, progress), a reference architecture for big organisations, team standards, and a "what to do when..." decision guide.

13.1 Reference architecture (large organisation)



Principles:

1. **The gateway owns connections, not business logic.** It authenticates, authorizes subscriptions (room joins), validates, and fans out. Domain services publish events to a broker; the gateway consumes and emits to rooms (or services use an emitter).
2. **Database is the source of truth.** Sockets deliver *notifications of change*; clients can always re-sync via REST/acks.

3. **Commands via socket or HTTP, both fine** — but commands must be idempotent and acknowledged. Many teams send *writes via* HTTP (easier caching, auth, retries, observability) and use sockets only for *server → client push*. Both are valid; be consistent.
4. **Stateless gateway pods** + adapter + sticky sessions → horizontal scaling.
5. **Contracts** (event names, payload schemas) are versioned and shared.

13.2 Pattern: Presence (online/offline) — robust version

Requirements: a user is online if **any** of their devices is connected; no flicker on brief reconnections; correct across many nodes; self-healing after crashes.

```
// Redis keys:
//   presence:conn:<userId>  -> sorted set of connectionIds, score = lastSeen (ms)
const GRACE_MS = 10_000;     // wait before announcing "offline"
const STALE_MS = 60_000;     // connections not refreshed for this long are dead (crashed node)

io.on("connection", async (socket) => {
  const userId = socket.data.user.id;
  const connKey = `presence:conn:${userId}`;
  const connId = `${process.env.HOSTNAME}:${socket.id}`;

  const wasOnline = (await redis.zCard(connKey)) > 0;
  await redis.zAdd(connKey, { score: Date.now(), value: connId });
  if (!wasOnline) io.to(`presence:${userId}`).emit("presence:update", { userId, online: true
});

  // heartbeat to refresh lastSeen (crash safety)
  const hb = setInterval(() => redis.zAdd(connKey, { score: Date.now(), value: connId }),
20_000);

  socket.on("disconnect", async () => {
    clearInterval(hb);
    await redis.zRem(connKey, connId);
    setTimeout(async () => {
      await redis.zRemRangeByScore(connKey, 0, Date.now() - STALE_MS); // drop stale entries
      if ((await redis.zCard(connKey)) === 0) {
        await redis.set(`presence:lastSeen:${userId}`, Date.now());
        io.to(`presence:${userId}`).emit("presence:update", { userId, online: false });
      }
    }, GRACE_MS);
  });

  // subscribe to presence of contacts (authorized!)
  socket.on("presence:subscribe", async ({ userIds }, ack) => {
    const allowed = await contacts.filterVisible(userId, userIds.slice(0, 500));
    await socket.join(allowed.map((id) => `presence:${id}`));
    const states = await Promise.all(allowed.map(async (id) => ({
```

```
      userId: id, online: (await redis.zCard(`presence:conn:${id}`)) > 0,
    })));
    ack?.({ ok: true, data: states });
  });
});
```

Key ideas to explain in interviews: **count connections per user**, **grace period**, **TTL/heartbeat for crash safety**, **subscribe to presence rooms** instead of broadcasting every presence change to everyone ($O(N^2)$ problem), **authorize** who can see whose presence, **last seen** timestamp.

13.3 Pattern: Typing indicator

```
// client: emit at most every 2 s while typing; stop event when idle
let typingTimer, lastSent = 0;
input.addEventListener("input", () => {
  const now = Date.now();
  if (now - lastSent > 2000) {
    socket.volatile.emit("typing:start", { conversationId });
    lastSent = now;
  }
  clearTimeout(typingTimer);
  typingTimer = setTimeout(() => socket.volatile.emit("typing:stop", { conversationId }),
3000);
});

// server: relay to others in the conversation (membership checked)
socket.on("typing:start", ({ conversationId }) => {
  if (!socket.rooms.has(`conversation:${conversationId}`)) return;
  socket.to(`conversation:${conversationId}`).volatile.emit("typing", { conversationId, userId:
socket.data.user.id, typing: true });
});

// receiving client: auto-expire the indicator after 5 s without refresh (in case "stop" is
lost)
```

Properties: **volatile** (losing one is fine), **throttled**, **self-expiring** on the receiver, **excludes sender**, no DB writes.

13.4 Pattern: Chat message flow with optimistic UI, acks and dedupe

```
Client                                    Server                                  Others
  | render "sending…" (clientMsgId)         |                                       |
  |-- message:send {clientMsgId,text} ---->| validate, authorize                    |
  |                                         | INSERT … ON CONFLICT(clientMsgId) DO NOTHING
(idempotent)
  |                                         |-- io.to(conv).emit(message:new) --->| render
  |<-- ack {ok, id, createdAt} ------------|                                       |
  | replace temp message with server data   |                                       |
  | (also receives message:new → dedupe by id/clientMsgId)                          |
  | on timeout → retry with SAME clientMsgId (safe) or show "tap to retry"          |
```

- **Ordering in the conversation**: server-assigned monotonic ids/timestamps; clients sort by server order, not by arrival.
- **Missed messages**: on `connect`, `sync` from last known id (Chapter 5).
- **Read receipts**: client emits `conversation:read { conversationId, upToMessageId }` (throttled); server stores per-user pointer and emits `receipt:update` to the room — store a **pointer** per user, not a flag per message.
- **Delivery receipts** ("delivered" ✓✓): the recipient client acks receipt or reports its latest received id on sync.
- **Unread counts**: computed from the read pointer; push `unread:update` to `user:<id>`.

13.5 Pattern: Notifications system

1. Service creates a notification → **persist** in DB (`notifications` table with `user_id`, `read_at`).
2. Publish event → gateway → `io.to("user:" + userId).emit("notification:new", n)`.
3. Clients online get it instantly; offline users see it on next load (REST `GET /notifications?unread=true`) and/or via **push notifications** (FCM/APNs/Web Push) — Socket.IO doesn't reach closed apps.
4. Mark as read via event or REST; broadcast `notification:read` to the user's other devices (`user:<id>` room) so badges sync across tabs.
5. Don't push to users who aren't connected through sockets *and* via push for the same notification without dedupe on the client.

13.6 Pattern: Live data subscriptions (dashboards, tickers, sports scores)

```
socket.on("ticker:subscribe", async ({ symbols }, ack) => {
  const allowed = symbols.filter(isValidSymbol).slice(0, 50);   // cap per socket
  await socket.join(allowed.map((s) => `ticker:${s}`));
  ack?.({ ok: true, data: await snapshot(allowed) });          // initial snapshot
});
socket.on("ticker:unsubscribe", ({ symbols }) => symbols.forEach((s) =>
socket.leave(`ticker:${s}`)));

// market data consumer (one per node or via a single publisher + adapter)
feed.on("tick", (t) => latest.set(t.symbol, t));
setInterval(() => {                                              // coalesce to 4 updates/s
  for (const [symbol, t] of latest) io.to(`ticker:${symbol}`).volatile.emit("tick", t);
  latest.clear();
}, 250);
```

Patterns: **snapshot + deltas**, **subscription rooms**, **coalescing**, **volatile**, **caps**. If every node consumes the full feed itself, emit with `io.local` to avoid re-broadcasting through the adapter (each node delivers to its own sockets).

13.7 Pattern: Collaborative editing

Naively relaying edits causes conflicts. Use a convergence algorithm:

- **OT (Operational Transformation)** — Google Docs style; server transforms concurrent ops (e.g. ShareDB).
- **CRDTs** — e.g. **Yjs** or **Automerge**: every client applies updates in any order and converges. Socket.IO can be the transport (e.g. the `y-socket.io` provider, or relay binary Yjs updates yourself).

Socket.IO responsibilities: rooms per document (`doc:<id>`), authorization, relaying binary updates (`socket.to(room).emit("doc:update", update)`), awareness (cursors — volatile, throttled), persistence of document state (periodically snapshot to DB), initial sync on join (state vector exchange in Yjs). Mention **binary** payloads and **ordering per doc**.

13.8 Pattern: Multiplayer game (server-authoritative)

```
// server: authoritative simulation at a fixed tick rate
const TICK_MS = 50; // 20 Hz
setInterval(() => {
  for (const match of matches.values()) {
    match.applyInputs();               // inputs queued from clients
    match.step(TICK_MS);
    io.to(`match:${match.id}`).volatile.compress(false).emit("state", match.snapshot()); //
compact snapshot
  }
}, TICK_MS);

socket.on("input", ({ seq, keys }) => {
  const match = matches.get(socket.data.matchId);
  match?.queueInput(socket.data.playerId, seq, keys);  // validate: rate, legal moves (anti-
cheat)
});
```

- Clients send **inputs**, not positions (anti-cheat); server validates.
- Clients do **prediction + reconciliation** using input sequence numbers, and **interpolation** of other players between snapshots.
- `volatile` for state snapshots; binary/msgpack for compactness.
- One match = one room; matchmaking assigns players to a node (or route all players of a match to the same node — sharding by match id — to keep simulation local).
- For twitchy shooters, TCP-based transports (WebSocket) suffer head-of-line blocking; consider WebRTC data channels / WebTransport datagrams.

13.9 Pattern: Live location tracking (delivery/ride apps)

- Driver app emits location every N seconds (adaptive: faster when moving), **volatile**, throttled; server validates (speed sanity), stores **latest** in Redis (GEOADD) and appends to a history store in batches.
- Riders/customers join `trip:<id>` (authorized) and receive throttled updates.
- Dispatchers subscribe to geo areas (`geo:<geohash-prefix>` rooms) — update the driver's area room membership when it moves across cells.
- Background location on mobile requires native background modes + HTTP fallback (sockets die in background).

13.10 Pattern: Chunked file upload with flow control

```
// client
async function upload(file) {
  const CHUNK = 256 * 1024; // < maxHttpBufferSize
  const { uploadId } = await socket.timeout(5000).emitWithAck("upload:start", { name:
file.name, size: file.size, type: file.type });
  for (let offset = 0; offset < file.size; offset += CHUNK) {
    const chunk = await file.slice(offset, offset + CHUNK).arrayBuffer();
    await socket.timeout(10000).emitWithAck("upload:chunk", { uploadId, offset, chunk }); //
ack = backpressure
    onProgress(Math.min(1, (offset + CHUNK) / file.size));
  }
  return socket.timeout(10000).emitWithAck("upload:end", { uploadId });
}

// server
socket.on("upload:chunk", async ({ uploadId, offset, chunk }, ack) => {
  const u = uploads.get(uploadId);
  if (!u || u.owner !== socket.id || offset !== u.received) return ack({ ok: false, error:
"BAD_STATE" });
  await u.stream.write(chunk);    // Buffer
  u.received += chunk.length;
  ack({ ok: true });
});
```

Each chunk waits for its ack → natural **backpressure**; offsets allow **resume** after reconnection. For large files, HTTP (multipart/resumable/pre-signed S3 URLs) is usually better; use this pattern when you need upload over an existing socket.

13.11 Pattern: Long-running job progress

1. Client starts the job via HTTP → gets `jobId`.
2. Client emits `job:subscribe { jobId }` (authorized) → joins `job:<jobId>`; ack returns current status (so a late subscriber isn't stuck).
3. Workers publish progress via **emitter**: `emitter.to("job:" + id).emit("job:progress", { pct })`, throttled (e.g. at most every 500 ms).
4. On completion: `job:done` with result URL; job status persisted so a reconnecting client can query it.

13.12 Pattern: Auctions / inventory / anything with races

- Server is the authority; clients send **bids** (commands with ack).
- Use **atomic DB operations** (`UPDATE … SET price = $1 WHERE id = $2 AND price < $1` or Redis Lua scripts) — never "read, compare in JS, write" across nodes.
- Broadcast the **accepted** state with a version number; clients ignore older versions.
- Server time for deadlines (send `serverNow` so clients can compute clock offset).
- Anti-sniping extensions handled server-side.

13.13 Pattern: Multi-tenancy

Options:

1. **Room prefixes per tenant** (`t:<tenantId>:…`), tenant id from the authenticated user — simplest, works with one namespace.
2. **Dynamic namespaces per tenant** (`/tenant-<id>`) with tenant-specific middleware; `cleanupEmptyChildNamespaces: true`.
3. **Separate clusters** per large tenant/region (isolation, data residency).

Always derive the tenant from auth, never from the client's choice, and include the tenant in every room name to prevent cross-tenant leaks.

13.14 Pattern: RPC over sockets

Acks give you request/response. A typed RPC helper:

```
async function rpc<T>(event: string, payload: unknown, ms = 8000): Promise<T> {
  const res = await socket.timeout(ms).emitWithAck(event, payload);
  if (!res.ok) throw Object.assign(new Error(res.error.message ?? res.error.code), { code:
res.error.code });
  return res.data as T;
}
```

Use for interactions that are inherently bound to the live session (join room + get history). For generic CRUD prefer HTTP (caching, tooling).

13.15 Organisational standards (put these in your engineering handbook)

Code structure

- Feature-based handler modules: `registerXHandlers(io, socket, deps)`.
- Handlers are thin: validate → authorize → call service → emit/ack.
- Services are transport-agnostic (callable from HTTP, jobs, tests).
- A single `createRealtimeServer(httpServer, config)` factory used by app and tests.

Event contract standards

- Naming: `domain:action` lowercase; commands imperative (`message:send`), facts past tense (`message:created`) — pick and document one style.
- Single **object payload**; ids as strings; times as ISO strings or epoch ms (documented).
- **Ack envelope**: `{ ok: true, data } | { ok: false, error: { code, message?, details? } }`.
- **Error codes** catalogue: `VALIDATION`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `RATE_LIMITED`, `CONFLICT`, `INTERNAL`, `UPGRADE_REQUIRED`.
- Every command has a timeout on the client and is idempotent on the server (`clientMsgId` / `requestId`).
- Contracts in a shared package (TS types + zod schemas), documented with AsyncAPI; breaking changes require a new event name/version and a deprecation window.

Room naming

- `type:id` (`user:42`, `conversation:9`, `org:3:role:admin`), never bare ids.
- Room membership changes only on the server, after authorization.

Reliability standards

- Clients: one socket instance; listeners registered once; re-sync on every `connect`; handle `connect_error` with `socket.active`; UI shows connection state.
- Servers: wrapped handlers; graceful shutdown; adapter + stickiness; CSR where useful.

Observability standards

- Required metrics & alerts (Chapter 10); logs include `socketId`, `userId`, `event`.
- Dashboards per environment; runbooks linked from alerts.

Security standards

- Chapter 6 checklist as a PR template section for any realtime change.

Code review checklist for Socket.IO changes

- [ ] Event name follows convention and exists in the contract package.
- [ ] Payload validated; ack always called (all paths); timeout on sender side.
- [ ] Authorization check for every data access/room join.
- [ ] Right broadcast target (`socket.to` vs `io.to` vs `socket.emit`), right namespace.
- [ ] Works with multiple nodes (no local maps), works after reconnection.
- [ ] No listener/timer leak; cleanup on `disconnect` /unmount.
- [ ] High-frequency events throttled/volatile.
- [ ] Backward compatible with old clients.
- [ ] Tests: integration test for the event (happy path + auth failure + validation failure).
- [ ] Metrics/logging added.

Ownership & SLOs

- A team owns the realtime gateway; others integrate via events.
- SLO examples: 99.9% of connection attempts succeed within 3 s; p95 event delivery latency < 300 ms; < 0.1% handler error rate.

13.16 "What do I do when...?" — decision guide

Situation	Do this
You need to notify a specific user from a REST endpoint	`io.to("user:" + id).emit(...)` (user room joined at connection)
Same, from a worker process	Redis emitter / broker → gateway
You need to know if a user is online	Presence in Redis (Section 13.2), not `fetchSockets` per request
Users complain messages are missing after network blips	Persist + `sync` on connect + dedupe; optionally CSR
Duplicate messages in the UI	Listener cleanup / single socket / dedupe by id
You added a 2nd server and things broke	Sticky sessions + adapter

Situation	Do this
Server CPU spikes when a popular room is active	Coalesce/throttle, smaller payloads, volatile, shard rooms, notification + HTTP fetch
Token expires during long sessions	Expiry timer + refresh flow; `auth` as a function
A user must be kicked out everywhere	`io.in("user:" + id).disconnectSockets(true)`
You must send a 50 MB file	HTTP upload (pre-signed URL); socket only for progress
Mobile app needs messages while backgrounded	Push notifications (FCM/APNs)
Corporate users can't connect via WebSocket	Keep polling fallback + sticky sessions
You want zero sticky-session complexity	`transports: ["websocket"]` (accept no fallback)
Need guaranteed processing of client commands	Ack + retries + idempotent handler
Need server push guaranteed	DB persistence + offsets + client sync
Thousands of tenants	Room prefixes or dynamic namespaces + cleanup
Need to broadcast to rooms A AND B (intersection)	Dedicated combined room or filter via `fetchSockets` (rarely)
Need to deploy without disconnect storms	Graceful shutdown, slow rolling deploy, jitter
Debugging "it doesn't connect"	Chapter 12 first-aid kit

13.17 Full mini-project: production-style chat server (all pieces together)

```
// src/realtime/index.js
const { Server } = require("socket.io");
const { createAdapter } = require("@socket.io/redis-adapter");
const { z } = require("zod");

function safe(socket, handler) {
  return async (...args) => {
    const ack = typeof args.at(-1) === "function" ? args.pop() : null;
    try {
      const data = await handler(...args);
      ack?.({ ok: true, data });
    } catch (err) {
      const code = err.code ?? (err instanceof z.ZodError ? "VALIDATION" : "INTERNAL");
      if (code === "INTERNAL") socket.data.log.error({ err }, "handler failed");
```

```
      ack?.({ ok: false, error: { code, message: code === "INTERNAL" ? "Internal error" :
err.message } });
    }
  };
}

const forbidden = () => Object.assign(new Error("Forbidden"), { code: "FORBIDDEN" });

exports.createRealtimeServer = async function ({ httpServer, redis, auth, chat, logger, config
}) {
  const pub = redis.duplicate(); const sub = redis.duplicate();
  pub.on("error", (err) => logger.error({ err }, "redis pub"));
  sub.on("error", (err) => logger.error({ err }, "redis sub"));
  await Promise.all([pub.connect(), sub.connect()]);

  const io = new Server(httpServer, {
    cors: { origin: config.webOrigins, credentials: true },
    allowRequest: (req, cb) => cb(null, !req.headers.origin ||
config.webOrigins.includes(req.headers.origin)),
    serveClient: false,
    maxHttpBufferSize: 1e5,
    adapter: createAdapter(pub, sub, { key: "chat-rt" }),
    connectionStateRecovery: undefined, // classic Redis adapter doesn't support CSR → we use
sync
  });

  io.use(async (socket, next) => {
    try {
      socket.data.user = await auth.verifyAccessToken(socket.handshake.auth?.token);
      socket.data.log = logger.child({ sid: socket.id, uid: socket.data.user.id });
      next();
    } catch (e) {
      next(Object.assign(new Error(e.code === "EXPIRED" ? "TOKEN_EXPIRED" : "UNAUTHORIZED"), {
data: { code: e.code } }));
    }
  });

  io.use(async (socket, next) => {
    // load the rooms BEFORE "connection", so handlers can be registered synchronously
    try {
      socket.data.conversationIds = await chat.conversationIdsFor(socket.data.user.id);
      next();
    } catch (err) {
      next(new Error("INTERNAL"));
    }
  });

  io.on("connection", (socket) => {   // NOT async: register every handler synchronously
    const { id: userId } = socket.data.user;
    socket.data.log.info({ transport: socket.conn.transport.name }, "connected");
    socket.join(`user:${userId}`);
    // re-join conversations from the source of truth (works after any reconnection)
    socket.join(socket.data.conversationIds.map((id) => `conversation:${id}`));

    const Send = z.object({ conversationId: z.string(), text:
```

```
z.string().trim().min(1).max(4000), clientMsgId: z.string().uuid() }).strict();
    const Sync = z.object({ conversationId: z.string(), afterId: z.string().nullable()
}).strict();

    socket.on("message:send", safe(socket, async (raw) => {
      const p = Send.parse(raw);
      if (!socket.rooms.has(`conversation:${p.conversationId}`)) throw forbidden();
      const { message, created } = await chat.createMessageIdempotent(userId, p);
      if (created) io.to(`conversation:${p.conversationId}`).emit("message:new", message);
      return { id: message.id, createdAt: message.createdAt };
    }));

    socket.on("conversation:sync", safe(socket, async (raw) => {
      const p = Sync.parse(raw);
      if (!socket.rooms.has(`conversation:${p.conversationId}`)) throw forbidden();
      return chat.messagesAfter(p.conversationId, p.afterId, 500);
    }));

    socket.on("typing:start", ({ conversationId } = {}) => {
      if (typeof conversationId !== "string" ||
!socket.rooms.has(`conversation:${conversationId}`)) return;
      socket.to(`conversation:${conversationId}`).volatile.emit("typing", { conversationId,
userId });
    });

    socket.on("disconnect", (reason) => socket.data.log.info({ reason }, "disconnected"));
  });

  // used by other modules (REST controllers) to push events
  chat.on("conversation:member-added", ({ conversationId, userId }) =>
    io.in(`user:${userId}`).socketsJoin(`conversation:${conversationId}`)); // cluster-wide
  chat.on("conversation:member-removed", ({ conversationId, userId }) =>
    io.in(`user:${userId}`).socketsLeave(`conversation:${conversationId}`));

  return io;
};
```

```
// client (browser)
import { io } from "socket.io-client";

export const socket = io(RT_URL, {
  autoConnect: false,
  transports: ["websocket", "polling"],
  auth: (cb) => cb({ token: tokenStore.get() }),
  withCredentials: true,
});

const lastIds = new Map(); // conversationId -> last message id
const seen = new Set();

socket.on("message:new", onMessage);              // registered ONCE
socket.on("typing", showTyping);
```

```
socket.on("connect", async () => {
  ui.setConnected(true);
  for (const conversationId of openConversations()) {
    try {
      const res = await socket.timeout(10000).emitWithAck("conversation:sync", {
conversationId, afterId: lastIds.get(conversationId) ?? null });
      if (res.ok) res.data.forEach(onMessage);
    } catch { /* timeout: will retry on next connect or via user action */ }
  }
});

socket.on("disconnect", () => ui.setConnected(false));

socket.on("connect_error", async (err) => {
  if (!socket.active) {
    if (err.message === "TOKEN_EXPIRED") { await tokenStore.refresh(); socket.connect(); }
    else ui.showLogin();
  }
});

function onMessage(m) {
  if (seen.has(m.id)) return;
  seen.add(m.id);
  lastIds.set(m.conversationId, m.id);
  ui.renderMessage(m);
}

export async function sendMessage(conversationId, text) {
  const clientMsgId = crypto.randomUUID();
  ui.renderPending(clientMsgId, text);
  for (let attempt = 0; attempt < 3; attempt++) {
    try {
      const res = await socket.timeout(8000).emitWithAck("message:send", { conversationId,
text, clientMsgId });
      if (res.ok) return ui.confirmPending(clientMsgId, res.data);
      return ui.failPending(clientMsgId, res.error.code);   // business error: don't retry
    } catch { /* timeout → retry with the SAME clientMsgId (idempotent server) */ }
  }
  ui.failPending(clientMsgId, "TIMEOUT");
}
```

This small codebase demonstrates almost every best practice in this book: auth in middleware, async loading done in middleware so the `connection` handler registers every listener synchronously (events emitted by the client right after `connect` are never lost), origin check, CORS allow-list, payload size limit, Redis adapter with error handlers, user rooms, re-joining from DB on every connection, cluster-wide room management with `socketsJoin`, validated & authorized handlers with an ack envelope, idempotent writes, volatile typing, client single registration, sync on connect, dedupe, token-expiry handling, retries with the same id.

13.18 Chapter summary

- Architecture: stateless gateway + adapter + broker + DB as source of truth.
- Patterns: presence (per-user count, grace, TTL), typing (volatile, throttled), chat flow (optimistic UI, idempotent, ack, sync, dedupe), notifications (persist + push), live data (snapshot + coalesced deltas), collaboration (CRDT/OT), games (authoritative ticks), tracking, chunked uploads, job progress, auctions (atomic ops), multi-tenancy.
- Standards: naming, ack envelope, error codes, contracts, review checklist, SLOs.

Next: Chapter 14 — Interview question bank

CHAPTER 14

Interview Question Bank (Entry Level → Senior/Architect)

> **How to use this chapter:** questions are grouped by topic and tagged by level: **[E]** entry/junior, **[M]** mid-level, **[S]** senior/lead/architect. Answers are written the way you should *say* them: start with a one-line direct answer, then the depth. Chapter references point to the full explanation.
>
> The question list was assembled from the topics interviewers and hiring guides consistently probe for Socket.IO / real-time roles — WebSocket vs Socket.IO, rooms vs namespaces, reconnection and dropped connections, authentication and data leaks, horizontal scaling with the Redis adapter and sticky sessions, and practical scenarios such as chat apps with rooms and presence or multiplayer games — plus the standard "design a real-time chat system" system-design round (stateless servers, pub/sub fan-out, offline delivery via last-seen ids, presence with TTL heartbeats, ordering, backoff), and the version-specific details that appear in technical screens.

Part A — Fundamentals

A1. [E] What is Socket.IO? A library for low-latency, bidirectional, event-based communication between clients and a server. It has a Node.js server (`socket.io`) and clients (`socket.io-client` for browser/Node, plus Java, Swift, C++, Python, etc.). It's built on Engine.IO, which handles the low-level connection (HTTP long-polling, WebSocket, WebTransport), heartbeats and transport upgrades; Socket.IO adds named events, acknowledgements, rooms, namespaces, broadcasting, automatic reconnection, buffering, middleware and multi-server adapters.

A2. [E] Is Socket.IO the same as WebSocket? Can a WebSocket client connect to a Socket.IO server? No. WebSocket is a protocol (RFC 6455) for a full-duplex channel. Socket.IO is a library with its own protocol that *may use* WebSocket as a transport. Every Socket.IO message carries metadata (Engine.IO packet type, Socket.IO packet type, namespace, ack id), e.g. `42["chat","hi"]` . A plain WebSocket client doesn't speak that protocol (it would have to handle the `0{…}` open packet, send `40` to connect to a namespace, answer pings `2` with `3` , encode events), and a Socket.IO client can't connect to a plain WebSocket server.

A3. [E] Why use Socket.IO instead of raw WebSockets? Polling fallback when WebSocket is blocked; automatic reconnection with exponential back-off; heartbeat-based disconnection detection; buffering while offline; named events; acknowledgements (request/response, with timeouts and promises); rooms and broadcasting; namespaces/multiplexing; middleware for auth; binary support with automatic serialization; horizontal scaling via adapters; connection state recovery; a typed API; an admin UI. With raw WebSockets you'd build all that yourself.

A4. [E] When would you NOT use Socket.IO? When clients must be standard WebSocket clients (third-party devices, `graphql-ws`, MQTT-over-WS); when only one-way server→client updates are needed (SSE is simpler); for extremely latency-sensitive/high-frequency use where every byte matters (custom binary over raw WS, WebRTC/WebTransport datagrams); for durable messaging needs (use a broker behind it); in serverless runtimes that can't hold connections; or when a managed service is preferable.

A5. [E] Explain polling, long polling, SSE and WebSocket. Short polling: client requests every N seconds — simple, wasteful, latency up to N. Long polling: server holds the request until data is available, client immediately re-requests — near-real-time over plain HTTP, but header overhead and a separate request for sending. SSE: a long-lived HTTP response streaming text events server→client with automatic reconnection — one-way, text-only. WebSocket: an HTTP request upgraded (101 Switching Protocols) to a persistent full-duplex framed channel — lowest overhead/latency, text and binary, no built-in reconnection. Socket.IO uses long polling and WebSocket (and optionally WebTransport). (Ch. 1)

A6. [E] What are the main components on the server and client? Server: `Server` (io), `Namespace`, `Socket` (one per client per namespace), internal `Client` (one per connection), `Adapter` (rooms/broadcast), parser, Engine.IO server and socket, transport. Client: `Socket` (per namespace) and `Manager` (owns the Engine.IO connection and reconnection, accessible via `socket.io`). (Ch. 7)

A7. [E] How do you set up Socket.IO with Express? Create an HTTP server from the Express app and pass it to Socket.IO; listen on the HTTP server:

```
const app = express();
const httpServer = http.createServer(app);
const io = new Server(httpServer, { cors: { origin: "https://app.example.com" } });
io.on("connection", (socket) => { /* … */ });
httpServer.listen(3000);   // NOT app.listen(3000)
```

`app.listen()` would create a second server Socket.IO isn't attached to → 404s on `/socket.io/`.

A8. [E] What does `io.on("connection", socket => …)` give you? It fires once for each client connecting to the main namespace (after middleware accepted it). `socket` is that client's server-side Socket: `socket.id`, `socket.handshake` (headers, query, auth, address…), `socket.data`, `socket.rooms`, `socket.emit/on/join/leave/disconnect`, etc. You register that client's event handlers inside it.

A9. [E] What is `socket.id`? Can I use it as a user id? A random identifier of a Socket.IO socket in a namespace. It changes on every reconnection (unless connection state recovery restored the session), it's different per namespace and per tab. So no — use your authenticated user id, and a `user:<id>` room to reach all of a user's sockets. Also, since v3 it differs from the Engine.IO session id (`socket.conn.id`) for security.

A10. [E] What's the difference between `socket.emit`, `io.emit`, `socket.broadcast.emit`? `socket.emit` → only that client. `io.emit` → every client in the main namespace (incl. the "sender"). `socket.broadcast.emit` → every client in the socket's namespace except that socket. Add rooms:

`io.to(room).emit` (everyone in room), `socket.to(room).emit` (everyone in room except sender). (Ch. 2 cheat sheet)

A11. [E] Can a client broadcast to other clients directly? No. `broadcast`, rooms and `to()` exist only on the server. The client emits to the server, which relays (after validating/authorizing).

A12. [E] Which transports does Socket.IO use and in which order? By default the client starts with HTTP long-polling, then tries to upgrade to WebSocket (client default list `["polling", "websocket", "webtransport"]`; WebTransport only if the server enables it). You can change the order or restrict (`transports: ["websocket"]`).

A13. [M] Why does Socket.IO start with polling and not WebSocket? Reliability: polling works almost everywhere and gives an immediately usable connection; some proxies/antivirus accept a WebSocket handshake but then fail, and waiting for that to time out would delay the user. Once connected, the client tests WebSocket in the background (`2probe` / `3probe`) and upgrades without losing or reordering packets. The trade-off: more initial requests and the need for sticky sessions in multi-node setups.

A14. [M] What changed between Socket.IO v2 and v3/v4? v3 (Nov 2020) was the breaking release: CORS no longer enabled by default; heartbeat direction reversed (server pings, client pongs); reconnection events moved to the Manager (`socket.io.on("reconnect")`); `socket.rooms` is a `Set`; `auth` option and `socket.handshake.auth`; middleware errors → `connect_error` with `err.data`; no implicit default-namespace connection; `maxHttpBufferSize` 1 MB default (was 100 MB); `cookie` and `perMessageDeflate` off by default; Engine.IO v4 payload encoding (`\x1e` separator); socket id ≠ engine id; rewritten in TypeScript. v4 is compatible with v3 clients; it added typed events, `io.except`, `fetchSockets/socketsJoin/socketsLeave/disconnectSockets`, `serverSideEmit`, ack timeouts, broadcast acks, `emitWithAck`, connection state recovery, client retries, `io.engine.use`, WebTransport, uWebSockets.js support. v2 clients need `allowEIO3: true` on a v3/v4 server.

A15. [M] Are Socket.IO client and server versions independent? Yes, they're separate packages, but they must speak compatible protocols: a v3/v4 client with a v3/v4 server; a v2 client with a v4 server only with `allowEIO3`; never a v3/v4 client with a v2 server. Third-party clients have their own compatibility tables.

A16. [E] What is `serveClient`? A server option (default `true`) that serves the client bundle at `/socket.io/socket.io.js`. Set it to `false` when you bundle `socket.io-client` yourself.

A17. [E] What's the difference between the URL path and the `path` option? In `io("https://host/admin")`, `/admin` is the **namespace**. The HTTP path where the server listens is the `path` option (default `/socket.io/`), which must match on both sides. Confusing the two causes "Invalid namespace" or 404 errors.

A18. [E] What is the default `maxHttpBufferSize` and what happens if exceeded? 1 MB (`1e6` bytes). A single message larger than that makes the server close the connection. Increase it deliberately, chunk the data, or use HTTP for large files.

Part B — Events, data and acknowledgements

B1. [E] How do you send and receive events? `socket.emit("name", ...args)` and `socket.on("name", (...args) ⇒ {})`, symmetric on both sides. Any number of arguments; prefer one object.

B2. [E] What data types can be sent? Anything JSON-serializable plus binary (`Buffer`, `ArrayBuffer`, typed arrays, `Blob` / `File`). Dates become ISO strings, `Map` / `Set` become `{}`, `undefined` properties are dropped, `NaN` / `Infinity` → `null`, `BigInt` throws, functions can't be sent except as the last argument which becomes an ack callback. (Ch. 3)

B3. [E] What is an acknowledgement? A callback passed as the last argument of `emit`; the receiver gets it as the last argument of its handler and calls it to send a response back to the sender — request/response over the socket.

```
socket.emit("create", data, (res) ⇒ console.log(res));
socket.on("create", (data, cb) ⇒ cb({ ok: true }));
```

B4. [M] What happens if the receiver never calls the ack? How do you handle it? The sender waits forever and keeps the callback in memory. Use `socket.timeout(ms).emit(...)` — the callback then receives an error as first argument on timeout — or `await socket.timeout(ms).emitWithAck(...)` which rejects. On the receiver, always call the ack in every code path (including errors), and verify `typeof ack ≡ "function"`.

B5. [M] What's `emitWithAck`? (v4.6+) A Promise-based version of emit-with-ack: `const res = await socket.emitWithAck("x", data)`. Combined with `timeout(ms)` it rejects on timeout. On the server with broadcast (`io.timeout(ms).emitWithAck`) it resolves to an array of responses.

B6. [M] Can you use acknowledgements with broadcasts? Yes, since v4.5, with a timeout: `io.timeout(5000).emit("x", (err, responses) ⇒ {})` — `responses` is an array, one per client; `err` is set if some clients didn't answer in time. Without `.timeout()` the callback fires almost immediately with a timeout error. Works across nodes with supporting adapters.

B7. [M] What does the `.timeout()` modifier change in the callback signature? It adds an `err` first parameter: `(err, response)`. Forgetting this is a classic bug — people read the response from the first argument.

B8. [M] What are volatile events? Give use cases. `socket.volatile.emit(...)` — the packet is dropped (instead of buffered) if the underlying transport isn't currently writable — on the server (e.g. no pending polling request) and on the client (which includes being disconnected). For superseded high-frequency data: cursor/mouse positions, typing indicators, game state ticks, live prices, sensor readings.

B9. [M] What happens to events emitted by the client while disconnected? They're buffered in `socket.sendBuffer` and sent upon reconnection (in order). Risk: flooding the server with stale events after long offline periods. Mitigate with volatile, `if (socket.connected)` checks, or clearing `sendBuffer` on reconnect.

B10. [M] What are reserved event names? `connect`, `connect_error`, `disconnect`, `disconnecting`, `newListener`, `removeListener`. Emitting them throws. Server-side namespace events (`connection`, `connect`, `new_namespace`) are also reserved for `serverSideEmit`.

B11. [M] What are catch-all listeners? `socket.onAny((event, ...args) ⇒ {})` for incoming events and (4.5+) `socket.onAnyOutgoing(...)` for outgoing; plus `prependAny`, `offAny`, `listenersAny`. Useful for logging, metrics, debugging. They don't receive reserved events.

B12. [M] Does Socket.IO guarantee message ordering? Yes — per connection, in both directions, regardless of transport and across the upgrade. Not across different sockets or across nodes. Async handlers can still finish out of order; serialize with a per-key queue if processing order matters.

B13. [M] What is the delivery guarantee by default? At-most-once: messages can be lost if the connection breaks or the client is disconnected when the server emits. Plus ordering. The client buffers its own emits while disconnected (in memory only).

B14. [S] How would you implement at-least-once delivery in both directions? Client → server: acks + retries (v4.6 `retries` and `ackTimeout` options, or manual) + idempotent server handlers (client-generated id + unique constraint) + optional persistent outbox on the client. Server → client: persist messages with monotonic ids; client tracks the last processed id; on every (re)connection the client syncs "everything after id X"; client dedupes by id. Optionally connection state recovery for short disconnections. "Exactly-once" is achieved as at-least-once + idempotent processing.

B15. [M] How do client `retries` work? With `ackTimeout` and `retries` options set, each emit waits for an ack; if none in `ackTimeout`, it's re-sent, up to `retries` times. Packets are queued and sent one at a time to preserve order. The server may receive duplicates → idempotency required.

B16. [M] How do you send binary/files? Pass a `Buffer` / `ArrayBuffer` / `Blob` / `File` as an argument; Socket.IO detects binary and sends it as attachments (binary WebSocket frames, base64 in polling). Respect `maxHttpBufferSize`; for large files chunk with acks (backpressure, resume) or use HTTP uploads.

B17. [M] What does `socket.compress(false)` do? Disables compression for that packet (relevant if `perMessageDeflate` is enabled or for HTTP compression in polling). Useful for already-compressed data to save CPU.

B18. [S] Why is `perMessageDeflate` disabled by default? Compression contexts consume significant memory per connection and CPU per message, and most real-time messages are small. Enable it only for large compressible payloads, with a threshold and `noContextTakeover` options.

B19. [M] How do you handle errors in event handlers? Socket.IO doesn't catch them: a sync throw propagates (can crash) and async rejections become unhandled rejections (crash by default in modern Node). Wrap handlers in a helper that validates, try/catches, logs, and always acks with an error envelope `{ ok: false, error: { code } }`.

B20. [M] What's `socket.send()`? Shorthand for `emit("message", ...)`; listened with `on("message")`.

Part C — Rooms, namespaces, broadcasting

C1. [E] What is a room? A server-side named group of sockets used for broadcasting to a subset of clients. Sockets join/leave rooms only on the server; rooms are created on first join and deleted when empty; sockets leave all rooms on disconnect; each socket is in a room named after its id.

C2. [E] How do you send a private message to one user? Join each socket to `user:<userId>` at connection (after auth) and emit `io.to("user:" + userId).emit(...)` — reaches all their tabs/devices and works across nodes and reconnections. To a single socket: `io.to(socketId).emit(...)`.

C3. [M] Difference between `io.to(room).emit` and `socket.to(room).emit`? `io.to` sends to all sockets in the room (including the current one if it's in it); `socket.to` excludes the sender.

C4. [M] What happens with `io.to("a").to("b").emit()`? Does a socket in both receive it twice? It's a union (sockets in a OR b); each socket receives it once. There's no built-in intersection.

C5. [M] What is a namespace? How is it different from a room? A namespace is a communication channel the **client chooses** to connect to, with its own event handlers, middleware and rooms, multiplexed over the same underlying connection. A room is a **server-managed** group inside a namespace used for broadcasting. Use rooms for grouping users (chats, documents, users); namespaces for separating concerns or permissions (`/admin` with its own auth middleware), or multi-tenancy via dynamic namespaces.

C6. [M] What is multiplexing? Multiple namespace sockets sharing a single underlying Engine.IO connection on the client (one Manager). `io(url)` and `io(url + "/admin")` share one WebSocket. `forceNew: true` or `multiplex: false` creates separate connections. Calling `io()` twice for the same URL **and same namespace** also creates a new connection.

C7. [M] What are dynamic namespaces? Namespaces matched by a RegExp or function, created on demand: `io.of(/^\/tenant-\w+$/)`. The parent namespace's middleware and handlers apply to all children; `socket.nsp` is the child; `parent.emit` broadcasts to all children. `cleanupEmptyChildNamespaces: true` deletes empty children.

C8. [M] How do you get all sockets in a room, across several servers? `await io.in("room").fetchSockets()` — returns local `Socket`s and `RemoteSocket`s (id, handshake, rooms, data; can emit/join/leave/disconnect). Requires an adapter that supports it (Redis etc.). Expensive — don't use on hot paths.

C9. [M] How do you make a user (all their sockets, any server) join/leave a room or disconnect? `io.in("user:" + id).socketsJoin("room")`, `socketsLeave("room")`, `disconnectSockets(true)`.

C10. [M] What's the difference between `disconnecting` and `disconnect`? `disconnecting` fires before the socket leaves its rooms (so `socket.rooms` is still populated — useful to notify rooms); `disconnect` fires after (rooms empty).

C11. [M] What are adapter room events? `io.of("/").adapter.on("create-room" | "delete-room" | "join-room" | "leave-room", …)`. They fire for private id-rooms too and are local to each node.

C12. [S] How would you list "real" rooms (not socket-id rooms)? `[...adapter.rooms.keys()].filter(r ⇒ !adapter.sids.has(r))` (local node only).

C13. [M] What does `io.local.emit` do? Broadcasts only to sockets connected to the current node, skipping the adapter's cross-node forwarding.

C14. [M] What's `io.except()`? Excludes sockets in given rooms from a broadcast: `io.except("banned").emit(...)`, `io.to("room1").except("room2").emit(...)`.

C15. [M] Is `socket.join` sync or async? With the default in-memory adapter it's synchronous; with some adapters it can return a Promise (`Promise<void> | void`). `await socket.join(room)` is safe in both cases.

C16. [S] Room design for a Slack-like app? `user:<id>` (all devices), `channel:<id>` (members), `workspace:<id>` (announcements), maybe `presence:<id>` (subscribers to someone's presence), `thread:<id>` if threads are subscribed separately. Joins authorized on the server, re-joined from the DB on every connection; membership changes propagated cluster-wide with `socketsJoin/socketsLeave`.

Part D — Connection lifecycle and reliability

D1. [E] Which events does the client receive on connection problems? `connect_error` (couldn't connect or rejected by middleware), `disconnect` (with reason), and Manager events `reconnect_attempt`, `reconnect`, `reconnect_error`, `reconnect_failed` on `socket.io`.

D2. [M] List the client disconnect reasons and which ones auto-reconnect. `io server disconnect` (server called disconnect — no auto-reconnect), `io client disconnect` (client called disconnect — no), `ping timeout`, `transport close`, `transport error`, `parse error` (all auto-reconnect).

D3. [M] List server-side disconnect reasons. `server namespace disconnect`, `client namespace disconnect`, `server shutting down`, `ping timeout`, `transport close`, `transport error`, `parse error`, `forced close`, `forced server close`.

D4. [M] How does the heartbeat work in v4? The server sends a PING every `pingInterval` (25 s); the client must answer PONG within `pingTimeout` (20 s), otherwise the server closes with `ping timeout`. The client expects a PING within `pingInterval + pingTimeout` (45 s) or closes with `ping timeout` and reconnects. The values are sent in the handshake. In v2 the direction was reversed.

D5. [S] Why did v3 reverse the heartbeat direction? Browsers throttle timers in background tabs; a client-driven ping could be delayed and the server would wrongly consider the client dead. Server-driven pings only require the client to respond to a received message, which isn't affected the same way.

D6. [M] Why are heartbeats needed if TCP exists? TCP may not detect a dead peer for a long time (sleeping laptop, lost mobile network, NAT dropping state — no FIN/RST sent). Heartbeats detect zombies, update presence, and keep proxies/LBs from closing idle connections.

D7. [M] Explain the reconnection algorithm. Exponential back-off with jitter: delay starts at `reconnectionDelay` (1 s), doubles each attempt, randomized ± `randomizationFactor` (50%), capped at `reconnectionDelayMax` (5 s), unlimited attempts by default (`reconnectionAttempts: Infinity`). Each attempt has a `timeout` (20 s). Jitter avoids thundering-herd reconnection storms.

D8. [M] After reconnecting, my user no longer receives room messages. Why? A reconnection creates a new server-side socket with a new id and no rooms or data. Re-join rooms in the `connection` handler based on the authenticated user (DB), or have the client re-subscribe on every `connect` , or use connection state recovery (and still handle the non-recovered case).

D9. [M] Why do I get duplicate messages after reconnection? Listeners registered inside `socket.on("connect")` (or React effects without cleanup) accumulate. Register listeners once outside `connect` , or remove them with `off` using the same reference.

D10. [M] `socket.on("reconnect")` **doesn't fire — why?** Since v3 reconnection events are on the Manager: `socket.io.on("reconnect", …)` .

D11. [M] What's `socket.active` **?** True if the socket is connected or will try to reconnect automatically. In `connect_error` , `socket.active` `===` `false` means the server rejected the connection (middleware) and you must fix and call `socket.connect()` yourself.

D12. [S] What is connection state recovery and how does it work? (v4.6+) Enabled with `connectionStateRecovery: { maxDisconnectionDuration, skipMiddlewares }` . The server gives the client a private session id (`pid`); each emitted event gets an offset appended (client tracks the last one). On a recoverable disconnect (transport error/close, ping timeout, forced close, server shutting down), the adapter stores `{sid, pid, rooms, data}` . On reconnection the client sends `{pid, offset}` ; if the session exists and hasn't expired, the server restores id, rooms, data, sets `socket.recovered = true` , and replays missed packets. Limits: not for packets with acks or volatile packets; memory for stored packets; best-effort; requires a supporting adapter (in-memory, Redis Streams, MongoDB — not the classic Redis pub/sub adapter, not Postgres).

D13. [S] Clients should not miss chat messages even after a 10-minute outage. Design it. Persist messages in the DB before broadcasting (server-assigned increasing ids per conversation). Clients store the last seen id per conversation; on every `connect` they call `sync(afterId)` via ack or HTTP (paginated); live events continue meanwhile; dedupe by id; render sorted by server id. Push notifications for apps in background. CSR can reduce sync load for short blips but isn't relied on.

D14. [M] What happens on page reload? The browser closes the connection; the server sees `transport close` ; the new page creates a new connection with a new socket id. The server can't distinguish reload from network loss → presence grace periods.

D15. [M] What timeouts exist? (pingInterval, pingTimeout, upgradeTimeout, connectTimeout, client timeout…) See Ch. 5 table: 25 s / 20 s / 10 s / 45 s on the server; client `timeout` 20 s; reconnection delays 1–5 s; `ackTimeout` none by default; Redis adapter `requestsTimeout` 5 s.

D16. [M] What's `connectTimeout` on the server? Time (45 s) a client has, after the Engine.IO handshake, to successfully join a namespace; if it hasn't, the server closes the low-level connection. Relevant when middleware never calls `next()`.

D17. [S] How do you handle access-token expiry on long-lived connections? Validate at connection; store `exp` in `socket.data`; schedule a disconnect at expiry (or emit `auth:expired`); client refreshes the token and reconnects (use `auth` as a function so each attempt reads the latest token) or performs in-band refresh; revoke with `io.in("user:" + id).disconnectSockets(true)`; re-check permissions on sensitive events.

D18. [M] How do you detect on the client that the connection is down and show a banner? Listen to `disconnect` (show "reconnecting…") and `connect` (hide), use `socket.connected` for initial state; optionally Manager `reconnect_attempt` for attempt counts and `reconnect_failed` for a final error.

D19. [S] How do mobile apps receive messages in the background? They don't via Socket.IO — the OS suspends the app and the connection dies. Use push notifications (APNs/FCM) for background delivery, and sync on foreground.

Part E — Middleware, authentication and security

E1. [E] How do you authenticate Socket.IO connections? Client sends credentials in the `auth` option (`io(url, { auth: { token } })`); server middleware `io.use((socket, next) ⇒ …)` verifies `socket.handshake.auth.token`, sets `socket.data.user`, and calls `next()` or `next(new Error("unauthorized"))`. Alternatively cookies/sessions via `io.engine.use(sessionMiddleware)`.

E2. [M] Why `auth` instead of query string or headers? Query strings end up in logs/history (token leakage). Browser WebSocket API can't set custom headers (`extraHeaders` only applies to polling in browsers). `auth` is sent in the CONNECT packet body and is per-namespace.

E3. [M] What does the client receive when middleware rejects? A `connect_error` event with `err.message` and `err.data` (from `err.data` set on the server). The client doesn't auto-reconnect (`socket.active === false`).

E4. [M] What are the middleware types? `io.engine.use()` (Express-style, raw HTTP requests, v4.6+), `io.use()` / `nsp.use()` (once per connection per namespace), `socket.use()` (per incoming event; `next(err)` emits `error` on the server socket), plus the `allowRequest` option (earliest gate).

E5. [M] How do you share an Express session with Socket.IO? `io.engine.use(sessionMiddleware)` then `socket.request.session` in handlers. For Passport: apply session + `passport.session()` for the handshake request only, then `socket.request.user`. (Pre-4.6: wrap middleware with `io.use(wrap(mw))`.)

E6. [S] What is Cross-Site WebSocket Hijacking and how do you prevent it? A malicious site opens a WebSocket to your server from the victim's browser; cookies are attached automatically, and WebSocket handshakes aren't protected by CORS/SOP. If you authenticate only via cookies, the attacker gets an authenticated channel. Prevent by checking `Origin` (`allowRequest`), using tokens (not ambient cookies), SameSite cookies, short-lived tickets.

E7. [M] How do you configure CORS and what's a common mistake? `new Server(httpServer, { cors: { origin: ["https://app.com"], credentials: true } })`; client `withCredentials: true` if cookies are needed. Mistakes: `origin: "*"` with credentials, trailing slashes/ports mismatches, believing CORS protects the WebSocket transport (it doesn't).

E8. [M] How do you authorize actions, not just connections? Every event checks permissions server-side using `socket.data.user` (never ids from the payload), especially room joins and data access; per-event middleware (`socket.use`) for role checks; re-validate for sensitive operations.

E9. [M] How do you rate-limit Socket.IO events? `socket.use` middleware with `rate-limiter-flexible` keyed by user id (Redis-backed in clusters); reply with `RATE_LIMITED` via ack; disconnect repeat offenders; cap connections per user/IP; edge rate limits for handshakes.

E10. [M] What security risks exist with broadcasting user content? XSS (render with `textContent`/framework escaping), data leaks to wrong rooms (authorize joins, prefix rooms), oversized payloads (validation, `maxHttpBufferSize`), injection into DB queries (validate/parameterize).

E11. [S] How would you protect against DoS on a Socket.IO server? Edge protections (WAF, connection/request rate limits), `maxHttpBufferSize` lowered, per-user rate limits, connection caps, validation of payload size/shape, timeouts on acks, limiting dynamic room/namespace creation, slow-consumer disconnects, keeping `engine.io` / `socket.io-parser` / `ws` patched (historic crash CVEs), autoscaling and process restarts.

E12. [M] How do you get the real client IP behind a proxy? `socket.handshake.address` gives the proxy IP; read `x-forwarded-for` (first entry) or `cf-connecting-ip` only when your proxy is trusted and sets it.

E13. [M] Is `socket.handshake.auth` trustworthy? It's client-supplied — trustworthy only after you verify it (e.g. JWT signature check).

E14. [S] Name some real vulnerabilities in the Socket.IO ecosystem. Permissive default CORS in `socket.io` < 2.4.0; an unhandled `error` event that could crash `socket.io` < 4.6.2; uncaught exceptions from crafted requests in `engine.io` (fixed in 6.2.1 / 6.4.2) and polling connection exhaustion (fixed in 6.6.7); `socket.io-parser` decoding/validation bugs and attachment-based memory exhaustion (fixed in 4.0.5, 4.2.3, 4.2.6, 4.2.7); `ws` DoS via many headers (8.17.1), memory disclosure (8.20.1) and fragment-based memory exhaustion (8.21.0). Pattern: malicious clients crashing or exhausting servers with malformed/oversized packets. Lesson: keep the whole stack patched (check the lockfile with `npm ls ws engine.io socket.io-parser`), keep `maxHttpBufferSize` small, rate-limit, and restart crashed processes. (Ch. 6.10 has the full table with advisory ids.)

Part F — Internals and protocol

F1. [M] What is Engine.IO? The low-level layer: handshake, transports (polling/WebSocket/WebTransport), upgrade mechanism, heartbeats, packet encoding (Engine.IO protocol v4). Socket.IO (protocol v5) runs on top of it.

F2. [M] Describe the handshake. Client: `GET /socket.io/?EIO=4&transport=polling`. Server: `0{"sid":"…","upgrades":["websocket"], "pingInterval":25000,"pingTimeout":20000,"maxPayload":1000000}`. Then the client sends a Socket.IO CONNECT `40` (with auth payload if any), server responds `40{"sid":"<socket id>"}` or `44{"message":…}` on error.

F3. [S] Explain the upgrade mechanism. After connecting over polling, the client opens a WebSocket with the same `sid`, sends `2probe`, server answers `3probe`; the client pauses polling (waits for pending requests; the server closes a pending GET with a `6` noop), sends `5` (upgrade), and all traffic moves to the WebSocket; buffered packets flush. No loss/reordering. `upgradeTimeout` (10 s) bounds it.

F4. [M] Engine.IO packet types? 0 open, 1 close, 2 ping, 3 pong, 4 message, 5 upgrade, 6 noop.

F5. [M] Socket.IO packet types? 0 CONNECT, 1 DISCONNECT, 2 EVENT, 3 ACK, 4 CONNECT_ERROR, 5 BINARY_EVENT, 6 BINARY_ACK.

F6. [M] Decode `42/admin,7["q"]` and `431[{"ok":true}]`. First: Engine.IO message (4) + Socket.IO EVENT (2) in namespace `/admin` with ack id 7, event `q`. Second: message (4) + ACK (3) for ack id 1 with response `{ok:true}`.

F7. [S] How is binary data transmitted? The parser replaces binary values with placeholders `{"_placeholder":true,"num":i}`, sends a BINARY_EVENT header like `451-["bin", {"_placeholder":true,"num":0}]` (1 attachment), then each attachment as a separate Engine.IO message: a raw binary WebSocket frame, or `b<base64>` over polling. The receiver reassembles.

F8. [M] How are multiple packets sent over polling? Concatenated in one HTTP body separated by the record separator `\x1e` (Engine.IO v4).

F9. [M] What's the parser and why change it? It encodes/decodes Socket.IO packets. Default: JSON + binary attachments. Alternatives: `socket.io-msgpack-parser` (smaller for numeric/binary data), custom parsers. Both sides must match.

F10. [S] What is the adapter and what does the default one do? Per-namespace component storing room membership (`rooms`, `sids` maps) and performing broadcasts (computing recipients, encoding once, writing to each). The default in-memory adapter works within one process; distributed adapters forward broadcasts and requests (fetchSockets, socketsJoin, serverSideEmit, broadcast acks) between nodes, and some persist sessions for CSR.

F11. [S] Why does `io.emit` to 10k sockets not serialize 10k times? The adapter encodes the packet once and writes the same encoded buffer/string to each target client.

F12. [M] What are `socket.conn` and `socket.request`? `socket.conn` is the underlying Engine.IO socket (transport, `remoteAddress`, `id`, `readyState`, `upgrade` event). `socket.request` is the first HTTP request (headers, session/user if middleware attached them).

F13. [M] How can you see which transport a client uses? Server: `socket.conn.transport.name` and `socket.conn.once("upgrade", t ⇒ …)`. Client: `socket.io.engine.transport.name` and `socket.io.engine.on("upgrade", …)`.

F14. [S] What is `rememberUpgrade`? Client option: if a previous connection successfully used WebSocket (static flag in the page), start directly with WebSocket next time.

F15. [M] What's WebTransport support? Since 4.7, an HTTP/3-based transport (requires HTTPS and an HTTP/3 server, enabled in server `transports`). Benefits: no head-of-line blocking, faster setup; limited browser support so WebSocket stays the default.

Part G — Scaling and architecture

G1. [M] How do you scale Socket.IO horizontally? Run multiple processes/servers behind a load balancer with (1) **sticky sessions** (or WebSocket-only transport) so all HTTP requests of a session reach the same node, and (2) a distributed **adapter** (Redis, Redis Streams, MongoDB, Postgres, cluster, cloud pub/sub) so broadcasts and room operations span all nodes. Keep nodes stateless (state in DB/Redis), use user rooms, and handle reconnection storms.

G2. [M] Why are sticky sessions needed? With HTTP long-polling, a session is many separate HTTP requests; the session lives in the memory of the node that did the handshake. If a request hits another node, it answers `400 {"code":1,"message":"Session ID unknown"}`. Even the WebSocket upgrade request must reach the node holding the session.

G3. [M] When are sticky sessions not needed? When clients connect with `transports: ["websocket"]` only — a single long-lived TCP connection stays on one node. Trade-off: no polling fallback.

G4. [M] How do you implement sticky sessions? IP hash (nginx `ip_hash` / `hash $remote_addr consistent`, HAProxy `balance source`), cookie-based affinity (AWS ALB stickiness, HAProxy `cookie`, ingress-nginx `affinity: cookie`, nginx Plus `sticky`), Socket.IO's `cookie` option for cookie hashing, `@socket.io/sticky` for Node cluster, `@socket.io/pm2` for PM2. Cross-origin cookie stickiness needs `withCredentials: true` on the client.

G5. [M] Why isn't the Node.js cluster module enough by itself? It distributes TCP connections round-robin; a polling session's requests use different TCP connections and would hit different workers. `@socket.io/sticky` makes the primary route by `sid`, and `@socket.io/cluster-adapter` lets workers broadcast to each other.

G6. [M] How does the Redis adapter work? Each node publishes broadcasts to Redis channels (`socket.io#/#` or `socket.io#/#<room>#` for single-room broadcasts) and pattern-subscribes to `socket.io#/#*` ; on receiving, a node broadcasts to its local sockets (ignoring its own messages). Requests (fetchSockets, socketsJoin, serverSideEmit, broadcast acks) use request/response channels with a `requestsTimeout` (5 s). It needs two Redis connections (pub and sub).

G7. [S] Limitations of the Redis pub/sub adapter? Pub/sub is fire-and-forget: a node disconnected from Redis misses messages; every node receives every broadcast (pattern subscription) → fan-out cost; no connection state recovery; Redis `client-output-buffer-limit pubsub` can disconnect slow subscribers. Alternatives: sharded adapter (Redis 7 sharded pub/sub, dynamic per-room subscriptions), Redis Streams adapter (resumes from offset, supports CSR).

G8. [M] How do you emit from a process that isn't a Socket.IO server (worker/cron/microservice)? Use an emitter (`@socket.io/redis-emitter` , Postgres/Mongo emitters) that publishes into the adapter's channels: `emitter.to("user:1").emit("x", data)` . Or publish to a broker consumed by the realtime gateway. Emitters can't receive acks.

G9. [M] What is `serverSideEmit` ? Events between Socket.IO servers in a cluster: `io.serverSideEmit("reload", data)` → other nodes receive via `io.on("reload")` ; with ack and timeout you get responses from each node (4.5+). Use for cache invalidation/config coordination.

G10. [M] How do you configure nginx for Socket.IO? `proxy_http_version 1.1; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade";` plus Host/X-Forwarded-For headers, `proxy_read_timeout` above the ping interval (e.g. 120 s+), sticky upstream (`hash $remote_addr consistent` or `ip_hash`), and `proxy_buffering off` .

G11. [S] Clients disconnect every 60 seconds in production — what's happening? Almost certainly an idle timeout in a proxy/LB (nginx `proxy_read_timeout` 60 s default, ALB 60 s idle timeout) shorter than the traffic gap — e.g. heartbeat interval increased above it, or some proxy not counting traffic. Fix timeouts > `pingInterval` ; check disconnect reason (`transport close`) and the periodicity.

G12. [S] How many connections can one Node server handle? How do you size a cluster? Typically tens of thousands per process; depends on memory per connection, message rates, fan-out (messages × recipients), compression, handler work. Load-test real traffic, measure memory per connection, CPU per message, event loop lag; size with headroom (2–3× peak) and for reconnection storms when a node dies (remaining nodes absorb its clients). Raise `ulimit -n` , watch ephemeral ports on proxies.

G13. [S] How do you deploy without disconnecting everyone at once? Rolling deploys a few nodes at a time; readiness probe fails first so LB stops new connections; graceful shutdown (`io.close()`) after a drain delay; clients reconnect with jitter to other nodes; CSR with a shared store (Redis Streams) to recover sessions; watch connection metrics between steps.

G14. [S] How do you autoscale socket servers? On connection count / memory / event loop lag (custom metrics), not just CPU. Remember scale-out only affects new connections; scale-in disconnects clients (graceful shutdown). Pre-scale for known peaks.

G15. [S] Design presence for millions of users. Track connections per user in Redis (set/sorted set with heartbeat timestamps, TTL for crash safety); user online if ≥1 live connection; grace period before "offline"; publish presence changes only to subscribers (`presence:<userId>` rooms for contacts/visible users) rather than to everyone; store last seen; batch/coalesce updates; shard Redis.

G16. [S] How do you avoid the "everyone receives everything" fan-out problem with big rooms? Avoid huge chatty rooms; send compact notifications and let clients fetch details over HTTP/CDN; coalesce updates per tick; shard large rooms; use sharded Redis adapter (dynamic subscriptions); `io.local` when each node consumes the source itself; separate read-heavy broadcast infrastructure (e.g. a dedicated fan-out tier).

G17. [S] How would you integrate Socket.IO in a microservices architecture? A dedicated realtime gateway service owns connections, auth and rooms; domain services publish domain events to a broker (Kafka/RabbitMQ/SNS-SQS); the gateway consumes and emits to rooms (or services use the Redis emitter). Commands go via HTTP or socket → gateway → service (RPC/broker). DB remains the source of truth; clients re-sync on reconnect.

G18. [M] Can Socket.IO run on serverless (Lambda, Vercel functions)? Not as a server — those runtimes are short-lived and don't keep connections. Use long-running containers/VMs, or managed services (Azure Web PubSub for Socket.IO, API Gateway WebSockets with a different protocol, Ably/Pusher).

G19. [S] Multi-region deployment? Route users to the nearest region (geo-DNS); each region has its own Socket.IO cluster and adapter; cross-region events go through a global broker/replication (with higher latency); keep users of the same conversation in one "home" region if strong ordering is needed, or accept per-region ordering + server ids for convergence.

Part H — Performance and production

H1. [M] How do you improve Socket.IO performance? Install `bufferutil` / `utf-8-validate` ; consider uWebSockets.js via `io.attachApp` ; keep `perMessageDeflate` off unless needed; discard `socket.request` if unused; broadcast via rooms (encode once); smaller payloads/diffs; throttle/coalesce/volatile for high-frequency data; msgpack parser if measured beneficial; WebSocket-only where possible; keep the event loop free (worker threads); horizontal scaling.

H2. [M] Common causes of memory leaks in Socket.IO apps? Listeners added to global emitters per connection and never removed; per-socket timers not cleared; maps of sockets never cleaned; pending acks without timeouts; client listener accumulation; large `socket.data` ; CSR packet storage; dynamic namespaces; compression contexts.

H3. [M] What causes mass `ping timeout` disconnections? Blocked event loop (CPU-heavy sync work, large JSON, GC pressure) preventing timely PING/PONG processing; overloaded servers; network issues. Check event-loop lag metrics.

H4. [M] How do you shut down gracefully? Fail readiness, wait for LB to drain new connections, optionally notify clients, call `await io.close()` (disconnects sockets with `server shutting down` and closes the HTTP server), close Redis/DB, exit. Handle SIGTERM.

H5. [S] How do you version a Socket.IO event API? Additive changes only; new event names for breaking changes (`x:v2`) with a deprecation window; send client version in `auth` and reject too-old clients with `UPGRADE_REQUIRED` ; shared contract package + AsyncAPI docs.

H6. [M] What metrics do you monitor? Connected clients (`io.engine.clientsCount`), connect/disconnect rates, disconnect reasons, `connect_error` rates, transport distribution, events per second by name, ack latency, handler errors, event loop lag, memory, Redis health. Low-cardinality labels only.

H7. [M] How do you debug Socket.IO? `DEBUG=socket.io*,engine` on the server, `localStorage.debug = "socket.io-client:*"` in the browser; DevTools Network (handshake response, 101 WebSocket, Messages tab); `onAny/onAnyOutgoing` logging; `io.engine.on("connection_error")` codes; Admin UI; curl the handshake.

H8. [M] What does `io.engine.on("connection_error")` tell you? Low-level failures with codes: 0 Transport unknown, 1 Session ID unknown (sticky sessions!), 2 Bad handshake method, 3 Bad request, 4 Forbidden (`allowRequest`), 5 Unsupported protocol version.

H9. [M] What is the Socket.IO Admin UI? `@socket.io/admin-ui` : `instrument(io, { auth: {...}, mode, readonly })` creates an `/admin` namespace; the UI (admin.socket.io or self-hosted) shows servers, sockets, rooms, events, and can join/leave/disconnect sockets. Protect it.

H10. [M] How do you test Socket.IO code? Unit-test thin handlers with fakes; integration tests with a real server on port 0 and real `socket.io-client` instances (wait for events with promises, use ack timeouts, always close); E2E with Playwright (two contexts); load tests with Artillery's Socket.IO engine or custom clients through the real LB.

Part I — Frameworks, TypeScript, clients

I1. [M] How do you type Socket.IO events in TypeScript? Define `ServerToClientEvents` , `ClientToServerEvents` , `InterServerEvents` , `SocketData` interfaces; `new Server<ClientToServerEvents, ServerToClientEvents, InterServerEvents, SocketData>()` on the server; `Socket<ServerToClientEvents, ClientToServerEvents>` on the client (reversed order: listen, emit). Share them in a package; still validate at runtime.

I2. [M] How do you use Socket.IO in React correctly? Single socket instance (module/context), `autoConnect: false` and connect after login; register listeners in `useEffect` with cleanup `socket.off(event, sameHandler)` ; avoid stale closures (functional updates/refs); StrictMode-safe; don't create sockets in render.

I3. [M] Why do I see two connections in React development? `io()` called in a component/effect that StrictMode runs twice (mount → unmount → mount), or on every render, or HMR. Create the socket outside components; cleanup properly.

I4. [M] How do you integrate with Angular? An injectable service wrapping the socket; expose events as Observables that `off` on unsubscribe; run handlers in `NgZone` (or update signals) so change detection runs.

I5. [M] How does NestJS support Socket.IO? `@WebSocketGateway()` classes with `@SubscribeMessage()` handlers (return value = ack), `@WebSocketServer()` for the server/namespace, lifecycle hooks (`afterInit` , `handleConnection` , `handleDisconnect`), guards/pipes/filters per message, `IoAdapter` subclass for the Redis adapter.

I6. [M] How do you use Socket.IO with Next.js? Client code only in client components/effects; the server either a custom Node server attaching Socket.IO to the same HTTP server, or (better) a separate long-running service. Not on serverless functions.

I7. [M] What's special about React Native? Use `transports: ["websocket"]` ; real device IPs; HTTPS (cleartext blocked by default); handle AppState (background disconnects) and push notifications.

I8. [M] Which clients exist for other platforms? Official: JavaScript, Java (Android), Swift (iOS), C++ — plus Python (`python-socketio`), Dart/Flutter, .NET, Rust, Go community clients. Check protocol compatibility tables.

Part J — Scenario / debugging questions (very common for experienced roles)

J1. [M] Users report the app works at home but not at the office. Corporate proxy/firewall blocks/breaks WebSocket. If you forced WebSocket-only, add polling fallback (with sticky sessions); use TLS (`wss`) since proxies tamper less with encrypted traffic; check `connect_error` telemetry by network.

J2. [M] After adding a second server, clients randomly reconnect and some messages are missing. Missing sticky sessions (`Session ID unknown`) and/or missing adapter (broadcasts don't cross nodes). Add both; verify with clients on different nodes.

J3. [M] Messages appear twice in the chat UI. Duplicate listeners (registered in `connect` handler or effects without cleanup), multiple socket instances, or retries without dedupe. Fix registration, single instance, dedupe by message id/clientMsgId.

J4. [M] The client never receives the "welcome" event emitted right at connection. The client registered the listener after connecting (race). Register listeners before connecting (`autoConnect: false` → register → `connect()`), or request initial state via an ack.

J5. [S] The server misses the first event the client sends after connecting. Handlers registered after an `await` in an async `connection` handler — events arriving meanwhile have no listener and are dropped. Register handlers synchronously first; do async loading in middleware or afterwards.

J6. [M] CPU spikes to 100% and mass ping timeouts happen when a big room is active. Fan-out of a high-frequency event to many recipients and/or big payloads; event loop saturation. Throttle/coalesce, volatile, smaller payloads, shard rooms, scale out, profile.

J7. [M] Clients disconnect exactly every 60 s with "transport close". Proxy idle timeout — see G11.

J8. [M] `connect_error: xhr poll error` in production only. URL/port/path wrong, server down, CORS-masked proxy errors, TLS issues, proxy not routing `/socket.io/`. Curl the handshake, check proxy logs.

J9. [M] The WebSocket never upgrades — polling every 25 seconds in DevTools. Proxy missing Upgrade headers / HTTP 1.1; LB without WebSocket support. Fix proxy config.

J10. [S] Memory grows linearly with time even with constant connections. A leak: listeners/timers not cleaned on disconnect, pending acks without timeout, maps, CSR storage. Heap snapshots comparing before/after many connect/disconnect cycles.

J11. [S] After a deploy, the cluster went down repeatedly. Reconnection storm / thundering herd: all clients reconnect to remaining nodes at once, expensive handshakes (DB lookups) overwhelm them. Jitter + larger max delay, cheap handshakes (JWT), edge rate limits, slower rolling deploys, pre-scaling.

J12. [M] An ack callback receives `Error: operation has timed out` immediately on a broadcast. Broadcast with ack without `.timeout()`. Use `io.timeout(ms).emit(...)`.

J13. [S] A user removed from a group still receives group messages. Their sockets are still in the room. On membership change call `io.in("user:" + id).socketsLeave("group:" + gid)` (cluster-wide), and check membership in handlers rather than relying solely on room membership.

J14. [S] Some users see other tenants' notifications. Room name collisions (`"42"` used for several entity types/tenants), client-chosen rooms without authorization. Prefix rooms by type and tenant, authorize joins.

J15. [M] After upgrading the server to v4, old mobile apps can't connect. They use a v2 client (Engine.IO v3). Enable `allowEIO3: true` temporarily, then update the apps (compatible client versions).

J16. [M] `socket.emit` throws "is a reserved event name". Using `connect`, `disconnect`, etc. as custom names. Rename.

J17. [S] Presence flickers online/offline when users refresh. No grace period; per-socket presence. Count connections per user and delay offline announcements by a few seconds.

J18. [S] Redis restarted and cross-node broadcasts silently stopped on one node. Subscriber didn't resubscribe / client in a broken state; missing error handling. Ensure reconnect logic and error listeners, monitor adapter health, consider Streams adapter.

J19. [M] Large file upload disconnects the client. Exceeds `maxHttpBufferSize`. Chunk with acks or use HTTP upload; adjust limit carefully.

J20. [M] Node test runner/Jest hangs after Socket.IO tests. Open handles: clients not disconnected / server not closed. Clean up in `after`.

Part K — Coding questions (write the code)

K1. [E] Write a server that echoes every message back to the sender and broadcasts a join notice.

```
io.on("connection", (socket) => {
  socket.broadcast.emit("user:joined", socket.id);
  socket.on("echo", (msg, ack) => {
    socket.emit("echo", msg);
    if (typeof ack === "function") ack({ ok: true });
  });
});
```

K2. [M] Implement JWT auth middleware with an error payload.

```
io.use((socket, next) => {
  try {
    const p = jwt.verify(socket.handshake.auth?.token, PUBLIC_KEY, { algorithms: ["RS256"] });
    socket.data.user = { id: p.sub, roles: p.roles };
    next();
  } catch (e) {
    const err = new Error(e.name === "TokenExpiredError" ? "TOKEN_EXPIRED" : "UNAUTHORIZED");
    err.data = { reason: e.name };
    next(err);
  }
});
```

K3. [M] Implement chat rooms with join authorization and history in the ack. (See Ch. 4 worked example: `channel:join` checks ACL, `await socket.join`, notifies others with `socket.to`, acks with history.)

K4. [M] Implement a typing indicator with throttling and auto-expiry. (Ch. 13.3.)

K5. [M] Write a promise-based `request()` helper with timeout and error envelope on the client.

```
async function request(event, payload, ms = 5000) {
  const res = await socket.timeout(ms).emitWithAck(event, payload); // rejects on timeout
  if (!res.ok) throw Object.assign(new Error(res.error.code), res.error);
  return res.data;
}
```

K6. [M] Count online users across a cluster. Approximate & cheap: Redis counter/sets maintained in connection/disconnect (with TTL heartbeats). Exact but expensive: `(await io.fetchSockets()).length`; distinct users: `new Set((await io.fetchSockets()).map(s => s.data.user.id)).size`.

K7. [M] Kick a user from all devices.

```
io.in(`user:${userId}`).emit("session:revoked");
io.in(`user:${userId}`).disconnectSockets(true);
```

K8. [M] Write a per-socket rate limiter without libraries (token bucket).

```
io.on("connection", (socket) => {
  let tokens = 10, last = Date.now();
  socket.use(([event, ...args], next) => {
    const now = Date.now();
    tokens = Math.min(10, tokens + ((now - last) / 1000) * 5); // refill 5/s, burst 10
    last = now;
    if (tokens < 1) {
      const ack = typeof args.at(-1) === "function" ? args.at(-1) : null;
      return ack?.({ ok: false, error: { code: "RATE_LIMITED" } });
    }
    tokens -= 1;
    next();
  });
});
```

K9. [S] Write a handler wrapper that validates with zod, catches errors and always acks. (Ch. 3.10 / 13.17 `safe()`.)

K10. [S] Implement message sync on reconnect with dedupe on the client. (Ch. 5.9 option B.)

K11. [M] Test that a broadcast excludes the sender. (Ch. 10.3 test "room broadcast excludes sender".)

K12. [S] Set up the Redis adapter with error handling. (Ch. 8.4.)

K13. [M] React hook that subscribes to an event safely. (Ch. 9.2 `useSocketEvent`.)

K14. [S] Broadcast to all clients and collect their responses within 3 seconds.

```
try {
  const responses = await io.timeout(3000).emitWithAck("health:check");
```

```
  console.log(`${responses.length} clients answered`);
} catch (err) {
  // some clients did not respond in time
}
```

K15. [S] Chunked upload with acks and resume. (Ch. 13.10.)

Part L — System design rounds

L1. [S] Design a WhatsApp/Slack-like chat system. Structure your answer: 1. **Requirements:** 1:1 and group chat, delivery/read receipts, presence, typing, media, history, push notifications, multi-device; scale (e.g. 10M concurrent), latency (< 300 ms p95), durability (no message loss), ordering per conversation. 2. **Connection tier:** Socket.IO gateway nodes (stateless) behind L7 LB with sticky sessions (or WS-only for mobile), TLS; auth via token in `auth`; `user:<id>` rooms; heartbeat defaults; capacity ~50–100k connections/node → hundreds of nodes; graceful deploys. 3. **Message flow:** client `message:send` with `clientMsgId` → validate/authorize → persist (Cassandra/Scylla/Postgres partitioned by conversation, server-assigned time-ordered ids) → publish to fan-out (adapter or broker) → `io.to(conversation room or member user rooms).emit` → ack to sender. Idempotency via `clientMsgId`. 4. **Offline/missed delivery:** last-seen id sync on reconnect; push notifications for offline/background users; dedupe. 5. **Fan-out:** small groups → per-member user rooms; large channels → room subscriptions or pull-on-notify; Redis sharded pub/sub / Kafka for cross-node. 6. **Presence/typing:** Redis TTL heartbeats, grace periods, subscription-based presence broadcast; typing volatile & throttled. 7. **Receipts:** per-user read pointer per conversation; batched updates. 8. **Media:** HTTP upload to object storage (pre-signed URLs), message carries the URL. 9. **Ordering & consistency:** per-conversation ordering via server ids; clients sort. 10. **Security:** authz per conversation, rate limits, E2E encryption option. 11. **Observability & ops:** metrics, alerts, rolling deploys, multi-region.

L2. [S] Design a live notifications service for 50M users. Persist notifications; gateway with user rooms; event-driven pipeline from services via broker; presence-aware routing (only push via socket if online, else mobile push); read-state sync across devices; unread counters in Redis; backpressure and batching; priority lanes.

L3. [S] Design a live sports score/stock ticker for millions of viewers. Subscription rooms per match/symbol, snapshot on subscribe + coalesced deltas (e.g. 4/s), volatile delivery, each node consuming the feed and using `io.local` (avoid adapter fan-out), CDN for heavy data, WebSocket-only for efficiency, horizontal scaling by viewer count.

L4. [S] Design collaborative document editing (Google Docs). Rooms per document; CRDT (Yjs) or OT (ShareDB) for convergence; binary update relay; awareness/cursors volatile & throttled; persistence snapshots + update log; initial sync via state vectors; authorization per doc; ordering per doc (route a doc's editors to the same node or rely on CRDT commutativity); offline editing merges on reconnect.

L5. [S] Design a real-time multiplayer game backend. Matchmaking assigns matches to nodes (all players of a match on one node); authoritative simulation tick loop; clients send inputs with sequence numbers; server broadcasts compact snapshots (volatile, binary/msgpack) at 10–30 Hz; client prediction and interpolation; anti-cheat validation; reconnect within grace period resumes the same player slot (CSR or game-level session tokens).

L6. [S] Design a ride-hailing live location system. Drivers emit throttled volatile location; gateway writes latest to Redis GEO and batches history; trip rooms for riders; geo-cell rooms for dispatch; adaptive frequency; push and HTTP fallback for background; scale by region.

L7. [S] Design a live auction platform. Bids as acknowledged commands; atomic conditional DB/Redis updates; broadcast accepted state with versions; server-authoritative timers with clock-offset; anti-sniping; idempotency; audit log; high-traffic rooms coalesced.

Part M — Experience / behavioural questions (prepare your stories)

- "**Tell me about a real-time feature you built.**" Cover requirements, why Socket.IO, architecture (rooms, auth, scaling), a problem you hit (e.g. duplicate listeners, sticky sessions, proxy timeouts, reconnection storms), how you diagnosed it (metrics, DEBUG, DevTools), the fix, and the measurable result.
- "**How did you ensure reliability?**" Acks, idempotency, sync on reconnect, CSR, monitoring disconnect reasons, graceful deploys.
- "**How did you scale it?**" Adapter choice and why, stickiness approach, load tests and numbers, bottleneck found (fan-out, event loop) and fix.
- "**A production incident involving WebSockets?**" Use the runbook structure from Ch. 12.9.
- "**How do you keep the event API maintainable across teams?**" Contracts package, naming conventions, ack envelope, AsyncAPI docs, review checklist, versioning.

Part N — Rapid-fire (one-liners)

1. Default path? → `/socket.io/`.
2. Default pingInterval / pingTimeout? → 25 000 / 20 000 ms.
3. Default maxHttpBufferSize? → 1 MB (1e6).
4. Default connectTimeout? → 45 000 ms.
5. Default upgradeTimeout? → 10 000 ms.
6. Default client reconnection delay / max / jitter? → 1 000 ms / 5 000 ms / 0.5.

7. Default reconnectionAttempts? → Infinity.
8. Default client connection timeout? → 20 000 ms.
9. Is CORS enabled by default (v3+)? → No.
10. Is perMessageDeflate enabled by default? → No.
11. Is the `io` cookie enabled by default? → No.
12. Default transports (server)? → `["polling", "websocket"]`.
13. What is `io.sockets`? → Alias of `io.of("/")`.
14. Room every socket joins automatically? → Its own id.
15. Are rooms client-side? → No, server-only.
16. Event to read rooms before leaving? → `disconnecting`.
17. Where are reconnection events (v3+)? → `socket.io` (Manager).
18. Property telling if the client will auto-reconnect? → `socket.active`.
19. Promise-based ack API? → `emitWithAck`.
20. Ack timeout API? → `socket.timeout(ms)`.
21. Drop-if-not-ready modifier? → `volatile`.
22. Broadcast only to sockets connected to THIS node? → `io.local`.
23. Server-to-server events? → `serverSideEmit`.
24. Fetch sockets cluster-wide? → `fetchSockets()`.
25. Accept v2 clients? → `allowEIO3: true`.
26. Express middleware on engine? → `io.engine.use()` (4.6+).
27. Engine error "Session ID unknown" code? → 1.
28. Engine error for disabled transport? → 0 "Transport unknown".
29. Packet `42` means? → Engine.IO message + Socket.IO event.
30. Polling payload separator (EIO4)? → `\x1e`.
31. Binary over polling prefix? → `b` + base64.
32. Upgrade probe packets? → `2probe` / `3probe`, then `5`.
33. CSR introduced in? → 4.6.
34. WebTransport support introduced in? → 4.7.
35. uWebSockets.js support via? → `io.attachApp(app)` (4.4+).
36. Adapters supporting CSR? → in-memory, Redis Streams, MongoDB.
37. Package for sticky sessions in Node cluster? → `@socket.io/sticky`.
38. Package to emit from non-Socket.IO processes? → `@socket.io/redis-emitter` (and others).
39. Admin dashboard package? → `@socket.io/admin-ui`.
40. Can `socket.emit("disconnect")` be used? → No, reserved.

CHAPTER 15

Cheat Sheet, API Reference and Glossary

> Quick reference for daily work and last-minute interview revision. Verified against `socket.io@4.8.4` / `socket.io-client@4.8.4`.

15.1 Server API

`Server` (`io`)

API	Description
`new Server(httpServer?, options?)` / `new Server(port, options?)`	Create a server
`io.listen(port | httpServer)` / `io.attach(...)`	Attach/listen
`io.attachApp(uwsApp)`	Attach to uWebSockets.js (4.4+)
`io.on("connection", (socket) ⇒ …)`	New connection on main namespace (alias `connect`)
`io.use((socket, next) ⇒ …)`	Connection middleware (main namespace)
`io.of(name | RegExp | fn)`	Get/create a namespace (dynamic with RegExp/fn)
`io.sockets`	Main namespace (`io.of("/")`)
`io.emit(ev, ...args)`	Broadcast to all in main namespace
`io.to(room | room[])` / `io.in(...)`	Target rooms
`io.except(room | room[])`	Exclude rooms
`io.local`	Only this node
`io.volatile`	Droppable
`io.compress(bool)`	Compression flag
`io.timeout(ms)`	Ack timeout for broadcast with acks
`io.emitWithAck(...)` (with `timeout`)	Promise of responses array

API	Description
`io.fetchSockets()`	`Promise<(Socket|RemoteSocket)[]>` cluster-wide
`io.socketsJoin(rooms)` / `io.socketsLeave(rooms)`	Bulk join/leave
`io.disconnectSockets(close?)`	Bulk disconnect
`io.serverSideEmit(ev, ...args)` / `serverSideEmitWithAck`	Events to other servers
`io.on(ev, handler)` for server-side events	Receive `serverSideEmit` events
`io.allSockets()`	`Promise<Set<SocketId>>` (older API; prefer `fetchSockets`)
`io.close()`	Close all connections and the HTTP server (Promise)
`io.engine`	Engine.IO server: `clientsCount`, `generateId`, `use()`, events `connection`, `connection_error`, `headers`, `initial_headers`
`io.adapter(Adapter)`	Set adapter (or `adapter` option)
`io.path()`, `io.serveClient()`, `io.connectTimeout()`	Getters/setters

`Namespace`

`nsp.name`, `nsp.sockets` (Map of local sockets), `nsp.adapter`, `nsp.use()`, `nsp.on("connection")`, `nsp.emit`, `nsp.to/in/except`, `nsp.local`, `nsp.volatile`, `nsp.compress`, `nsp.timeout`, `nsp.fetchSockets`, `nsp.socketsJoin/socketsLeave`, `nsp.disconnectSockets`, `nsp.serverSideEmit`.

Server-side `Socket`

API	Description
`socket.id`	Socket id (per namespace)
`socket.handshake`	`{ headers, time, address, xdomain, secure, issued, url, query, auth }`
`socket.data`	Arbitrary per-socket data (serialized for `fetchSockets`)
`socket.rooms`	`Set<Room>` (includes own id)
`socket.nsp`	Namespace
`socket.conn`	Engine.IO socket (`transport.name`, `remoteAddress`, `on("upgrade")`, `writeBuffer`)

API	Description
`socket.request`	First HTTP request
`socket.connected` / `socket.disconnected`	State
`socket.recovered`	CSR succeeded (4.6+)
`socket.emit(ev, ...args[, ack])`	Emit to this client
`socket.emitWithAck(ev, ...args)`	Promise ack (use with `timeout`)
`socket.timeout(ms)`	Ack timeout (4.5+)
`socket.volatile`, `socket.compress(bool)`	Flags
`socket.broadcast`	Broadcast to all except this socket
`socket.to(room)` / `socket.in(room)` / `socket.except(room)`	Broadcast to rooms, excluding sender
`socket.join(room | room[])` / `socket.leave(room)`	Room membership
`socket.on/once/off/removeAllListeners/listeners`	Listeners
`socket.onAny/prependAny/offAny/listenersAny`	Catch-all incoming
`socket.onAnyOutgoing/prependAnyOutgoing/offAnyOutgoing/listenersAnyOutgoing`	Catch-all outgoing
`socket.use(([ev, ...args], next) ⇒ …)`	Per-event middleware
`socket.send(...args)`	Emit `"message"`
`socket.disconnect(close?)`	Leave namespace; `true` closes the connection
Events	`disconnecting`, `disconnect`, `error`

15.2 Client API

`io(url?, options?)` → `Socket`

API	Description
`socket.id`	Undefined until connected; changes on reconnect (unless recovered)

API	Description
`socket.connected` / `socket.disconnected` / `socket.active` / `socket.recovered`	State
`socket.io`	The Manager
`socket.auth`	Auth payload (object or function)
`socket.sendBuffer` / `socket.receiveBuffer`	Buffers
`socket.connect()` / `socket.open()`	Connect manually
`socket.disconnect()` / `socket.close()`	Disconnect (no auto-reconnect)
`socket.emit`, `emitWithAck`, `timeout`, `volatile`, `compress`, `send`	Emitting
`socket.on/once/off/listeners`, `onAny…`, `onAnyOutgoing…`	Listening
Events	`connect`, `connect_error`, `disconnect`

`Manager` (`socket.io`)

`socket.io.engine` (Engine.IO client: `transport.name`, `on("upgrade")`), `socket.io.opts`, `socket.io.reconnection(bool)`, `reconnectionAttempts(n)`, `reconnectionDelay(ms)`, `reconnectionDelayMax(ms)`, `randomizationFactor(n)`, `timeout(ms)`, `socket.io.socket(nsp)`. Events: `open`, `error`, `ping`, `packet`, `close`, `reconnect_attempt`, `reconnect`, `reconnect_error`, `reconnect_failed`.

15.3 Options with defaults

Server

Option	Default
`path`	`"/socket.io/"`
`serveClient`	`true`
`adapter`	in-memory
`parser`	default parser
`connectTimeout`	`45000`

Option	Default
`connectionStateRecovery`	off (`{ maxDisconnectionDuration: 120000, skipMiddlewares: true }` when enabled)
`cleanupEmptyChildNamespaces`	`false`
`pingInterval`	`25000`
`pingTimeout`	`20000`
`upgradeTimeout`	`10000`
`maxHttpBufferSize`	`1e6`
`allowRequest`	—
`transports`	`["polling", "websocket"]`
`allowUpgrades`	`true`
`perMessageDeflate`	`false`
`httpCompression`	`true` (threshold 1024)
`wsEngine`	`ws` Server
`cors`	— (disabled)
`cookie`	`false`
`allowEIO3`	`false`

Client

Option	Default
`reconnection`	`true`
`reconnectionAttempts`	`Infinity`
`reconnectionDelay`	`1000`
`reconnectionDelayMax`	`5000`
`randomizationFactor`	`0.5`
`timeout`	`20000`
`autoConnect`	`true`
`auth`	—
`ackTimeout` / `retries`	—

Option	Default
`path`	`"/socket.io/"`
`transports`	`["polling", "websocket", "webtransport"]`
`upgrade`	`true`
`rememberUpgrade`	`false`
`tryAllTransports`	`false`
`withCredentials`	`false`
`forceNew`	`false`
`multiplex`	`true`
`closeOnBeforeunload`	`false`
`query`, `extraHeaders`	—

15.4 Emit cheat sheet

```
socket.emit("e", data);                    // this client
io.emit("e", data);                        // everyone (main namespace)
socket.broadcast.emit("e", data);          // everyone except sender
io.to("r").emit("e", data);                // room r
socket.to("r").emit("e", data);            // room r except sender
io.to(["a", "b"]).emit("e");               // a ∪ b, once each
io.except("banned").emit("e");             // everyone except room
io.to(socketId).emit("e");                 // one socket
io.of("/admin").to("r").emit("e");         // room in namespace
io.local.emit("e");                        // this node only
socket.volatile.emit("e");                 // droppable
socket.compress(false).emit("e");          // no compression
socket.emit("e", data, (res) => {});       // ack
socket.timeout(5000).emit("e", (err, res) => {});          // ack + timeout
await socket.timeout(5000).emitWithAck("e", data);         // promise
io.timeout(5000).emit("e", (err, responses) => {});        // broadcast ack
io.serverSideEmit("e", data);              // other servers
```

15.5 Disconnect reasons

Server	Client
`server namespace disconnect`	`io server disconnect` (no auto-reconnect)
`client namespace disconnect`	`io client disconnect` (no auto-reconnect)
`server shutting down`	`ping timeout`
`ping timeout`	`transport close`
`transport close`	`transport error`
`transport error`	`parse error`
`parse error`	
`forced close`	
`forced server close`	

15.6 Protocol cheat sheet

Engine.IO: `0` open · `1` close · `2` ping · `3` pong · `4` message · `5` upgrade · `6` noop. Socket.IO: `0` CONNECT · `1` DISCONNECT · `2` EVENT · `3` ACK · `4` CONNECT_ERROR · `5` BINARY_EVENT · `6` BINARY_ACK. Format: `<type>[<attachments>-][<nsp>,][<ackId>][<json>]` → `42/admin,7["q"]`. Engine errors: `0` Transport unknown · `1` Session ID unknown · `2` Bad handshake method · `3` Bad request · `4` Forbidden · `5` Unsupported protocol version.

15.7 Ecosystem packages

Package	Purpose
`socket.io` / `socket.io-client`	Server / client
`engine.io` / `engine.io-client` / `engine.io-parser`	Low-level layer
`socket.io-parser` / `socket.io-msgpack-parser`	Parsers
`socket.io-adapter`	Base & in-memory adapter, `ClusterAdapter` base
`@socket.io/redis-adapter`	Redis pub/sub (and sharded) adapter
`@socket.io/redis-streams-adapter`	Redis Streams adapter

Package	Purpose
`@socket.io/mongo-adapter` / `@socket.io/postgres-adapter`	DB adapters
`@socket.io/cluster-adapter` / `@socket.io/sticky` / `@socket.io/pm2`	Single-machine multi-core
`@socket.io/gcp-pubsub-adapter` / `@socket.io/aws-sqs-adapter` / `@socket.io/azure-service-bus-adapter`	Cloud adapters
`@socket.io/redis-emitter` / `@socket.io/postgres-emitter` / `@socket.io/mongo-emitter`	Emit from other processes
`@socket.io/admin-ui`	Admin dashboard
`rate-limiter-flexible`, `zod`	Rate limiting, validation (third-party)
Artillery	Load testing

15.8 Glossary

- **Ack (acknowledgement):** callback-based reply to an emitted event.
- **Adapter:** stores rooms and performs broadcasts; distributed adapters link servers.
- **At-most-once / at-least-once / exactly-once:** delivery guarantees; Socket.IO default is at-most-once.
- **Back-off (exponential) & jitter:** growing randomized delays between reconnection attempts.
- **Broadcast:** one emit to many sockets.
- **Connection state recovery (CSR):** restoring id, rooms, data and missed packets after a short disconnection.
- **CSWSH:** Cross-Site WebSocket Hijacking.
- **Emitter:** library to emit through an adapter from non-Socket.IO processes.
- **Engine.IO:** low-level transport layer under Socket.IO.
- **Fan-out:** number of deliveries a broadcast generates.
- **Handshake:** first request establishing the session (`sid`, ping settings).
- **Heartbeat:** ping/pong to detect dead connections.
- **Idempotency:** processing a duplicate request has no extra effect.
- **Long polling:** server holds HTTP requests open until data is available.
- **Manager:** client object owning the connection and reconnection.
- **Multiplexing:** several namespaces sharing one connection.
- **Namespace:** client-chosen channel with its own handlers, middleware, rooms.
- **Parser:** encodes/decodes Socket.IO packets.
- **Presence:** online/offline tracking.

- **Room:** server-side group of sockets for broadcasting.
- **Sticky session (session affinity):** routing a client's requests to the same server.
- **Thundering herd:** many clients reconnecting simultaneously.
- **Transport:** polling, WebSocket, WebTransport.
- **Upgrade:** switching from polling to WebSocket.
- **Volatile:** an emit that may be dropped instead of buffered.

15.9 The 30-second final revision

1. Socket.IO ≠ WebSocket; Engine.IO underneath; polling → WebSocket upgrade.
2. Events + acks (+ timeout, emitWithAck), volatile, binary, reserved names.
3. Rooms (server-side, auto id-room, auto-leave) vs namespaces (client-chosen, own middleware).
4. `socket.to` excludes sender; `io.to` doesn't; multiple `to` = union.
5. Heartbeat: server pings 25 s / 20 s timeout; reconnection 1–5 s with jitter.
6. Disconnect reasons; no auto-reconnect after server/client disconnect or middleware rejection.
7. At-most-once by default; at-least-once = acks + retries + idempotency + sync.
8. CSR (4.6): restore session; limitations; adapter support.
9. Auth via `auth` + `io.use`; CORS ≠ WebSocket protection; check Origin; authorize every action.
10. Scale: sticky sessions + adapter (Redis/Streams…); emitters; proxy Upgrade headers & timeouts.
11. Production: graceful shutdown, metrics (disconnect reasons, event loop lag), leaks, versioning.
12. Common bugs: `app.listen`, CORS, path vs namespace, duplicate listeners, rooms lost on reconnect, async connection handler, missing sticky sessions/adapter, ack without timeout.

You've finished Part I. Part II goes beyond the core: Chapter 16 — Native mobile clients. Or go back to the table of contents.

CHAPTER 16

Native Mobile Clients: Android (Java/Kotlin), iOS (Swift) and Flutter (Dart)

> **Goal:** connect native mobile apps to a Socket.IO server correctly — installation, version compatibility, authentication, events, acknowledgements with timeouts, reconnection, threading, app lifecycle, background behaviour, and the bugs that only happen on phones.
>
> **How this chapter was checked:** the **Java** client code was compiled with Maven and run against a real Socket.IO 4.8.4 server (connect with `auth`, emit with an ack + timeout, server→client ack). The **Swift** and **Dart** APIs were checked line by line against the libraries' current source code and READMEs (`socket.io-client-swift` master, `socket_io_client` 3.1.6 on pub.dev); they could not be executed in this environment (no Xcode / Dart SDK).

16.1 ELI5

Your phone app wants the same walkie-talkie the website has. There are official walkie-talkies built for each kind of phone: one for Android (Java/Kotlin), one for iPhone (Swift), and a community one for Flutter (Dart). They all speak the same Socket.IO language — **but only if their version matches the server's version**, just like two walkie-talkies must be on the same channel.

Phones are special because: - they **sleep** — when the app goes to the background, the operating system pauses it and the connection dies; - their network **changes** all the time (Wi-Fi → 4G → elevator → no signal); - UI code must run on the **main thread**, but Socket.IO callbacks arrive on a background thread.

16.2 Version compatibility (check this FIRST)

Client	Version	Works with Socket.IO server
Java / Android (`io.socket:socket.io-client`)	1.x	2.x
	2.x (latest **2.1.2**)	**3.x and 4.x**
Swift (`socket.io-client-swift`)	15.x	2.x

Client	Version	Works with Socket.IO server
	16.x	2.x (with `.version(.two)`), **3.x and 4.x** (default)
Dart / Flutter (`socket_io_client`)	0.9.x – 1.x	2.x
	2.x	3.x – 4.6.x
	3.x (latest **3.1.6**)	**4.7.x – 4.x**

The Dart table is copied from the package's README. "Works in the browser but not on Android/iOS" is very often a client/server **protocol mismatch** — check this table before anything else.

16.3 Android — Java

Installation (Gradle)

```
// app/build.gradle.kts
dependencies {
    implementation("io.socket:socket.io-client:2.1.2") {
        // Android already ships org.json — exclude the duplicate to avoid conflicts
        exclude(group = "org.json", module = "json")
    }
}
```

```
<!-- AndroidManifest.xml -->
<uses-permission android:name="android.permission.INTERNET" />
```

Android 9+ blocks **cleartext HTTP** by default. Use `https://` in production; for local development either use HTTPS or allow cleartext only for your dev host with a `network_security_config`. On the **Android emulator**, your computer's `localhost` is `10.0.2.2`; on a real device use your computer's LAN IP.

A complete, verified example

This exact code (minus the Android-specific UI parts) was compiled and run against a Node.js Socket.IO 4.8.4 server:

```
import io.socket.client.Ack;
import io.socket.client.AckWithTimeout;
import io.socket.client.IO;
import io.socket.client.Socket;
```

```java
import org.json.JSONObject;

import java.net.URI;
import java.util.Collections;

IO.Options options = IO.Options.builder()
    .setAuth(Collections.singletonMap("token", accessToken)) // → socket.handshake.auth on the server
    .setTransports(new String[] { "websocket" })             // recommended on mobile (see 16.6)
    .setReconnectionDelay(1000)
    .setReconnectionDelayMax(5000)
    .build();

Socket socket = IO.socket(URI.create("https://api.example.com"), options);

socket.on(Socket.EVENT_CONNECT, args -> {
    System.out.println("connected, id=" + socket.id());
});

socket.on(Socket.EVENT_CONNECT_ERROR, args -> {
    // args[0] is an Exception; for middleware rejections its message is the server's err.message
    System.out.println("connect_error: " + args[0]);
});

socket.on(Socket.EVENT_DISCONNECT, args -> {
    System.out.println("disconnect: " + args[0]);   // e.g. "io server disconnect", "transport close"
});

// receive an event
socket.on("chat:message", args -> {
    JSONObject msg = (JSONObject) args[0];          // JSON objects arrive as org.json.JSONObject
    String text = msg.optString("text");
});

// server → client with acknowledgement: the Ack is the LAST argument
socket.on("question", args -> {
    Ack ack = (Ack) args[args.length - 1];
    ack.call("from java");
});

socket.connect();

// emit with acknowledgement AND timeout (AckWithTimeout, client 2.1+)
JSONObject payload = new JSONObject().put("a", 2).put("b", 40);
socket.emit("sum", new Object[] { payload }, new AckWithTimeout(5000) {
    @Override public void onSuccess(Object... res) { System.out.println("sum ack: " + res[0]); } // 42
    @Override public void onTimeout() { System.out.println("no answer in 5 s"); }
});

// simple emit and emit with a plain Ack (no timeout)
```

```
socket.emit("typing", new JSONObject().put("conversationId", "c1"));
socket.emit("ping-me", "hello", (Ack) res -> System.out.println(res[0]));
```

Real output of that run:

```
connected, id=Uu7oWvS3c96eMpVEAAAA
sum ack: 42
(server log) server got answer from java client: from java
```

Data type mapping (Java ↔ JSON)

JavaScript (server)	Java (client)
object	`org.json.JSONObject`
array	`org.json.JSONArray`
string / number / boolean	`String` / `Integer` / `Long` / `Double` / `Boolean`
`null`	`JSONObject.NULL`
`Buffer` / `ArrayBuffer`	`byte[]`

To send an object, build a `JSONObject` (or `JSONArray`). Binary: pass a `byte[]`.

Threading — the #1 Android bug

All Socket.IO callbacks run on the client's **internal event thread**, *not* the Android main (UI) thread. Touching views from them throws `CalledFromWrongThreadException` ("Only the original thread that created a view hierarchy can touch its views").

```
socket.on("chat:message", args -> {
    JSONObject msg = (JSONObject) args[0];
    runOnUiThread(() -> adapter.add(msg));     // Activity
    // or: new Handler(Looper.getMainLooper()).post(() -> ...);
    // or: liveData.postValue(msg);  (postValue is thread-safe; setValue is NOT)
});
```

Other Java client APIs worth knowing

```
socket.id();                    // null until connected
socket.connected();             // boolean
socket.isActive();              // will it reconnect automatically?
socket.off("chat:message");     // remove all listeners of an event
socket.off("chat:message", listener); // remove one (keep a reference to the Emitter.Listener!)
```

```
socket.off();                    // remove ALL listeners
socket.disconnect();             // no auto-reconnect after this
socket.io().on(Manager.EVENT_RECONNECT_ATTEMPT, args -> {});   // Manager-level events
socket.io().reconnection(false);

// other namespace on the same connection (multiplexing)
Socket admin = socket.io().socket("/admin");

// extra headers / query (Manager-level)
IO.Options.builder()
    .setExtraHeaders(Collections.singletonMap("X-App-Version",
Collections.singletonList("5.2.0")))
    .setQuery("platform=android")
    .build();
```

Note the difference between `setAuth(...)` (sent in the Socket.IO CONNECT packet — recommended for tokens) and `setQuery(...)` / `setExtraHeaders(...)` (HTTP-level; with WebSocket on Android, headers *can* be sent, unlike in browsers).

16.4 Android — Kotlin (same library, idiomatic usage)

Kotlin uses the same `io.socket:socket.io-client` library. A production-style wrapper with coroutines/Flow:

```
import io.socket.client.AckWithTimeout
import io.socket.client.IO
import io.socket.client.Socket
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.suspendCancellableCoroutine
import org.json.JSONObject
import java.net.URI
import kotlin.coroutines.resume
import kotlin.coroutines.resumeWithException

enum class ConnState { CONNECTING, CONNECTED, DISCONNECTED }

class RealtimeClient(url: String, private val tokenProvider: () -> String) {

    // The client keeps a reference to THIS map and reads it on every connection attempt,
    // so updating it before connect() sends the fresh token (verified with client 2.1.2).
    private val auth = HashMap<String, String>()

    private val socket: Socket = IO.socket(
```

```
        URI.create(url),
        IO.Options.builder()
            .setAuth(auth)
            .setTransports(arrayOf("websocket"))
            .setReconnectionDelayMax(10_000)
            .build()
    )

    private val _state = MutableStateFlow(ConnState.DISCONNECTED)
    val state: StateFlow<ConnState> = _state

    init {
        socket.on(Socket.EVENT_CONNECT) { _state.value = ConnState.CONNECTED }
        socket.on(Socket.EVENT_DISCONNECT) { _state.value = ConnState.DISCONNECTED }
        socket.on(Socket.EVENT_CONNECT_ERROR) { args ->
            _state.value = ConnState.DISCONNECTED
            if (!socket.isActive) {
                // rejected by server middleware → refresh the token and try again (or log out)
            }
        }
    }

    fun connect() {
        // refresh auth before EVERY connect so new connections use the latest token
        auth["token"] = tokenProvider()
        _state.value = ConnState.CONNECTING
        socket.connect()
    }

    fun disconnect() = socket.disconnect()

    /** Events as a cold Flow: listener added on collect, removed on cancel (no leaks). */
    fun events(name: String): Flow<JSONObject> = callbackFlow {
        val listener = io.socket.emitter.Emitter.Listener { args -> trySend(args[0] as
JSONObject) }
        socket.on(name, listener)
        awaitClose { socket.off(name, listener) }
    }

    /** Request/response with timeout, as a suspend function. */
    suspend fun request(event: String, payload: JSONObject, timeoutMs: Long = 5_000): Any? =
        suspendCancellableCoroutine { cont ->
            socket.emit(event, arrayOf(payload), object : AckWithTimeout(timeoutMs) {
                override fun onSuccess(vararg args: Any?) { cont.resume(args.firstOrNull()) }
                override fun onTimeout() { cont.resumeWithException(RuntimeException("ack
timeout: $event")) }
            })
        }
}
```

> **Refreshing the token (verified):** `Socket`'s own `auth` field is private, so you can't assign a new map. But the socket stores a reference to the map you passed to `setAuth(...)` and serializes it on every CONNECT. Keep that map in a field and update it before calling `connect()` — a test run printed `token-1` for the first connection and `token-2` for the second after `auth.put("token", "token-2")`. Note that *automatic* reconnections reuse whatever the map contains at that moment, so update it as soon as you refresh the token.

Usage in a ViewModel:

```
class ChatViewModel(private val rt: RealtimeClient) : ViewModel() {
    val messages = rt.events("chat:message")
        .map { Message.fromJson(it) }
        .shareIn(viewModelScope, SharingStarted.WhileSubscribed(5_000))

    fun send(text: String) = viewModelScope.launch {
        try {
            rt.request("message:send", JSONObject().put("text", text).put("clientMsgId",
UUID.randomUUID().toString()))
        } catch (e: Exception) { /* show retry */ }
    }
}
```

Collecting a Flow on the main dispatcher solves the threading problem automatically (`trySend` is thread-safe, collection happens where you collect).

Lifecycle on Android

- **Where to own the socket:** one instance for the whole app (Application-scoped singleton / DI `@Singleton`), not per Activity — otherwise rotations create new connections and duplicate listeners.
- **Foreground/background:** observe `ProcessLifecycleOwner`:

```
ProcessLifecycleOwner.get().lifecycle.addObserver(object : DefaultLifecycleObserver {
    override fun onStart(owner: LifecycleOwner) = rt.connect()     // app visible
    override fun onStop(owner: LifecycleOwner) = rt.disconnect()   // app in background
})
```

Keeping a socket open in the background drains battery and Android (Doze, App Standby) will cut network access anyway. - **Background delivery:** use **Firebase Cloud Messaging (FCM).** A Socket.IO connection is not a substitute for push notifications. If you truly need a long-lived connection while in background (e.g. a ride in progress), use a **foreground service** with a visible notification — and expect the OS to still be strict. - **Network changes:** register a `ConnectivityManager.NetworkCallback`; on

`onAvailable` call `socket.connect()` if not connected (faster than waiting for the back-off). - **ProGuard/R8:** the library works with R8; if you use reflection-based JSON mapping on top, keep your model classes.

16.5 iOS — Swift (`socket.io-client-swift`)

Installation

Swift Package Manager (Xcode → File → Add Packages), or `Package.swift`:

```
.package(url: "https://github.com/socketio/socket.io-client-swift", .upToNextMinor(from:
"16.1.1"))
// target dependency: "SocketIO"
```

CocoaPods: `pod 'Socket.IO-Client-Swift', '~> 16.1.1'`. The library depends on **Starscream** (WebSocket implementation).

Core concepts

- `SocketManager` = the connection (like the JS `Manager`): one per server URL. **You must keep a strong reference** to it (e.g. a property of a long-lived service). If the manager is deallocated, the socket silently disconnects — the most common iOS bug.
- `SocketIOClient` = one namespace socket: `manager.defaultSocket` (namespace `/`) or `manager.socket(forNamespace: "/admin")`.

A complete example

```
import SocketIO

final class RealtimeService {
    static let shared = RealtimeService()

    // keep STRONG references for the lifetime of the app
    private let manager: SocketManager
    private let socket: SocketIOClient

    private init() {
        manager = SocketManager(
            socketURL: URL(string: "https://api.example.com")!,
            config: [
                .log(false),
                .compress,
```

```
                .forceWebsockets(true),     // skip long-polling (see 16.6)
                .reconnects(true),
                .reconnectAttempts(-1),     // -1 = never give up
                .reconnectWait(1),          // seconds (min delay)
                .reconnectWaitMax(10),      // seconds (max delay)
                .randomizationFactor(0.5),
                .extraHeaders(["X-App-Version": "5.2.0"]),
                .connectParams(["platform": "ios"]),   // query string parameters
            ]
        )
        socket = manager.defaultSocket
        registerHandlers()
    }

    private func registerHandlers() {
        socket.on(clientEvent: .connect) { data, ack in
            print("connected", self.socket.sid ?? "")
        }
        socket.on(clientEvent: .disconnect) { data, ack in
            print("disconnected", data)
        }
        socket.on(clientEvent: .error) { data, ack in
            // connection errors, including server middleware rejections (connect_error)
            print("error", data)
        }
        socket.on(clientEvent: .reconnectAttempt) { data, ack in
            print("reconnect attempt", data)
        }

        socket.on("chat:message") { data, ack in
            guard let msg = data.first as? [String: Any], let text = msg["text"] as? String
else { return }
            print("message:", text)
        }

        // server → client with acknowledgement
        socket.on("question") { data, ack in
            ack.with("from swift")
        }
    }

    func connect(token: String) {
        // the payload becomes socket.handshake.auth on the server
        socket.connect(withPayload: ["token": token])
    }

    func disconnect() { socket.disconnect() }

    func send(text: String, conversationId: String) {
        socket.emitWithAck("message:send", ["conversationId": conversationId, "text": text])
            .timingOut(after: 5) { data in
                if data.first as? String ?? "passed" == SocketAckStatus.noAck {
                    print("server did not answer within 5 s")   // timeout
                    return
                }
```

```
                print("ack:", data)
            }
        }
    }
```

Facts to remember (verified in the library source):

JS concept	Swift API
`io(url, opts)`	`SocketManager(socketURL:config:)` + `manager.defaultSocket`
`auth` option	`socket.connect(withPayload: [...])`
`query`	`.connectParams([...])`
`transports: ["websocket"]`	`.forceWebsockets(true)` (`.forcePolling(true)` for polling only)
`reconnectionDelay` / `Max`	`.reconnectWait(sec)` / `.reconnectWaitMax(sec)` — **seconds**, not ms
`reconnectionAttempts`	`.reconnectAttempts(n)` (`-1` = infinite)
`forceNew`	`.forceNew(true)`
`path`	`.path("/socket.io/")`
v2 server	`.version(.two)` (default is `.three` = Socket.IO 3/4)
Self-signed cert (dev)	`.selfSigned(true)`; certificate pinning: `.security(...)`
`socket.on("connect")`	`socket.on(clientEvent: .connect)`
Client events	`.connect`, `.disconnect`, `.error`, `.ping`, `.pong`, `.reconnect`, `.reconnectAttempt`, `.statusChange`, `.websocketUpgrade`
emit with ack	`socket.emitWithAck("e", items).timingOut(after: seconds) { data in }` — timeout detected by comparing with `SocketAckStatus.noAck`
reply to an ack	`ack.with(items...)`
other namespace	`manager.socket(forNamespace: "/admin")`
`socket.id`	`socket.sid`

No offline buffering in the Swift client

Unlike the JavaScript and Java clients, the Swift client **does not buffer** emits while disconnected: `emit` on a socket that isn't connected triggers the `.error` client event with `"Tried emitting when not connected"` and the event is dropped (from `SocketIOClient.swift`). Check `socket.status == .connected` before emitting, and keep important messages in your own outbox (Chapter 21) until they're acknowledged.

Also: a server middleware rejection (`CONNECT_ERROR`) arrives as the `"error"` event on the Swift socket, with the error payload as data.

Threading on iOS

By default handlers are called on the **main queue** (`handleQueue` defaults to `DispatchQueue.main`), so you can update UIKit/SwiftUI directly. For heavy processing, set `.handleQueue(someBackgroundQueue)` and hop back to main for UI.

SwiftUI integration

```
@MainActor
final class ChatStore: ObservableObject {
    @Published var messages: [String] = []
    @Published var connected = false
    private let manager = SocketManager(socketURL: URL(string: "https://api.example.com")!,
                                        config: [.forceWebsockets(true)])
    private var socket: SocketIOClient { manager.defaultSocket }

    init() {
        socket.on(clientEvent: .connect) { [weak self] _, _ in self?.connected = true }
        socket.on(clientEvent: .disconnect) { [weak self] _, _ in self?.connected = false }
        socket.on("chat:message") { [weak self] data, _ in
            if let m = data.first as? [String: Any], let t = m["text"] as? String {
self?.messages.append(t) }
        }
    }
    func start(token: String) { socket.connect(withPayload: ["token": token]) }
}
```

Use `[weak self]` in handlers to avoid retain cycles (the socket holds the closures; the store holds the socket).

iOS lifecycle

- iOS suspends apps a few seconds after they go to background; the connection dies. Observe `scenePhase` (SwiftUI) or `UIApplication.didEnterBackgroundNotification` and disconnect cleanly; reconnect on `.active` .
- Use **APNs push notifications** for background delivery.
- **App Transport Security (ATS)** blocks plain `http://` by default — use HTTPS in production; for local dev add an ATS exception only for your dev host.
- Simulator can reach your Mac's `localhost` directly; a real device needs your Mac's LAN IP.

16.6 Why `websocket` -only is common on mobile

- Mobile apps control their networking stack (no corporate proxies inside the app as with enterprise browsers), and WebSocket is reliable on 4G/5G/Wi-Fi.
- It removes the need for **sticky sessions** on your load balancer.
- Fewer HTTP requests → less battery and data.
- On iOS/Android, polling implementations differ subtly from browsers' XHR; WebSocket-only avoids a class of issues.

Keep polling enabled only if your users are known to be behind WebSocket-hostile networks (some enterprise MDM VPNs).

16.7 Flutter / Dart (`socket_io_client`)

Installation

```
# pubspec.yaml
dependencies:
  socket_io_client: ^3.1.6   # for Socket.IO server 4.7+; see the compatibility table
```

A complete example

```
import 'package:socket_io_client/socket_io_client.dart' as IO;

class RealtimeService {
  late final IO.Socket socket;

  RealtimeService(String url, String Function() tokenProvider) {
    socket = IO.io(
      url,
      IO.OptionBuilder()
          .setTransports(['websocket'])      // REQUIRED for Flutter / Dart VM (see note)
          .disableAutoConnect()              // connect explicitly
          .setAuth({'token': tokenProvider()})
          .setReconnectionDelay(1000)
          .setReconnectionDelayMax(5000)
          .setExtraHeaders({'x-app-version': '5.2.0'})
          .build(),
    );

    socket.onConnect((_) => print('connected ${socket.id}'));
    socket.onConnectError((err) => print('connect_error $err'));
```

```
    socket.onDisconnect((reason) => print('disconnected $reason'));

    socket.on('chat:message', (data) {
      final text = (data as Map)['text'];
      print('message: $text');
    });

    // server -> client with ack. When an event carries MORE than one value (here: the
    // payload + the ack function), the handler receives ONE List; the ack is its last item.
    socket.on('question', (data) {
      final list = data as List;
      final question = list.first;
      final ack = list.last as Function;
      ack('from dart');     // callable once; later calls are ignored
    });
  }

  void connect() => socket.connect();     // don't call connect() if autoConnect is enabled
  void disconnect() => socket.disconnect();

  // emit with ack (callback)
  void send(String text) {
    socket.emitWithAck('message:send', {'text': text}, ack: (res) {
      print('ack: $res');
    });
  }

  // emit with ack (Future) + timeout
  Future<dynamic> request(String event, dynamic data) {
    return socket.timeout(5000).emitWithAckAsync(event, data);
  }

  void dispose() {
    socket.dispose();   // removes listeners and closes — call in your widget/service dispose
  }
}
```

Facts (verified in the package source, v3.1.6):

- `OptionBuilder` methods include `setTransports`, `setAuth` (and `setAuthFn` for a function computed per connection), `setQuery`, `setPath`, `setExtraHeaders`, `enableAutoConnect` / `disableAutoConnect`, `enableReconnection` / `disableReconnection`, `setReconnectionAttempts`, `setReconnectionDelay`, `setReconnectionDelayMax`, `setRandomizationFactor`, `setTimeout`, `enableForceNew`, `enableMultiplex` / `disableMultiplex`, `enableWithCredentials`, `setRememberUpgrade`, `setUpgrade`, `setParser`, `enableAddTrailingSlash` / `disableAddTrailingSlash`.
- `emitWithAck(event, data, {ack})` and `emitWithAckAsync(event, data)` (a `Future`); `socket.timeout(ms)` exists; with a timeout, the Future completes with an error when the server doesn't answer.
- Client `retries` (at-least-once) are supported as in the JS client.
- `socket.active` mirrors the JS property.

- `setTransports(['websocket'])` **is required on Flutter mobile/desktop** (the README states it); the polling transport is meant for Flutter Web.
- **Receiving multiple arguments:** the Dart client calls your handler with a single value when the event has one argument, but with a `List` **of all arguments** when it has more than one (including the ack function). So a server `socket.emit("move", x, y)` arrives as `[x, y]` in Dart. Prefer a single object payload on the server — it keeps every platform simple.
- **Data shape:** if you emit a `List`, its items are spread as separate arguments (the source does `sendData.addAll(data)` for iterables). To send a list as **one** argument, wrap it: `socket.emit('items', [myList])` or send a `Map`.

Flutter lifecycle

- Own the socket in a long-lived service (Provider/Riverpod/GetIt), not in a widget's `build`.
- If a `StatefulWidget` registers handlers, remove them in `dispose()` (`socket.off('event', handler)`), or use `socket.dispose()` when the whole service goes away.
- Use `WidgetsBindingObserver.didChangeAppLifecycleState` to disconnect in `paused` and reconnect in `resumed`.
- Background delivery → FCM/APNs (e.g. `firebase_messaging`).

16.8 Mobile-specific production checklist

- [ ] Client library version compatible with server version (16.2 table).
- [ ] HTTPS/WSS (ATS on iOS, cleartext policy on Android).
- [ ] One socket instance per app (singleton/DI); strong reference to `SocketManager` on iOS.
- [ ] Handlers update UI on the main thread (Android) / handlers don't create retain cycles (iOS).
- [ ] Listeners removed when screens are destroyed (or use Flows/Observables with teardown).
- [ ] Token passed via `auth` (`setAuth`, `connect(withPayload:)`), refreshed before reconnecting.
- [ ] Disconnect on background, reconnect on foreground; push notifications for background delivery.
- [ ] Re-sync missed data on every connect (last message id) — mobile networks drop often.
- [ ] `websocket`-only transport unless polling is genuinely needed.
- [ ] Send app version in `auth`/headers; server rejects too-old clients with a clear error.
- [ ] Test on real devices: airplane mode on/off, Wi-Fi ↔ cellular switch, lock screen, low-power mode.

16.9 Interview questions for this chapter

Q1. [M] Which Java client version works with a Socket.IO v4 server? The 2.x line (latest 2.1.2). 1.x speaks the v2 protocol.

Q2. [M] My Android app crashes with "Only the original thread that created a view hierarchy can touch its views" when a message arrives. Why? Socket.IO Java callbacks run on the client's event thread, not the main thread. Post to the UI thread (`runOnUiThread`, `Handler(Looper.getMainLooper())`, `LiveData.postValue`, or collect a Flow on the main dispatcher).

Q3. [M] My iOS socket disconnects immediately after connecting, with no error. Why? Most often the `SocketManager` was a local variable and got deallocated. Keep a strong reference (property of a long-lived object).

Q4. [M] How do you send an auth token from Swift / Java / Dart? Swift: `socket.connect(withPayload: ["token": t])`. Java: `IO.Options.builder().setAuth(map)`. Dart: `OptionBuilder().setAuth({...})` or `setAuthFn`. All land in `socket.handshake.auth` on the server.

Q5. [M] How do you do an ack with timeout on each platform? Java: `AckWithTimeout(ms)` with `onSuccess` / `onTimeout`. Swift: `emitWithAck(...).timingOut(after: seconds) { data in }` and check `SocketAckStatus.noAck`. Dart: `socket.timeout(ms).emitWithAckAsync(...)`. JS: `socket.timeout(ms).emitWithAck(...)`.

Q6. [S] How do mobile apps get messages while in the background? They don't, through Socket.IO — the OS suspends the app. Use FCM/APNs push, then sync over the socket/HTTP when the app returns to the foreground.

Q7. [M] Why do many teams use WebSocket-only transport on mobile? Reliability, no sticky sessions needed, fewer requests (battery), and polling implementations on mobile platforms are less battle-tested than browsers'.

Q8. [M] Why is `Swift .reconnectWait(1)` different from JS `reconnectionDelay: 1000`? Same meaning, different unit — Swift uses seconds, JS/Java use milliseconds.

Q9. [M] A Flutter client connects in Flutter Web but not on Android. What's the first thing to check? `setTransports(['websocket'])` (required on Flutter mobile per the package docs), then the package/server version compatibility table and cleartext/HTTPS settings.

Next: Chapter 17 — Servers and clients in other languages

CHAPTER 17

Socket.IO Servers and Clients in Other Languages (Python, Go, Rust, Java)

> **Goal:** run Socket.IO servers outside Node.js — Python (FastAPI/ASGI, aiohttp, Flask-SocketIO), Go, Rust and Java — and know exactly what is the same, what differs, and what breaks when you mix languages.
>
> **How this chapter was checked:** every server below was **built and run**, and a real **Node.js** `socket.io-client` **4.8.4** connected to it and exercised authentication rejection, events, acknowledgements (both directions where supported) and room broadcasts excluding the sender. Versions used: `python-socketio` 5.17.0 / `python-engineio` 4.14.0 / `Flask-SocketIO` 5.7.0, Go `github.com/zishang520/socket.io/servers/socket/v3` v3.0.6, Rust `socketioxide` 0.18.7 with `axum` 0.8, Java `netty-socketio` 2.0.13.

17.1 ELI5

Socket.IO is a *language* (a protocol). The Node.js library is the most famous speaker, but people have written speakers in Python, Go, Rust, Java and more. As long as both sides speak the **same version** of the language (Engine.IO v4 + Socket.IO v5 for modern clients), a browser using `socket.io-client` can't tell which language the server is written in.

What does **not** travel between languages: the **multi-server plumbing**. A Python server's Redis "message queue" and the Node Redis adapter speak different internal formats — you can't put Python and Node servers in the same cluster and expect broadcasts to cross.

17.2 Choosing a server implementation

Language	Library	Protocol (modern clients)	Scaling across servers	Notes
Node.js	`socket.io`	✅ reference implementation	Adapters (Redis, Streams, Mongo, Postgres, ...)	Everything in this book

Language	Library	Protocol (modern clients)	Scaling across servers	Notes
Python	`python-socketio` (+ `python-engineio`)	✅	"Client managers": Redis, Kafka, RabbitMQ/AioPika, Kombu, ZeroMQ	asyncio (ASGI/aiohttp) and WSGI (Flask, threading/eventlet/gevent)
Python/Flask	`Flask-SocketIO`	✅ (built on python-socketio)	Same managers via `message_queue=`	Flask-style decorators, request context
Go	`github.com/zishang520/socket.io` (v3 module path `…/servers/socket/v3`)	✅ (Socket.IO v4+ protocol)	Adapters (see its docs)	Close port of the Node API: `On`, `Emit`, `To`, `Use`, `Of`
Go	`googollee/go-socket.io`	⚠️ historically v2-era protocol	—	Check protocol support before choosing
Rust	`socketioxide`	✅ v5 (v4 behind a feature flag)	`socketioxide-redis`, `-mongodb`, `-postgres`	Tower layer; axum/hyper integration; extractors
Java	`netty-socketio`	✅ claims client 1.x–4.x (verified events/acks/rooms with a 4.8.4 client)	Redisson / Hazelcast store	Netty-based; see caveats in 17.6
Deno	official Deno port	✅	—	See Chapter 20

Choose based on your team's language, and check: protocol version, acknowledgement support, binary support, namespaces, scaling story, maintenance activity.

17.3 Python — `python-socketio`

Install

```
pip install python-socketio uvicorn fastapi        # asyncio + ASGI
pip install python-socketio aiohttp                # asyncio + aiohttp
pip install flask flask-socketio simple-websocket  # Flask (threading mode with WebSocket support)
pip install redis                                  # for the Redis manager
```

Concepts mapped from Node.js

Node.js	python-socketio
`new Server(opts)`	`socketio.AsyncServer(async_mode="asgi", ...)` (asyncio) or `socketio.Server(...)` (WSGI/threads)
`io.on("connection", socket ⇒ …)`	`@sio.event async def connect(sid, environ, auth)`
`socket` object	**a `sid` string** — every API takes the sid
`socket.data`	`await sio.save_session(sid, {...})` / `await sio.get_session(sid)` / `async with sio.session(sid) as s:`
`next(new Error("x"))` in middleware	`raise socketio.exceptions.ConnectionRefusedError("x")` (or `return False`) in `connect`
`socket.on("e", (data, ack) ⇒ ack(x))`	`@sio.on("e") async def h(sid, data): return x` — **the return value is the ack**
`socket.emit("e", data)`	`await sio.emit("e", data, to=sid)`
`io.to(room).emit`	`await sio.emit("e", data, room=room)` (or `to=room`)
`socket.to(room).emit` (exclude sender)	`await sio.emit("e", data, room=room, skip_sid=sid)`
`socket.join(room)` / `leave`	`await sio.enter_room(sid, room)` / `await sio.leave_room(sid, room)`
`socket.timeout(ms).emitWithAck(...)`	`await sio.call("e", data, to=sid, timeout=5)` (seconds)
emit with ack callback	`await sio.emit("e", data, to=sid, callback=fn)`
`socket.disconnect()`	`await sio.disconnect(sid)`
namespaces	`namespace="/admin"` argument everywhere, or class-based `socketio.AsyncNamespace`
`pingInterval: 25000` (ms)	`ping_interval=25` (**seconds**)
`maxHttpBufferSize`	`max_http_buffer_size=1000000`
`cors: { origin }`	`cors_allowed_origins=[...]` (`"*"` for all; `[]` to disable checks)
adapter (Redis)	`client_manager=socketio.AsyncRedisManager(url)`
emitter (Redis)	`socketio.AsyncRedisManager(url, write_only=True)`

Defaults (from the `python-engineio` 4.14 constructor): `ping_interval=25`, `ping_timeout=20`, `max_http_buffer_size=1000000`, `allow_upgrades=True`, `http_compression=True`, `compression_threshold=1024`, `cors_credentials=True`, `async_handlers=True` — the same values as Node's defaults, but in seconds.

Ordering difference: with `async_handlers=True` (the default), python-socketio runs each incoming event in its **own task**, so two events from the same client can be *processed* concurrently and finish out of order — even though they *arrived* in order. If order matters, set `async_handlers=False` (events of a client are then processed one after another) or serialize per key yourself.

FastAPI + Socket.IO on one port (ASGI) — verified

```
# server.py
import socketio
from fastapi import FastAPI

sio = socketio.AsyncServer(async_mode="asgi", cors_allowed_origins=["http://localhost:5173"])
api = FastAPI()

@api.get("/health")
async def health():
    return {"ok": True}

# Socket.IO handles /socket.io/, everything else goes to FastAPI
app = socketio.ASGIApp(sio, other_asgi_app=api)

@sio.event
async def connect(sid, environ, auth):
    if not auth or auth.get("token") ≠ "good":
        raise socketio.exceptions.ConnectionRefusedError("unauthorized")   # → client
connect_error
    await sio.save_session(sid, {"user_id": "u1"})                         # ≈ socket.data
    await sio.enter_room(sid, "user:u1")                                   # per-user room
    await sio.emit("welcome", {"sid": sid}, to=sid)

@sio.on("sum")
async def sum_(sid, data):
    return data["a"] + data["b"]          # return value = acknowledgement

@sio.on("join")
async def join(sid, room):
    await sio.enter_room(sid, room)
    await sio.emit("joined", {"room": room}, room=room, skip_sid=sid)     # room, excluding
sender
    return {"ok": True}

@sio.on("ask-client")
async def ask_client(sid):
    answer = await sio.call("question", "2+2?", to=sid, timeout=5)     # server → client with
ack
    return {"client_said": answer}

@sio.event
```

```
async def disconnect(sid, reason):
    print("disconnect", sid, reason)
```

Run: `uvicorn server:app --port 8765`.

What the Node.js client observed (real output):

```
bad token → unauthorized active: false        ← middleware-style rejection, no auto-retry
welcome string
sum ack: 5
join ack: { ok: true }
call: { client_said: 4 }                      ← server-initiated ack round trip
transport: websocket                          ← upgraded from polling
(server) disconnect GE5M_tNyD_-xOgj_AAAD client disconnect
```

> Note the `disconnect(sid, reason)` signature: recent python-socketio versions pass the reason (e.g. `"client disconnect"`, `"server disconnect"`, `"ping timeout"`, `"transport close"`); older versions call `disconnect(sid)` only.

Class-based namespaces

```
class ChatNamespace(socketio.AsyncNamespace):
    async def on_connect(self, sid, environ, auth):
        await self.enter_room(sid, "lobby")

    async def on_message(self, sid, data):          # handles the "message" event
        await self.emit("message", data, room="lobby", skip_sid=sid)
        return {"ok": True}

    async def on_disconnect(self, sid, reason):
        pass

sio.register_namespace(ChatNamespace("/chat"))
```

Method name = `on_` + event name. This is the Python equivalent of feature-based handler modules.

aiohttp

```
from aiohttp import web
import socketio

sio = socketio.AsyncServer(async_mode="aiohttp")
app = web.Application()
sio.attach(app)          # serves /socket.io/ on the aiohttp app
```

```
if __name__ == "__main__":
    web.run_app(app, port=8000)
```

Flask-SocketIO — verified

```
from flask import Flask, request
from flask_socketio import SocketIO, emit, join_room, ConnectionRefusedError

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins=["http://localhost:5173"])

@socketio.on("connect")
def on_connect(auth):
    if not auth or auth.get("token") != "good":
        raise ConnectionRefusedError("unauthorized")
    join_room("lobby")                              # uses request.sid implicitly
    emit("welcome", {"sid": request.sid})           # to this client

@socketio.on("chat")
def on_chat(data):
    emit("chat", data, to="lobby", include_self=False)   # room broadcast excluding sender
    return {"ok": True}                                   # ack

if __name__ == "__main__":
    socketio.run(app, port=8780, allow_unsafe_werkzeug=True)  # dev server only
```

Observed with two Node clients: `welcome received`, the second client got `{ text: 'hi' }`, and the sender received the ack `{ ok: true }`.

Flask-SocketIO specifics: - `request.sid` is the current client's sid; `emit` / `send` / `join_room` without `to=` target the current client (context-aware, like Flask's `request`). - `include_self=False` ≈ Node's `socket.to(room)` / `socket.broadcast`. - **Production async modes:** `threading` (with `simple-websocket` for WebSocket), `eventlet` or `gevent` (with monkey patching). The Werkzeug dev server (`allow_unsafe_werkzeug=True`) is for development only — use gunicorn with an appropriate worker (e.g. `gunicorn -k eventlet -w 1 app:app` or `--threads` for threading mode). **One worker process per gunicorn instance**: multiple workers in one gunicorn need sticky sessions, which gunicorn's load balancing can't do — run several single-worker instances behind nginx with sticky sessions + a `message_queue` instead. - Scale-out: `SocketIO(app, message_queue="redis://...")`.

Scaling Python servers with Redis — verified

```
import socketio

mgr = socketio.AsyncRedisManager("redis://localhost:6379/0")
sio = socketio.AsyncServer(async_mode="asgi", client_manager=mgr)
app = socketio.ASGIApp(sio)
```

```
@sio.event
async def connect(sid, environ, auth):
    await sio.enter_room(sid, "news")

@sio.on("shout")
async def shout(sid, msg):
    await sio.emit("news", msg, room="news")    # reaches clients on ALL servers
```

Test: two uvicorn processes on ports 8771 and 8772, client A on 8771 emits `shout`, client B on 8772 printed `client on server B got: hello from server A`.

Other managers: `AsyncAioPikaManager` (RabbitMQ), `KafkaManager`, `KombuManager`, `ZmqManager` (and sync versions `RedisManager` etc.). As in Node, **sticky sessions are required** when polling is enabled.

Emitting from a worker process (write-only manager) — verified

```
import asyncio, socketio

async def main():
    external = socketio.AsyncRedisManager("redis://localhost:6379/0", write_only=True)
    await external.emit("news", "hello from a worker process", room="news")

asyncio.run(main())
```

A Node client connected to a Python server received `client got: hello from a worker process`. This is the Python equivalent of `@socket.io/redis-emitter`. (Celery tasks commonly use this to push progress updates.)

The Python client

```
import asyncio, socketio

async def main():
    sio = socketio.AsyncClient(reconnection=True)

    @sio.event
    async def connect():
        print("connected", sio.sid)

    @sio.on("news")
    async def on_news(msg):
        print("got", msg)

    await sio.connect("https://api.example.com", auth={"token": "..."}, transports=
["websocket"])
    result = await sio.call("sum", {"a": 1, "b": 2}, timeout=5)    # emit with ack
    await sio.emit("typing", {"conversationId": "c1"})
    await sio.wait()                                               # keep running
```

```
asyncio.run(main())
```

It works against Node servers too (the Python client and server both speak the standard protocol). There's also `socketio.SimpleClient` / `AsyncSimpleClient` for scripts: `client.connect(url)`, `client.emit(...)`, `event = client.receive()`.

Python-specific pitfalls

- **Units:** seconds everywhere (`ping_interval`, `timeout`), Node uses ms.
- **Blocking calls** in async handlers (sync DB drivers, `time.sleep`, `requests`) block the event loop → ping timeouts. Use async libraries or `asyncio.to_thread`.
- **Monkey patching** (eventlet/gevent) must happen before other imports.
- `cors_allowed_origins` default (`None`) only allows the same origin as the request's host (and the `X-Forwarded-Proto/Host` variant behind a proxy); set it explicitly.
- **Origin checks apply to WebSocket too in Python — unlike Node.** Tested with a raw WebSocket client sending a forged `Origin`:

Server	Allowed origin	`Origin: https://evil.example`
python-socketio (`cors_allowed_origins=["http://localhost:5173"]`)	accepted	**rejected, HTTP 403**
Node socket.io (`cors: { origin: "http://localhost:5173" }`)	accepted	**accepted**

So python-engineio gives you Cross-Site WebSocket Hijacking protection out of the box, while in Node you must add an `allowRequest` origin check yourself (Chapter 6.5). - Concurrent handler execution (`async_handlers=True`) → ordering surprises (see above). - Testing: `socketio.AsyncClient` against a server started in the test, or `flask_socketio.SocketIOTestClient` (`socketio.test_client(app)`) for Flask.

17.4 Go — `github.com/zishang520/socket.io` — verified

```
go get github.com/zishang520/socket.io/servers/socket/v3
```

```
package main

import (
    "log"
    "net/http"
```

```
    "github.com/zishang520/socket.io/servers/socket/v3"
)

func main() {
    server := socket.NewServer(nil, nil)

    // connection middleware: reject unauthenticated clients with a CONNECT_ERROR
    server.Use(func(client *socket.Socket, next func(*socket.ExtendedError)) {
        if token, _ := client.Handshake().Auth["token"].(string); token != "good" {
            next(socket.NewExtendedError("unauthorized", map[string]any{"code": "AUTH"}))
            return
        }
        next(nil)
    })

    server.On("connection", func(clients ...any) {
        client := clients[0].(*socket.Socket)
        client.Join("lobby")
        client.Emit("welcome", map[string]any{"sid": string(client.Id())})

        // the ack callback is the LAST argument
        client.On("sum", func(args ...any) {
            p := args[0].(map[string]any)
            ack := args[len(args)-1].(socket.Ack)
            ack([]any{p["a"].(float64) + p["b"].(float64)}, nil) // JSON numbers decode as
float64
        })

        client.On("chat", func(args ...any) {
            client.To("lobby").Emit("chat", args[0]) // room, excluding sender
            if ack, ok := args[len(args)-1].(socket.Ack); ok {
                ack([]any{map[string]any{"ok": true}}, nil)
            }
        })

        client.On("disconnect", func(reason ...any) {
            log.Println("disconnect", client.Id(), reason[0])
        })
    })

    http.Handle("/socket.io/", server.ServeHandler(nil))
    log.Fatal(http.ListenAndServe(":8795", nil))
}
```

Real output with Node clients: `bad → unauthorized active: false`, `sum: 42`, `chat ack: { ok: true }`, the other client received `{ text: 'hi go' }`, the sender did not; server logged `disconnect … client namespace disconnect`.

Go-specific notes: - Handlers are variadic `func(args ...any)`; you type-assert. **JSON numbers arrive as `float64`**; objects as `map[string]any`. - The ack is `socket.Ack = func([]any, error)`; call it with a slice of response values. - API mirrors Node: `server.Of("/admin", nil)`, `server.To(room).Emit`,

`client.Timeout(d)`, `client.EmitWithAck`, `socket.DefaultServerOptions()` with `SetPingInterval`, `SetMaxHttpBufferSize`, `SetCors(&types.Cors{...})`. - Integrates with `net/http`, fasthttp and Fiber (via adaptors) through `server.ServeHandler(opts)`. - `DEBUG=socket.io*` enables logs, like Node.

17.5 Rust — `socketioxide` — verified

```toml
# Cargo.toml
[dependencies]
socketioxide = { version = "0.18", features = ["state"] }
axum = "0.8"
tokio = { version = "1", features = ["full"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
anyhow = "1"
```

```rust
use axum::routing::get;
use serde::Deserialize;
use serde_json::{json, Value};
use socketioxide::{
    extract::{AckSender, Data, SocketRef, TryData},
    handler::ConnectHandler,
    SocketIo,
};

#[derive(Debug, Deserialize)]
struct Auth { token: String }

#[derive(Debug, Deserialize)]
struct Sum { a: i64, b: i64 }

// connect middleware: Err(...) rejects the connection with a CONNECT_ERROR (message = Display)
async fn auth_middleware(TryData(auth): TryData<Auth>) -> Result<(), anyhow::Error> {
    match auth {
        Ok(a) if a.token == "good" => Ok(()),
        _ => Err(anyhow::anyhow!("unauthorized")),
    }
}

async fn on_connect(socket: SocketRef) {
    socket.join("lobby");
    socket.emit("welcome", &json!({ "sid": socket.id.to_string() })).ok();

    socket.on("sum", async |Data(p): Data<Sum>, ack: AckSender| {
        ack.send(&(p.a + p.b)).ok();
    });

    socket.on("chat", async |s: SocketRef, Data(msg): Data<Value>, ack: AckSender| {
```

```
        s.to("lobby").emit("chat", &msg).await.ok();   // to room, excluding sender
        ack.send(&json!({ "ok": true })).ok();
    });

    socket.on_disconnect(async |s: SocketRef| {
        println!("disconnected {}", s.id);
    });
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let (layer, io) = SocketIo::new_layer();
    io.ns("/", on_connect.with(auth_middleware));      // namespace + connect middleware

    let app = axum::Router::new()
        .route("/health", get(|| async { "ok" }))
        .layer(layer);                                  // Socket.IO is a Tower layer

    let listener = tokio::net::TcpListener::bind("0.0.0.0:8790").await?;
    axum::serve(listener, app).await?;
    Ok(())
}
```

Real output with Node clients: `bad → unauthorized active: false`, `sum: 42`, `chat ack: { ok: true }`, the other client got `{ text: 'hi rust' }`, the sender didn't.

Rust-specific notes: - **Extractors** (axum-style): `SocketRef`, `Data<T>` (deserialized payload), `TryData<T>` (deserialization result, lets you handle bad payloads), `AckSender`, `State<T>` (shared state, `state` feature), `Extension`, `HttpExtension`. - Payload types are your own `serde` structs → **type-checked deserialization** of client data, a nice security property (malformed payloads fail to deserialize). - In 0.18, connect middlewares must be **async** functions returning `Result<(), E>` where `E: Display` (a synchronous fn fails to compile with "Invalid ConnectMiddleware"). - Room broadcasts are async (`.emit(...).await`) because they may go through an adapter. - Store per-socket data with `socket.extensions.insert(...)`. - Scaling adapters: `socketioxide-redis`, `socketioxide-mongodb`, `socketioxide-postgres`; emitting from other processes: `socketioxide-emitter`. - Supports protocol v5 by default and the old v4 protocol (v2 JS clients) behind the `v4` feature flag.

17.6 Java — `netty-socketio` (server) — verified with caveats

```
<dependency>
  <groupId>com.corundumstudio.socketio</groupId>
  <artifactId>netty-socketio</artifactId>
```

```
  <version>2.0.13</version>
</dependency>
```

```
import com.corundumstudio.socketio.AuthorizationResult;
import com.corundumstudio.socketio.Configuration;
import com.corundumstudio.socketio.SocketIOServer;
import java.util.Map;

Configuration config = new Configuration();
config.setHostname("0.0.0.0");
config.setPort(8810);
// runs on the HTTP handshake (before any namespace CONNECT): here, a query parameter check
config.setAuthorizationListener(data ->
    "web".equals(data.getSingleUrlParam("platform"))
        ? AuthorizationResult.SUCCESSFUL_AUTHORIZATION
        : AuthorizationResult.FAILED_AUTHORIZATION);

SocketIOServer server = new SocketIOServer(config);

server.addConnectListener(client -> client.joinRoom("lobby"));

server.addEventListener("sum", Map.class, (client, data, ackRequest) -> {
    int a = ((Number) data.get("a")).intValue(), b = ((Number) data.get("b")).intValue();
    if (ackRequest.isAckRequested()) ackRequest.sendAckData(a + b);
});

server.addEventListener("chat", Map.class, (client, data, ackRequest) -> {
    server.getRoomOperations("lobby").sendEvent("chat", client, data);  // `client` is excluded
    ackRequest.sendAckData(Map.of("ok", true));
});

server.addDisconnectListener(client -> { /* ... */ });
server.start();
```

Observed with Node `socket.io-client` 4.8.4: `sum: 42`, `chat ack: { ok: true }`, room broadcast reached the other client and excluded the sender. **Caveats seen in the test:**

1. The authorization listener runs at the **HTTP handshake**, so a rejection reaches the JS client as a transport failure (`connect_error: "xhr poll error"`) — not as a middleware-style `connect_error` with your message, and `socket.active` stays `true` (the client keeps retrying). Plan your client error handling accordingly.
2. In this test, `client.getHandshakeData().getAuthToken()` returned `null` for the main namespace even though the client sent an `auth` payload. Pass tokens in the query string (over HTTPS) or verify `auth` support for your exact version and namespace before relying on it.

Scaling: `netty-socketio` supports Redisson (Redis) and Hazelcast "stores" for multi-node setups — internal to netty-socketio, not compatible with Node adapters.

> For Spring Boot teams, alternatives include running a Node/Rust/Go Socket.IO gateway next to the Java services, or using Spring's native WebSocket/STOMP support when you control both clients and server.

17.7 Mixing languages — rules that matter

1. **Clients ↔ servers interoperate across languages** as long as protocol versions match (all verified above with the Node client).
2. **Servers of different languages cannot share a cluster.** Node's Redis adapter, python-socketio's Redis manager, socketioxide-redis and netty-socketio's Redisson store all use their own message formats/channels. Pick one server implementation per cluster.
3. **Emitters are implementation-specific too:** use `@socket.io/redis-emitter` with Node adapter clusters, `AsyncRedisManager(write_only=True)` with Python clusters, etc. For a polyglot backend, put a **message broker** (Kafka/RabbitMQ/SQS) in front of the realtime gateway and let any service publish domain events there.
4. **Units differ:** Node/Java/JS clients use milliseconds; Python and Swift use seconds.
5. **Data types differ:** JSON numbers become `float64` in Go, `Number` / `Integer` in Java, `int` / `float` in Python; dates are strings everywhere (unless you use a custom parser — Chapter 19).
6. **Ack styles differ:** return value (Python, NestJS), `AckSender` (Rust), `socket.Ack` last arg (Go), `AckRequest` (netty-socketio), callback last arg (Node).

17.8 Interview questions for this chapter

Q1. [M] Can a Python Socket.IO server serve a React app using `socket.io-client`? Yes — `python-socketio` implements the same protocol; the client can't tell the difference. Verified: auth rejection, acks, rooms, server→client calls, upgrade to WebSocket.

Q2. [M] How do you acknowledge an event in python-socketio? Return a value from the handler; the client's ack callback receives it. For server→client acks use `await sio.call(event, data, to=sid, timeout=s)`.

Q3. [S] Can I run Node.js and Python Socket.IO servers in the same cluster with Redis? No. Their Redis adapter/manager formats are different, so cross-server broadcasts won't be understood. Use one implementation per cluster, and a broker for polyglot event publishing.

Q4. [M] What's the python-socketio equivalent of `socket.to(room).emit()`? `await sio.emit(event, data, room=room, skip_sid=sid)`; in Flask-SocketIO `emit(event, data, to=room, include_self=False)`.

Q5. [S] Why might events from one client be processed out of order in python-socketio? `async_handlers=True` (default) runs each event in its own task. Set it to `False` or serialize per key when order matters.

Q6. [M] How do you push from a Celery worker to Socket.IO clients of a Python cluster? `socketio.RedisManager(url, write_only=True).emit(...)` (or the async variant) — the Python counterpart of `@socket.io/redis-emitter`.

Q7. [M] What's specific about Rust's socketioxide? Tower layer usable with axum/hyper; axum-like extractors (`Data<T>`, `TryData<T>`, `AckSender`, `State<T>`); typed payload deserialization via serde; async connect middlewares; Redis/Mongo/Postgres adapters.

Q8. [M] In the Go server, how do you get the ack callback? It's the last variadic argument, of type `socket.Ack` (`func([]any, error)`).

Next: Chapter 18 — WebTransport in practice

CHAPTER 18

WebTransport in Practice (HTTP/3)

> **Goal:** understand what WebTransport is, when it's worth it, and set it up end to end with Socket.IO — certificates, the HTTP/3 server, browser and Node clients, upgrades, deployment and debugging.
>
> **How this chapter was checked:** the server and client code below was run with `socket.io` 4.8.4, `@fails-components/webtransport` 1.6.10 (+ its quiche HTTP/3 transport) and Node 22. Results: a WebTransport-only client connected and exchanged acks; a client using the default transport order connected over polling and **upgraded directly to WebTransport**. Browser behaviour is described from the WebTransport specification and browser documentation (not executed here — no browser in this environment).

18.1 ELI5

WebSocket is a walkie-talkie that runs over **one rope** (a TCP connection). If one message gets stuck on the rope (a lost network packet), **every message behind it waits** — even unrelated ones. That's called *head-of-line blocking*.

WebTransport runs over **HTTP/3 (QUIC)**, which is like having **many small ropes** inside one connection: a stuck message only blocks its own rope. It can also send "throw-away" messages (datagrams) that are never retried. It connects faster too.

Socket.IO can use WebTransport as a third transport, next to polling and WebSocket.

18.2 What WebTransport brings (and what Socket.IO uses)

Property	WebSocket	WebTransport
Underlying protocol	TCP (+ TLS)	QUIC over **UDP** (TLS 1.3 built in)
Head-of-line blocking	Yes (one TCP stream)	Not across streams
Connection setup	TCP + TLS handshakes	Faster (QUIC combines them; 0-RTT resumption possible)

Property	WebSocket	WebTransport
Network changes (Wi-Fi ↔ 4G)	Connection breaks	QUIC supports connection migration
Unreliable datagrams	No	Yes
Requires TLS	Optional (`ws://`)	**Always** (HTTPS only)
Proxy/firewall friendliness	Good (TCP 443)	UDP 443 is blocked on some networks

Socket.IO uses a WebTransport **bidirectional stream** as its transport (reliable, ordered — the same guarantees as WebSocket), so your application code doesn't change. The benefits you get are mostly faster connection setup and better behaviour on lossy or changing networks. Socket.IO's API does not expose unreliable datagrams.

Is it worth it? For most apps, WebSocket is fine. Consider WebTransport for mobile-heavy audiences on lossy networks, latency-sensitive interactive apps, or when you already run HTTP/3 infrastructure. Always keep WebSocket/polling as fallbacks — UDP may be blocked and browser support isn't universal (check current support tables, e.g. caniuse, before relying on it; Safari historically lagged behind Chromium and Firefox).

18.3 Requirements

1. **HTTPS with a valid certificate.** Browsers refuse WebTransport without TLS. - In production: a normal CA-signed certificate for your domain. - In development: a self-signed certificate **pinned by hash** with the `serverCertificateHashes` option. The WebTransport spec restricts such certificates: **ECDSA** keys (e.g. P-256) and a **validity of at most 14 days**.
2. **An HTTP/3 server.** Node.js has no built-in HTTP/3 server (as of Node 22), so you add one: `@fails-components/webtransport` (with its quiche-based transport package).
3. **UDP open** on the port (typically 443) through firewalls, security groups and load balancers.
4. `"webtransport"` added to the server's `transports` option (it is **disabled by default** on the server).

18.4 Step 1 — certificates

Development certificate (ECDSA P-256, 14 days), plus its SHA-256 fingerprint for pinning:

```
openssl req -new -x509 -nodes \
  -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 \
  -keyout key.pem -out cert.pem -days 14 \
```

```
  -subj '/CN=localhost' -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"

# SHA-256 of the DER-encoded certificate, as hex (used by serverCertificateHashes)
node -e 'const {X509Certificate,createHash}=require("crypto");const fs=require("fs");
const c=new X509Certificate(fs.readFileSync("cert.pem"));
console.log(createHash("sha256").update(c.raw).digest("hex"))' > cert.hash
```

Remember to regenerate before the 14 days expire (and update the pinned hash in clients).

18.5 Step 2 — the server (verified)

```
// wt-server.mjs
import { readFileSync } from "node:fs";
import { createServer } from "node:https";
import { Server } from "socket.io";
import { Http3Server } from "@fails-components/webtransport";

const key = readFileSync("./key.pem");
const cert = readFileSync("./cert.pem");

// 1) a classic HTTPS server for polling + websocket (TCP)
const httpsServer = createServer({ key, cert });
const io = new Server(httpsServer, {
  transports: ["polling", "websocket", "webtransport"],   // WebTransport must be enabled
explicitly
});

io.on("connection", (socket) => {
  console.log("server: connected via", socket.conn.transport.name);
  socket.conn.on("upgrade", (t) => console.log("server: upgraded to", t.name));
  socket.on("ping-me", (msg, ack) => ack("pong: " + msg));
});

httpsServer.listen(3443);

// 2) an HTTP/3 server on the same port number — but UDP
const h3Server = new Http3Server({
  port: 3443,
  host: "0.0.0.0",
  secret: "change-me",   // used by QUIC for stateless tokens — use a real random secret
  cert,
  privKey: key,
});
h3Server.startServer();

// 3) hand every WebTransport session opened on /socket.io/ to Engine.IO
const stream = await h3Server.sessionStream("/socket.io/");
const reader = stream.getReader();
```

```
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    io.engine.onWebTransportSession(value);
  }
```

Key points: - TCP 3443 (HTTPS: polling/WebSocket) and UDP 3443 (HTTP/3: WebTransport) can share the same port number because they are different protocols. - `io.engine.onWebTransportSession(session)` is the Engine.IO hook (added in `engine.io` 6.5, shipped with `socket.io` 4.7) that adopts a WebTransport session. - The session path must match the Socket.IO `path` (`/socket.io/` by default).

18.6 Step 3 — clients

Browser

```
import { io } from "socket.io-client";

const socket = io("https://localhost:3443", {
  // default order is ["polling", "websocket", "webtransport"]; you can keep it
  transportOptions: {
    webtransport: {
      // only for self-signed dev certificates; omit in production with a CA-signed cert
      serverCertificateHashes: [{ algorithm: "sha-256", value: hexToBytes(CERT_HASH_HEX) }],
    },
  },
});

function hexToBytes(hex) {
  return new Uint8Array(hex.match(/../g).map((h) => parseInt(h, 16)));
}

socket.on("connect", () => {
  console.log("transport:", socket.io.engine.transport.name);
  socket.io.engine.on("upgrade", (t) => console.log("upgraded to", t.name)); // "webtransport"
});
```

Node.js (uses a WebTransport implementation from the same package) — verified

Node has no global `WebTransport`, so the test client installs the one from `@fails-components/webtransport`:

```
// wt-client.mjs
import { readFileSync } from "node:fs";
import { WebTransport, quicheLoaded } from "@fails-components/webtransport";
import { io } from "socket.io-client";

await quicheLoaded;
globalThis.WebTransport = WebTransport;               // browsers have this natively

const hash = Buffer.from(readFileSync("./cert.hash", "utf8").trim(), "hex");

const socket = io("https://localhost:3443", {
  transports: ["webtransport"],                       // WebTransport only
  transportOptions: {
    webtransport: { serverCertificateHashes: [{ algorithm: "sha-256", value: hash }] },
  },
});

socket.on("connect", async () => {
  console.log("client: transport =", socket.io.engine.transport.name);
  console.log("client: ack =", await socket.timeout(3000).emitWithAck("ping-me", "hi"));
});
```

Real output:

```
client: transport = webtransport
client: ack = pong: hi
server: connected via webtransport
```

Upgrade behaviour — verified

With the default transport order (`["polling", "websocket", "webtransport"]`) and the server advertising WebTransport, the connection **started on polling and upgraded straight to WebTransport**, skipping WebSocket:

```
client: connected via polling
client: upgraded to webtransport
server: connected via polling
server: upgraded to webtransport
```

This matches the client source: when `"webtransport"` is among the server's advertised upgrades, the client probes it in preference to WebSocket. If the WebTransport probe fails (UDP blocked, unsupported browser), the client stays on polling; with `tryAllTransports: true` (4.8+) a failing initial transport makes the client try the next one in the list.

Recommended client configurations:

Goal	Client option
Best compatibility (default)	default `transports`; WebTransport used when possible
Skip polling, prefer WT, fall back to WS	`transports: ["webtransport", "websocket"]`, `tryAllTransports: true`
WT only (testing)	`transports: ["webtransport"]`

18.7 Deployment considerations

- **Load balancers:** you need one that forwards **UDP/QUIC** (e.g. AWS NLB with UDP listeners, or an HTTP/3-aware proxy). Many classic L7 load balancers terminate HTTP/1.1 and HTTP/2 only. Check whether your edge (CDN, ALB, ingress) supports WebTransport *pass-through*; HTTP/3 support for normal requests does not imply WebTransport support.
- **Sticky sessions:** a WebTransport-only connection is a single QUIC connection, so like WebSocket-only it doesn't need stickiness — but a client that **starts on polling and upgrades** needs the upgrade to reach the same node, i.e. sticky routing that works across TCP and UDP (often by client IP). QUIC connection migration can change the client address mid-connection — consider this when routing by IP.
- **Firewalls:** open UDP 443 (and your health checks). Corporate networks frequently block UDP → keep fallbacks.
- **Certificates:** production must use CA-signed certs; `serverCertificateHashes` is for dev/self-signed scenarios with the ≤14-day/ECDSA constraints.
- **Monitoring:** add the transport name to your metrics (`socket.conn.transport.name`) to see what share of users actually get WebTransport.

18.8 Troubleshooting WebTransport

Symptom	Likely cause	Fix
Always stays on polling/WebSocket	`"webtransport"` missing from server `transports`; session stream not wired to `io.engine.onWebTransportSession`; UDP blocked	Enable it; wire the session stream; open UDP

Symptom	Likely cause	Fix
`serverCertificateHashes is not valid fingerprint` (Node polyfill)	Hash not the SHA-256 of the DER certificate, or wrong encoding	Hash `X509Certificate.raw`; pass bytes (`Uint8Array` / `Buffer`), not a hex string
Browser refuses the connection with a self-signed cert	Certificate is RSA or valid for more than 14 days	ECDSA P-256, `-days 14` (or less), regenerate regularly
Works on localhost, fails in production	LB/CDN doesn't pass QUIC/WebTransport; UDP closed	UDP-capable LB, open ports, keep fallback
`connect_error: webtransport error`	Server unreachable on UDP, TLS mismatch, path mismatch	Check `sessionStream` path = `path`; cert; port

18.9 Interview questions for this chapter

Q1. [M] What is WebTransport and how does it differ from WebSocket? A browser API over HTTP/3 (QUIC/UDP) offering multiple independent streams (no head-of-line blocking across streams), unreliable datagrams, faster connection setup and connection migration; always encrypted. WebSocket is a single ordered TCP stream.

Q2. [M] How do you enable WebTransport in Socket.IO? Add `"webtransport"` to the server's `transports`, run an HTTP/3 server (`@fails-components/webtransport` in Node), and pass each session from `h3Server.sessionStream("/socket.io/")` to `io.engine.onWebTransportSession()`. Clients use it automatically when advertised (or `transports: ["webtransport"]`).

Q3. [S] Does Socket.IO's WebTransport transport give you unreliable delivery? No — it uses a reliable bidirectional stream, so semantics match WebSocket. The gains are setup latency and resilience on lossy/changing networks.

Q4. [M] What certificate constraints apply to `serverCertificateHashes`? ECDSA certificate with a validity of at most 14 days; you pin the SHA-256 of the DER certificate. In production use a normal CA-signed certificate.

Q5. [S] Why should you keep WebSocket/polling fallbacks? UDP is blocked on some networks, and browser/platform support for WebTransport is not universal.

Q6. [M] With default client settings, which transport does a client end up on when the server offers WebTransport? It starts on polling and upgrades directly to WebTransport (verified), falling back to staying on polling if the probe fails.

Next: Chapter 19 — Custom adapters and custom parsers

CHAPTER 19

Writing a Custom Adapter and a Custom Parser

> **Goal:** understand the two main extension points of Socket.IO well enough to build your own: an **adapter** (how servers in a cluster talk to each other) and a **parser** (how packets are encoded on the wire).
>
> **How this chapter was checked:** both the adapter and the parser below were run. The adapter was used by two Socket.IO 4.8.4 servers in separate processes connected through Redis 7: a broadcast crossed servers, `fetchSockets()` returned sockets from both servers, `serverSideEmit()` reached the other server, and `serverCount()` returned 2. The parser round-tripped a JavaScript `Date` as a real `Date` in both directions.

19.1 ELI5

- An **adapter** is the post office inside each server. It knows which customers are in which rooms, and when there are several post offices it sends letters between them. You write a custom one when you want the post offices to talk through a different "road" (NATS, Kafka, a database, a cloud service...).
- A **parser** is the envelope format. The default one writes JSON. You write a custom one when you want a different envelope (MessagePack, Protobuf, JSON that keeps Dates...).

19.2 When should you write a custom adapter?

Almost never — first check the official ones (Redis, Redis Streams, MongoDB, Postgres, cluster, GCP Pub/Sub, AWS SQS, Azure Service Bus) and community ones. Valid reasons:

- your organisation's standard bus is something else (NATS, Kafka, RabbitMQ, ZeroMQ);
- you need special routing (per-tenant isolation, cross-region rules);
- you're building a platform/product on top of Socket.IO.

Writing one is also the best way to *really* understand clustering — a frequent senior interview topic.

19.3 The adapter API

Every namespace has one adapter instance. The base class `Adapter` (in `socket.io-adapter`) is the in-memory implementation; distributed adapters override parts of it.

Method	Called when
`addAll(id, rooms)` / `del(id, room)` / `delAll(id)`	a local socket joins/leaves rooms
`broadcast(packet, opts)`	`io.emit`, `io.to(...).emit`, `socket.broadcast.emit`...
`broadcastWithAck(packet, opts, clientCountCallback, ack)`	broadcast with `timeout()` + ack
`addSockets(opts, rooms)` / `delSockets(opts, rooms)`	`io.socketsJoin` / `io.socketsLeave`
`disconnectSockets(opts, close)`	`io.disconnectSockets`
`fetchSockets(opts)`	`io.fetchSockets`
`serverSideEmit(packet)`	`io.serverSideEmit`
`serverCount()`	number of servers in the cluster
`persistSession(session)` / `restoreSession(pid, offset)`	connection state recovery
`init()` / `close()`	lifecycle

Writing all of that by hand is error-prone. Since `socket.io-adapter` 2.5.3, two base classes do the hard part:

- `ClusterAdapter` — implements every cluster operation (broadcasts, requests with responses, acks, fetchSockets...) on top of two abstract methods:
- `doPublish(message)` → send a message to **all other** servers; returns an *offset* (only meaningful for connection state recovery);
- `doPublishResponse(requesterUid, response)` → send a response to **one** server. Your transport calls `this.onMessage(message, offset)` and `this.onResponse(response)` when something arrives.
- `ClusterAdapterWithHeartbeat` — the same, plus heartbeats so each server knows how many servers are alive (needed to know how many responses to wait for in `fetchSockets()` / `serverSideEmit` with ack). Options: `heartbeatInterval` (default 5 000 ms), `heartbeatTimeout` (default 10 000 ms).

Each server has a random `uid`; messages from yourself are ignored automatically.

19.4 A complete adapter in ~50 lines (verified)

A Redis pub/sub adapter written from scratch on `ClusterAdapterWithHeartbeat`, using the `redis` (node-redis v4+) client and MessagePack (`notepack.io`) so binary payloads survive:

```
// mini-redis-adapter.js
const { ClusterAdapterWithHeartbeat } = require("socket.io-adapter");
const { encode, decode } = require("notepack.io"); // msgpack: keeps Buffers (binary) intact

function createAdapter(pubClient, subClient, opts = {}) {
  const prefix = opts.channelPrefix ?? "mini-sio";
  const namespaceToAdapters = new Map(); // nsp name → adapter (one adapter per namespace)

  // one shared subscription for every namespace of this server
  subClient.pSubscribe(`${prefix}#*`, (raw, channel) => {
    const [, nspName, target] = channel.toString().split("#"); // "mini-sio#/#all" or "mini-
sio#/#<uid>"
    const adapter = namespaceToAdapters.get(nspName);
    if (!adapter) return;
    const message = decode(raw);
    if (target === "all") adapter.onMessage(message);   // a message for every server
    else adapter.onResponse(message);                   // a response addressed to this server
  }, true /* receive Buffers */);

  return function (nsp) {
    const adapter = new MiniRedisAdapter(nsp, pubClient, prefix, opts);
    namespaceToAdapters.set(nsp.name, adapter);
    const close = adapter.close.bind(adapter);
    adapter.close = () => { namespaceToAdapters.delete(nsp.name); close(); };
    return adapter;
  };
}

class MiniRedisAdapter extends ClusterAdapterWithHeartbeat {
  constructor(nsp, pubClient, prefix, opts) {
    super(nsp, opts);           // opts: { heartbeatInterval, heartbeatTimeout }
    this.pub = pubClient;
    this.prefix = prefix;
  }

  // send a message to ALL other servers (broadcasts, socketsJoin, fetchSockets requests,
heartbeats…)
  async doPublish(message) {
    await this.pub.publish(`${this.prefix}#${this.nsp.name}#all`, encode(message));
    return ""; // an offset, only meaningful for adapters that support connection state
recovery
  }

  // send a response to ONE server (the one that made a request, e.g. fetchSockets)
  async doPublishResponse(requesterUid, response) {
    await this.pub.publish(`${this.prefix}#${this.nsp.name}#${requesterUid}`,
encode(response));
  }
```

```
}

module.exports = { createAdapter };
```

Using it:

```
const { Server } = require("socket.io");
const { createClient } = require("redis");
const { createAdapter } = require("./mini-redis-adapter");

const pub = createClient({ url: process.env.REDIS_URL });
const sub = pub.duplicate();
pub.on("error", console.error);
sub.on("error", console.error);
await Promise.all([pub.connect(), sub.connect()]);

const io = new Server(port, { adapter: createAdapter(pub, sub, { heartbeatInterval: 1000 }) });

io.on("connection", (socket) => {
  socket.data.name = socket.handshake.auth.name;
  socket.join("room1");
  socket.on("go", async (ack) => {
    io.to("room1").emit("hello", `from ${port}`);                          // cross-server
broadcast
    const names = (await io.fetchSockets()).map((s) => s.data.name).sort(); // cross-server
fetch
    io.serverSideEmit("ping-servers", port);                                // server-to-server
    ack({ names, servers: await io.of("/").adapter.serverCount() });
  });
});
io.on("ping-servers", (from) => console.log(`server ${port} got serverSideEmit from ${from}`));
```

Real output with two servers (ports 4001 and 4002), Alice on 4001 and Bob on 4002:

```
bob (on server 4002) received: from 4001
ack: { names: [ 'alice', 'bob' ], servers: 2 }
(server 4002 log) server 4002 got serverSideEmit from 4001
```

A bug I hit while writing it (and what it teaches)

The first version did `channel.split("#")` and crashed with `TypeError: channel.split is not a function`. With node-redis' **buffer mode** (the `true` argument to `pSubscribe`, needed to receive binary message bodies), the **channel name is a Buffer too.** Fix: `channel.toString()`. Lesson: in pub/sub adapters, be explicit about binary vs string at every boundary — payloads must stay binary (msgpack, attachments), channel names are text.

Design decisions you must make in a real adapter

1. **Channel layout.** One channel for everything (simple, every server gets everything) vs per-room channels (servers only receive rooms they host — what the sharded Redis adapter does in `dynamic` mode).
2. **Encoding.** MessagePack (binary-safe, compact) vs JSON (must handle Buffers yourself).
3. **Delivery guarantees.** Pub/sub loses messages during disconnects. A log-based transport (Redis Streams, Kafka, NATS JetStream) can resume from an offset and enables **connection state recovery**: return the stored message id from `doPublish`, and implement `persistSession` / `restoreSession` (see the Redis Streams adapter source as a reference).
4. **Request timeouts.** `fetchSockets()` and acks wait for every live server; heartbeats decide who is alive. Tune `heartbeatInterval` / `heartbeatTimeout`.
5. **Message size limits** of your bus (Postgres `NOTIFY` is limited to 8000 bytes — the Postgres adapter stores bigger payloads in a table).
6. **Cleanup.** Unsubscribe and clear timers in `close()`.

Testing a custom adapter

Run 2–3 servers (separate processes, or several `Server` instances on different ports in one test) against a real bus (Redis in Docker/Testcontainers) and assert:

- broadcast to all / to a room / with `except` reaches clients on every server exactly once;
- `socket.to(room)` excludes the sender everywhere;
- `fetchSockets()` returns sockets from all servers, with `data`;
- `socketsJoin` / `socketsLeave` / `disconnectSockets` affect remote sockets;
- `serverSideEmit` with and without ack;
- broadcast with ack collects responses from all servers;
- a server going down is removed after `heartbeatTimeout` (no hanging requests);
- binary payloads survive.

19.5 Parsers

The contract

A parser module exports two classes, `Encoder` and `Decoder`. Both client and server must be configured with the **same** parser (`new Server({ parser })`, `io(url, { parser })`).

```
class Encoder {
  /** packet: { type, nsp, data?, id? } → array of strings and/or binary chunks */
```

```
  encode(packet) { /* … */ return [chunk1, chunk2]; }
}

class Decoder extends Emitter {           // from "@socket.io/component-emitter"
  add(chunk) {                            // called for every chunk received from the transport
    /* … when a whole packet has been decoded: */ this.emit("decoded", packet);
  }
  destroy() {}                            // called when the connection closes
}
```

Packet types: `0` CONNECT, `1` DISCONNECT, `2` EVENT, `3` ACK, `4` CONNECT_ERROR (plus `5` / `6` binary variants used by the default parser). **Validate decoded packets:** a throw inside `add()` makes the connection close, which is the right reaction to garbage from a malicious client — and historic security advisories in `socket.io-parser` were exactly about insufficient validation (Chapter 6.10).

Example: a JSON parser that preserves `Date` objects (verified)

The default parser turns Dates into ISO strings. This parser tags them so the receiver gets real `Date` objects back (JSON-only: no binary support, which is fine for many apps):

```
// date-parser.js
const { Emitter } = require("@socket.io/component-emitter");

const DATE_TAG = "__date__";

const replacer = function (key, value) {
  // JSON.stringify calls Date#toJSON() BEFORE the replacer, so read the original value from
`this`
  const original = this[key];
  return original instanceof Date ? { [DATE_TAG]: original.toISOString() } : value;
};
const reviver = (key, value) =>
  value && typeof value === "object" && typeof value[DATE_TAG] === "string" ? new
Date(value[DATE_TAG]) : value;

class Encoder {
  encode(packet) {
    return [JSON.stringify(packet, replacer)];      // must return an ARRAY of encoded chunks
  }
}

class Decoder extends Emitter {
  add(chunk) {
    if (typeof chunk !== "string") throw new Error("binary not supported by this parser");
    const packet = JSON.parse(chunk, reviver);
    if (!isPacketValid(packet)) throw new Error("invalid packet");   // a throw closes the
connection
    this.emit("decoded", packet);
  }
  destroy() {}
}
```

```
function isPacketValid({ type, data, nsp, id }) {
  const isNamespaceValid = typeof nsp === "string";
  const isAckIdValid = id === undefined || Number.isInteger(id);
  if (!isNamespaceValid || !isAckIdValid) return false;
  switch (type) {
    case 0: return data === undefined || typeof data === "object";     // CONNECT
    case 1: return data === undefined;                                 // DISCONNECT
    case 2: return Array.isArray(data) && typeof data[0] === "string"; // EVENT
    case 3: return Array.isArray(data);                                // ACK
    case 4: return typeof data === "object";                           // CONNECT_ERROR
    default: return false;
  }
}

module.exports = { Encoder, Decoder };
```

Test (real output):

```
const parser = require("./date-parser");
const io = new Server(4400, { parser });
io.on("connection", (s) =>
  s.on("when", (d, ack) => ack({ isDate: d instanceof Date, plusOneDay: new Date(+d + 86400000)
})));

const c = ioClient("http://localhost:4400", { parser });
const r = await c.timeout(2000).emitWithAck("when", new Date("2026-01-01T00:00:00Z"));
// → server saw a Date: true | client got a Date back: true 2026-01-02T00:00:00.000Z
```

The subtle part (a classic JavaScript gotcha): `JSON.stringify` calls `Date#toJSON()` **before** passing the value to the replacer, so `value` is already a string — you must look at `this[key]` to see the original `Date`.

MessagePack parser (official)

```
// server
const msgpackParser = require("socket.io-msgpack-parser");
const io = new Server(httpServer, { parser: msgpackParser });

// client
import msgpackParser from "socket.io-msgpack-parser";
const socket = io(URL, { parser: msgpackParser });
// or the prebuilt browser bundle: socket.io.msgpack.min.js
```

Binary-encodes everything (no separate attachments). Smaller for numeric/binary-heavy data; similar for text-heavy data. Not human-readable in DevTools. Every client (including mobile clients) must support the same parser — check before adopting.

Other ideas for custom parsers

- **Protobuf / FlatBuffers** with a schema per event (very compact, typed, but every client must share the schemas).
- **Compression** of large textual payloads (but prefer `perMessageDeflate` first).
- **Per-message encryption** (prefer TLS + application-level E2EE, Chapter 21).
- **Validation in the parser** (reject unknown event names early).

19.6 Interview questions for this chapter

Q1. [S] What do you need to implement to build a cluster adapter today? Extend `ClusterAdapterWithHeartbeat` (or `ClusterAdapter`) from `socket.io-adapter` and implement `doPublish(message)` (to all servers) and `doPublishResponse(uid, response)` (to one server); feed incoming traffic to `onMessage()` / `onResponse()`. The base class implements broadcasts, acks, `fetchSockets`, `socketsJoin/Leave`, `disconnectSockets`, `serverSideEmit`, and server counting via heartbeats.

Q2. [S] Why do heartbeats matter in a cluster adapter? Requests like `fetchSockets()` wait for one response per server; heartbeats tell each node how many servers are alive so it neither waits for dead nodes nor misses live ones.

Q3. [S] How would an adapter support connection state recovery? Use a log-based bus (streams), return the stored message id as the offset from `doPublish`, and implement `persistSession` / `restoreSession` so missed packets can be replayed from the offset.

Q4. [M] What must a custom parser export? `Encoder` with `encode(packet)` returning an array of chunks, and `Decoder` (an event emitter) with `add(chunk)` emitting `"decoded"` and `destroy()`. Same parser on both sides.

Q5. [M] Why validate packets in the decoder? Malformed input from malicious clients must not crash or confuse the server; throwing closes that connection. Past parser vulnerabilities came from insufficient validation.

Q6. [M] Why does a naive "keep Dates" replacer not work with `JSON.stringify`? `toJSON()` runs before the replacer, so the replacer sees a string; inspect `this[key]`.

Next: Chapter 20 — Other runtimes: Electron, service workers, Bun, Deno

CHAPTER 20

Other Runtimes: Electron, Web/Shared/Service Workers, Edge, Bun and Deno

> **Goal:** use Socket.IO outside the classic "browser page + Node.js server" setup.
>
> **How this chapter was checked:** - **Fetch transport** (for service workers): verified — a client configured with `transports: [Fetch, WebSocket]` connected over fetch-based polling and exchanged acks. - **Bun 1.4.2** running the npm `socket.io` server: verified with a Node client (connect, acks, WebSocket upgrade). - **Deno 2.9.6** running `npm:socket.io@4.8.4`: verified (connect, acks, upgraded to WebSocket). - The official Deno port on `deno.land/x` could not be downloaded from this sandbox (host blocked), and **Electron** could not be run here (no display) — those sections are written from the libraries' and Electron's documented behaviour and are marked as such.

20.1 ELI5

The Socket.IO client is like a phone that normally lives in a web page. But you can also put that phone in other places: in a desktop app (Electron), in a helper that runs behind the web page (workers), or in other JavaScript engines (Bun, Deno). Each place has its own rules — some places don't allow certain phone features, some go to sleep, some can't keep a line open at all.

20.2 Web Workers and SharedWorkers

`socket.io-client` works inside **dedicated workers** and **shared workers** — they have `WebSocket` and `XMLHttpRequest`.

One connection for all tabs with a SharedWorker

Ten open tabs normally mean ten connections (ten times the server load, ten copies of every notification). A **SharedWorker** can own a single connection and relay to tabs:

```
// shared-socket-worker.js  (bundle socket.io-client into it, or importScripts the UMD build)
import { io } from "socket.io-client";
```

```
const socket = io("https://api.example.com", {
  transports: ["websocket"],
  auth: (cb) => cb({ token: currentToken }),
});
let currentToken = null;
const ports = new Set();

socket.onAny((event, ...args) => {
  for (const port of ports) port.postMessage({ type: "event", event, args });
});
socket.on("connect", () => broadcastStatus(true));
socket.on("disconnect", () => broadcastStatus(false));

function broadcastStatus(connected) {
  for (const port of ports) port.postMessage({ type: "status", connected });
}

onconnect = (e) => {
  const port = e.ports[0];
  ports.add(port);
  port.onmessage = async ({ data }) => {
    if (data.type === "token") { currentToken = data.token; if (!socket.connected)
socket.connect(); }
    if (data.type === "emit") {
      try {
        const res = await socket.timeout(8000).emitWithAck(data.event, data.payload);
        port.postMessage({ type: "ack", requestId: data.requestId, res });
      } catch (err) {
        port.postMessage({ type: "ack", requestId: data.requestId, error: "TIMEOUT" });
      }
    }
    if (data.type === "close") ports.delete(port);   // tabs should send this on "pagehide"
  };
  port.postMessage({ type: "status", connected: socket.connected });
};
```

```
// in each tab
const worker = new SharedWorker(new URL("./shared-socket-worker.js", import.meta.url), { type:
"module" });
worker.port.start();
worker.port.postMessage({ type: "token", token: accessToken });
worker.port.onmessage = ({ data }) => { if (data.type === "event") dispatch(data.event,
data.args); };
addEventListener("pagehide", () => worker.port.postMessage({ type: "close" }));
```

Caveats: - **Browser support:** SharedWorker isn't available everywhere (notably some mobile browsers). Feature-detect and fall back to one socket per tab. - Rooms on the server now represent the *browser*, not the tab; per-tab subscriptions must be multiplexed by the worker (reference-count subscriptions). - An alternative without SharedWorker: **leader election** between tabs via `BroadcastChannel` / Web Locks API — one tab holds the socket and rebroadcasts.

20.3 Service workers and the fetch transport

Service workers have **no** `XMLHttpRequest`, so the classic polling transport can't run there. Since `socket.io-client` 4.8, you can choose transport **implementations**, including a polling transport based on `fetch()`:

```
import { io, Fetch, WebSocket } from "socket.io-client";

const socket = io("https://api.example.com", {
  transports: [Fetch, WebSocket],   // constructors instead of names
});
```

Verified in Node 22 (which has `fetch`): with `transports: [Fetch, WebSocket]` and `upgrade: false`, the client connected and the server reported transport `polling`; acks worked (`transport: polling | ack: hello polling`).

`socket.io-client` 4.8 exports these implementations: `Fetch`, `XHR`, `NodeXHR`, `WebSocket`, `NodeWebSocket`, `WebTransport`.

But should a service worker hold a socket? Usually no: - Browsers **terminate idle service workers** after a short time (tens of seconds); an open WebSocket doesn't reliably keep one alive. A long-lived connection is not what service workers are for. - For background notifications use the **Push API** (Web Push) — that's the platform's mechanism to wake a service worker. - Valid uses: short-lived exchanges during `sync` / `push` event handling, or runtimes that lack XHR but aren't service workers.

20.4 Edge and serverless runtimes

- **Cloudflare Workers, Vercel Edge Functions, AWS Lambda:** you **cannot host a Socket.IO server** there — no long-running Node HTTP server, and request lifetimes are limited. (Cloudflare Durable Objects can hold raw WebSockets, but that's not the Socket.IO protocol.)
- **Clients** in edge runtimes: the fetch transport (20.3) makes short-lived client usage possible, but invocation time limits make it unsuitable for persistent connections.
- **Pattern:** keep Socket.IO servers on long-running infrastructure (containers/VMs), and have serverless functions publish through an **emitter** (Redis) or a broker (Chapter 8.5) — e.g. a Lambda that processes a payment publishes `emitter.to("user:42").emit("payment:done")`.

20.5 Electron (desktop apps)

(Not executed in this environment — based on Electron's documented process model and security guidance.)

Electron apps have two kinds of processes, and you can run the Socket.IO client in either:

Where	Pros	Cons
Renderer (a Chromium page)	Same code as your web app; DevTools Network tab shows frames	Reloading the window drops the connection; subject to the page's security settings
Main process (Node.js)	Survives window reloads/closing; one connection for many windows; can set custom headers (Node client)	You must relay events to windows via IPC

Renderer with secure defaults

With `contextIsolation: true`, `sandbox: true` and `nodeIntegration: false` (the recommended settings), import `socket.io-client` in your **bundled renderer code** like in any web app. Don't expose Node APIs to the page.

Add your API host to the Content Security Policy:

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; connect-src 'self' https://api.example.com
wss://api.example.com">
```

Main process + IPC (recommended for apps that should stay connected)

```
// main.js
const { app, BrowserWindow, ipcMain, powerMonitor } = require("electron");
const { io } = require("socket.io-client");

let socket;
app.whenReady().then(() => {
  socket = io("https://api.example.com", {
    transports: ["websocket"],
    auth: (cb) => cb({ token: loadTokenFromSecureStorage() }),    // e.g. safeStorage / OS
keychain
    extraHeaders: { "x-client": "desktop/5.2.0" },              // allowed in the Node client
  });

  // relay server events to every window
  socket.onAny((event, ...args) => {
    for (const win of BrowserWindow.getAllWindows()) win.webContents.send("rt:event", event,
args);
```

```
  });

  // windows send commands through IPC; the main process applies a timeout and returns the ack
  ipcMain.handle("rt:emit", (_e, event, payload) => socket.timeout(8000).emitWithAck(event,
payload));

  // laptops sleep: reconnect promptly on resume instead of waiting for back-off
  powerMonitor.on("resume", () => { if (!socket.connected) socket.connect(); });
});
```

```
// preload.js — expose a minimal, safe API (contextBridge), never the raw socket
const { contextBridge, ipcRenderer } = require("electron");
contextBridge.exposeInMainWorld("realtime", {
  emit: (event, payload) => ipcRenderer.invoke("rt:emit", event, payload),
  on: (handler) => {
    const listener = (_e, event, args) => handler(event, args);
    ipcRenderer.on("rt:event", listener);
    return () => ipcRenderer.removeListener("rt:event", listener);   // unsubscribe
  },
});
```

Optionally allow-list event names in the preload/main process so a compromised renderer can't emit arbitrary events.

Origins and CORS in Electron

- Pages loaded from `file://` don't have a normal web origin. Requests from them carry an unusual `Origin` (an opaque/`file://`-style origin) that your `cors` allow-list won't match by default. **Log `req.headers.origin` on the server** to see what your build sends, and avoid simply allowing `"null"` (sandboxed iframes on any website also send `null`).
- Better options: create the socket in the **main process** (no browser CORS rules; send an app identifier header), or serve the UI from a **custom privileged scheme** (`protocol.registerSchemesAsPrivileged` with `standard` / `secure` privileges) and allow that exact origin.
- Authenticate with tokens, not origin alone — desktop apps can be inspected and modified by their users.

20.6 Bun

The npm `socket.io` server runs on Bun through its Node.js compatibility layer. Verified with Bun 1.4.2:

```
// bun-server.js
const { Server } = require("socket.io");
const io = new Server(4101);
io.on("connection", (s) => s.on("hi", (ack) => ack("hello from bun via " +
s.conn.transport.name)));
```

```
bun bun-server.js
```

A Node `socket.io-client` connected and got `hello from bun via websocket` (and still `websocket` after the upgrade window).

Notes: - Compatibility is good for the core features shown here; test adapters (Redis), native add-ons (`bufferutil`) and `uWebSockets.js` separately on your Bun version. - Bun also has its own fast native WebSocket server (`Bun.serve({ websocket })`) — that's plain WebSocket, not the Socket.IO protocol.

20.7 Deno

npm `socket.io` on Deno — verified (Deno 2.9.6)

```
// npm-server.ts
import { Server } from "npm:socket.io@4.8.4";

const io = new Server(4500, { cors: { origin: "*" } });
io.on("connection", (s) =>
  s.on("hi", (ack: (m: string) => void) => ack("hello from Deno (npm:socket.io) via " +
s.conn.transport.name)));
```

```
// deno.json — needed when the folder also has a package.json
{ "nodeModulesDir": "auto" }
```

```
deno install --entrypoint npm-server.ts
deno run -A npm-server.ts     # or narrower: --allow-net --allow-env --allow-read
```

Result with a Node client: `hello from Deno (npm:socket.io) via websocket`.

The first attempt failed with "*Could not find a matching package for 'npm:socket.io@4.8.4' in the node_modules directory*" because the folder contained a `package.json`; Deno then expects dependencies to be installed. Adding `"nodeModulesDir": "auto"` (or running `deno install`) fixed it.

The official Deno port

There is also an official Socket.IO server **port written for Deno** (published on `deno.land/x/socket_io`), which plugs into Deno's HTTP server via `io.handler()`:

```
// not executed here — deno.land was unreachable from the test sandbox
import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { Server } from "https://deno.land/x/socket_io@0.2.1/mod.ts";

const io = new Server();
io.on("connection", (socket) => { /* same API style as Node */ });
await serve(io.handler(), { port: 3000 });
```

It implements a subset of the Node server's features; check its README for what's supported (adapters, etc.) before choosing it over `npm:socket.io`.

20.8 Node.js-only client features (for completeness)

When the client runs in Node.js (CLI tools, tests, Electron main, services):

- `extraHeaders` apply to **all** transports (browsers can't set WebSocket headers).
- TLS options: `ca`, `cert`, `key`, `pfx`, `passphrase`, `rejectUnauthorized` (never `false` in production), `ciphers`.
- `agent` for HTTP proxies (e.g. `https-proxy-agent`).
- `autoUnref: true` → the heartbeat timer doesn't keep the process alive.
- With `withCredentials: true`, the Node client keeps cookies between requests (this is what made HAProxy cookie-based stickiness work in the Chapter 23 lab).

20.9 Interview questions for this chapter

Q1. [M] How can several browser tabs share one Socket.IO connection? A SharedWorker owning the socket and relaying via `MessagePort`s, or leader election between tabs (BroadcastChannel / Web Locks). Mind browser support and per-tab subscriptions.

Q2. [M] Can you use Socket.IO in a service worker? The client can run there using the fetch-based polling transport (`transports: [Fetch, WebSocket]`, 4.8+), but service workers are terminated when idle, so they're unsuitable for persistent connections; use Web Push for background notifications.

Q3. [M] Can you host a Socket.IO server on Lambda / Cloudflare Workers / Vercel functions? No — they don't keep long-lived connections. Host servers on long-running infrastructure and publish from serverless code through an emitter or broker.

Q4. [S] Where should an Electron app hold its socket? Usually the main process (survives reloads, one connection, Node client features) with a narrow IPC API exposed via `contextBridge`; or the renderer for simple apps. Use secure renderer settings, CSP `connect-src`, and token auth (file:// pages have unusual origins).

Q5. [M] Does `npm:socket.io` work on Bun and Deno? Yes for core functionality (verified: connect, acks, WebSocket upgrade on Bun 1.4.2 and Deno 2.9.6). Test adapters and native add-ons on your exact runtime version.

Next: Chapter 21 — Offline-first clients and end-to-end encryption

CHAPTER 21

Offline-First Clients and End-to-End Encryption

> **Goal:** two advanced client-side topics that come up in serious products and senior interviews: (1) never losing a user's action, even across offline periods and page reloads (**outbox pattern**), and (2) making messages unreadable to the server (**end-to-end encryption, E2EE**) while still using Socket.IO as the transport.
>
> **How this chapter was checked:** both implementations were run in Node 22 against a Socket.IO 4.8.4 server. The outbox test used `fake-indexeddb` to provide the browser's IndexedDB API: two messages written while the server was down survived a simulated page reload and were delivered exactly once when the server came back. The E2EE test used the standard Web Crypto API (identical in browsers): the server only saw a 12-byte IV and a 32-byte ciphertext; the recipient decrypted the original text.

21.1 ELI5

Outbox: when you write a letter but the post office is closed, you put it in a box by your door (saved on disk). Every time the post office opens, you hand over the letters from the box one by one, and you only throw a letter away when the post office gives you a receipt. If you move house (reload the page), the box comes with you.

E2EE: you and your friend each have a special padlock and key. You lock your letter with a padlock only your friend can open. The postman (the server) carries the locked box but can't read what's inside.

21.2 Why Socket.IO's built-in buffering isn't enough

Recap (Chapter 5): the JS client keeps emits made while disconnected in `socket.sendBuffer` and sends them on reconnection. But:

Situation	Built-in buffer	Outbox
Short network blip	✅ delivered	✅ delivered

Situation	Built-in buffer	Outbox
Page reload / tab closed / app killed while offline	❌ lost (memory only)	✅ persisted in IndexedDB
Server received it but the ack was lost	⚠️ not retried → you don't know	✅ retried with the **same id** → server dedupes
Server rejected it (validation)	you see the ack, nothing stored	✅ removed with an error state for the UI
Swift client (no offline buffering at all, Chapter 16)	❌	✅

Use the outbox for **user-generated commands that matter:** chat messages, form submissions, likes, edits in collaborative apps, field data entry.

21.3 The outbox pattern — design

1. **Persist first, send second.** The UI writes the command to durable local storage (IndexedDB) with a **client-generated unique id** before sending.
2. **Optimistic UI.** Show the item immediately as "pending".
3. **Send in order, one at a time**, with an **ack + timeout**.
4. **Remove on success** (or on a *permanent* rejection, surfaced to the user).
5. **Retry on reconnect** (`connect` event), and on app start.
6. **Server is idempotent:** a unique constraint on the client id makes retries harmless.
7. **Stop at the first transient failure** to preserve order.

21.4 Implementation (verified)

```
// outbox.mjs — offline-first outbox: messages survive offline periods AND page reloads.
import { get, set } from "idb-keyval";       // tiny IndexedDB wrapper (works in browsers)

const KEY = "outbox:v1";

export class Outbox {
  constructor(socket, { event = "message:send", ackTimeout = 8000 } = {}) {
    this.socket = socket;
    this.event = event;
    this.ackTimeout = ackTimeout;
    this.flushing = false;
    socket.on("connect", () => this.flush());           // retry everything pending on every
(re)connect
```

```
  }

  async #load() { return (await get(KEY)) ?? []; }
  async #save(items) { await set(KEY, items); }

  // 1) persist FIRST, 2) then try to send
  async enqueue(payload) {
    const item = { id: crypto.randomUUID(), payload, createdAt: Date.now() };
    await this.#save([...(await this.#load()), item]);
    this.flush();
    return item.id;                                  // use it as the key of the "pending"
UI item
  }

  async pending() { return this.#load(); }

  async flush() {
    if (this.flushing || !this.socket.connected) return;
    this.flushing = true;
    try {
      for (const item of await this.#load()) {           // in order, one at a time
        if (!this.socket.connected) break;
        try {
          const res = await this.socket.timeout(this.ackTimeout)
            .emitWithAck(this.event, { ...item.payload, clientMsgId: item.id }); // SAME id on
every retry
          if (res?.ok || res?.error?.code === "VALIDATION") {
            // delivered (or permanently rejected → show it to the user) → remove from the
outbox
            await this.#save((await this.#load()).filter((i) => i.id !== item.id));
          } else {
            break;                                       // transient error: keep order, retry
later
          }
        } catch {
          break;                                         // ack timeout: retry on next connect
/ flush
        }
      }
    } finally {
      this.flushing = false;
    }
  }
}
```

Server side — idempotent handler:

```
socket.on("message:send", async (m, ack) => {
  // UNIQUE index on client_msg_id → the second insert of a retried message does nothing
  const { inserted, message } = await db.insertMessageIfAbsent(m.clientMsgId,
socket.data.user.id, m);
  if (inserted) io.to(`conversation:${m.conversationId}`).emit("message:new", message);
```

```
  ack({ ok: true, data: { id: message.id }, duplicate: !inserted });
});
```

Test scenario and real output

```
// phase 1: server is DOWN; the user sends 2 messages, then the page "reloads" (client
destroyed)
let c1 = io(URL, { reconnectionDelay: 200, reconnectionDelayMax: 300 });
let ob1 = new Outbox(c1);
await ob1.enqueue({ text: "written offline #1" });
await ob1.enqueue({ text: "written offline #2" });
console.log("pending before reload:", (await ob1.pending()).length);
c1.close();                                  // the in-memory sendBuffer would be lost here

// phase 2: new page load, server comes back
startServer();
const c2 = io(URL);
const ob2 = new Outbox(c2);                   // reads the persisted items and flushes on
connect
await sleep(800);
console.log("pending after reconnect:", (await ob2.pending()).length);
console.log("server stored:", [...stored.values()]);
```

```
pending before reload: 2
pending after reconnect: 0
server stored: [ 'written offline #1', 'written offline #2' ]
```

Production refinements

- **Multiple tabs:** two tabs flushing the same IndexedDB outbox can double-send (harmless thanks to idempotency, but wasteful). Use the Web Locks API (`navigator.locks.request("outbox", …)`) so only one tab flushes at a time.
- **Per-conversation ordering** instead of one global queue, so one stuck item doesn't block unrelated conversations.
- **Expiry & poison messages:** after N attempts or a max age, mark as failed and let the user retry or discard.
- **Large items** (attachments): store blobs separately; upload via HTTP; the outbox item references the upload.
- **Service worker Background Sync** can trigger a flush after connectivity returns even if the page is closed (where supported) — it would use HTTP, not the socket.
- **Encryption at rest:** IndexedDB isn't encrypted; don't store secrets there.
- **Mobile equivalents:** SQLite/Room (Android), Core Data/SQLite (iOS), Hive/Isar/sqflite (Flutter) with the same algorithm.

Server → client: the inbox side

The outbox covers client → server. For server → client completeness, combine with the sync pattern (Chapter 5.9): persist on the server, client syncs "after last id" on every connect, dedupes by id. Together they give at-least-once delivery both ways with effectively-once processing.

21.5 End-to-end encryption (E2EE)

What E2EE means here

TLS (`wss://`) already encrypts traffic **between client and server**, but the server sees plaintext. With E2EE, clients encrypt **before** emitting; the server stores and relays **ciphertext** only. Socket.IO is just the transport: binary payloads (`Uint8Array`) travel as attachments.

Things the server still sees: who talks to whom, when, message sizes, room membership — **metadata**. E2EE protects content, not metadata.

The building blocks (Web Crypto API — available in browsers and Node 20+)

- **ECDH (P-256)** key agreement: each user has a key pair; combining my private key with your public key yields the same shared secret on both sides.
- **HKDF** to derive a proper AES key from that secret (with a salt/context, e.g. the conversation id).
- **AES-GCM (256)** for authenticated encryption, with a **random 12-byte IV per message** and **associated data** (AAD) binding the ciphertext to its context.

```
// e2ee.mjs — identical code runs in browsers and Node 20+
const subtle = globalThis.crypto.subtle;
const enc = new TextEncoder(), dec = new TextDecoder();

export async function createIdentity() {
  // private key not extractable: it can't be exported by scripts
  const keyPair = await subtle.generateKey({ name: "ECDH", namedCurve: "P-256" }, false,
["deriveBits"]);
  const publicJwk = await subtle.exportKey("jwk", keyPair.publicKey);
  return { keyPair, publicJwk };
}

export async function deriveConversationKey(myKeyPair, theirPublicJwk, conversationId) {
  const theirKey = await subtle.importKey("jwk", theirPublicJwk, { name: "ECDH", namedCurve:
"P-256" }, false, []);
  const sharedBits = await subtle.deriveBits({ name: "ECDH", public: theirKey },
myKeyPair.privateKey, 256);
```

```
  const hkdfKey = await subtle.importKey("raw", sharedBits, "HKDF", false, ["deriveKey"]);
  return subtle.deriveKey(
    { name: "HKDF", hash: "SHA-256", salt: enc.encode(conversationId), info: enc.encode("chat-
v1") },
    hkdfKey, { name: "AES-GCM", length: 256 }, false, ["encrypt", "decrypt"],
  );
}

export async function encrypt(key, plaintext, aad) {
  const iv = crypto.getRandomValues(new Uint8Array(12));           // NEVER reuse an IV with
the same key
  const ct = await subtle.encrypt({ name: "AES-GCM", iv, additionalData: enc.encode(aad) },
key, enc.encode(plaintext));
  return { iv, ct: new Uint8Array(ct) };                            // binary → sent as
Socket.IO attachments
}

export async function decrypt(key, { iv, ct }, aad) {
  const pt = await subtle.decrypt({ name: "AES-GCM", iv, additionalData: enc.encode(aad) },
key, ct);
  return dec.decode(pt);                                            // throws if tampered with
}
```

Server: an untrusted relay

```
const publicKeys = new Map();   // in production: a DB table of (userId, deviceId, publicKey)

io.on("connection", (s) => {
  const user = s.data.user.id;              // from auth middleware
  s.join(`user:${user}`);
  s.on("keys:publish", (jwk, ack) => { publicKeys.set(user, jwk); ack(true); });
  s.on("keys:get", (u, ack) => ack(publicKeys.get(u) ?? null));
  s.on("e2e:send", (msg) => {
    // the server only sees the routing info and opaque bytes
    io.to(`user:${msg.to}`).emit("e2e:message", { from: user, iv: msg.iv, ct: msg.ct });
  });
});
```

Clients

```
const me = await createIdentity();
await socket.emitWithAck("keys:publish", me.publicJwk);

const theirKey = await socket.emitWithAck("keys:get", "bob");
const key = await deriveConversationKey(me.keyPair, theirKey, "dm:alice:bob");

// send
const { iv, ct } = await encrypt(key, "meet at 7pm 🍕", "dm:alice:bob");
socket.emit("e2e:send", { to: "bob", iv, ct });
```

```
// receive (Bob)
socket.on("e2e:message", async (m) => {
  const iv = new Uint8Array(m.iv), ct = new Uint8Array(m.ct);   // Buffers in Node,
ArrayBuffers in browsers
  console.log(await decrypt(bobKey, { iv, ct }, "dm:alice:bob"));
});
```

Real output of the test run:

```
server sees: { to: 'bob', ivBytes: 12, ciphertextBytes: 32 }
bob decrypted: meet at 7pm 🍕
```

(32 bytes = 16 bytes of UTF-8 text including the emoji + 16-byte GCM authentication tag.)

What this demo deliberately leaves out (read before shipping E2EE)

This is a **teaching example**, not a complete secure messenger. Real systems add:

1. **Key authenticity.** The server hands out public keys — a malicious server could hand out its own (man-in-the-middle). Real apps let users **verify safety numbers / QR codes**, or use key transparency logs.
2. **Forward secrecy and post-compromise security.** One static key per pair means a stolen private key decrypts all past messages. The **Signal protocol** (X3DH + Double Ratchet) rotates keys per message. Use a vetted library (e.g. libsignal) rather than inventing a protocol.
3. **Multiple devices** per user (a key per device; messages encrypted per device).
4. **Groups:** sender keys or MLS (Messaging Layer Security, RFC 9420) instead of pairwise encryption to every member.
5. **Key storage:** non-extractable `CryptoKey` objects can be stored in IndexedDB (structured clone) so they survive reloads without exposing raw key material.
6. **Replay protection & ordering:** include message ids/counters in the AAD.
7. **Server-side features you lose:** search, moderation, link previews and push notification content on the server. Plan alternatives (client-side search, generic push text, client-side reporting).

Interplay with Socket.IO features

- Binary payloads use attachments (Chapter 7) — fine; with the msgpack parser they're inline.
- `maxHttpBufferSize` still applies to ciphertext size (encryption adds IV + 16-byte tag).
- Connection state recovery and offline outboxes work unchanged — they store opaque bytes.
- Logging: never log decrypted content on the client in production; on the server there's nothing to log but metadata.

21.6 Interview questions for this chapter

Q1. [S] Socket.IO already buffers emits while offline — why build an outbox? The buffer is in memory: lost on reload/app kill; and it doesn't retry when an ack is lost. An outbox persists commands with client-generated ids, retries with the same id on every reconnect, and relies on server idempotency → at-least-once delivery, effectively-once processing.

Q2. [M] How does the server make retries safe? A unique constraint on the client-generated id (`INSERT … ON CONFLICT DO NOTHING`), and it acks duplicates as success.

Q3. [M] Why send outbox items one at a time? To preserve order and avoid flooding; stop at the first transient failure and resume later.

Q4. [S] What is the difference between TLS and E2EE in a Socket.IO app? TLS protects the hop client ↔ server (server sees plaintext). E2EE encrypts on the sender device and decrypts on the recipient device; the server only relays ciphertext but still sees metadata.

Q5. [S] Which primitives would you use for a simple E2EE chat over Socket.IO, and what's missing for production? ECDH (P-256/X25519) + HKDF + AES-GCM with random IVs and AAD. Missing: key verification (MITM by server), forward secrecy (Double Ratchet/Signal), multi-device and group key management (sender keys/MLS), secure key storage, replay protection.

Q6. [M] How do encrypted bytes travel through Socket.IO? As binary arguments (`Uint8Array`) → sent as binary attachments over WebSocket frames (or base64 in polling); arrive as `Buffer` in Node and `ArrayBuffer` in browsers.

Next: Chapter 22 — Complete options reference

CHAPTER 22

Complete Options Reference (Server and Client)

> **Goal:** every configuration option of the Socket.IO server and client — not just the common ones — with its default, what it does, and when you'd change it.
>
> **Source of truth:** the option lists were extracted from the TypeScript declarations shipped in `socket.io` 4.8.4, `engine.io` 6.6.11, `socket.io-client` 4.8.4 and `engine.io-client` 6.6.x, and the defaults from the code that applies them. Where the declaration comment and runtime behaviour differ, the runtime behaviour is given.

The Socket.IO server's options are the union of three layers: **Socket.IO options** + **Engine.IO server options** + **Engine.IO attach options**. The client's options are **Manager options** + **Socket options** + **Engine.IO client options**. All can be passed in the single options object.

22.1 Server — Socket.IO layer

Option	Default	What it does	When to change
`path`	`"/socket.io/"`	HTTP path the server intercepts	Serving under a prefix (`/api/socket.io/`), or several Socket.IO servers on one host. Must match the client's `path`
`serveClient`	`true`	Serve `/socket.io/socket.io.js` (and `.map` , `.esm` variants)	Set `false` when you bundle `socket.io-client` (smaller attack surface, no version disclosure)
`adapter`	in-memory `Adapter`	Room storage and broadcasting implementation	Multi-server: Redis, Redis Streams, Mongo, Postgres, cluster... (Chapter 8)
`parser`	default `socket.io-parser`	Packet encoder/decoder	msgpack or custom parser (Chapter 19) — same on clients
`connectTimeout`	`45000` ms	Time for a client to join a namespace after the Engine.IO handshake before the raw connection is closed	Lower to drop half-open connections faster; it's also the delay before a client stuck in a middleware that never calls `next()` is cut

Option	Default	What it does	When to change
`connectionStateRecovery`	disabled	`{ maxDisconnectionDuration: 120000, skipMiddlewares: true }` when enabled — restore id, rooms, data, missed packets	Enable for short-disconnection resilience with a compatible adapter (Chapter 5.8)
`cleanupEmptyChildNamespaces`	`false`	Remove dynamic child namespaces once they have no sockets	Enable with dynamic namespaces per tenant/room

22.2 Server — Engine.IO layer

Option	Default	What it does	When to change
`pingInterval`	`25000` ms	Interval between server PINGs	Faster dead-connection detection (lower) or fewer wakeups (higher); keep below every proxy idle timeout
`pingTimeout`	`20000` ms	Time to wait for a PONG before closing (`ping timeout`)	Raise on very slow networks; lower for faster detection
`upgradeTimeout`	`10000` ms	Max time to complete a transport upgrade	Rarely
`maxHttpBufferSize`	`1e6` (1 MB)	Max size of one message (and polling body); larger → connection closed	Lower (e.g. `1e5`) for small JSON apps (DoS protection); raise deliberately for bigger payloads
`allowRequest`	—	`(req, cb) ⇒ cb(err, allowed)` called for handshake/upgrade requests	Origin checks (CSWSH), IP rules, early rejection
`transports`	`["polling", "websocket" ]`	Allowed transports (`"webtransport"` opt-in)	WebSocket-only setups; enable WebTransport
`allowUpgrades`	`true`	Allow polling → WebSocket/WebTransport upgrades	Rarely disabled (debugging)
`perMessageDeflate`	`false`	WebSocket compression (`true` or `ws` options; threshold defaults to 1024)	Large compressible messages; costs memory/CPU per connection
`httpCompression`	`true` (threshold 1024 bytes)	gzip/deflate for polling responses	Disable if a proxy already compresses

Option	Default	What it does	When to change
`wsEngine`	`require("ws").Server`	WebSocket server implementation (must match the `ws` interface)	Alternative engines (e.g. the C++ `eiows` addon mentioned in engine.io's docs) — verify maintenance
`initialPacket`	—	Extra packet concatenated to the handshake response	Rare
`cookie`	`false`	Set an `io` cookie (`name` , `path` , `httpOnly` , `sameSite` , …; defaults when enabled: `name: "io"` , `path: "/"` , `httpOnly: true` , `sameSite: "lax"`)	Cookie-based sticky sessions at a load balancer
`cors`	— (disabled)	Options for the `cors` package, applied to HTTP (polling) requests	Cross-origin browsers. **Doesn't protect WebSocket** (Chapter 6.5)
`allowEIO3`	`false`	Accept Socket.IO v2 clients (Engine.IO protocol 3)	During v2 → v4 migrations

22.3 Server — attach layer

Option	Default	What it does	When to change
`path`	(see above)	Path to capture	—
`destroyUpgrade`	`true`	For WebSocket upgrade requests on **other** paths, end the socket if nothing has written a response within `destroyUpgradeTimeout` (engine.io checks `socket.bytesWritten ⩽ 0`)	Set `false` if another upgrade handler on the same HTTP server may take **longer than the timeout** to answer (e.g. it does async authentication before completing the handshake). A second WebSocket server that answers immediately is not affected
`destroyUpgradeTimeout`	`1000` ms	Delay before unhandled upgrade requests are ended	Rarely
`addTrailingSlash`	`true`	Normalize the path with a trailing slash (`/socket.io/`)	Set `false` (and on the client) if a proxy/router strips trailing slashes and you get 404s

When `destroyUpgrade` bites: "my other WebSocket endpoint on the same server fails to connect, but only when its handshake is slow" → its upgrade handler awaited something (DB/auth) for more than 1 s before answering, and engine.io ended the idle socket. Set `destroyUpgrade: false` or raise `destroyUpgradeTimeout` . (Behaviour read from `engine.io` 6.6.11 `attach()` .)

22.4 Client — Manager options (the connection)

Option	Default	What it does
`forceNew`	`false`	Always create a new Manager (separate connection)
`multiplex`	`true`	Reuse the cached Manager for other namespaces on the same URL
`path`	`"/socket.io/"`	HTTP path (must match server)
`reconnection`	`true`	Automatic reconnection
`reconnectionAttempts`	`Infinity`	Max attempts before `reconnect_failed`
`reconnectionDelay`	`1000` ms	Initial back-off
`reconnectionDelayMax`	`5000` ms	Max back-off (cap applied after jitter)
`randomizationFactor`	`0.5`	Jitter (±50%)
`timeout`	`20000` ms	Timeout of each connection attempt (`connect_error: timeout`)
`autoConnect`	`true`	Connect immediately in `io()` ; set `false` to call `socket.connect()` later (e.g. after login, after registering listeners)
`parser`	default	Must match the server's parser

22.5 Client — Socket options (per namespace)

Option	Default	What it does
`auth`	—	Object or `(cb) ⇒ cb({...})` sent in the CONNECT packet → `socket.handshake.auth` . Use the function form to refresh tokens on every attempt
`ackTimeout`	—	Default timeout for acks; enables the error-first ack signature
`retries`	—	Number of retries for unacknowledged emits (requires acks; packets queued one at a time)

22.6 Client — Engine.IO options (transport level)

Option	Default	What it does / when to use
`transports`	`["polling", "websocket", "webtransport"]`	Order of transports. Accepts names or (4.8+) implementations: `[Fetch, WebSocket]`
`tryAllTransports`	`false`	(4.8+) If the first transport fails, try the next ones instead of aborting
`upgrade`	`true`	Try to upgrade from polling
`rememberUpgrade`	`false`	If WebSocket worked before (in this page), start directly with it
`query`	—	Extra query parameters on every Engine.IO request (`socket.handshake.query`)
`extraHeaders`	—	Extra HTTP headers. **In browsers only applies to polling**; Node applies them to all transports
`withCredentials`	`false`	Send cookies with cross-origin polling requests (needed for cookie auth and LB-cookie stickiness — verified in Chapter 23). In Node, also enables the client's cookie jar
`timestampRequests`	timestamps on (the client adds `t=` unless set to `false`)	Cache-busting parameter on polling requests
`timestampParam`	`"t"`	Name of that parameter
`requestTimeout`	—	Timeout for polling HTTP requests (ms)
`transportOptions`	`{}`	Per-transport overrides, e.g. `{ polling: { extraHeaders }, webtransport: { serverCertificateHashes } }`
`closeOnBeforeunload`	`false`	Close silently on `beforeunload` (uniform behaviour across browsers)
`addTrailingSlash`	`true`	Match the server's setting
`protocols`	`[]`	WebSocket sub-protocols
`perMessageDeflate`	`{ threshold: 1024 }`	WebSocket compression settings (Node client)
`useNativeTimers`	`false`	Use native timers even if mocked (fake timers in tests)
`autoUnref`	`false`	Heartbeat timer doesn't keep the Node process alive
`host` , `hostname` , `secure` , `port`	from URL	Override parts of the URL
`agent`	`false`	HTTP agent (Node), e.g. for proxies

Option	Default	What it does / when to use
`pfx`, `key`, `passphrase`, `cert`, `ca`, `ciphers`	—	TLS options (Node)
`rejectUnauthorized`	`true`	Verify TLS certificates (Node). Never `false` in production

22.7 Configuration recipes

Small internal app, single server

```
new Server(httpServer, { serveClient: false, maxHttpBufferSize: 1e5, cors: { origin: APP_ORIGIN
} });
```

Public SPA behind a load balancer, cookies for auth

```
// server
new Server(httpServer, {
  serveClient: false,
  cors: { origin: ["https://app.example.com"], credentials: true },
  allowRequest: (req, cb) => cb(null, req.headers.origin === "https://app.example.com"),
  adapter: createAdapter(pub, sub),
  maxHttpBufferSize: 1e5,
});
// client
io("https://rt.example.com", { withCredentials: true });
```

Mobile apps only, no sticky sessions

```
// server
new Server(httpServer, { transports: ["websocket"], serveClient: false, adapter:
createAdapter(pub, sub) });
// clients: transports ["websocket"] / forceWebsockets(true) / setTransports(['websocket'])
```

High-churn mobile networks, recovery

```
new Server(httpServer, {
  pingInterval: 20000, pingTimeout: 15000,
  connectionStateRecovery: { maxDisconnectionDuration: 60_000 },
```

```
  adapter: createAdapter(redisClient),   // from "@socket.io/redis-streams-adapter" (supports recovery)
});
```

Sharing the HTTP server with another WebSocket service whose handshake is slow

```
new Server(httpServer, { path: "/socket.io/", destroyUpgrade: false });
```

Large fleets (thundering herd protection) — client

```
io(URL, { reconnectionDelay: 2000, reconnectionDelayMax: 30000, randomizationFactor: 0.5 });
```

Next: Chapter 23 — Hands-on exercises and labs (with solutions)

CHAPTER 23

Hands-On Labs and Exercises (with Solutions)

> **Goal:** turn reading into skill. Four labs let you *see* the protocol, the load-balancer behaviour and session recovery with your own eyes; then 18 exercises from beginner to senior, each with a solution.
>
> **Every lab output below is real** (Socket.IO 4.8.4, Node 22, nginx 1.24, HAProxy 2.8, Redis 7). Exercises 5, 6 and 7 were run as automated tests and pass; the others reuse code already verified in earlier chapters (references given).

Setup for all labs:

```
mkdir sio-labs && cd sio-labs && npm init -y
npm install socket.io socket.io-client @socket.io/redis-adapter redis
```

Lab A — Speak the protocol by hand with `curl`

Goal: understand exactly what a Socket.IO client does, by doing it yourself.

```
// lab-server.js
const { Server } = require("socket.io");
const io = new Server(3000);
io.on("connection", (socket) => {
  socket.on("hello", (name, ack) => ack(`hi ${name}!`));
  socket.emit("welcome", "you are connected");
});
```

Run `node lab-server.js`, then in another terminal:

```
B="http://localhost:3000/socket.io/?EIO=4&transport=polling"

# 1) Engine.IO handshake
OPEN=$(curl -s "$B"); echo "$OPEN"
SID=$(echo "${OPEN:1}" | python3 -c "import sys,json;print(json.load(sys.stdin)['sid'])")

# 2) Socket.IO CONNECT to the main namespace
curl -s -X POST -d '40' "$B&sid=$SID"

# 3) Receive (long-poll): the CONNECT reply + the "welcome" event, separated by \x1e
```

```
curl -s "$B&sid=$SID" | cat -v

# 4) Emit "hello" with ack id 1
curl -s -X POST -d '421["hello","curl"]' "$B&sid=$SID"

# 5) Receive the ack
curl -s "$B&sid=$SID"

# 6) A made-up session id (what you get without sticky sessions)
curl -s "http://localhost:3000/socket.io/?EIO=4&transport=polling&sid=doesnotexist"

# 7) An old (v2) client protocol version
curl -s "http://localhost:3000/socket.io/?EIO=3&transport=polling"
```

Real output:

```
1) 0{"sid":"wZjDFReArxzgbgpgAAAA","upgrades":
["websocket"],"pingInterval":25000,"pingTimeout":20000,"maxPayload":1000000}
2) ok
3) 40{"sid":"d2PtE1O5hpksWhMjAAAA"}^^42["welcome","you are connected"]
4) ok
5) 431["hi curl!"]
6) {"code":1,"message":"Session ID unknown"}
7) {"code":5,"message":"Unsupported protocol version"}
```

(`^^` is how `cat -v` prints the `\x1e` record separator.)

Questions to answer yourself (answers in Chapter 7): Why are the two `sid` values different? What does the `4` prefix mean? What would happen if you waited 45+ seconds before step 3 without answering pings? (*The server sends `2` pings in the long-poll responses; if you don't POST `3` back within `pingTimeout`, the session is closed.*)

Lab B — Load balancers, sticky sessions and the Redis adapter

Goal: reproduce the two scaling problems (Chapter 8) and their fixes.

Requirements: Redis on `localhost:6379` (here: port 6390), nginx, HAProxy.

```
// node.js — one Socket.IO node; run 3 copies on ports 5001-5003
const { Server } = require("socket.io");
const { createClient } = require("redis");
const { createAdapter } = require("@socket.io/redis-adapter");

(async () => {
```

```
  const pub = createClient({ url: "redis://localhost:6390" }); const sub = pub.duplicate();
  pub.on("error", console.error); sub.on("error", console.error);
  await Promise.all([pub.connect(), sub.connect()]);
  const port = Number(process.argv[2]);
  const io = new Server(port, { adapter: createAdapter(pub, sub) });
  io.engine.on("connection_error", (e) => console.log(`[${port}] engine error code=${e.code}
${e.message}`));
  io.on("connection", (s) => {
    s.join("all");
    s.on("whoami", (ack) => ack({ node: port, transport: s.conn.transport.name }));
    s.on("shout", (m) => io.to("all").emit("shout", `${m} (via node ${port})`));
  });
})();
```

```
# nginx.conf (listen 8080) — the Chapter 8 configuration
events { worker_connections 1024; }
http {
  map $http_upgrade $connection_upgrade { default upgrade; '' close; }
  upstream socket_nodes {
    hash $remote_addr consistent;
    server 127.0.0.1:5001; server 127.0.0.1:5002; server 127.0.0.1:5003;
  }
  server {
    listen 8080;
    location /socket.io/ {
      proxy_pass http://socket_nodes;
      proxy_http_version 1.1;
      proxy_set_header Upgrade $http_upgrade;
      proxy_set_header Connection $connection_upgrade;
      proxy_set_header Host $host;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_read_timeout 120s;
      proxy_send_timeout 120s;
      proxy_buffering off;
    }
  }
}
```

```
# haproxy-cookie.cfg (port 8081): round-robin + cookie stickiness
defaults
  mode http
  timeout connect 5s
  timeout client 120s
  timeout server 120s
  timeout tunnel 1h
frontend http_front
  bind *:8081
  default_backend socket_nodes
backend socket_nodes
  balance roundrobin
  cookie SERVERID insert indirect nocache
  server app01 127.0.0.1:5001 check cookie app01
```

```
  server app02 127.0.0.1:5002 check cookie app02
  server app03 127.0.0.1:5003 check cookie app03
```

Make a second HAProxy config on port 8082 **without** the two `cookie` parts (pure round-robin). Start: three nodes, `nginx -c $PWD/nginx.conf`, `haproxy -f haproxy-cookie.cfg`, `haproxy -f haproxy-nosticky.cfg`. Then connect clients:

```
// lbtest.js (abridged)
const test = (label, url, opts) => /* connect with reconnection:false, call "whoami" */;
await test("nginx (polling→websocket)", "http://localhost:8080");
await test("haproxy cookie sticky, polling only", "http://localhost:8081", { transports:
["polling"], withCredentials: true });
await test("haproxy cookie sticky, NO withCredentials", "http://localhost:8081", { transports:
["polling"] });
await test("haproxy NO sticky, polling only", "http://localhost:8082", { transports:
["polling"] });
await test("haproxy NO sticky, websocket only", "http://localhost:8082", { transports:
["websocket"] });
// cross-node broadcast: client A on node 5001 shouts, client B on node 5002 listens
```

Real results:

```
nginx (polling→websocket): OK node=5001 transport=websocket (client: websocket)
haproxy cookie sticky, polling only: OK (3 runs → nodes 5001, 5002, 5003)
haproxy cookie sticky, NO withCredentials: connect_error xhr poll error / xhr post error (3 of
3 runs)
haproxy NO sticky, polling only: connect_error xhr poll error
haproxy NO sticky, websocket only: OK node=5002 transport=websocket
cross-node broadcast received by B: hello (via node 5001)

node logs:  [5002] engine error code=1 Session ID unknown   (several, from the failing cases)
```

What you learned: (1) polling needs stickiness, WebSocket-only doesn't; (2) cookie stickiness only works if the client sends the cookie back (`withCredentials: true` — in browsers this matters for cross-origin setups); (3) the failure signature is always `Session ID unknown` (engine error code 1); (4) the Redis adapter makes broadcasts cross nodes.

Extension: remove `proxy_http_version 1.1` and the `Upgrade` / `Connection` headers from the nginx config and watch the client stay on `polling`.

Lab C — Connection state recovery

```
const { Server } = require("socket.io");
const { io } = require("socket.io-client");
```

```
const srv = new Server(4600, { connectionStateRecovery: { maxDisconnectionDuration: 60_000 }
});
srv.on("connection", (socket) => {
  console.log(`server: connection id=${socket.id} recovered=${socket.recovered}
rooms=${[...socket.rooms]}`);
  if (!socket.recovered) {
    socket.join("news");
    socket.data.name = "alice";
  }
  socket.on("cut-me", () => socket.conn.close());     // simulate a network drop (recoverable)
});

const client = io("http://localhost:4600", { reconnectionDelay: 300, reconnectionDelayMax: 300
});
client.on("connect", () => console.log(`client: connect id=${client.id}
recovered=${client.recovered}`));
client.on("disconnect", (r) => {
  console.log("client: disconnect", r);
  srv.to("news").emit("news", "sent while you were offline");   // broadcast during the outage
});
client.on("news", (m) => console.log("client: news →", m));

let step = 0;
client.on("connect", async () => {
  if (++step === 1) {
    srv.to("news").emit("news", "first news");
    setTimeout(() => client.emit("cut-me"), 200);
  } else {
    const [s] = await srv.fetchSockets();
    console.log("server: socket.data after recovery =", s.data);
    setTimeout(() => { client.close(); srv.close(); }, 200);
  }
});
```

Real output:

```
server: connection id=kZ7vlpsnXqdTmdgRAAAA recovered=false rooms=kZ7vlpsnXqdTmdgRAAAA
client: connect id=kZ7vlpsnXqdTmdgRAAAA recovered=false
client: news → first news
client: disconnect transport close
server: connection id=kZ7vlpsnXqdTmdgRAAAA recovered=true rooms=kZ7vlpsnXqdTmdgRAAAA,news
client: news → sent while you were offline
client: connect id=kZ7vlpsnXqdTmdgRAAAA recovered=true
server: socket.data after recovery = { name: 'alice' }
```

What you learned: after recovery the **same socket id**, **rooms** and `socket.data` come back, and the event broadcast during the outage is replayed. Note the order: the missed `news` arrived **before** the client's `connect` handler ran — so register your event listeners before connecting, and don't assume `connect` comes first.

Extension: change `socket.conn.close()` to `socket.disconnect()` (a deliberate server-side disconnect) — recovery won't happen (`io server disconnect` is not a recoverable reason and the client doesn't even reconnect automatically).

Lab D — WebTransport

Follow Chapter 18 step by step (certificate, HTTP/3 server, Node client with the `@fails-components/webtransport` implementation). Expected: `client: transport = webtransport`, and with default client transports, polling → **upgrade to webtransport**.

Exercises

Levels: **[E]** entry, **[M]** mid, **[S]** senior. Try each before reading the solution.

Exercise 1 [E] — Echo with acknowledgement

Write a server event `echo` that returns whatever it receives, and a client that prints the response with a 3-second timeout.

Solution

```
// server
io.on("connection", (socket) => {
  socket.on("echo", (data, ack) => { if (typeof ack === "function") ack(data); });
});
// client
const res = await socket.timeout(3000).emitWithAck("echo", { hello: "world" });
console.log(res); // { hello: "world" }
```

Exercise 2 [E] — Join/leave rooms with notifications

Implement `room:join` / `room:leave`; notify **other** members (not the sender) with `room:user-joined` / `room:user-left`; ack with `{ ok: true }`.

Solution

```
socket.on("room:join", async (room, ack) => {
  await socket.join(room);
  socket.to(room).emit("room:user-joined", { id: socket.id, room });
  ack?.({ ok: true });
});
socket.on("room:leave", async (room, ack) => {
  await socket.leave(room);
  socket.to(room).emit("room:user-left", { id: socket.id, room });
  ack?.({ ok: true });
});
socket.on("disconnecting", () => {
  for (const room of socket.rooms) if (room !== socket.id)
socket.to(room).emit("room:user-left", { id: socket.id, room });
});
```

In production: authorize the join first (Chapter 4.7).

Exercise 3 [E] — Private messages that reach every device

User 42 has a phone and two tabs open. Send them a DM from a REST endpoint.

Solution

Join a user room at connection (after auth) and emit to it:

```
io.on("connection", (socket) => socket.join(`user:${socket.data.user.id}`));
app.post("/api/dm", (req, res) => {
  io.to(`user:${req.body.to}`).emit("dm:new", { from: req.user.id, text: req.body.text });
  res.sendStatus(202);
});
```

Never map userId → socket.id (breaks with multiple devices, reconnections, multiple servers).

Exercise 4 [M] — Typing indicator

Throttle to one event per 2 s per user, droppable, excluded sender, auto-expiring on the receiver.

Solution

See Chapter 13.3 (client throttle + `volatile`, server `socket.to(room).volatile.emit`, receiver timeout of ~5 s).

Exercise 5 [M] — Auth with token expiry *(tested)*

Reject connections without a valid token. Disconnect the client exactly when its token expires; the client must not auto-reconnect with the expired token.

Solution

```
io.use((socket, next) => {
  const payload = verifyToken(socket.handshake.auth.token);   // throws/returns null if
invalid
  if (!payload) return next(new Error("UNAUTHORIZED"));
  socket.data.exp = payload.exp * 1000;
  next();
});

io.on("connection", (socket) => {
  const timer = setTimeout(() => {
    socket.emit("auth:expired");
    socket.disconnect(true);
  }, Math.max(0, socket.data.exp - Date.now()));
  socket.on("disconnect", () => clearTimeout(timer));
});

// client
socket.on("disconnect", async (reason) => {
  if (reason === "io server disconnect") {        // no automatic reconnection for this
reason
    await refreshToken();                          // auth is a function → picks up the
new token
    socket.connect();
  }
});
```

Test result: bad token → `connect_error: UNAUTHORIZED`; valid short token → client `disconnect` with reason `io server disconnect` and `socket.active === false`.

Exercise 6 [M] — Rate limiter without libraries *(tested)*

Allow bursts of 5 events, refilling 5 tokens/second; over-limit events must receive `{ ok: false, error: "RATE_LIMITED" }` via their ack and must not reach handlers.

Solution

```
io.on("connection", (socket) => {
  let tokens = 5, last = Date.now();
  socket.use(([event, ...args], next) => {
    const now = Date.now();
```

```
    tokens = Math.min(5, tokens + ((now - last) / 1000) * 5);
    last = now;
    if (tokens < 1) {
      const ack = args.at(-1);
      if (typeof ack === "function") ack({ ok: false, error: "RATE_LIMITED" });
      return;                                  // don't call next(): the handler never
runs
    }
    tokens -= 1;
    next();
  });
});
```

Test result: 8 simultaneous emits → 5 succeeded, 3 got `RATE_LIMITED`. In a cluster, key the limiter by user id in Redis (Chapter 6.7).

Exercise 7 [M] — A live poll with broadcast acks *(tested)*

A host starts a poll; every connected client answers; the host receives all answers within 1 s — and still gets the answers that arrived if someone doesn't respond.

Solution

```
socket.on("start-poll", (question, ack) => {
  // callback form: on timeout `err` is set, but `answers` still has every reply received
  io.timeout(1000).emit("poll", question, (err, answers) => ack({ complete: !err, answers
}));
});

// voters
socket.on("poll", (question, ack) => ack("yes"));
```

Test result: with one connected client that never answered, the result was `{ complete: false, answers: [3 answers] }`. The Promise form (`await io.timeout(1000).emitWithAck(...)`) **rejected** in the same situation and lost the 3 answers — that's why the callback form is the right choice here.

Exercise 8 [M] — Fix the bug: duplicated messages

```
socket.on("connect", () => {
  socket.on("chat", (m) => render(m));
});
```

Solution

Each reconnection adds another listener. Register `socket.on("chat", render)` once, outside the `connect` handler (Chapter 5.6).

Exercise 9 [M] — Fix the bug: events lost right after connecting

```
io.on("connection", async (socket) ⇒ {
  const rooms = await db.roomsFor(socket.data.user.id);
  socket.join(rooms);
  socket.on("message:send", handler);
});
```

Solution

Events sent by the client during the `await` find no listener and are dropped (verified in Chapter 12, C3). Register handlers synchronously first, or load the rooms in an `io.use` middleware and keep the `connection` handler synchronous (Chapter 13.17).

Exercise 10 [M] — Fix the bug: "Cannot GET /socket.io/"

```
const app = express();
const server = http.createServer(app);
const io = new Server(server);
app.listen(3000);
```

Solution

`app.listen()` creates a second HTTP server that Socket.IO isn't attached to. Use `server.listen(3000)`.

Exercise 11 [M] — Fix the bug: the ack callback receives `null` as the response

```
socket.timeout(5000).emit("getUser", 42, (user) ⇒ console.log(user.name));
```

Solution

With `.timeout()`, the first callback argument is the error: `(err, user) ⇒ { if (err) …; console.log(user.name); }`.

Exercise 12 [M] — Write an integration test

Test that a broadcast to a room excludes the sender.

Solution

Chapter 10.3, test "room broadcast excludes sender" (and the lesson about not keeping "the current socket" in a shared variable).

Exercise 13 [S] — Online users across a cluster

Show "N users online" (distinct users, multiple tabs count once), correct with 10 nodes.

Solution

Per-user connection sets in Redis with heartbeats and a grace period (Chapter 13.2); count users with `SCARD` on a set of online user ids updated when a user's connection count goes 0→1 / 1→0. Avoid `fetchSockets()` for this at scale.

Exercise 14 [S] — Never lose a chat message

Messages must survive offline periods, reloads and lost acks, without duplicates.

Solution

Client outbox in IndexedDB with client ids + idempotent server insert (Chapter 21, verified across a simulated reload) + server-side persistence with sync-after-last-id on every connect + client dedupe (Chapter 5.9).

Exercise 15 [S] — Upload a 20 MB file over the socket

Solution

Chunk it (e.g. 256 KB) and await an ack per chunk for backpressure and resumability (Chapter 13.10) — or better, upload over HTTP to object storage and use the socket only for progress.

Exercise 16 [S] — Kick a banned user everywhere, now

Solution

`io.in("user:" + id).emit("session:revoked"); io.in("user:" + id).disconnectSockets(true);` — cluster-wide with an adapter; also revoke tokens so reconnection fails in middleware.

Exercise 17 [S] — Protect against Cross-Site WebSocket Hijacking

Solution

`allowRequest` origin allow-list (verified: forged origin → HTTP 403 for polling / 400 for the WebSocket upgrade, engine code 4), token auth in `auth` rather than ambient cookies, SameSite cookies (Chapter 6.5).

Exercise 18 [S] — Design review

Your team proposes: "one namespace per chat conversation, created dynamically; the client connects to `/conv-<id>` to join a conversation." Review it.

Solution

Namespaces are client-chosen, so authorization must be done in namespace middleware for every connection; thousands of dynamic namespaces cost memory (need `cleanupEmptyChildNamespaces`), each one is a separate Socket object and CONNECT round-trip, and server-side membership changes (removing someone from a conversation) are harder. Rooms (`conversation:<id>`) joined by the server after authorization are simpler and cheaper; keep namespaces for separating concerns (`/admin`) or tenants.

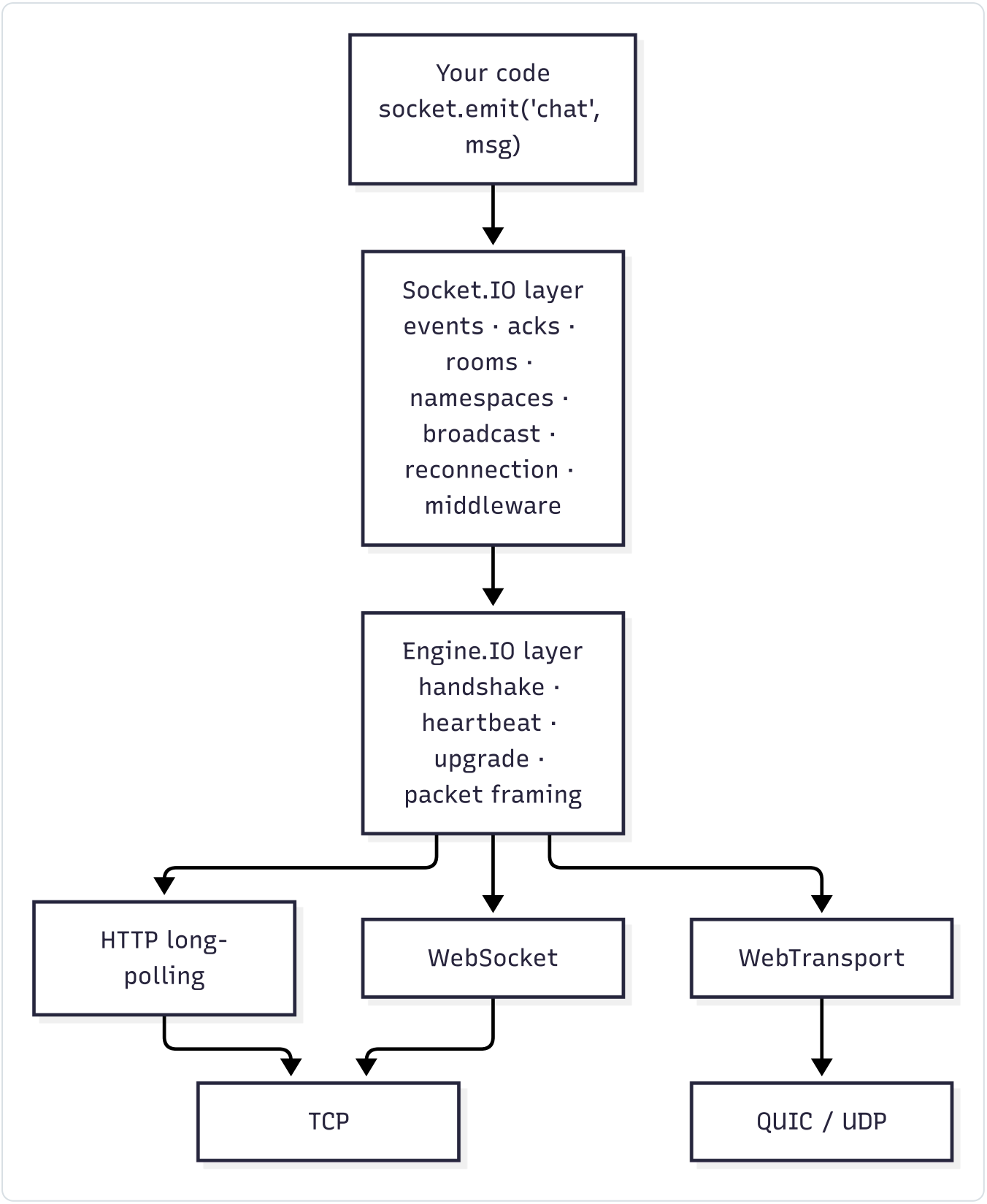
Next: Chapter 24 — Visual guide (diagrams)

CHAPTER 24

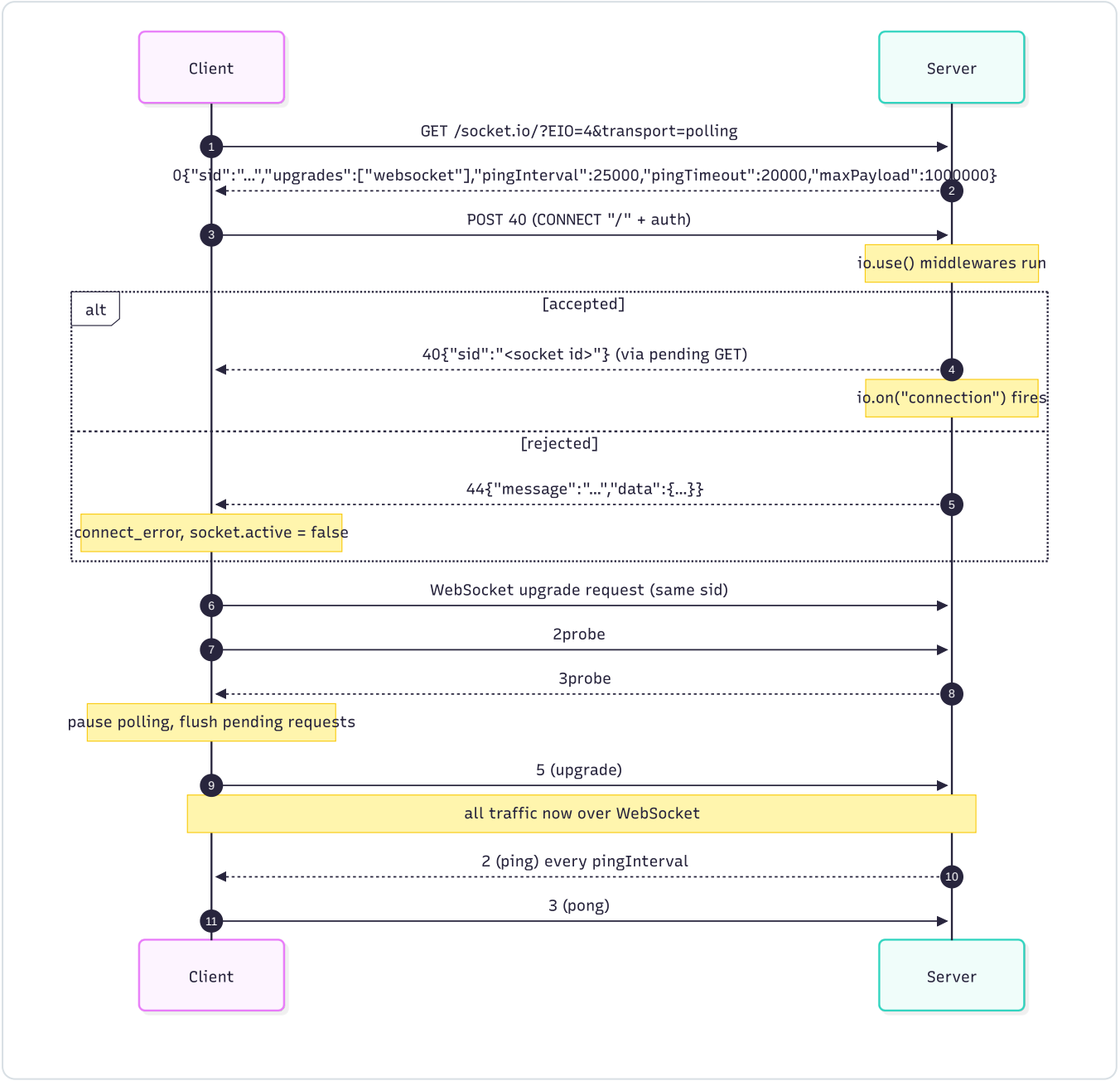
Visual Guide (Diagrams)

> **Goal:** the most important mechanisms of the book as pictures. The diagrams are written in **Mermaid**, which GitHub, GitLab, many IDEs and documentation tools render as real diagrams. Every diagram here was rendered successfully with the Mermaid CLI. The flows match behaviour verified in earlier chapters (packet captures in Chapter 7, recovery lab in Chapter 23).

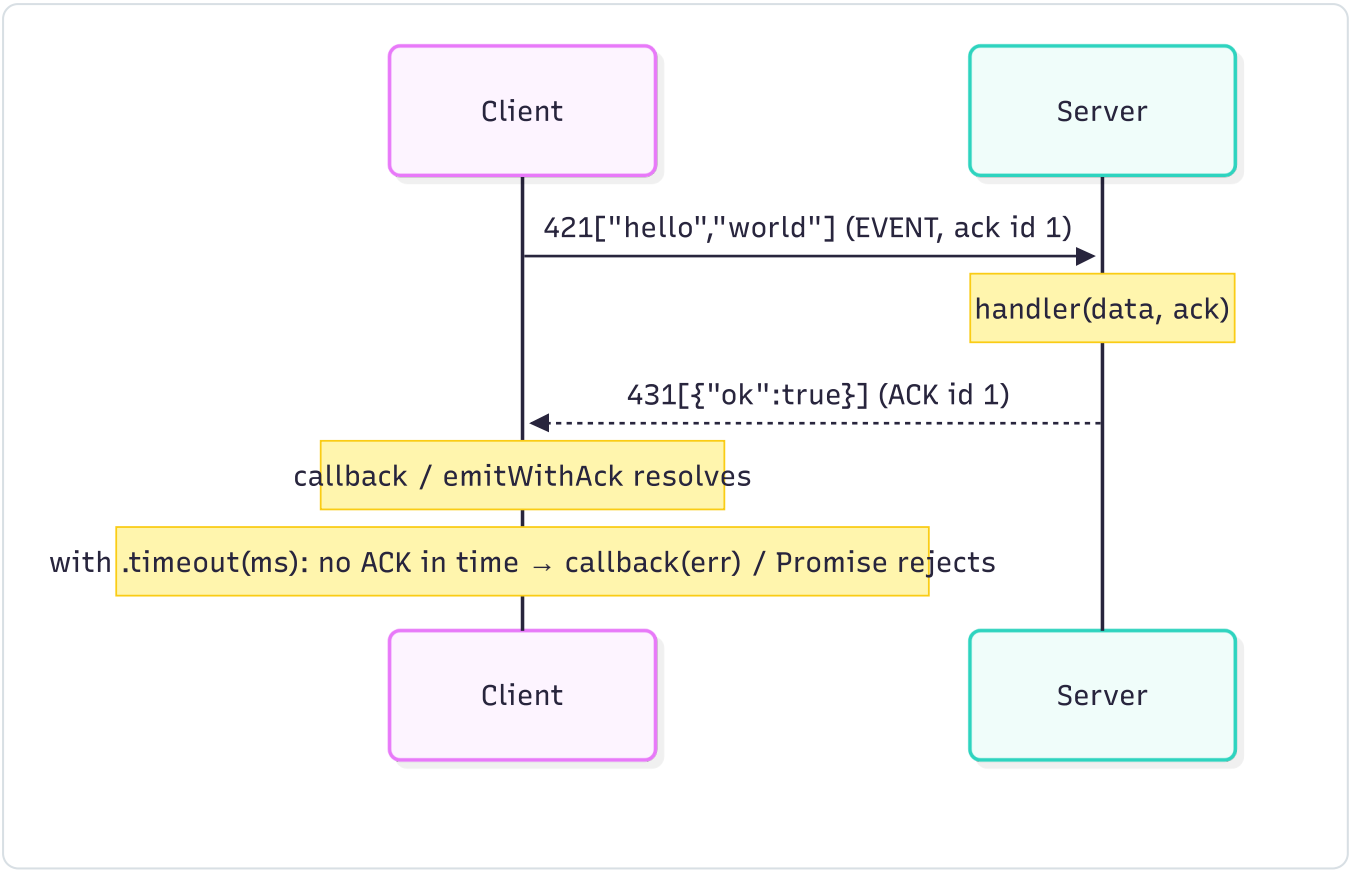
24.1 The layers



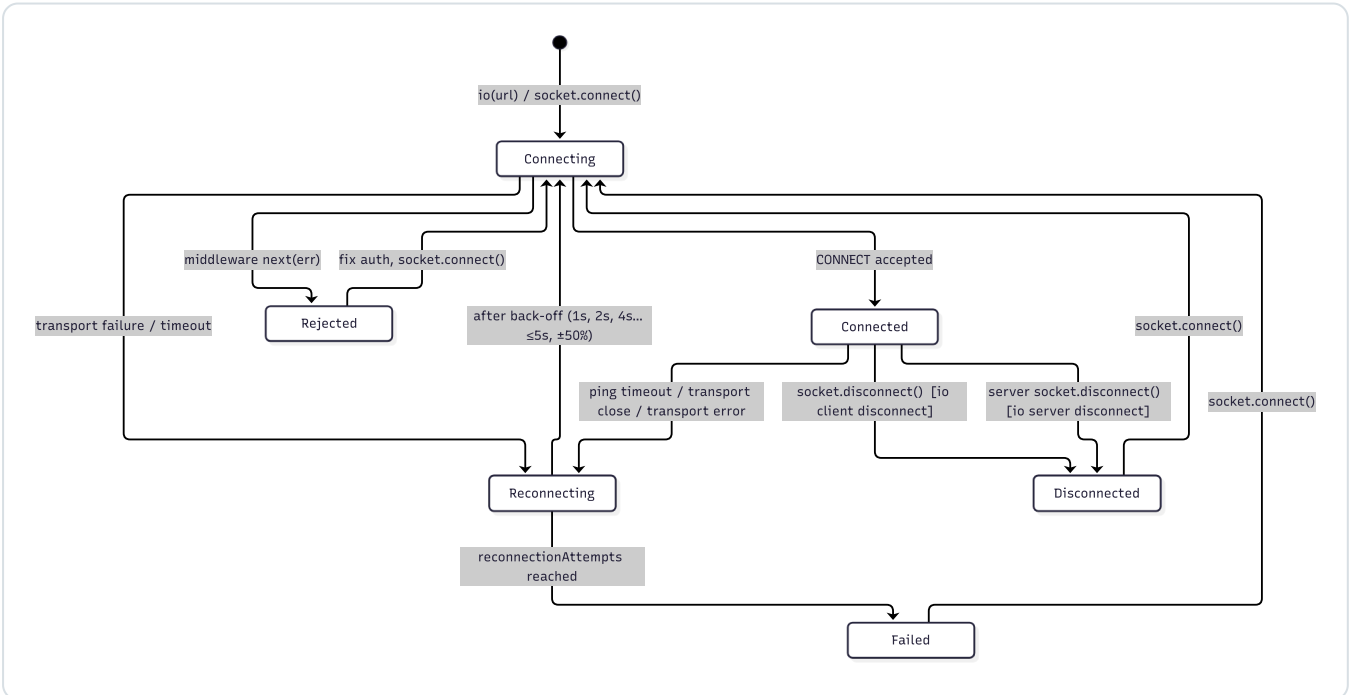
24.2 Connection: handshake, CONNECT and upgrade



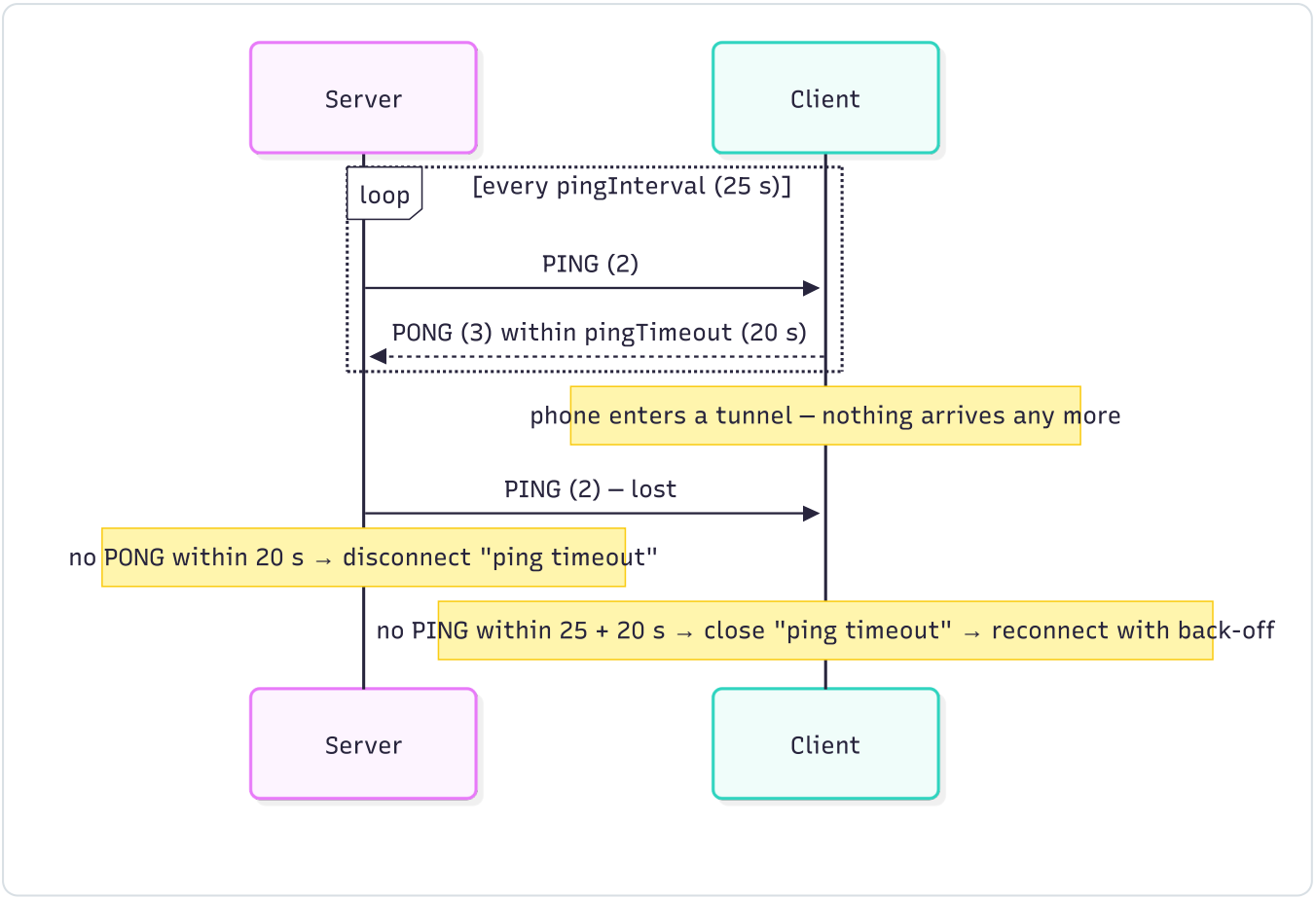
24.3 Event with acknowledgement



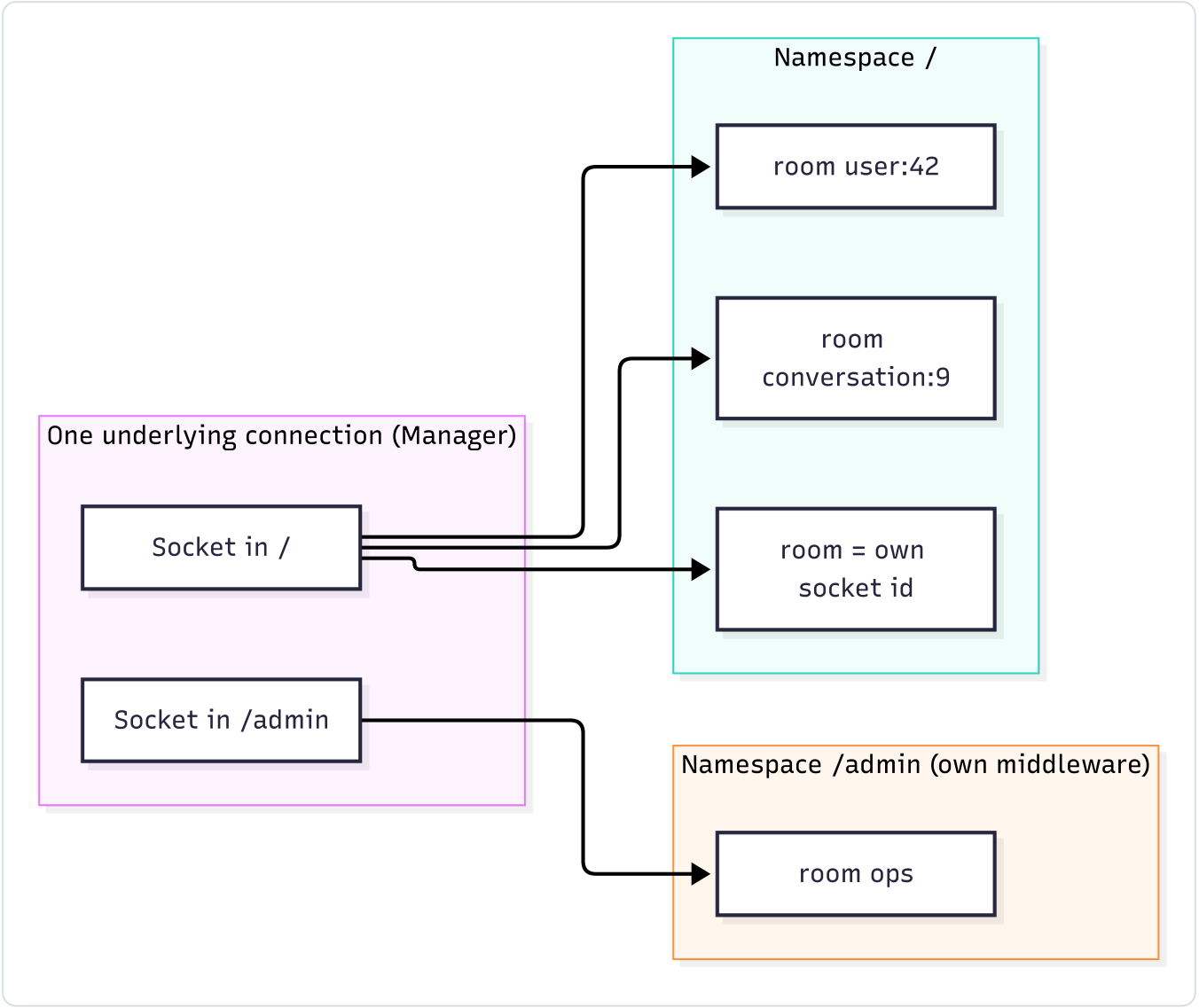
24.4 Client connection state machine



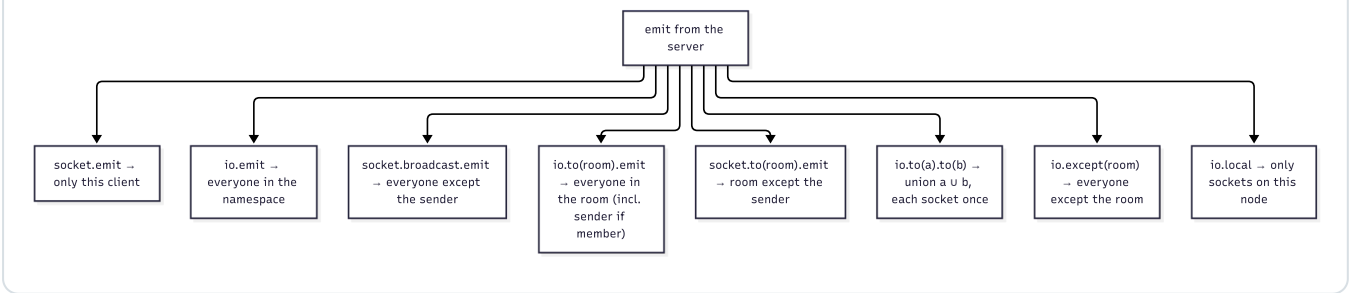
24.5 Heartbeat and dead-connection detection



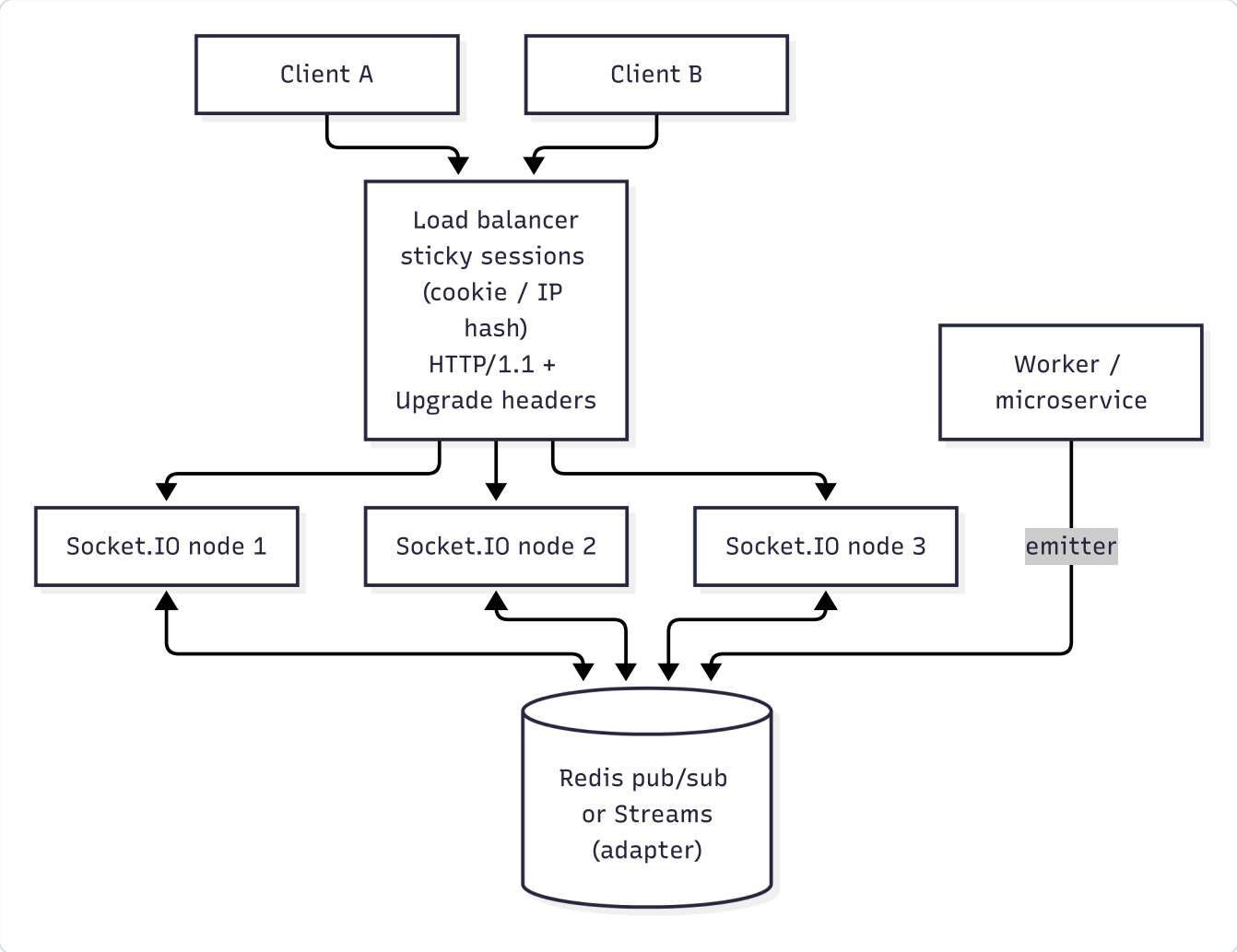
24.6 Namespaces, rooms and sockets



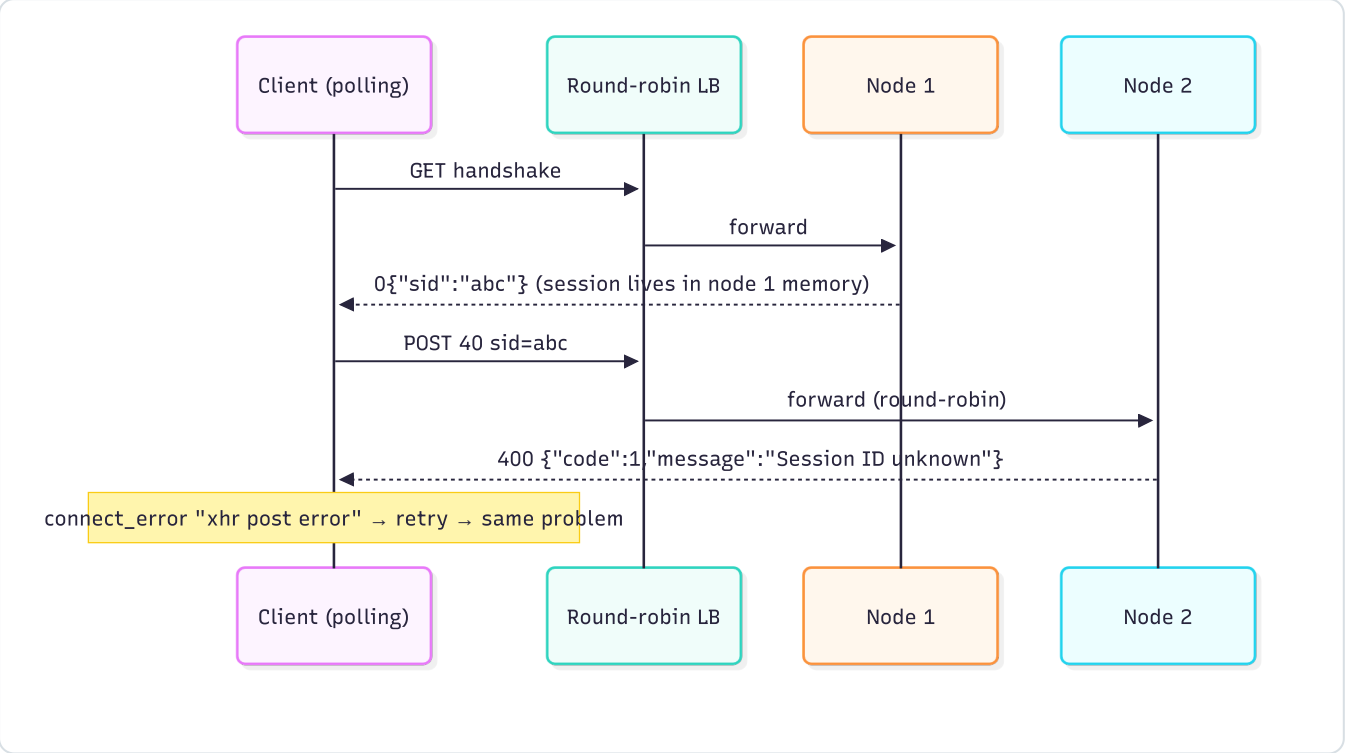
24.7 Broadcast targets



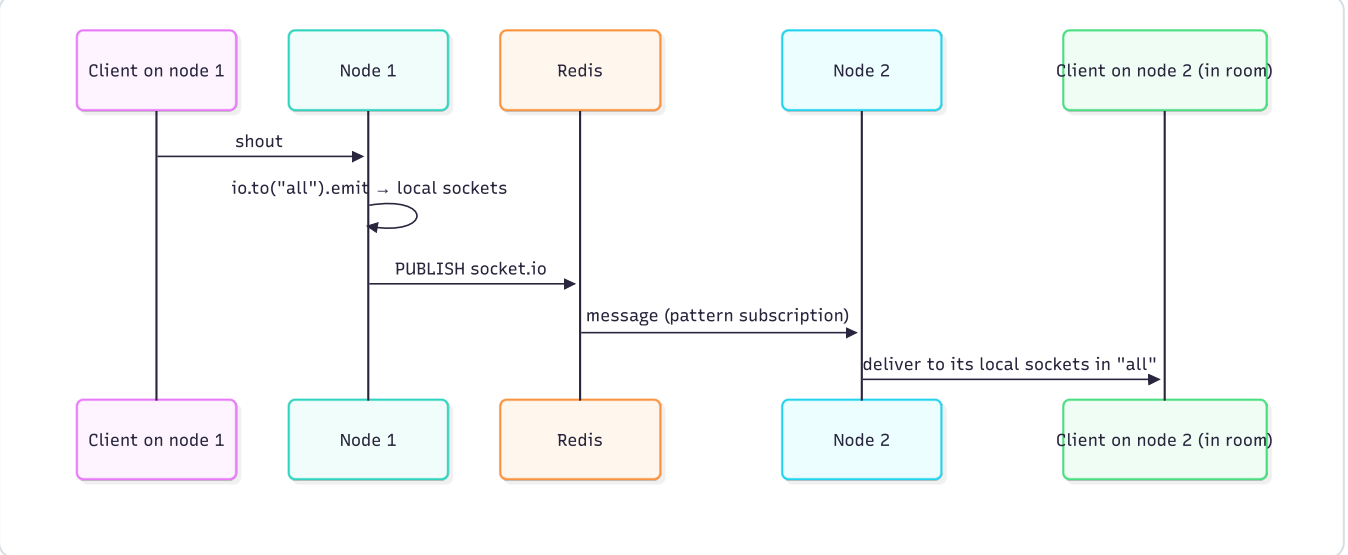
24.8 Scaling: load balancer, sticky sessions and adapter



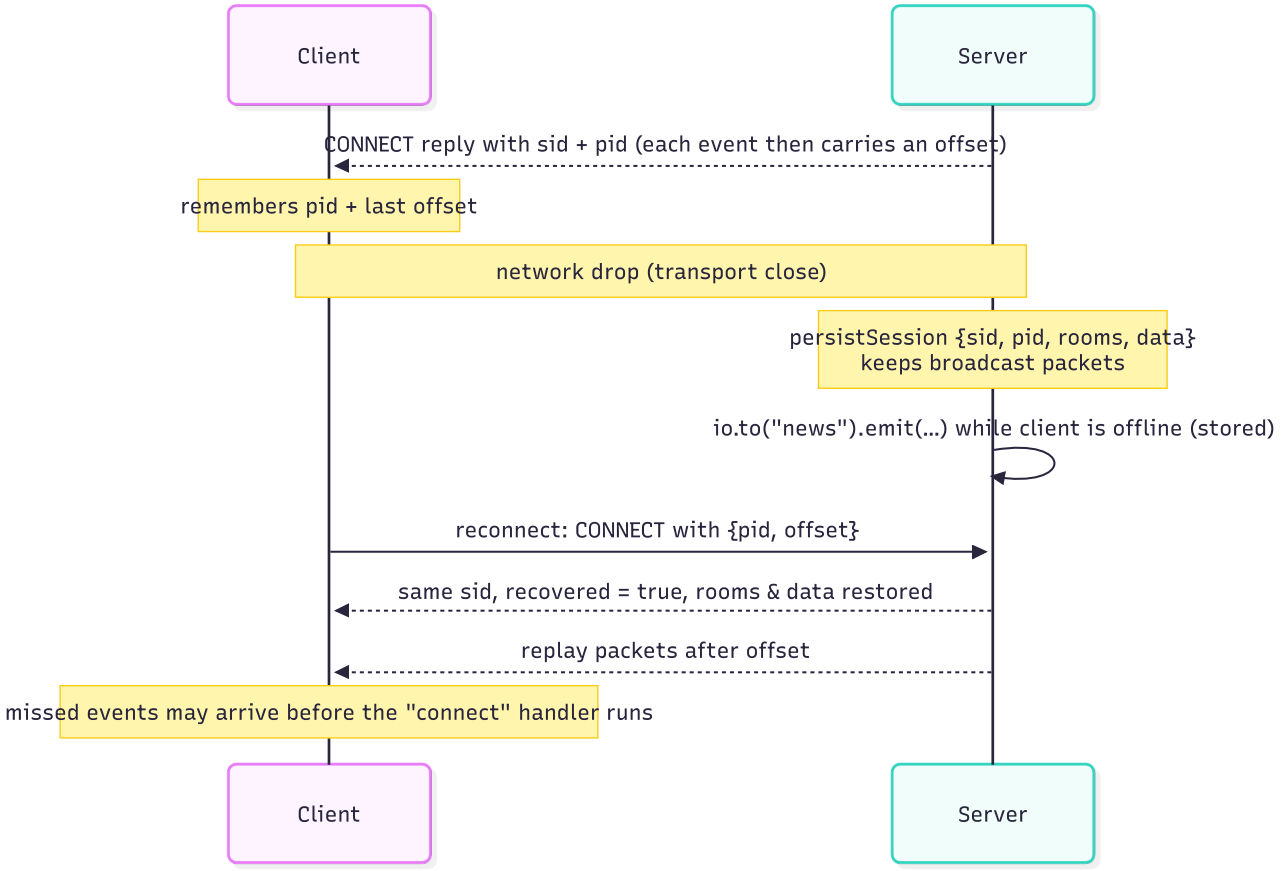
24.9 What happens without sticky sessions (verified in Lab B)



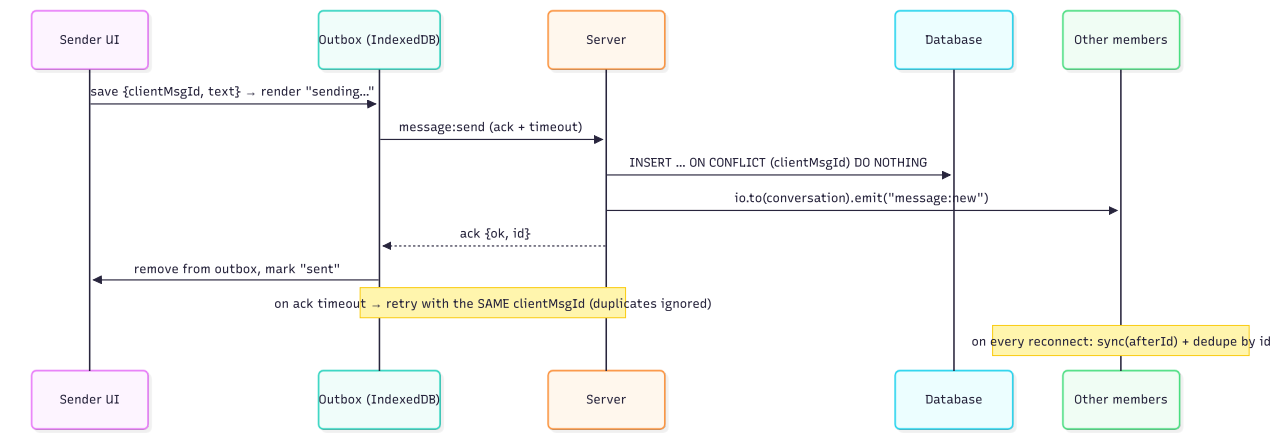
24.10 Cross-node broadcast through the adapter



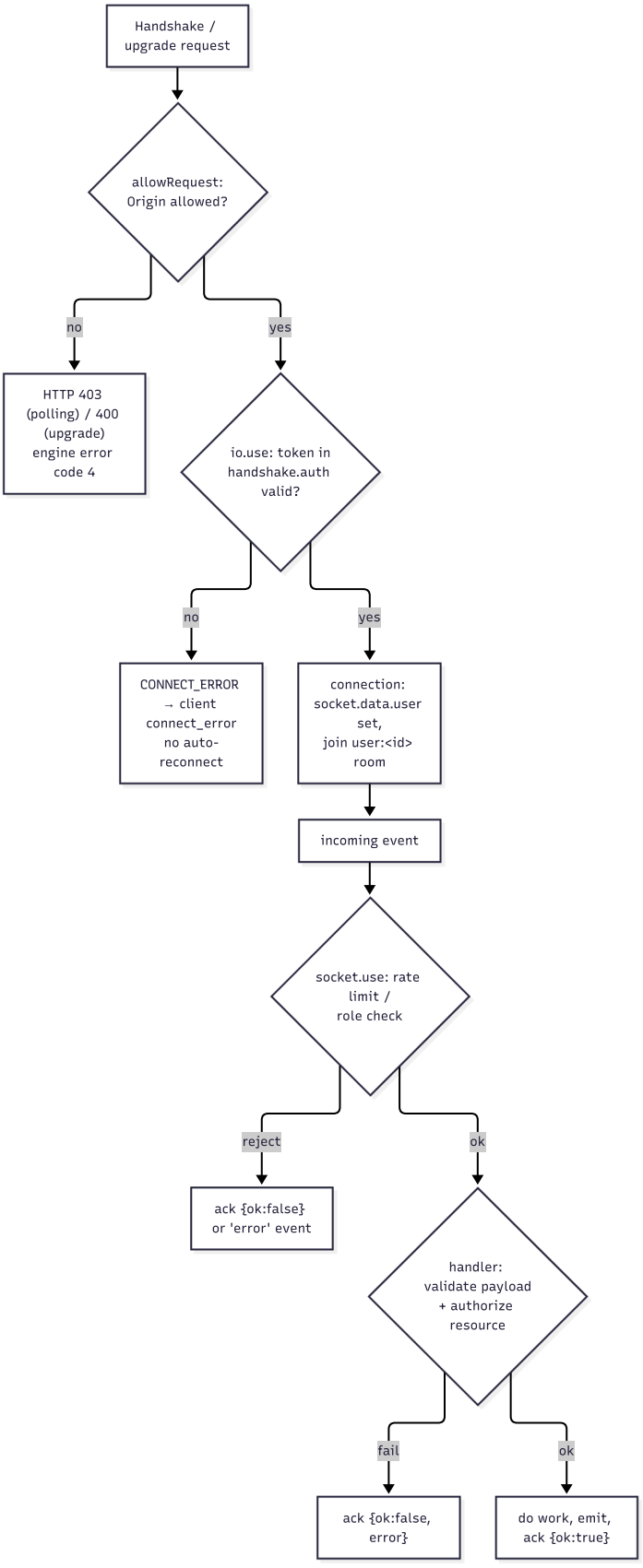
24.11 Connection state recovery (verified in Lab C)



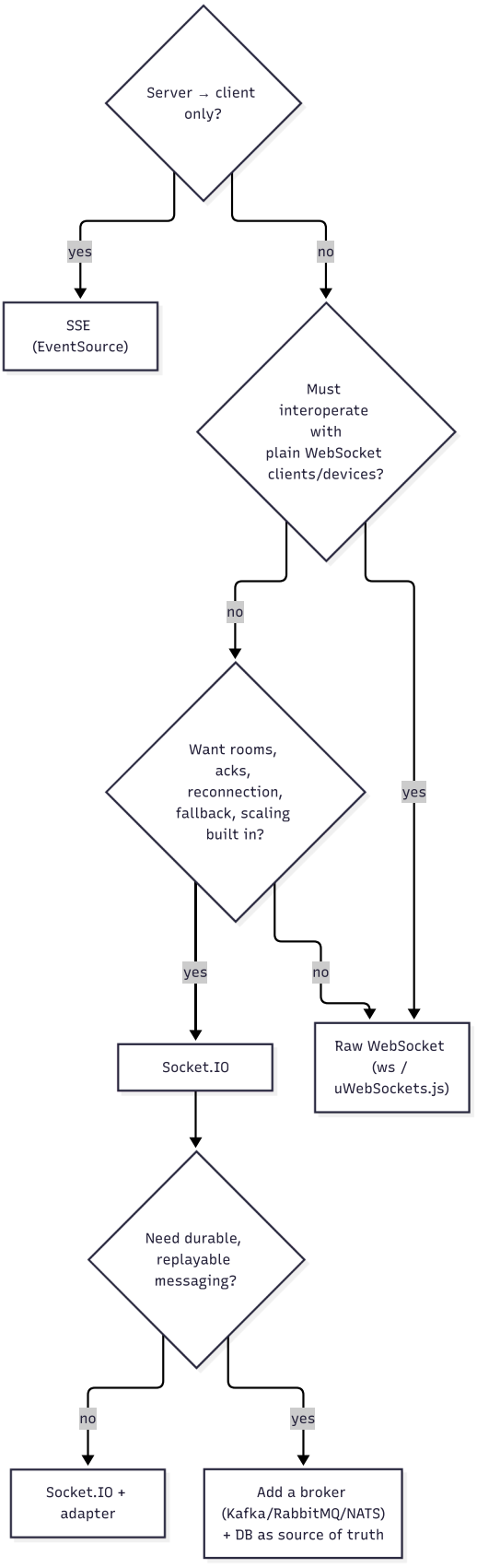
24.12 Chat message flow (optimistic UI, idempotency, sync)



24.13 Authentication and authorization



24.14 Choosing a real-time technology



Next: back to the table of contents.

End of the book. Verified against socket.io 4.8.4, socket.io-client 4.8.4, engine.io 6.6.x, October 2026.